

**Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse
von Rich-Client-Webanwendungen mittels Tool-Reports,
Playwright-Interaktionen und Codeanalyse**

Bachelorarbeit

vorgelegt am 11.05.2026

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

Kurs WWI2023

von

JOEL YERAI MARTINEZ CAMPO

Betreuung in der Ausbildungsstätte:

BITBW

Ilko Hoffmann

Betreuer der Bachelorarbeit

Unterschrift

DHBW Stuttgart:

Sascha Alexander Ströbel

Abstract

Web-Barrierefreiheit ist trotz gesetzlicher Verpflichtung durch den European Accessibility Act (EAA) und das Barrierefreiheitsstärkungsgesetz (BFSG) in der Praxis nach wie vor unzureichend umgesetzt. 94,8 % der meistbesuchten Webseiten weisen nachweisbare Web Content Accessibility Guidelines (WCAG)-Verstöße auf. Bestehende automatisierte Prüfwerkzeuge wie axe, WAVE und Lighthouse erfassen lediglich syntaktische Verstöße und übersehen semantische sowie zustandsabhängige Barrieren in dynamischen Rich-Client-Anwendungen systembedingt. Obwohl LLM-gestützte Analysen diese semantischen Lücken schließen könnten, scheidet der Transfer sensibler Anwendungsdaten an cloudbasierte LLM-APIs für öffentliche Verwaltungen aus datenschutzrechtlichen Gründen weitgehend aus.

Diese Bachelorarbeit konzipiert und implementiert nach dem Design Science Research (DSR)-Paradigma das lokal ausführbare Assistenzsystem *Accessibility Context Engine* (ACE). Das System kombiniert axe-core-Violation-Reports, Playwright-Interaktionsdaten und statische Quellcodeanalysen in einer zweistufigen Datenpipeline. In der ersten Stufe analysiert ein LLM-Detektor den Quellcode der zu prüfenden React-Anwendung chunkweise auf semantische Barrierefreiheitsmuster und erzeugt strukturierte Findings, bevor in der zweiten Stufe ein weiterer LLM-Schritt alle Quellen quellenübergreifend priorisiert und datei- sowie zeilenspezifische Handlungsempfehlungen generiert.

Die Evaluation auf drei kontrollierten Benchmarksuiten (test-12: 12 Violations, test-50: 50 Violations, test-100: 100 Violations) mit drei lokalen Modellen (`qwen2.5-coder:7b`, `qwen2.5-coder:14b`, `qwen3:32b`) sowie eine ergänzende Erprobung am realen BW-Empfangsclient (BWEC) zeigen modellabhängig deutliche Recall-Gewinne gegenüber der reinen Tool-Baseline: Mit `qwen3:32b` steigt der Recall auf test-12 von 66,7 % auf 100 %, auf test-50 ebenfalls auf 100 % und auf test-100 von 34 % auf 63 %. Kleinere Modelle (`qwen2.5-coder:7b`) können bei hoher Violations-Dichte durch tokenlimitbedingte Kürzung der Eingabe hingegen unter das Baseline-Niveau fallen. Auf den kleineren Benchmarksuiten werden für die Mehrzahl der Violations datei- und zeilenspezifische Handlungsempfehlungen auf Fix-Qualitätsstufe 2 erzielt. Das Local-First-Prinzip wird vollständig eingehalten: Kein Quellcode und kein DOM-Inhalt verlässt das lokale System.

Schlüsselwörter: Web-Barrierefreiheit, WCAG, Large Language Models, Prompt-Engineering, axe-core, Playwright, React, Rich-Client-Anwendungen, Lokale KI, Design Science Research, Ollama, Accessibility-Automatisierung, BITV, EAA.

Inhaltsverzeichnis

Abkürzungsverzeichnis	V
Abbildungsverzeichnis	VII
Tabellenverzeichnis	IX
1 Einleitung	1
1.1 Motivation und Kontext	1
1.2 Problemstellung	2
1.3 Zielsetzung und Forschungsfragen	3
1.4 Aufbau der Arbeit	5
2 Theoretische Grundlagen	6
2.1 Barrierefreiheit von Webanwendungen	6
2.1.1 Begriffsdefinition und Relevanz	6
2.1.2 WCAG: Aufbau, Prinzipien und Konformitätsstufen	7
2.1.3 Rechtlicher Rahmen	8
2.2 Automatisierte Barrierefreiheitsanalyse	9
2.2.1 Klassische Tool-Ansätze	9
2.2.2 Grenzen bestehender Tools	10
2.2.3 Taxonomie der Verstöße: syntaktisch, semantisch, Layout	11
2.3 Rich-Client-Webanwendungen und Testautomatisierung	12
2.3.1 Charakteristika und Abgrenzung	12
2.3.2 Herausforderungen für die Barrierefreiheitsprüfung	13
2.3.3 Playwright als Testautomatisierungswerkzeug	14
2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen	14
2.4.1 Grundlagen: Large Language Models	14
2.4.2 Prompt-Engineering-Strategien	15
2.4.3 RAG und Fine-Tuning als Erweiterungsansätze	17
2.4.4 Datenschutz bei KI-gestützten Accessibility-Services	17
3 Stand der Forschung	19
3.1 Vorgehen der Literaturrecherche	19
3.2 Von frühen KI-Ansätzen zur LLM-basierten Erkennung	19
3.3 LLM-basierte Korrektur und Prompt-Engineering	19
3.4 Hybride Pipelines und Ensemble-Testing	20
3.5 Forschungslücken und Einordnung der vorliegenden Arbeit	21
4 Methodik	23
4.1 Wissenschaftliche Methode	23
4.2 Untersuchungsgegenstand und Scope-Abgrenzung	24
4.3 Evaluationsdesign und Kriterien	24
4.3.1 Evaluationslogik	25
4.3.2 Bewertungskriterien	25
4.3.3 Limitationen des Evaluationsdesigns	26

5	Analyse und Anforderungen	27
5.1	Problemkontext und Handlungsbedarf	27
5.2	Funktionale Anforderungen	27
5.3	Nicht-funktionale Anforderungen	28
5.4	Lokale Inferenzinfrastruktur und Modellkandidaten	29
5.4.1	Ollama als Laufzeitumgebung	29
5.4.2	Anforderungen an das Sprachmodell	30
5.4.3	Ausgewählte Modellkandidaten für die Evaluation	30
5.4.4	Zusammenfassung der Anforderungen	30
6	Konzeption des Assistenzsystems	32
6.1	Zielarchitektur und Datenpipeline	32
6.1.1	Analysequellen	32
6.1.2	Unified Finding Schema	34
6.2	Prompt-Strategie	35
6.2.1	Zweistufige Prompt-Strategie	35
6.2.2	Wahl der Prompt-Engineering-Strategien	35
6.2.3	Token-Budget-Steuerung	36
6.3	Ausgabeformat: entwicklernahe To-Do-Liste	36
7	Implementierung des Proof of Concept	38
7.1	Technischer Rahmen und Projektstruktur	38
7.2	axe-core-Modul	39
7.3	Playwright-Modul	40
7.4	Code-Modul	40
7.4.1	Anreicherung bestehender Findings	40
7.4.2	Pattern-Findings	41
7.5	LLM-Detektor-Modul	41
7.5.1	Abgrenzung zu grep und Nutzen der Kombination	42
7.5.2	Beispielhafter Ablauf eines LLM-Findings	42
7.6	Prompt-Builder und System-Prompt	42
7.6.1	System-Prompt	43
7.6.2	Token-Budget-Steuerung	43
7.6.3	Serialisierung	44
7.7	Ollama-Client und Output-Formatter	44
7.8	Pipeline-Orchestrierung	44
7.9	Beispielabläufe am Untersuchungsobjekt	45
8	Evaluation und Diskussion	47
8.1	Versuchsdesign	47
8.1.1	Testobjekte und Ground Truth	47
8.1.2	Modelle, Konfiguration und Runs	48
8.1.3	Metriken und Messverfahren	48
8.2	Ergebnisse: Detektionsqualität (Recall)	49
8.2.1	Baseline: Tool-Pipeline ohne LLM-Detektor	49
8.2.2	Recall nach LLM-Detektor-Augmentierung	49
8.2.3	Erkennungsleistung nach Verstößkategorie	49
8.2.4	Precision und F1-Score	50
8.3	Ergebnisse: Priorisierungsqualität	51
8.3.1	Modellvergleich Must-have und Nice-to-have	51
8.4	Ergebnisse: Fix-Qualität (qualitativ)	52

8.4.1	Bewertungsmaßstab	52
8.4.2	Fallbeispiele nach Modell	52
8.5	Ergebnisse: Effizienz und Robustheit	53
8.5.1	Laufzeit und NFA-02-Konformität	53
8.5.2	Konsistenz und Parsing-Stabilität	53
8.5.3	Token-Nutzung	54
8.6	Limitationen und Validität	54
8.6.1	Externe Validität	54
8.6.2	Interne Validität und Token-Limit-Effekt	54
8.6.3	Weitere Einschränkungen	55
8.7	Belastungsprüfung: test-50 und test-100	55
8.7.1	Evaluation an test-50	55
8.7.2	Evaluation an test-100	56
8.7.3	Evaluation am BWEC-Untersuchungsobjekt	58
8.8	Beantwortung der Forschungsfragen	59
8.8.1	Übergeordnete Forschungsfrage	59
8.8.2	TF1: Erkennungsleistung nach Verstoßtyp	59
8.8.3	TF2: Mehrwert der Kontextkombination und Ablationsanalyse	60
8.9	Modellwahl und Einsatzempfehlungen	61
9	Fazit	63
9.1	Zusammenfassung der Ergebnisse	63
9.1.1	Beitrag zur übergeordneten Forschungsfrage	63
9.1.2	Erfüllung der funktionalen und nicht-funktionalen Anforderungen	63
9.2	Kritische Reflexion	64
9.2.1	Grenzen der externen Validität	64
9.2.2	Methodische Einschränkungen	64
9.3	Ausblick	65
9.3.1	Kurz- und mittelfristige Weiterentwicklungen	65
9.3.2	Langfristige Forschungsperspektiven	65
	Anhang	67
	Literaturverzeichnis	175

Abkürzungsverzeichnis

ACE	<i>Accessibility Context Engine</i>
API	Application Programming Interface
AST	Abstract Syntax Tree
BFSG	Barrierefreiheitsstärkungsgesetz
BITBW	IT Baden-Württemberg
BITV	Barrierefreie-Informationstechnik-Verordnung
BWEC	BW-Empfangsclient
CoT	Chain-of-Thought Prompting
CI	Continuous Integration
CI/CD	Continuous Integration/Continuous Delivery
CLI	Command Line Interface
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DOM	Document Object Model
DSGVO	Datenschutz-Grundverordnung
DSR	Design Science Research
EAA	European Accessibility Act
FA	Funktionale Anforderungen
FP	False Positives
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IQR	Interquartile Range
JSON	JavaScript Object Notation
JSX	JavaScript XML
KI	Künstliche Intelligenz
KPI	Key Performance Indicator
LLM	Large Language Model
NFA	Nicht-funktionale Anforderungen
PoC	Proof of Concept

PW	Playwright
RAG	Retrieval-Augmented Generation
RAM	Random-Access Memory
SEO	Search Engine Optimization
SPA	Single-Page Application
TF	Teilfragen
TP	True Positive
UFS	Unified Finding Schema
UI	User Interface
URL	Uniform Resource Locator
W3C	World Wide Web Consortium
WAI	Web Accessibility Initiative
WCAG	Web Content Accessibility Guidelines

Abbildungsverzeichnis

1	Prozessüberblick des geplanten Assistenzsystems	4
2	WCAG-Schichtenmodell (WCAG 2.1)	8
3	Taxonomie der Barrierefreiheitsverstöße	12
4	Zero-Shot und Few-Shot Prompting im Vergleich	17
5	Komplexität von Home Pages	71
6	Framework Popularity über die letzten Jahre	71
7	Schematische Übersicht der ACE-Pipeline	74
8	Aufteilung des Token-Budgets	80
9	Anmeldefenster der test-12-Suite	101
10	Kontaktformular der test-12-Suite	102
11	Dashboard-Übersicht der test-50-Suite	122
12	Profil-Tab (Profil bearbeiten) der test-50-Suite	123
13	Serverstatus der test-50-Suite	124
14	Benachrichtigungen der test-50-Suite	124
15	Pareto nicht erkannter Violations	154
16	Boxplot Laufzeiten test-100	155
17	Pipeline der Findings-Verarbeitung	156
18	Chat-Modul der test-100-Suite	157
19	Suche-Modul der test-100-Suite	157
20	Einstellungen-Modul der test-100-Suite	158
21	Kalender-Modul der test-100-Suite	158
22	Hilfe-Center-Modul der test-100-Suite	159
23	Datei-Upload-Modul der test-100-Suite	159
24	Recall-Degradation im Suite-Vergleich	162
25	Gesamtlaufzeit-Skalierung im Suite-Vergleich	163
26	Large Language Model (LLM)-Konsistenz σ im Suite-Vergleich	164
27	Recall und Überschuss im Suite-Vergleich	165
28	Truncation-Rate im Suite-Vergleich	166
29	Heatmap der Recall-Werte	167
30	Must/Nice-Verteilung je Modell und Suite	168

Tabellenverzeichnis

1	Erkennungsleistung von Accessibility-Tools	10
2	Konzeptmatrix (Hauptmatrix, hoch priorisierte Studien)	22
3	Zuordnung der DSRM-Phasen zu den Kapiteln der Arbeit.	23
4	Anforderungsübersicht: funktionale und nicht-funktionale Anforderungen	31
5	Zuordnung der Analysesequenzen	33
6	Durchgeführte Benchmark-Runs je Modell und Testsuite	48
7	Recall-Vergleich je Bedingung und Modell	49
8	Recall nach Verstößkategorie und Modell	50
9	Precision, Recall und F1 auf test-50 und test-100	51
10	Priorisierungsverhalten der drei Modelle (test-12)	52
11	Ablationsanalyse: Recall-Beitrag der Pipeline-Stufen	60
12	Modellwahl nach Anwendungsszenario.	61
13	Übersicht: generell vs. suite-spezifisch	70
14	Konzeptmatrix (vollständige Übersicht)	73
15	Implementierte Playwright-Checks mit WCAG-Zuordnung und Kategorie	75
16	Implementierte grep-Patterns mit WCAG-Zuordnung	75
17	Modellempfehlungen nach Anwendungsszenario	81
18	Bewertungsskala der Empfehlungsqualität	82
19	Erfüllungsgrad der funktionalen und nicht-funktionalen Anforderungen	84
20	Umgebungs- und Konfigurationsparameter	86
21	Threats to Validity	88
22	Navigationsübersicht test-12-Suite	90
23	Accessibility-Verstöße (test-12)	91
24	Erkennungsrate der Violations (test-12)	92
25	Recall-Matrix (test-12)	93
26	Rohdaten Benchmark-Runs (test-12)	94
27	Modell-Leistungsvergleich (test-12)	95
28	Token-Nutzung pro Modell (test-12)	96
29	Laufzeiten je Pipeline-Phase (test-12)	97
30	LLM-Findings je Sekunde (test-12)	97
31	Effizienzvergleich der Modelle (test-12)	97
32	Über-Detektion (test-12)	98
33	Detektor-Konsistenz (test-12)	98
34	Empfehlungsqualität (test-12)	99
35	CSV-Spaltenschema	100
36	Beispielzeilen aus dem CSV-Export (test-12)	100
37	Navigationsübersicht test-50-Suite	103
38	Accessibility-Verstöße (test-50)	104
39	Erkennungsrate der Violations (test-50)	107
40	Recall-Matrix (test-50)	110
41	Rohdaten Benchmark-Runs (test-50)	113
42	Modell-Leistungsvergleich (test-50)	114
43	Token-Nutzung pro Modell (test-50)	115
44	Effizienzvergleich der Modelle (test-50)	115
45	Über-Detektion (test-50)	116
46	Precision je Run: test-50-Suite	117

47	Detektor-Konsistenz (test-50)	117
48	Empfehlungsqualität (test-50)	118
49	Beispielzeilen aus dem CSV-Export (test-50)	121
50	Navigationsübersicht test-100-Suite	125
51	Accessibility-Verstöße (test-100)	126
52	Erkennungsrate der Violations (test-100)	130
53	Recall-Matrix (test-100)	135
54	Rohdaten Benchmark-Runs (test-100)	140
55	Modell-Leistungsvergleich (test-100)	141
56	Token-Nutzung pro Modell (test-100)	142
57	Effizienzvergleich der Modelle (test-100)	142
58	Über-Detektion (test-100)	143
59	Stichproben-Plausibilisierung des finalen Reports	144
60	Precision je Run: test-100-Suite	145
61	Detektor-Konsistenz (test-100)	146
62	Empfehlungsqualität (test-100)	147
63	Beispielzeilen aus dem CSV-Export (test-100)	153
64	Zentrale Metriken im Suite-Vergleich	161
65	Navigationsübersicht BWEC-Block	169
66	Detektionsquellen-Übersicht BWEC	170
67	BWEC Tool-Findings Katalog	170
68	Rohdaten BWEC (maxfiles =100)	171
69	Modell-Leistungsvergleich BWEC (maxfiles =100)	172
70	Rohdaten BWEC (maxfiles =500)	173
71	Modell-Leistungsvergleich BWEC (maxfiles =500)	174

1 Einleitung

Web-Barrierefreiheit ist eine zentrale Voraussetzung für den gleichberechtigten Zugang zu digitalen Informationen und Dienstleistungen – unabhängig von individuellen Einschränkungen. In der Praxis zeigt sich jedoch eine erhebliche Diskrepanz zwischen den normativen Anforderungen und der tatsächlichen Umsetzung: Barrierefreiheit wird im Entwicklungsprozess häufig erst spät berücksichtigt. Bestehende Prüfwerkzeuge erfassen zudem nur einen Bruchteil der tatsächlichen Barrieren, während manuelle Prüf- und Korrekturprozesse zeitaufwändig bleiben.¹ Diese Arbeit untersucht, inwieweit ein LLM-basierter Ansatz diese Lücke schließen kann, indem Barrierefreiheitsprobleme in Webanwendungen automatisiert erkannt und entwicklernahe Handlungsempfehlungen abgeleitet werden – lokal ausführbar und ohne den Transfer sensibler Daten an externe Dienste.²

1.1 Motivation und Kontext

Rund 1,3 Milliarden Menschen, also etwa 16% der Weltbevölkerung, leben mit einer Form von Behinderung.³ Für diese Personengruppe ist der Zugang zu digitalen Angeboten keine Komfortfrage, sondern eine grundlegende Voraussetzung gesellschaftlicher Teilhabe. Ohne barrierefreie Webanwendungen bleiben wesentliche Lebensbereiche verschlossen – von der Kommunikation mit Behörden über den Zugang zu Bildung und Gesundheitsinformationen bis hin zur Teilnahme am wirtschaftlichen und gesellschaftlichen Leben. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist völkerrechtlich in der UN-Behindertenrechtskonvention verankert und hat sich in den vergangenen Jahren zunehmend auch in verbindlichem nationalen und europäischen Recht niedergeschlagen.⁴

Auf europäischer Ebene verpflichtet der European Accessibility Act (EAA) die Mitgliedstaaten, Barrierefreiheitsanforderungen für digitale Produkte und Dienstleistungen durchzusetzen.⁵ In Deutschland wurde diese Richtlinie durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt, das seit dem 28. Juni 2025 auch für weite Teile der Privatwirtschaft gilt.⁶ Ergänzt wird dieser Rahmen durch die Barrierefreie-Informationstechnik-Verordnung (BITV), die für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Vorgaben festlegt.⁷

Trotz dieser eindeutigen Rechtslage ist die praktische Umsetzung barrierefreier Webanwendungen nach wie vor unzureichend. Die jährlich durchgeführte WebAIM-Million-Studie, die die Startseiten der meistbesuchten Webseiten weltweit analysiert, stellte 2025 fest, dass 94,8% der untersuchten Seiten nachweisbare Verstöße gegen die Web Content Accessibility Guidelines (WCAG)

¹ Vgl. Souza et al. 2024; Vgl. Abuaddous, Jali, Basir 2016, S. 175

² Vgl. Bassi et al. 2025, S. 297

³ Vgl. World Health Organization 2026

⁴ Vgl. United Nations 2006

⁵ Vgl. European Commission 2026

⁶ Vgl. Deutscher Bundestag 2026

⁷ Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025; Vgl. Heinemann 2024, S. 281

aufweisen.⁸ Gegenüber 2019 (97,8%) entspricht dies lediglich einer Verbesserung um drei Prozentpunkte.⁹ Für die öffentliche Verwaltung ist diese Situation besonders brisant. Während privatwirtschaftliche Anbieter erst seit Juni 2025 durch das BFSG verpflichtet wurden, galten für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Barrierefreiheitsanforderungen nach der BITV – dennoch weisen auch zahlreiche Webangebote der öffentlichen Hand weiterhin erhebliche Mängel auf.¹⁰ Damit bleibt Barrierefreiheit in der Entwicklungspraxis trotz gesetzlicher Verpflichtung systematisch unterrepräsentiert.

Die IT Baden-Württemberg (BITBW) nimmt als zentrale IT-Dienstleisterin der Landesverwaltung Baden-Württemberg in diesem Kontext eine besondere Stellung ein.¹¹ Als Entwicklerin und Betreiberin öffentlicher Webanwendungen für Ministerien, Polizeibehörden und Kommunen ist sie gesetzlich zur Einhaltung der BITV und des BFSG verpflichtet.¹² Drei Faktoren machen die BITBW zu einem besonders geeigneten Praxiskontext für die vorliegende Arbeit. Erstens unterliegen ihre Anwendungen strengeren Barrierefreiheitsanforderungen als privatwirtschaftliche Produkte. Zweitens schließen die Datenschutzvorgaben der öffentlichen Verwaltung die Nutzung cloudbasierter LLM-APIs weitgehend aus. Drittens erschweren gewachsene Systemlandschaften mit Legacy-Anteilen und begrenzten Entwicklungsressourcen die manuelle Barrierefreiheitsprüfung im erforderlichen Umfang. Die BITBW stellt somit den geeigneten praktischen Rahmen der Arbeit dar.

1.2 Problemstellung

Die mangelhafte Umsetzung von Barrierefreiheit in Softwareprojekten lässt sich nicht allein durch fehlende Motivation oder Ressourcen erklären. Wesentliche Gründe sind die Schwächen der verfügbaren Tools: automatisierte Barrierefreiheitsanalyse-Tools wie axe¹³, WAVE¹⁴ oder Lighthouse¹⁵ erkennen zwar syntaktische Verstöße, stoßen jedoch bei semantischen Problemen an ihre Grenzen. Ob ein Alternativtext den Bildinhalt angemessen beschreibt oder eine Schaltflächenbeschriftung für Screenreader-Nutzende verständlich ist, entzieht sich der regelbasierten Prüflogik dieser Tools.

Empirisch belegt wird diese Schwäche durch das Mutation-Testing-Framework **Mally**.¹⁶ Es injiziert gezielt Barrierefreiheitsfehler in reale Webanwendungen und prüft anschließend, ob bestehende Accessibility-Tools diese künstlich eingebrachten Defekte erkennen. In einer systematischen Evaluation mit 25 gezielt eingebrachten Barrierefreiheitsproblemen erkannte kein einziges

⁸Vgl. WebAIM 2025

⁹Vgl. WebAIM 2019

¹⁰Vgl. Heinemann 2024, S. 281

¹¹Vgl. BITBW 2021

¹²Vgl. Landesrecht BW 2015

¹³Vgl. Deque Systems 2026

¹⁴Vgl. WebAIM 2026a

¹⁵Vgl. Google 2025

¹⁶Vgl. Tafreshipour et al. 2024, S. 97

der sechs untersuchten Tools mehr als 50% der injizierten Fehler. Somit blieb im Durchschnitt nahezu die Hälfte aller Defekte unentdeckt.¹⁷

Eine zweite strukturelle Schwäche betrifft die Charakteristik moderner Webanwendungen. Rich-Client-Anwendungen auf Basis von Frameworks wie React erzeugen ihren Inhalt größtenteils clientseitig und verändern das Document Object Model (DOM) dynamisch in Abhängigkeit von Nutzerinteraktionen. Barrieren entstehen häufig erst in bestimmten Zuständen: wenn ein Modalfenster geöffnet wird, eine Fehlermeldung erscheint oder ein Dropdown-Menü aktiviert ist. Rein statische Analysen – oder solche, die ausschließlich den initialen Seitenladezustand prüfen – erfassen diese zustandsabhängigen Barrieren nicht.¹⁸

Jüngere Forschungsarbeiten deuten darauf hin, dass LLMs geeignet sein können, zentrale Grenzen regelbasierter Accessibility-Tools zu überwinden. Systeme wie **AccessGuru**¹⁹ und **GenA11y**²⁰ setzen das cloudbasierte Large Language Model (LLM) GPT-4o ein und erzielen bei der Erkennung und Korrektur von Barrierefreiheitsverstößen vielversprechende Ergebnisse. Beide Ansätze beschränken sich jedoch im Wesentlichen auf die Analyse statischen HTMLs und berücksichtigen weder dynamische Zustandswechsel noch den Quellcode der geprüften Anwendung. Der Schritt von der Fehlererkennung zu einer entwicklernahen, auf den Quellcode bezogenen Handlungsempfehlung wird in der Literatur als offenes Problem benannt,²¹ bleibt aber bislang ungelöst. Eine vertiefte Analyse dieser und weiterer Arbeiten erfolgt in Kapitel 3 „Stand der Forschung“.

Die vorliegende Forschungsliteratur weist auf drei zentrale Defizite hin: Vorhandene Ansätze adressieren Teilprobleme, nicht jedoch die vollständige Kombination aus (a) automatisierter Erfassung dynamischer Zustände in React-basierten Webanwendungen, (b) Verknüpfung von Tool-Reports mit dem Quellcode der geprüften Anwendung und (c) entwicklernaher, priorisierter Handlungsempfehlung – lokal ausführbar und ohne den Transfer sensibler Anwendungsdaten an externe Dienste.

1.3 Zielsetzung und Forschungsfragen

Ziel dieser Bachelorarbeit ist die Konzeption und prototypische Umsetzung einer lokal ausführbaren, kontextsensitiven Assistenzlösung zur Barrierefreiheitsanalyse von Rich-Client-Webanwendungen. Als Praxisrahmen und Motivationskontext dient der BW-Empfangsclient (BWEC), eine React-basierte Webanwendung der BITBW; die Hauptevaluation erfolgt jedoch an synthetisch konstruierten Testsuiten mit vollständig bekannter Ground Truth (d.h. vorab definierter und im System eingebetteter Menge aller vorhandenen Violations). Der Proof of Concept (PoC) soll drei Datenquellen in einer gemeinsamen Pipeline zusammenführen: (1) strukturierte Violation-Reports aus axe-core, (2) Interaktionsdaten aus Playwright-gesteuerten Browser-Tests sowie (3)

¹⁷Vgl. Tafreshipour et al. 2024, S. 100

¹⁸Vgl. Bassi et al. 2025, S. 303

¹⁹Vgl. Fathallah, Hernández, Staab 2025

²⁰Vgl. He, Huq, Malek 2025, FSE101:10–11

²¹Vgl. He, Huq, Malek 2025, FSE101:15–17

statische Quellcodeanalysen der geprüften React-Komponenten, etwa hinsichtlich fehlender `aria`-Attribute oder nicht-interaktiver Elemente mit `onClick`-Handlern. Auf dieser Grundlage werden priorisierte, entwicklernahe Handlungsempfehlungen abgeleitet und strukturiert aufbereitet. Abbildung 1 veranschaulicht diesen Prozess im Überblick.

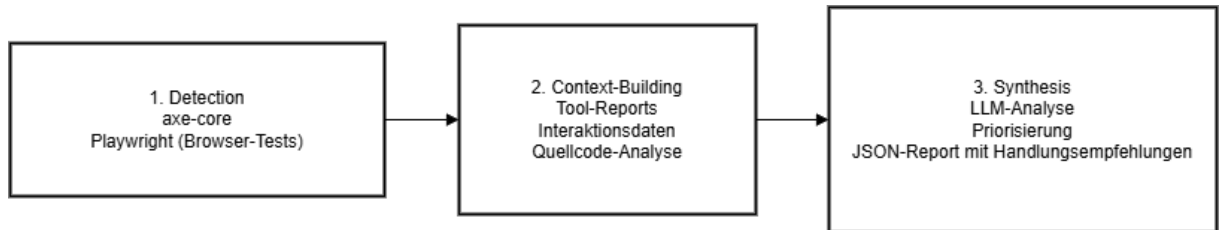


Abb. 1: Prozessüberblick: von den drei Datenquellen zur entwicklernahe Handlungsempfehlung.

Aus der Zielsetzung der Arbeit ergeben sich drei konkrete Beiträge:

1. Einen strukturierten Überblick darüber, wie unterschiedliche Datenquellen der Barrierefreiheitsanalyse systematisch kombiniert und interpretiert werden können.
2. Einen Proof of Concept (PoC), der das Vorgehen exemplarisch demonstriert.
3. Eine qualitative und quantitative Bewertung der Untersuchungsergebnisse anhand definierter Kriterien, die aus den Anforderungen und der einschlägigen Fachliteratur abgeleitet werden.

Die übergeordnete Forschungsfrage dieser Arbeit lautet:

Inwieweit kann ein kontextsensitives KI-Assistenzsystem, das Tool-Reports, Playwright-Interaktionsdaten und Quellcode kombiniert, Barrierefreiheitsverstöße in React-basierten Webanwendungen automatisiert erkennen und entwicklernahe Handlungsempfehlungen generieren?

Diese übergeordnete Forschungsfrage wird durch zwei Teilfragen operationalisiert:

- TF1: Welche Typen von Barrierefreiheitsverstößen – syntaktisch, semantisch und layout-bezogen – lassen sich durch den gewählten Ansatz zuverlässig erkennen und wo liegen die methodischen Grenzen?
- TF2: Inwiefern verbessert die vollständige Kontextkombination aus Tool-Reports, Playwright-Interaktionsdaten und Quellcode die Erkennungsleistung und Empfehlungsqualität gegenüber einer reinen regelbasierten Tool-Baseline ohne LLM-Kontextualisierung?

TF1 adressiert dabei das Erkennungsproblem und erlaubt eine differenzierte Aussage über die Leistungsfähigkeit und Grenzen des Systems entlang der Verstoß-Taxonomie von Fathallah et al.²² TF2 greift den in der Literatur identifizierten Mehrwert der Kontextualisierung auf und überprüft ihn durch einen systematischen Vergleich zwischen der reinen Tool-Baseline (axe-core,

²²Vgl. Fathallah, Hernández, Staab 2025, S. 114–115

Playwright, grep ohne LLM) und der vollständigen Pipeline (Baseline + LLM-gestützte Quellcodeanalyse) an den synthetischen Testsuiten mit bekannter Ground Truth.

1.4 Aufbau der Arbeit

Die Arbeit gliedert sich in neun Kapitel. Im Anschluss an die Einleitung werden in Kapitel 2 die theoretischen Grundlagen erarbeitet: Barrierefreiheit im Web, die WCAG-Standards und der rechtliche Rahmen, die Charakteristika von Rich-Client-Webanwendungen sowie Grundlagen zu Large Language Models und Prompt-Engineering-Strategien.

Kapitel 3 gibt einen systematischen Überblick über den Stand der Forschung, gliedert die relevanten Arbeiten thematisch – nach LLM-basierter Erkennung, Code-Korrektur, hybriden Pipelines und Evaluation – und schließt mit einer expliziten Einordnung des Beitrags der vorliegenden Arbeit.²³

Kapitel 4 legt das methodische Vorgehen dar. Es beschreibt die Wahl des Design-Science-Research-Paradigmas als wissenschaftliche Methode, definiert den Untersuchungsgegenstand und entwirft das Evaluationsdesign.²⁴

Kapitel 5 analysiert den Problemkontext anhand des BWEC als Praxisrahmen und leitet daraus funktionale und nicht-funktionale Anforderungen an das System ab. Dabei werden auch die Auswahlkriterien für die eingesetzten lokalen Sprachmodelle begründet.

Auf Basis dieser Anforderungen wird in Kapitel 6 das Assistenzsystem konzipiert: Systemarchitektur, Datenpipeline, Prompt-Strategie und Ausgabeformat werden begründet entworfen.

Kapitel 7 beschreibt die prototypische Implementierung des PoC und illustriert die Systemfunktionalität anhand von Beispielabläufen für syntaktische, semantische und dynamisch erzeugte Barrierefreiheitsprobleme.

Kapitel 8 präsentiert und diskutiert die Evaluationsergebnisse auf den drei synthetischen Benchmarksuiten sowie am realen BWEC. Es beantwortet die Forschungsfragen auf Basis der Messwerte und reflektiert die Limitationen des Ansatzes.

Das abschließende Kapitel 9 fasst die Kernaussagen zusammen, ordnet den Beitrag der Arbeit in die Forschungslandschaft ein und formuliert Empfehlungen für weiterführende Untersuchungen.

²³Vgl. Webster, Watson 2002

²⁴Vgl. Hevner, A. R. et al. 2004, S. 75

2 Theoretische Grundlagen

Dieses Kapitel bündelt die theoretischen Grundlagen für die Konzeption und Evaluation des Proof of Concept. Im Mittelpunkt stehen Web-Barrierefreiheit, die Grenzen automatisierter Prüfwerkzeuge, Rich-Client-Webanwendungen sowie LLM-basierte Assistenzsysteme einschließlich Prompting und Datenschutz.

2.1 Barrierefreiheit von Webanwendungen

2.1.1 Begriffsdefinition und Relevanz

Der Begriff *Web Accessibility*, im Deutschen Web-Barrierefreiheit, bezeichnet nach Definition der Web Accessibility Initiative (WAI) die Eigenschaft von Webangeboten, von allen Menschen unabhängig von körperlichen, motorischen oder sensorischen Einschränkungen sowie individuellen Fähigkeiten wahrgenommen, bedient und genutzt werden zu können.²⁵ Im Mittelpunkt der vorliegenden Arbeit stehen vor allem visuelle und motorische Beeinträchtigungen; der gesellschaftliche und rechtliche Kontext wurde in Abschnitt 1.1 dargelegt.

In der Forschungsliteratur werden vier wesentliche Behinderungsformen unterschieden, aus denen sich unterschiedliche Anforderungen an die Gestaltung barrierefreier Webangebote ergeben.²⁶ Für diese Arbeit sind insbesondere zwei Kategorien relevant – visuelle Beeinträchtigungen und motorische Einschränkungen –, da sie den Großteil der automatisiert prüfbaren WCAG-Verstöße betreffen:

Visuelle Beeinträchtigungen umfassen vollständige oder partielle Blindheit sowie Farbfehlsichtigkeit.²⁷ Betroffene Personen nutzen häufig Screenreader-Software (z. B. JAWS, NVDA, VoiceOver), deren Wirksamkeit auf einer korrekten semantischen HTML-Struktur beruht.²⁸ Fehlende Alternativtexte, unzureichende Farbkontraste und fehlende **aria**-Labels bilden die häufigsten Verstöße in dieser Kategorie²⁹ und zugleich den primären Erkennungsbereich des in dieser Arbeit entwickelten Systems.

Motorische Einschränkungen betreffen Personen, die konventionelle Eingabegeräte wie Maus oder Touchscreen nicht präzise bedienen können.³⁰ Sie sind auf Tastaturnavigation, Sprachsteuerung oder spezielle assistive Eingabegeräte angewiesen.³¹ Eine vollständige Bedienbarkeit über

²⁵Vgl. Abuaddous, Jali, Basir 2016, S. 172

²⁶Vgl. Lazar, Feng, Hochheiser 2017, S. 493–494

²⁷Vgl. Gubbi Mohanbabu, Pavel 2024

²⁸Vgl. Morris et al. 2016, S. 5508

²⁹Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.1.1

³⁰Vgl. Pérez et al. 2020, S. 110381–110382

³¹Vgl. Kouroupetroglou 2013, S. 208

die Tastatur ist daher eine zentrale Voraussetzung für barrierefreie Nutzung.³² Für das vorliegende System ist diese Kategorie besonders relevant, da Tastaturfallen und fehlende Fokussteuerung in (React-)Anwendungen häufig erst durch interaktionsbasierte Tests, etwa mit Playwright, sichtbar werden.³³

Auditive und kognitive Beeinträchtigungen werden im Folgenden nicht vertieft, da ihre Bewertung überwiegend menschliches Urteilsvermögen erfordert und sich nur begrenzt automatisieren lässt.³⁴

Assistive Technologien – darunter Screenreader, Bildschirmvergrößerungssoftware und alternative Eingabegeräte – bilden die technische Voraussetzung für die Nutzung digitaler Angebote durch die genannten Personengruppen.³⁵ Ihre Wirksamkeit hängt jedoch unmittelbar davon ab, dass Webanwendungen klar definierte strukturelle und semantische Anforderungen erfüllen.³⁶ Die Nichterfüllung dieser Anforderungen führt nicht nur zu einer Beeinträchtigung der betroffenen Personen, sondern kann auch wirtschaftliche und rechtliche Konsequenzen nach sich ziehen.³⁷

2.1.2 WCAG: Aufbau, Prinzipien und Konformitätsstufen

Die WCAG sind der internationale Standard für barrierefreie Webinhalte und werden vom World Wide Web Consortium (W3C) im Rahmen der WAI veröffentlicht. Die derzeit maßgebliche Referenzversion ist WCAG 2.2, die im Oktober 2023 WCAG 2.1 (2018) als aktuelle Referenzversion ablöste.³⁸ WCAG 2.2 führt neun zusätzliche Erfolgskriterien ein und berücksichtigt insbesondere kognitive Einschränkungen und mobile Nutzung.³⁹ Zu diesen Kriterien gehören konsistentere Fokusindikatoren (2.4.11–2.4.13) und die Vermeidung von Drag-and-Drop-Pflichtinteraktionen (WCAG 2.5.7).⁴⁰

Die WCAG ist in vier Ebenen gegliedert: Prinzipien, Richtlinien, Erfolgskriterien und unterstützende Techniken. Auf oberster Ebene stehen vier grundlegende Prinzipien, zusammengefasst im Akronym **POUR**: *Perceivable* (Wahrnehmbarkeit), *Operable* (Bedienbarkeit), *Understandable* (Verständlichkeit) und *Robust* (Robustheit).⁴¹ Typische Anforderungen sind Alternativtexte für Bilder (1.1.1), ausreichende Farbkontraste (1.4.3) und vollständige Tastaturerreichbarkeit aller Funktionen.⁴²

³²Vgl. WebAIM 2026b

³³Vgl. Chiou, Alotaibi, Halfond 2021, S. 855

³⁴Vgl. Abou-Zahra, Brewer, Cooper 2018

³⁵Vgl. World Wide Web Consortium (W3C) 2026b; Abou-Zahra, Brewer, Cooper 2018

³⁶Vgl. Morrison et al. 2017, S. 82

³⁷Vgl. Krok 2024, S. 864–865

³⁸Vgl. World Wide Web Consortium (W3C) 2018; World Wide Web Consortium (W3C) 2023

³⁹Vgl. Purwandari et al. 2025, S. 118; Vgl. Souza et al. 2024

⁴⁰Vgl. World Wide Web Consortium (W3C) 2023

⁴¹Vgl. Souza et al. 2024

⁴²Vgl. World Wide Web Consortium (W3C) 2023, Abschnitte 1–4

Die 13 Richtlinien unterhalb der Prinzipien werden durch insgesamt 87 testbare Erfolgskriterien konkretisiert (WCAG 2.2; gegenüber 78 in WCAG 2.1 kommen neun neue Kriterien hinzu, während Erfolgskriterium 4.1.1 als obsolet markiert wird).⁴³ Jedes Erfolgskriterium ist einer von drei Konformitätsstufen zugeordnet.⁴⁴ Stufe A umfasst grundlegende Anforderungen, deren Nichteinhaltung bestimmte Nutzergruppen vollständig ausschließt. Stufe AA ist der in Europa gesetzlich geforderte Mindeststandard für öffentliche Stellen und weitere Teile der Privatwirtschaft.⁴⁵ Stufe AAA beschreibt Anforderungen, die nicht für alle Inhalte generell erfüllbar sind. Die vorliegende Arbeit fokussiert sich entsprechend dem gesetzlichen Mindeststandard auf Stufe A und AA.

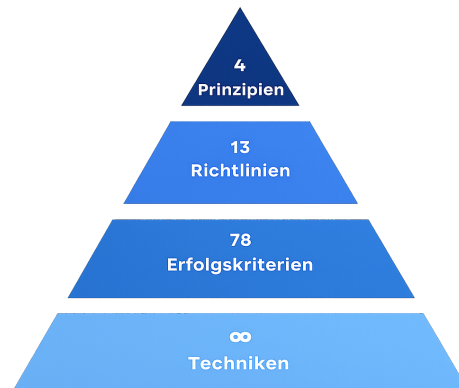


Abb. 2: WCAG-Schichtenmodell (dargestellt nach WCAG 2.1; in WCAG 2.2 strukturell unverändert).

Die praktische Umsetzung dieser Standards bleibt trotz ihrer langen Etablierung unzureichend.⁴⁶ Die WebAIM-Million-Studie 2025 identifizierte durchschnittlich 56,8 WCAG-Fehler pro Seite. Die häufigsten Verstoßkategorien betrafen fehlende Alternativtexte (54,5 % der untersuchten Seiten), unzureichende Farbkontraste (80,6 %), fehlende Labels (47,4 %), leere Links (44,6 %) und fehlende Sprachangaben im HTML (17,1 %).⁴⁷

2.1.3 Rechtlicher Rahmen

Der rechtliche Rahmen – EAA, BfSG und BITV 2.0 – wurde in Abschnitt 1.1 ausführlich dargestellt. Für die technische Umsetzung relevant ist, dass WCAG 2.2 auf Konformitätsstufe AA den verbindlichen Prüfmaßstab bildet, der in dieser Arbeit sowohl für die Testsuite-Konstruktion als auch für die Anforderungsableitung zugrunde gelegt wird.⁴⁸

⁴³Vgl. World Wide Web Consortium (W3C) 2023

⁴⁴Vgl. Fathallah, Hernández, Staab 2025

⁴⁵Vgl. Kerkmann, Lewandowski 2015, S. 40, 195–197

⁴⁶Vgl. Bassi et al. 2025, S. 297

⁴⁷Vgl. WebAIM 2025

⁴⁸Vgl. World Wide Web Consortium (W3C) 2023; Bundesministerium für Digitales und Staatsmodernisierung 2025; Deutscher Bundestag 2026

2.2 Automatisierte Barrierefreiheitsanalyse

Die in Abschnitt 1.2 skizzierten Ursachen für die mangelhafte Umsetzung von Barrierefreiheit – späte Integration in Entwicklungsprozesse, begrenztes WCAG-Wissen und Zeitdruck – begründen den Einsatz automatisierter Prüfwerkzeuge.⁴⁹ Dieser Abschnitt beschreibt deren Funktionsweise, Leistungsumfang und strukturelle Grenzen.

2.2.1 Klassische Tool-Ansätze

Für die automatisierte Überprüfung von Webanwendungen auf Barrierefreiheit existiert eine stetig wachsende Zahl von Werkzeugen. Die WAI führt in ihrer aktuellen Evaluation-Tool-Liste über 160 Werkzeuge⁵⁰, von denen 84 explizit die WCAG 2.1 adressieren.⁵¹ Alle gängigen Werkzeuge arbeiten regelbasiert, das heißt, sie prüfen den DOM einer geladenen Seite anhand eines vordefinierten Regelkatalogs gegen bekannte Verstoßmuster.⁵² Das Document Object Model (DOM) ist eine vom Browser erzeugte, baumartige Objektstruktur, die den HTML-Code einer Webseite als hierarchische Struktur aus Elementknoten repräsentiert.⁵³ Skripte und Analysewerkzeuge können auf diese Struktur über standardisierte Programmierschnittstellen zugreifen und sie gezielt auswerten.

Zu den verbreitetsten automatisierten Werkzeugen der Barrierefreiheitsprüfung gehören unter anderem:

- **axe-core**, entwickelt von Deque Systems, hat sich als Standardlösung für die automatische Integration von Accessibility-Tests in Entwicklungspipelines etabliert.⁵⁴
- **WAVE** (WebAIM) bietet visuelle Browser-Extensions, die Fehler direkt in der gerenderten Seitenansicht annotieren.⁵⁵
- **Lighthouse** ist in die Chrome-DevTools integriert und kombiniert Accessibility-Testing mit Performance- und SEO-Analysen.⁵⁶

Der typische Output solcher Werkzeuge ist ein strukturierter Violation Report, bei dem jedem erkannten Verstoß eine Bezeichnung, Beschreibung und ein Schweregrad zugewiesen wird (bei axe-core: minor, moderate, serious, critical) sowie, falls möglich, ein HTML-Ausschnitt des betroffenen Elements.⁵⁷ Dieser standardisierte Output bildet in der vorliegenden Arbeit die Grundlage für die erste Stufe der Datenpipeline.

⁴⁹Vgl. Bassi et al. 2025, S. 298; Vgl. Mowar 2024, S. 123

⁵⁰Vgl. World Wide Web Consortium (W3C) 2026a

⁵¹Vgl. Delnevo, Andruccioli, Mirri 2024

⁵²Vgl. Kodithuwakku, Wickramarachchi 2025; Manca et al. 2023, 3:9

⁵³Vgl. Robie 2026

⁵⁴Vgl. Deque Systems 2026

⁵⁵Vgl. WebAIM 2026a

⁵⁶Vgl. Google 2025

⁵⁷Vgl. Kodithuwakku, Wickramarachchi 2025; Deque Systems 2026

2.2.2 Grenzen bestehender Tools

Trotz ihrer weiten Verbreitung weisen regelbasierte Accessibility-Tools grundlegende strukturelle Grenzen auf. Diese lassen sich in drei Kategorien einteilen: begrenzte Regelabdeckung, semantische Blindheit und fehlende Dynamik.

Die Regelabdeckung der Werkzeuge ist deutlich geringer, als häufig angenommen wird. Kodithuwakku und Wickramaarachchi zeigen, dass selbst die leistungsfähigsten untersuchten Tools lediglich 21% aller WCAG-Tests automatisiert abdecken können (vgl. Tabelle 1).⁵⁸ Die in der Tabelle ausgewiesenen Werte CER und ICR messen davon abweichende Dimensionen: CER beschreibt, welcher Anteil der tatsächlich gemeldeten Befunde korrekt ist (Präzision), während ICR angibt, wie viele der vorhandenen Probleme vom Tool gefunden wurden (Recall). Für Tastaturzugänglichkeit und auditive Barrieren lag die Erkennungsrate in ihrer Studie bei null Prozent; hinzu kommt eine hohe Rate an *False Positives*.⁵⁹ Bassi et al. kommen zu dem Ergebnis, dass das beste verfügbare Einzeltool (**ARC Toolkit**) nur 26% der WCAG-Tests abdeckt.⁶⁰

Die Kombination mehrerer Tools in einem Ensemble verbessert die Abdeckung, beseitigt das strukturelle Problem jedoch nicht vollständig. Pool zeigt, dass selbst ein Ensemble aus acht gleichzeitig eingesetzten Tools, darunter auch **axe-core**, **WAVE** und **Qualweb**, erhebliche Lücken hinterlässt: jedes einzelne Tool erkennt mindestens sieben Violations exklusiv, die von keinem anderen Tool gefunden werden.⁶¹ Manca et al. stellen zudem fest, dass von elf untersuchten Werkzeugen lediglich drei explizit auf ihre Erkennungslücken hinweisen.⁶²

Tool	True Positives	False Positives	CER (%)	ICR (%)
Lighthouse	17	16	52.94	15.92
WAVE	16	12	35.29	14.65
SiteImprove	23	18	47.06	19.75
axe	15	10	52.94	15.29
Accessibility Insights	24	19	47.06	15.29
A11y	6	8	23.53	3.82
Includia Accessibility Checker	33	31	41.18	21.02

Tab. 1: Vergleich der Erkennungsleistung verschiedener Accessibility-Prüfwerkzeuge.⁶³ CER beschreibt den Anteil korrekt erkannter Verstöße im Verhältnis zu allen gemeldeten Verstößen, während ICR den Anteil tatsächlich vorhandener Accessibility-Probleme misst, die vom jeweiligen Tool erkannt werden.

Ein zentrales Defizit liegt in der **semantischen Blindheit** der Werkzeuge.⁶⁴ Regelbasierte Tools können lediglich prüfen, ob ein Accessibility-Attribut vorhanden ist, nicht aber, ob der Inhalt dem richtigen Zweck dient. Ein Bild mit dem Alternativtext *image* oder *IMG_0042.jpg* wird

⁵⁸Vgl. Kodithuwakku, Wickramaarachchi 2025

⁵⁹Vgl. Kodithuwakku, Wickramaarachchi 2025

⁶⁰Vgl. Bassi et al. 2025, S. 298

⁶¹Vgl. Pool 2023

⁶²Vgl. Manca et al. 2023, S. 4

⁶³Entnommen aus Kodithuwakku, Wickramaarachchi 2025.

⁶⁴Vgl. Fathallah, Hernández, Staab 2025

von allen Tools als regelkonform eingestuft, obwohl dieser Text für Screenreader-Nutzende keinen Mehrwert bietet. **Ma11y**, ein Mutation-Testing-Framework für Accessibility, hat dieses Problem empirisch quantifiziert: Bei der systematischen Injektion von 25 gezielten Defekten in reale Webanwendungen erkannten die sechs ausgewählten Tools semantische Verstöße nur zu 26% und syntaktische Verstöße zu 54% – im Durchschnitt blieb damit nahezu die Hälfte aller injizierten Defekte unentdeckt.⁶⁵

Ein weiteres strukturelles Problem besteht in der **fehlenden Dynamik** der Analyse. Viele Tools analysieren ausschließlich den initial geladenen DOM-Zustand einer Seite.⁶⁶ Barrieren, die erst durch Nutzerinteraktionen entstehen – etwa beim Öffnen eines Modalfensters, beim Aktivieren eines Dropdown-Menüs oder beim Erscheinen einer Fehlermeldung – bleiben dabei unsichtbar. Hinzu kommt, dass alle Violation Reports generischer Natur sind: Sie beschreiben das Problem auf der Ebene des gerenderten DOM, ohne Bezug zur Quellcodestruktur der geprüften Anwendung herzustellen.⁶⁷

2.2.3 Taxonomie der Verstöße: syntaktisch, semantisch, Layout

Um Barrierefreiheitsverstöße systematisch analysieren und gezielt adressieren zu können, legt diese Arbeit eine dreiteilige Taxonomie zugrunde, die auf Fathallah et al. zurückgeht (vgl. Abbildung 3).⁶⁸

- **Syntaktische Verstöße** bezeichnen Fälle, in denen ein erforderliches Accessibility-Element vollständig fehlt oder formal falsch gebildet ist.⁶⁹ Ein typisches Beispiel ist ein fehlendes `alt`-Attribut an einem `<img>`-Element. Diese Kategorie ist für regelbasierte Tools am besten prüfbar, da die Verletzung rein struktureller Natur ist.⁷⁰
- **Semantische Verstöße** liegen vor, wenn Accessibility-Attribute zwar vorhanden sind, ihren Zweck aber inhaltlich verfehlen.⁷¹ Ein Bild mit dem Alternativtext *Bild* oder ein Link mit dem Text *hier klicken* sind typische Beispiele. Da die Bewertung solcher Verstöße Sprachverständnis und Kontextwissen erfordert, bilden sie das primäre Anwendungsfeld für LLM-basierte Analysen.⁷²
- **Layout-Verstöße** betreffen visuelle Barrieren, insbesondere unzureichende Farbkontraste⁷³ und fehlende oder schwer erkennbare Fokusindikatoren.⁷⁴ Einige dieser Verstöße können algorithmisch geprüft werden; andere erfordern ein kontextuelles visuelles Urteil.

⁶⁵Vgl. Tafreshipour et al. 2024, S. 108

⁶⁶Vgl. Fathallah, Hernández, Staab 2025

⁶⁷Vgl. Delnevo, Andruccioli, Mirri 2024

⁶⁸Vgl. Fathallah, Hernández, Staab 2025

⁶⁹Vgl. Tafreshipour et al. 2024, S. 102

⁷⁰Vgl. Flanagan 2020, S. 569 ff.

⁷¹Vgl. World Wide Web Consortium (W3C) 2023

⁷²Vgl. He, Huq, Malek 2025, FSE101:2

⁷³Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.4.3

⁷⁴Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 2.4.7–2.4.13

⁷⁵Mit Änderungen entnommen aus: Fathallah, Hernández, Staab 2025

Taxonomie der Barrierefreiheitsverstöße

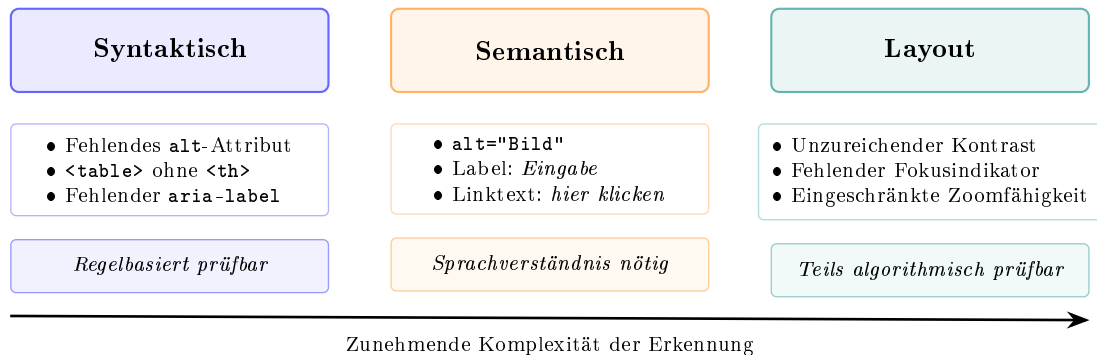


Abb. 3: Taxonomie der Barrierefreiheitsverstöße nach Fathallah et al. mit Beispielen und Erkennungscharakteristik je Kategorie.⁷⁵

2.3 Rich-Client-Webanwendungen und Testautomatisierung

2.3.1 Charakteristika und Abgrenzung

Im Rahmen dieser Arbeit bezeichnet eine **Rich-Client-Webanwendung** eine webbasierte Anwendung, bei der ein wesentlicher Teil der Anwendungslogik und Darstellungssteuerung clientseitig im Browser ausgeführt wird.⁷⁶ Im Gegensatz zu klassischen serverseitig gerenderten Anwendungen werden bei einer Single-Page Application (SPA) nach dem initialen Laden des HTML-Rahmens ausschließlich Daten zwischen Client und Server ausgetauscht, wobei die Darstellung durch clientseitige Skriptsprachen wie JavaScript oder TypeScript direkt im Browser durch Manipulation des DOM gesteuert wird.⁷⁷ Die zunehmende Komplexität solcher Anwendungen ist in Anhang 2 veranschaulicht.

React, ein von Meta entwickeltes JavaScript-Framework, ist derzeit einer der dominierenden Ansätze zur Entwicklung solcher Anwendungen. Laut Stack Overflow Developer Survey 2025 wird React von 44,7% aller befragten Entwicklerinnen und Entwickler eingesetzt⁷⁸ – seine anhaltende Dominanz gegenüber anderen Frameworks ist in Anhang 3 veranschaulicht.

Für die Barrierefreiheitsanalyse sind zwei Architekturmerkmale von React besonders relevant: Erstens werden Anwendungslogik und Darstellung in wiederverwendbaren **Komponenten** gekapselt, deren Kommunikation über **Props** (unveränderliche Eingabeparameter) und internen **State** (veränderlicher Zustand) erfolgt.⁷⁹ Zweitens wird die Darstellung der Komponenten mithilfe der Syntaxerweiterung JavaScript XML (JSX) (bzw. TSX bei Verwendung von TypeScript)

⁷⁶Vgl. Meta Open Source 2026b; Vgl. Sommerville 2016, S. 184–186, 499–501

⁷⁷Vgl. Meta Open Source 2026a

⁷⁸Vgl. Stack Overflow 2025

⁷⁹Vgl. Meta Open Source 2026b

definiert, die HTML-ähnliche Strukturen direkt in JavaScript- oder TypeScript-Code einbettet.⁸⁰

2.3.2 Herausforderungen für die Barrierefreiheitsprüfung

Komponentenbasierte Frameworks wie React, Vue oder Angular weisen gemeinsame Architekturmerkmale auf, die die Barrierefreiheitsanalyse grundsätzlich erschweren.⁸¹ Da die vorliegende Arbeit eine React-Anwendung als Untersuchungsgegenstand verwendet, werden die folgenden Herausforderungen anhand von React konkretisiert; sie gelten in ähnlicher Form jedoch auch für andere komponentenbasierte Frameworks.

Erstens entstehen viele Barrierefreiheitsprobleme **zustandsabhängig**, wie bereits in Abschnitt 2.2.2 erläutert wurde. Ein Modalfenster kann sich nach einem Klick öffnen, ohne den Tastaturfokus korrekt zu verwalten. Ebenso kann eine Fehlermeldung nach dem Absenden eines Formulars erscheinen, ohne über eine ARIA-Live-Region programmatisch angekündigt zu werden. Auch ein Dropdown-Menü kann für Tastaturnutzende nicht navigierbar sein. All diese Probleme sind im initialen Seitenzustand nicht vorhanden und damit für statische Analysewerkzeuge unsichtbar.⁸² In React wird diese Problematik durch das State-Management verstärkt: Zustandsänderungen lösen ein erneutes Rendering einzelner Komponenten aus, wobei sich die Accessibility-Eigenschaften des resultierenden DOM je nach State unterscheiden können.

Zweitens besteht eine **Diskrepanz zwischen Quellcode und gerendertem DOM**, die für die entwicklernahe Behebung von Befunden zentral ist. In React werden Benutzeroberflächen in JSX/TSX definiert; der im Browser gerenderte DOM ist das Ergebnis der React-Laufzeitumgebung, die diese Definitionen in DOM-Elemente übersetzt. Ein Accessibility-Tool analysiert den gerenderten DOM, nicht aber den Quellcode, aus dem er hervorging.⁸³ Für Entwicklerinnen und Entwickler ist jedoch der Quellcode der relevante Interventionspunkt.⁸⁴ Ein konkretes Beispiel: Eine React-Komponente `IconButton` rendert im DOM ein `<button>`-Element mit einem `<svg>`-Icon, jedoch ohne sichtbaren Text und ohne `aria-label`. `axe-core` meldet den Verstoß `button-has-visible-text` mit dem HTML-Ausschnitt `<button class="btn-icon»...</button>` – ohne Hinweis darauf, dass die Ursache in der Datei `IconButton.tsx` liegt, wo das `aria-label`-Prop nicht weitergereicht wird.

Drittens erschwert die **Komponentenabstraktion** die Ursachenanalyse: Eine wiederverwendbare Komponente mit einem Accessibility-Problem manifestiert sich im gerenderten DOM an jeder Stelle, an der sie eingesetzt wird. Accessibility-Tool-Reports melden jeden einzelnen Treffer separat, ohne den Bezug zur gemeinsamen Quellkomponente herzustellen, deren einmalige Korrektur alle Treffer beseitigen würde.⁸⁵

⁸⁰Vgl. Bai 2022

⁸¹Vgl. Tafreshipour et al. 2024, S. 110

⁸²Vgl. He, Huq, Malek 2025, S. 5–6; Vgl. World Wide Web Consortium (W3C) 2023, Erf.-krit. 2.4.3 & 4.1.3

⁸³Vgl. Fernandes et al. 2012, S. 31

⁸⁴Vgl. Fathallah, Hernández, Staab 2025

⁸⁵Vgl. Abou-Zahra, Brewer, Cooper 2018

2.3.3 Playwright als Testautomatisierungswerkzeug

Um diese dynamischen Zustände automatisiert reproduzierbar testen zu können, werden Werkzeuge zur Browserautomatisierung benötigt.

Playwright ist ein von Microsoft entwickeltes Open-Source-Framework für die End-to-End-Testautomatisierung von Webanwendungen.⁸⁶ Es ermöglicht die automatisierte Steuerung von Chromium, Firefox und WebKit über eine einheitliche Application Programming Interface (API) und führt Anwendungen in einem echten Browser-Kontext aus, sodass JavaScript-Ausführung, clientseitiges Rendering und dynamische DOM-Manipulationen vollständig nachgebildet werden.

Für die Barrierefreiheitsanalyse ist Playwright aus mehreren Gründen besonders geeignet: Erstens kann es durch programmierte Interaktionen – Klicks, Formularausfüllungen, Tastatureingaben, Hover-Aktionen – gezielt verschiedene Anwendungszustände auslösen.⁸⁷ Zweitens bietet Playwright mit `@axe-core/playwright` eine direkte Integration der axe-core-Bibliothek: Nach jeder Interaktion kann unmittelbar eine vollständige axe-Prüfung des aktuellen DOM-Zustands angestoßen werden.⁸⁸

Verschiedene Forschungsarbeiten nutzen diese Integration bereits erfolgreich für die Barrierefreiheitsanalyse dynamischer Webanwendungen (vgl. Kapitel 3). In der vorliegenden Arbeit dient Playwright als zentrales Werkzeug der Detection-Phase: Es löst definierte Interaktionssequenzen aus, erfasst die resultierenden DOM-Zustände und stößt axe-core-Prüfungen an, deren strukturierter JSON-Output als erster Input für die LLM-Analysepipeline dient.

2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen

Innerhalb der Künstliche Intelligenz (KI)-Forschung haben sich LLMs als leistungsfähige Klasse von Systemen für sprachbasierte Aufgaben etabliert.⁸⁹ Für die vorliegende Arbeit sind sie relevant, weil sie sowohl Code verstehen als auch natürlichsprachige Handlungsempfehlungen generieren können.

2.4.1 Grundlagen: Large Language Models

LLMs sind neuronale Netzwerke, die auf extrem großen Datensätzen mit dem Ziel trainiert werden, das nächste Token einer Textsequenz vorherzusagen.⁹⁰ Die zugrunde liegende Architektur ist der Transformer, der ausschließlich auf Attention-Mechanismen basiert und rekurren-

⁸⁶Vgl. Microsoft 2026

⁸⁷Vgl. Microsoft 2026

⁸⁸Vgl. Deque Systems 2026

⁸⁹Vgl. Russell, Norvig 2021, S. 22

⁹⁰Vgl. Brynjolfsson, Li, Raymond 2025, S. 5–6; Vgl. Jurafsky, Martin 2026, S. 146–148

te sowie konvolutionale Netze in der Sequenzmodellierung ablöst.⁹¹ Auf dieser Trainingsbasis entstehen Modelle, die komplexe Sprachstrukturen, semantische Zusammenhänge und Formen logischen Schlussfolgerns abbilden. Für den Zugriff auf externes Wissen werden häufig *Embedding*-basierte Retrieval-Methoden verwendet (Vektorraumsuche), die in Retrieval-Augmented Generation (RAG)-Systemen zentral sind.⁹² Zu den bekanntesten Modellfamilien zählen GPT-4o (OpenAI), Claude (Anthropic) und LLaMA (Meta).⁹³ Entscheidend für den Anwendungskontext dieser Arbeit sind beobachtete Fähigkeiten dieser Modelle, etwa Code-Generierung, mehrsprachige Übersetzung und kontextuelles Schlussfolgern, die in der Literatur ab einer bestimmten Modellgröße beschrieben werden.⁹⁴

Im Kontext der Barrierefreiheitsanalyse bieten LLMs gegenüber regelbasierten Tools wesentliche Vorteile.⁹⁵ Sie verfügen über ein Sprachverständnis, das die Bewertung semantischer Inhalte ermöglicht,⁹⁶ können Code generieren und transformieren⁹⁷ und verarbeiten den Gesamtkontext einer Seite.⁹⁸ Diesen Stärken stehen bekannte Schwächen gegenüber, insbesondere Halluzinationen,⁹⁹ Wissensgrenzen durch den Knowledge-Cutoff,¹⁰⁰ Nicht-Determinismus¹⁰¹ und mögliche Bias-Effekte.¹⁰²

Eine zentrale Frage der vorliegenden Arbeit ist, ob diese Fähigkeiten auch bei kleineren, lokal ausführbaren Modellen in ausreichendem Maße vorhanden sind. Alle in Kapitel 3 untersuchten Systeme – AccessGuru, GenA11y, ACCESS – setzen GPT-4o ein, ein Modell mit geschätzt über 200 Milliarden Parametern und Zugang zu umfangreichen Cloud-Ressourcen.¹⁰³ Lokal ausführbare Modelle der 7B-Klasse verfügen über deutlich weniger Parameter und damit potenziell über ein eingeschränkteres Sprachverständnis und schwächere Reasoning-Fähigkeiten. Ob ein solches Modell semantische Accessibility-Verstöße zuverlässig erkennen kann, ist empirisch bislang nicht belegt und wird in der vorliegenden Arbeit als explizite Forschungsfrage adressiert. Die Begründung der konkreten Modellauswahl erfolgt in Abschnitt 5.4.

2.4.2 Prompt-Engineering-Strategien

Die Qualität der Ausgaben eines LLM hängt maßgeblich von der Formulierung der Eingabe, also des sogenannten Prompts, ab. Prompt-Engineering bezeichnet die systematische Gestaltung dieser Eingaben, um die Ausgabequalität zu maximieren.¹⁰⁴ Im Folgenden werden die für diese

⁹¹ Vgl. Jurafsky, Martin 2026, S. 173–174; Vgl. Vaswani et al. 2023

⁹² Vgl. Lewis et al. 2020, S. 2–3

⁹³ Vgl. Anthropic 2026a; OpenAI 2023; Touvron et al. 2023

⁹⁴ Vgl. Wei, Tay et al. 2022, S. 2

⁹⁵ Vgl. Fathallah, Hernández, Staab 2025

⁹⁶ Vgl. Tafreshipour et al. 2024

⁹⁷ Vgl. Touvron et al. 2023, S. 3

⁹⁸ Vgl. Bassi et al. 2025, S. 298

⁹⁹ Vgl. Huang, L. et al. 2025, 1:2

¹⁰⁰ Vgl. Anthropic 2026b, S. 7; Huang, L. et al. 2025, 1:9–10

¹⁰¹ Vgl. OpenAI 2026; Vgl. Anthropic 2026a

¹⁰² Vgl. Bender et al. 2021, S. 614–615

¹⁰³ Vgl. Fathallah, Hernández, Staab 2025; Vgl. He, Huq, Malek 2025, FSE101:10; Vgl. Huang, C. et al. 2024, S. 7

¹⁰⁴ Vgl. Schulhoff et al. 2024, S. 4

Arbeit relevanten Strategien beschrieben, die in Kapitel 7 als Grundlage für das Prompt-Design des PoC dienen.

- **Zero-Shot Prompting** bezeichnet die direkte Anfrage ohne Beispiele oder Kontextinformationen. Das Modell nutzt ausschließlich sein vortrainiertes Wissen. Diese Strategie ist einfach umzusetzen, erzielt bei komplexen Aufgaben wie der Barrierefreiheitsanalyse jedoch häufig unzureichende Ergebnisse.¹⁰⁵
- **Few-Shot Prompting** ergänzt den Prompt um eine kleine Anzahl von Beispielen (Input-Output-Paaren), die dem Modell das erwartete Ausgabeformat und die inhaltlichen Erwartungen verdeutlichen. Dieser Ansatz verbessert die Konsistenz und Präzision der Ausgaben erheblich, erfordert jedoch sorgfältig ausgewählte Beispiele.¹⁰⁶ Abbildung 4 vergleicht Zero-Shot- und Few-Shot-Prompting.
- **Chain-of-Thought Prompting (CoT)** veranlasst das Modell, seinen Lösungsweg Schritt für Schritt zu erklären, bevor es zur finalen Antwort gelangt. Dieses explizite Reasoning reduziert nachweislich Halluzinationen bei komplexen Schlussfolgerungen.¹⁰⁷
- **ReAct-Prompting** kombiniert Reasoning (Re) und Action (Act) iterativ: Das Modell wechselt zwischen Denk- und Handlungsschritten, etwa durch das Einbeziehen von Tools und Zwischenergebnissen. Huang et al. demonstrieren, dass ReAct-Prompting bei der automatisierten Barrierefreiheitskorrektur (**ACCESS**) mit einem Severity Score Reduction von 51,3% alle anderen verglichenen Strategien übertrifft.¹⁰⁸
- **Role-Play Prompting** weist dem Modell eine Expertenrolle zu, die seine Ausgaben in Richtung domänenspezifischen Wissens lenkt. Ein Prompt, der dem Modell die Rolle eines „erfahrenen Barrierefreiheitsexperten und React-Entwicklers“ zuweist, verbessert nachweislich die Qualität accessibility-spezifischer Ausgaben.¹⁰⁹
- **Metacognitive Prompting** fordert das Modell auf, die eigene Ausgabe in einem zweiten Schritt zu reflektieren und zu bewerten, bevor diese finalisiert wird.¹¹⁰ In Verbindung mit Corrective Re-Prompting – einer Technik, bei der fehlerhafte Ausgaben dem Modell mit einer Fehlerbeschreibung zurückgespielt werden – ermöglicht dieser Ansatz eine iterative Qualitätssteigerung. Fathallah et al. belegen, dass Corrective Re-Prompting in **Access-Guru** den Violation Score Decrease von 0,72 auf 0,84 verbessert.¹¹¹

¹⁰⁵ Vgl. Brown et al. 2020, S. 4; Vgl. Njifenjou et al. 2024

¹⁰⁶ Vgl. Brown et al. 2020, S. 5

¹⁰⁷ Vgl. Wei, Wang, X. et al. 2022, S. 2; Fathallah, Hernández, Staab 2025

¹⁰⁸ Vgl. Huang, C. et al. 2024, S. 5; Yao et al. 2022, S. 1

¹⁰⁹ Vgl. Fathallah, Hernández, Staab 2025; Njifenjou et al. 2024; Schulhoff et al. 2024, S. 6, 12

¹¹⁰ Vgl. Wang, Y., Zhao 2024; Schulhoff et al. 2024, S. 8–9

¹¹¹ Vgl. Fathallah, Hernández, Staab 2025

¹¹² Entnommen aus Brown et al. 2020

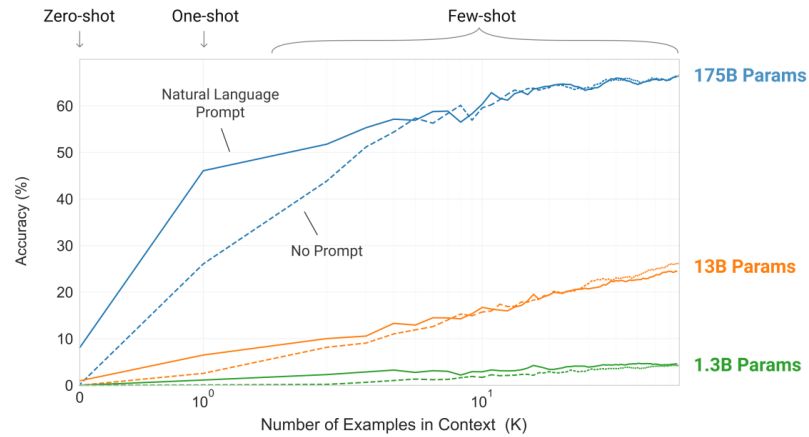


Abb. 4: Zero-Shot und Few-Shot Prompting im Vergleich.¹¹²

2.4.3 RAG und Fine-Tuning als Erweiterungsansätze

Retrieval-Augmented Generation (RAG) erweitert den Prompt zur Laufzeit mit thematisch passenden Dokumenten aus einer externen Wissensbasis und könnte im Accessibility-Kontext aktuelle WCAG-Kriterien unabhängig vom Trainings-Cutoff abrufbar machen.¹¹³ **Fine-Tuning** bezeichnet das domänenspezifische Training auf einem aufgabenspezifischen Datensatz; für den Kontext der BITBW wäre insbesondere ein Fine-Tuning auf BITV-spezifischen Anforderungen denkbar.¹¹⁴ Beide Ansätze liegen außerhalb des Untersuchungsumfangs dieser Arbeit und werden im Ausblick (Abschnitt 9.3) aufgegriffen.

2.4.4 Datenschutz bei KI-gestützten Accessibility-Services

Der Einsatz von LLMs im Kontext öffentlicher Verwaltungen wirft spezifische Datenschutzfragen auf. Webanwendungen der öffentlichen Verwaltung verarbeiten häufig personenbezogene Daten – etwa in Formularseiten, die persönliche Angaben von Bürgerinnen und Bürgern erfassen. Werden diese Seiten für eine KI-gestützte Accessibility-Analyse an externe LLM-APIs übermittelt, ist die Wahrung der Datenschutzgarantien fraglich.

Die Datenschutz-Grundverordnung (DSGVO) schreibt vor, dass personenbezogene Daten nur auf Basis einer Rechtsgrundlage und unter Wahrung der Grundsätze der Zweckbindung und Datenminimierung verarbeitet werden dürfen.¹¹⁵ Die Übermittlung von HTML-Quellcode der produktiven Anwendung, der im schlimmsten Fall personenbezogene Daten als Attributwerte enthalten kann, an externe API-Dienste wie die OpenAI API oder die Anthropic API ist ohne explizite Rechtsgrundlage und einen Auftragsverarbeitungsvertrag nach Art. 28 DSGVO für öffentliche Verwaltungen in Deutschland problematisch.¹¹⁶

¹¹³Vgl. Lewis et al. 2020, S. 1–2

¹¹⁴Vgl. Silvestre, Pietzuch 2025, S. 90

¹¹⁵Vgl. Europäisches Parlament und Rat der Europäischen Union 2016

¹¹⁶Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28

Als Lösungsansatz wird in dieser Arbeit ein Lokal-First-Prinzip verfolgt: Das KI-Assistenzsystem soll ohne externe API-Aufrufe betreibbar sein, indem lokal ausführbare Open-Source-Modelle – etwa Modelle der LLaMA-Familie oder Mistral – eingesetzt werden.¹¹⁷ Damit bleiben sämtliche Analyse- und Codedaten innerhalb der eigenen Infrastruktur. Diese Anforderung prägt sowohl die nicht-funktionalen Anforderungen in Kapitel 5 als auch die Architekturentscheidungen in Kapitel 7.

Die in diesem Kapitel erarbeiteten theoretischen Grundlagen – das normative Fundament der WCAG, die empirisch belegten Grenzen regelbasierter Tools, die Besonderheiten von React-Anwendungen sowie die Möglichkeiten und Grenzen von LLMs und des Prompt Engineerings – bilden den konzeptionellen Rahmen für den im folgenden Kapitel dargestellten Forschungsstand sowie für die anschließende Methodik.

¹¹⁷Vgl. Touvron et al. 2023

3 Stand der Forschung

Dieses Kapitel gibt einen thematisch strukturierten Überblick über den Forschungsstand zur KI-gestützten Barrierefreiheitsanalyse. Darauf aufbauend werden die verbleibenden Forschungslücken herausgearbeitet, die den Ausgangspunkt der vorliegenden Arbeit bilden. Die relevanten Arbeiten lassen sich drei Entwicklungslinien zuordnen: (1) LLM-basierte Erkennung von Accessibility-Verstößen, (2) LLM-basierte Korrektur und Code-Generierung sowie (3) hybride Pipelines, die regelbasierte Tools mit LLM-Analyse kombinieren.

3.1 Vorgehen der Literaturrecherche

Zur Einordnung des Forschungsstands wurde eine strukturierte Literaturrecherche in wissenschaftlichen Repositorien und Datenbanken durchgeführt (u. a. ACM Digital Library, IEEE Xplore, ScienceDirect und arXiv).¹¹⁸ Die Recherche erfolgte themenorientiert auf Basis von Suchbegriffen wie „web accessibility“, „LLM accessibility“, „accessibility violation detection“ und „axe-core“. Berücksichtigt wurden primär aktuelle Arbeiten mit Bezug zur automatisierten Erkennung oder Behebung von Barrierefreiheitsverstößen in Webanwendungen mit KI-Assistenzsystemen. Die identifizierten Arbeiten wurden thematisch gruppiert, priorisiert und vergleichend ausgewertet.

3.2 Von frühen KI-Ansätzen zur LLM-basierten Erkennung

Eine der frühen Arbeiten in diesem Kontext stammt von Delnevo et al. (2024). Die Autoren zeigen, dass GPT-3.5 einfache syntaktische Verstöße mit moderater Genauigkeit erkennt, bei semantischen Problemen jedoch inkonsistente Bewertungen liefert.¹¹⁹ Bassi et al. (2025) vergleichen GPT-4o, GPT-3.5 und LLaMA 3.1 in einer zweistufigen Evaluation. GPT-4o identifiziert dabei 74% der Verstöße korrekt, während 15% der Befunde als Halluzinationen eingestuft werden.¹²⁰ Eine nachgelagerte Chain-of-Verification-Stufe erhöht die Zuverlässigkeit deutlich; RAG wird nur als zukünftige Erweiterung diskutiert.¹²¹

3.3 LLM-basierte Korrektur und Prompt-Engineering

Abu Doush und Kassem (2024) zeigen, dass strukturierte Prompts die Qualität LLM-generierter barrierefreier HTML-Alternativen deutlich verbessern, bei komplexen Interaktionsmustern je-

¹¹⁸Vgl. Webster, Watson 2002

¹¹⁹Vgl. Delnevo, Andruccioli, Mirri 2024

¹²⁰Vgl. Bassi et al. 2025, S. 302

¹²¹Vgl. Bassi et al. 2025, S. 301

doch weiterhin Schwächen bestehen.¹²² Guriță und Vatavu (2025) zeigen, dass accessibility-orientierte Prompts paradoxerweise zunächst mehr Violations erzeugen können als neutrale Formulierungen – erst iteratives Prompting über fünf Runden senkte den Violation Score von 0,84 auf 0,32.¹²³

Einen zentralen Referenzpunkt des aktuellen Forschungsstands bildet **AccessGuru** von Fathallah et al. (2025).¹²⁴ Das System kombiniert eine zweistufige Pipeline: zunächst regelbasierte Detection via Playwright und **axe-core**, anschließend eine LLM-basierte Korrektur durch GPT-4o mit Role-Play-, Contextual- und Metacognitive-Prompting. Corrective Re-Prompting verbessert den Violation Score Decrease¹²⁵ von 0,72 auf 0,84 bei einer semantischen Ähnlichkeit von 77% zu menschlichen Referenzlösungen. Als offenes Problem benennen die Autoren die fehlende Verknüpfung korrigierter HTML-Snippets mit dem Quellcode der geprüften Anwendung.

Fernández-Navarro und Chicano (2026) erweitern diesen Ansatz um eine direkte Quellcode-Korrektur und adressieren damit explizit die Lücke zwischen DOM-Korrektur und entwicklerseitiger Umsetzung.¹²⁶ Ihr Tool injiziert Korrekturen auf Basis einer Quellcodeanalyse gezielt in tatsächliche Komponentendateien und erreicht in der Evaluation auf öffentlichen Websites und Open-Source-Angular-Projekten Remediation Rates¹²⁷ von über 80% bei vollständiger Build-Integrität. Der Ansatz beschränkt sich jedoch auf Angular und setzt auf cloudbasiertes GPT-4o; eine Übertragung auf React wird von den Autoren als kurzfristiger Folgeschritt benannt.¹²⁸

3.4 Hybride Pipelines und Ensemble-Testing

Huang et al. entwickeln 2024 mit **ACCESS** ein System, das Prompt-Engineering gezielt zur automatisierten Barrierefreiheitskorrektur einsetzt und dabei verschiedene Strategien systematisch vergleicht.¹²⁹ Der Vergleich von ReAct, Chain-of-Thought und Few-Shot Prompting zeigt, dass ReAct mit einem Severity Score Reduction¹³⁰ von 51,3% alle anderen Strategien übertrifft – ein Ergebnis, das die Bedeutung iterativer, werkzeuggestützter Reasoning-Strategien für diese Aufgabenklasse unterstreicht.

He et al. präsentieren 2025 mit **GenA11y** eines der methodisch ausgefeiltesten Detection-Systeme des aktuellen Forschungsstands.¹³¹ **GenA11y** extrahiert nach vollständigem clientseitigem Rendering den DOM via Playwright und prüft ihn anschließend kriteriumsspezifisch mit

¹²²Vgl. Doush, Kassem 2025, S. 343

¹²³Vgl. Guriță, Vatavu 2025, S. 1000

¹²⁴Vgl. Fathallah, Hernández, Staab 2025

¹²⁵Der *Violation Score Decrease* gibt an, um welchen Anteil die gewichtete Gesamtanzahl der Violations durch das System reduziert werden konnte (Wert zwischen 0 und 1).

¹²⁶Vgl. Fernández-Navarro, Chicano 2026

¹²⁷*Remediation Rate*: Anteil der Violations, die durch das System erfolgreich behoben wurden.

¹²⁸Vgl. Fernández-Navarro, Chicano 2026

¹²⁹Vgl. Huang, C. et al. 2024, S. 7, 9

¹³⁰*Severity Score Reduction*: prozentualer Rückgang des gewichteten Schweregradwerts aller Violations gegenüber dem Ausgangszustand.

¹³¹Vgl. He, Huq, Malek 2025, FSE101:20–21

GPT-4o. Eine Ablation Study belegt, dass weder DOM-Extraktion noch optimiertes Prompting allein die Gesamtleistung von Precision 94,5% und Recall 87,61% erreichen.¹³² **GenA11y** erkennt acht Verstoßtypen, die von keinem der verglichenen regelbasierten Tools gefunden werden.¹³³ Paternò et al. ergänzen dieses Bild mit **TeVal**, das semantische WCAG-Kriterien durch Role-Play Prompting mit Few-Shot-Beispielen und Confidence Score bewertet und dabei eine Effektivität von 92,4% erzielt.¹³⁴

Pool (2023) adressiert die Detection-Schicht mit **Testaro**, einem Ensemble aus acht Accessibility-Werkzeugen (u. a. axe-core, WAVE, QualWeb); die Analyse zeigt, dass jedes Tool exklusive Violations erkennt und die Werkzeuge damit komplementär wirken.¹³⁵

3.5 Forschungslücken und Einordnung der vorliegenden Arbeit

Die Analyse der bestehenden Forschungsarbeiten zeigt drei persistente Forschungslücken, die den Ausgangspunkt der vorliegenden Arbeit bilden.

Erstens berücksichtigt keiner der untersuchten Ansätze zustandsabhängige Barrieren systematisch, die erst durch Nutzerinteraktionen in React-basierten Single-Page Applications entstehen.¹³⁶ Zwar setzen einzelne Systeme wie **AccessGuru** Playwright zur DOM-Extraktion ein, jedoch beschränkt sich dies auf das initiale Rendering – interaktionsbedingte Zustandswechsel werden nicht geprüft.¹³⁷

Zweitens verknüpft keiner der analysierten Ansätze Tool-Reports systematisch mit dem Quellcode der geprüften Anwendung. **AccessGuru** selbst benennt dies als offenes Problem: die generierten Korrekturen beziehen sich auf isolierte DOM-Snippets und nicht auf die JSX-Komponenten, in denen Entwicklerinnen und Entwickler tatsächlich eingreifen müssten.¹³⁸ Die Lücke zwischen technischer Fehlererkennung und entwicklernaher Umsetzung bleibt damit bestehen.

Drittens adressiert keine der untersuchten Arbeiten explizit die datenschutzrechtlichen Anforderungen öffentlicher Verwaltungen: alle Systeme setzen auf cloudbasierte LLM-APIs (primär GPT-4o), was für die BITBW und vergleichbare Institutionen ohne vertiefte datenschutzrechtliche Prüfung nicht nutzbar ist.¹³⁹

Die Konzeptmatrix in Tabelle 2 fasst die zentralen Merkmale der untersuchten Studien im Vergleich zum eigenen PoC zusammen und macht die identifizierten Forschungslücken strukturell sichtbar.

¹³²Vgl. He, Huq, Malek 2025, FSE101:20

¹³³Vgl. He, Huq, Malek 2025, FSE101:20

¹³⁴Vgl. Paternò et al. 2025

¹³⁵Vgl. Pool 2023

¹³⁶Vgl. Tafreshipour et al. 2024, S. 110

¹³⁷Vgl. Fathallah, Hernández, Staab 2025

¹³⁸Vgl. Fathallah, Hernández, Staab 2025

¹³⁹Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28, 44 ff.

¹⁴⁰Vgl. Webster, Watson 2002.

Studie		Konzepte													
Autor(en) & Jahr	System	Ziel			Datenquellen			KI- / LLM-Ansatz			Evaluation			Kontext	
		WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA	Rechtl. Rahmen (BfSG etc.)
Hoch priorisierte Studien															
He et al. (2025)	GenA11y	x			x			x			x				
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x				
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x			x
Paternò et al. (2025)	TeVal	x		x	x			x	x			x			
Tafreshipour et al. (2024)	Ma11y	x			x	x					x				
Fernández-Navarro & Chicano (2026)	LLM Remediation	x	x	x	x		x	x			x			x	
Martinez Campo (2026)	ACE (PoC)	x	x	x	x	x	x	x	x	x		x		x	x

Tab. 2: Konzeptmatrix nach Webster Watson¹⁴⁰: hoch priorisierte Studien im Vergleich zum eigenen Proof of Concept (PoC). **x** = Konzept vorhanden/adressiert; leer = nicht adressiert. Für den eigenen PoC bezeichnen die Markierungen angestrebte Designziele, deren Umsetzung und Wirksamkeit in Kapitel 8 evaluiert wird. Die vollständige Matrix aller analysierten Studien findet sich in Anhang 4.

4 Methodik

Dieses Kapitel beschreibt das methodische Vorgehen der Arbeit. Zunächst wird die gewählte Forschungsmethode begründet (Abschnitt 4.1), anschließend der Untersuchungsgegenstand abgegrenzt (Abschnitt 4.2) und schließlich das Evaluationsdesign mit den zugehörigen Bewertungskriterien definiert (Abschnitt 4.3).

4.1 Wissenschaftliche Methode

Die vorliegende Arbeit folgt dem Forschungsparadigma des Design Science Research (DSR), das von Hevner et al. für die Wirtschaftsinformatik systematisiert wurde.¹⁴¹ DSR eignet sich für diese Arbeit aus zwei Gründen. Erstens steht die Konstruktion eines konkreten technischen Artefakts im Zentrum der Forschungsfrage, nicht die Überprüfung einer statistischen Hypothese. Zweitens fordert DSR explizit die Evaluation des Artefakts in einem realen Anwendungskontext, was dem praxisorientierten Charakter der Arbeit entspricht.¹⁴²

Alternative Ansätze wie Action Research oder kontrollierte Experimente wurden verworfen: Action Research setzt eine längerfristige Praktikerzusammenarbeit voraus, während kontrollierte Experimente eine Stichprobenbasis erfordern, die im Verwaltungskontext nicht verfügbar ist.¹⁴³

Als prozessualer Rahmen dient das sechshebige DSRM-Prozessmodell nach Peffers et al.¹⁴⁴ Die Zuordnung der einzelnen Phasen zu den Kapiteln dieser Arbeit zeigt Tabelle 3.

DSRM-Phase	Kapitel
Problem-Identifikation	Kapitel 1-3
Zieldefinition	Kapitel 1.3 & 3.5
Design & Entwicklung	Kapitel 5-7
Demonstration	Kapitel 7 (insb. Beispielabläufe)
Evaluation	Kapitel 8
Kommunikation	Gesamtarbeit, zusammenfassend Kapitel 9

Tab. 3: Zuordnung der DSRM-Phasen zu den Kapiteln der Arbeit.

Ergänzt wird dieser methodische Rahmen durch einen Fallstudienansatz nach Yin.¹⁴⁵ Der BWEC dient als Einzelfallstudie, da die Forschungsfrage auf ein zeitgenössisches Phänomen in einem spezifischen Praxiskontext abzielt und die Grenzen zwischen technischem System und organisatorischem Kontext nicht trennscharf sind.

¹⁴¹Vgl. Hevner, A. R. et al. 2004

¹⁴²Vgl. Hevner, A. R. et al. 2004, S. 80

¹⁴³Vgl. Baskerville, R. L. 1999, S. 1–2; Vgl. Baskerville, R., Myers 2004, S. 239–335

¹⁴⁴Vgl. Peffers et al. 2007

¹⁴⁵Vgl. Yin 2018

Die Wahl eines PoC als Artefakttyp ist durch den explorativen Charakter der Forschungsfrage begründet: Das Ziel ist die konzeptionelle Demonstration der technischen Machbarkeit, nicht die statistisch belastbare Generalisierung auf eine Grundgesamtheit.¹⁴⁶

4.2 Untersuchungsgegenstand und Scope-Abgrenzung

Als Praxisrahmen und Motivationskontext dient der BWEC, eine öffentlich zugängliche React-basierte Webanwendung der BITBW.¹⁴⁷ Er vereint drei für die Forschungsfrage zentrale Merkmale: gesetzlich relevante Anforderungen an digitale Barrierefreiheit, einen datenschutzsensiblen Einsatzkontext in der öffentlichen Verwaltung sowie typische Interaktionsmuster moderner Single-Page Applications. Die formale Hauptevaluation erfolgt jedoch an synthetisch konstruierten Testsuiten (test-12, test-50, test-100) mit vollständig bekannter Ground Truth. Der BWEC wird ergänzend als Feldtest ohne quantitativen Ground-Truth-Vergleich eingesetzt (vgl. Abschnitt 8.7.3).

Der Untersuchungsscope ist wie folgt abgegrenzt:

- **Eingeschlossen:** Öffentlich zugängliche Bereiche der Anwendung ohne Authentifizierung. Betrachtet werden WCAG 2.2-Verstöße auf Konformitätsstufe A und AA, soweit sie durch regelbasierte Prüfung, DOM-Analyse und Interaktionsausführung im Frontend beobachtbar sind.¹⁴⁸
- **Ausgeschlossen:** Backend-Logik, Datenbankstrukturen sowie eine vollständige rechtliche Bewertung im Sinne einer formalen BITV-Prüfung.¹⁴⁹

Die Einschränkungen hinsichtlich vollständiger WCAG-Abdeckung und Generalisierbarkeit auf beliebige Frameworks werden in Kapitel 8 als Limitationen diskutiert.

4.3 Evaluationsdesign und Kriterien

Ziel der Evaluation ist es, die technische Machbarkeit und den praktischen Nutzen des entwickelten PoC anhand synthetisch konstruierter Testsuiten mit vollständig bekannter Ground Truth zu untersuchen. Der BWEC dient ergänzend als Feldtest-Kontext ohne quantitativen Recall-Vergleich (vgl. Abschnitt 8.7.3).

¹⁴⁶Vgl. Hevner, A. R. et al. 2004

¹⁴⁷Vgl. Land Baden-Württemberg 2025

¹⁴⁸Vgl. World Wide Web Consortium (W3C) 2023

¹⁴⁹Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

4.3.1 Evaluationslogik

Die Evaluation folgt einem fünfstufigen Vorgehen:

1. **Testsuite-Konstruktion:** Drei synthetische React-SPAs werden mit gezielt eingebetteten WCAG-Violations entwickelt (test-12: 12, test-50: 50, test-100: 100 Violations), die alle drei Verstoßkategorien der in Abschnitt 2.2.3 eingeführten Taxonomie abdecken.¹⁵⁰
2. **Benchmarkläufe:** Jede Suite wird mit drei Modellen jeweils zehnmal analysiert (insgesamt 90 Runs). Pro Lauf werden Recall, Priorisierungsverhalten, Fix-Qualität und Laufzeit erfasst.
3. **Recall-Auswertung:** Die Ergebnisse werden mit der bekannten Ground Truth verglichen. Eine Violation gilt als erkannt, wenn sie in der Mehrzahl der zehn Runs im finalen LLM-Report erscheint.
4. **Qualitative Empfehlungsbewertung:** Die generierten Handlungsempfehlungen werden anhand einer dreistufigen Bewertungsskala (Stufe 0–2) nach Dateikonkretheit und Code-Fix-Qualität bewertet.
5. **BWEC-Feldtest:** Die Pipeline wird ergänzend auf dem realen BWEC-Untersuchungsobjekt eingesetzt, um Systemstabilität und qualitative Plausibilität unter realen Bedingungen zu prüfen.¹⁵¹

4.3.2 Bewertungskriterien

Die Evaluation erfolgt anhand von vier Kriterien, die aus den Forschungsfragen und der Fachliteratur abgeleitet wurden.

Kriterium 1 – Korrektheit der Erkennung: Die vom System identifizierten Verstöße werden mit der bekannten Ground Truth der synthetischen Testsuiten verglichen. Als Orientierungswerte dienen die Ergebnisse vergleichbarer Systeme, insbesondere **GenA11y** erreicht eine Precision von 94,5% und einen Recall von 87,61%.¹⁵²

Kriterium 2 – Qualität der Handlungsempfehlungen: Die generierten Empfehlungen werden anhand einer dreistufigen Skala (Stufe 0–2) qualitativ bewertet: Stufe 2 bezeichnet einen konkreten Fix mit Dateiname, Zeilennummer und umsetzbarem Code-Vorschlag; Stufe 1 einen zutreffenden Fix-Typ ohne Quellcode-Kontext; Stufe 0 eine generische Problembeschreibung. **AccessGuru** erreicht als Referenzwert einen Violation Score Decrease von 84%.¹⁵³

Kriterium 3 – Mehrwert der Kontextualisierung: Der Zusatznutzen der vollständigen Kontextkombination wird durch den systematischen Vergleich zweier Konfigurationen ermittelt:

¹⁵⁰Vgl. World Wide Web Consortium (W3C) 2023

¹⁵¹Vgl. Deque Systems 2026; Vgl. Kodithuwakku, Wickramaarachchi 2025

¹⁵²Vgl. He, Huq, Malek 2025, FSE101:20

¹⁵³Vgl. Fathallah, Hernández, Staab 2025

(a) die reine Tool-Baseline (axe-core, Playwright, grep ohne LLM-Kontextualisierung) und (b) die vollständige Pipeline (Baseline + LLM-gestützte Quellcodeanalyse). Verglichen werden Recall, Fix-Qualität sowie die Fähigkeit, datei- und zeilenspezifische Handlungsempfehlungen zu generieren.¹⁵⁴

Kriterium 4 – Technische Praktikabilität: Laufzeit, Stabilität und Qualität des JSON-Outputs werden unter realen Bedingungen dokumentiert.

4.3.3 Limitationen des Evaluationsdesigns

Zwei methodische Einschränkungen des Evaluationsdesigns sind vorab zu benennen. Erstens wird die Bewertung durch den Autor selbst durchgeführt und nicht durch unabhängige Gutachter validiert. Zweitens sind die genannten Referenzwerte von **AccessGuru**¹⁵⁵ und **GenA11y**¹⁵⁶ nur eingeschränkt vergleichbar, da sie auf anderen Datensätzen und unter anderen Rahmenbedingungen ermittelt wurden. Sie dienen daher als Orientierung, nicht als direkter Benchmark.

¹⁵⁴Siehe Abschnitt 1.3.

¹⁵⁵Vgl. Fathallah, Hernández, Staab 2025

¹⁵⁶Vgl. He, Huq, Malek 2025

5 Analyse und Anforderungen

Dieses Kapitel erarbeitet die Grundlagen für das in Kapitel 6 beschriebene Systemdesign. Zunächst werden Problemkontext und Handlungsbedarf analysiert. Anschließend werden die funktionalen und nicht-funktionalen Anforderungen an das zu entwickelnde System spezifiziert. Abschließend wird die Auswahl der eingesetzten Inferenzinfrastruktur und der Modellkandidaten begründet.

5.1 Problemkontext und Handlungsbedarf

Wie in Kapitel 1 dargelegt, unterliegen alle von der BITBW entwickelten und betriebenen Webanwendungen den gesetzlichen Anforderungen der BITV 2.0, für die WCAG 2.2 auf Konformitätsstufe AA den verbindlichen Maßstab bildet.¹⁵⁷ In der Praxis erfolgt die Barrierefreiheitsprüfung bislang überwiegend manuell oder mithilfe einzelner regelbasierter Tools. Dieses Vorgehen ist aus zwei Gründen problematisch: Manuelle Audits sind personalintensiv und skalieren nur unzureichend mit der Anzahl zu betreuender Anwendungen; regelbasierte Tools erfassen, wie in Kapitel 2.2.1 dargelegt, nur einen Bruchteil der tatsächlich vorhandenen Barrieren.¹⁵⁸

Die in Kapitel 3 identifizierten drei Forschungslücken bilden den Ausgangspunkt der Anforderungserhebung.¹⁵⁹ Dazu zählen die fehlende Zustandserfassung durch Interaktionen, die fehlende Verknüpfung von DOM-Befunden mit dem Quellcode sowie die Unvereinbarkeit cloudbasierter LLM-APIs mit den Datenschutzanforderungen öffentlicher Verwaltungen (vgl. Abschnitt 2.4.4). Daraus ergibt sich als Systemziel ein lokal ausführbares, LLM-gestütztes Assistenzsystem, das axe-core-Reports, Playwright-Interaktionsdaten und statische Quellcodeanalyse kombiniert (vgl. Abschnitt 1.3).¹⁶⁰

5.2 Funktionale Anforderungen

Funktionale Anforderungen (FA) beschreiben die Leistungen, die ein System erbringen soll.¹⁶¹ Die folgenden Anforderungen leiten sich aus der Problemanalyse (Abschnitt 5.1), den identifizierten Forschungslücken (Abschnitt 3.5) sowie der Analyse vergleichbarer Systeme aus der Literatur ab.

FA-01 Regelbasierte Accessibility-Erkennung

¹⁵⁷Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

¹⁵⁸Vgl. Tafreshipour et al. 2024, S. 8–9

¹⁵⁹Vgl. Hevner, A., Chatterjee 2010, S. 79; Vgl. Hevner, A., Chatterjee 2010, S. 9–11, 23–24

¹⁶⁰Vgl. Hevner, A., Chatterjee 2010, S. 9–11, 23–24

¹⁶¹Vgl. Sommerville 2016, S. 105–107

Das System muss den BVEC automatisiert auf Barrierefreiheitsverstöße gemäß WCAG 2.2 der Konformitätsstufen A und AA prüfen.¹⁶² Die Prüfung basiert auf axe-core als Detection-Backend.¹⁶³ Der Output je Verstoß umfasst mindestens: Regel-ID, Schweregrad (critical/serious/moderate/minor) sowie ein HTML-Ausschnitt des betroffenen Elements.

FA-02 Dynamische Zustandserfassung via Playwright

Das System muss mit Playwright definierte Interaktionssequenzen ausführen und zustandsabhängige Barrieren durch generische Interaktionschecks erfassen.¹⁶⁴ Damit werden Barrieren erfasst, die im initialen Seitenzustand nicht vorhanden sind.¹⁶⁵

FA-03 Statische Quellcodeanalyse

Das System muss den Quellcode der geprüften React-Anwendung automatisiert auf Muster untersuchen, die auf Barrierefreiheitsprobleme hindeuten, etwa fehlende `aria`-Attribute, `onClick`-Handler auf nicht-interaktiven Elementen, fehlende `alt`-Attribute sowie fehlende Formular-Assoziationen.¹⁶⁶

FA-04 LLM-gestützte Analyse und Handlungsempfehlung

Das System muss die Ergebnisse aus FA-01, FA-02 und FA-03 in einem strukturierten Kontext zusammenführen und an ein lokal ausgeführtes Sprachmodell übergeben, das daraus je Verstoß eine Handlungsempfehlung im Format Must-have/Nice-to-have unter Angabe der betroffenen Datei generiert.¹⁶⁷

FA-05 Strukturierter JSON-Output

Das System muss seine Ergebnisse in einem maschinenlesbaren JSON-Format bereitstellen.¹⁶⁸ Der Report enthält je Befund: Verstoß-ID, Kategorie (syntaktisch/semantisch/layout), Schweregrad, betroffene Datei(en), Handlungsempfehlung (Must-have/Nice-to-have), WCAG-Erfolgskriterium und – falls vorhanden – den relevanten Quellcode-Ausschnitt.

5.3 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen (NFA) beschreiben die Qualitätseigenschaften des Systems. Für das vorliegende System haben sie besonderes Gewicht, da der Betriebskontext der BITBW (öffentliche Verwaltung, lokale Infrastruktur) strenge Rahmenbedingungen setzt.

NFA-01 Datenschutz und Lokal-First

¹⁶²Vgl. Land Baden-Württemberg 2025; World Wide Web Consortium (W3C) 2023

¹⁶³Vgl. Deque Systems 2026

¹⁶⁴Vgl. Microsoft 2026

¹⁶⁵Siehe Kapitel 2.3.2

¹⁶⁶Siehe Kapitel 2.1

¹⁶⁷Siehe Abschnitt 6.2

¹⁶⁸Vgl. Paternò et al. 2025; Pool 2023

Das System muss vollständig ohne externe API-Aufrufe betreibbar sein. Weder Quellcode noch DOM-Inhalte noch Violation-Reports dürfen das interne Netzwerk verlassen.¹⁶⁹

NFA-02 Laufzeit und Praktikabilität

Der vollständige Analyselauf muss auf einem Entwicklungsrechner innerhalb einer für den Entwicklungsalltag praktikablen Zeit abgeschlossen werden. Als Richtwert gilt eine Gesamtlaufzeit von unter zehn Minuten für eine einzelne Seite mit bis zu 20 axe-Verstößen. Diese Anforderung schließt mehrstufige Reasoning-Schleifen aus dem Scope des PoC aus.¹⁷⁰

NFA-03 Wartbarkeit und Erweiterbarkeit

Die Systemarchitektur muss eine modulare Trennung der Komponenten (Detection, Context-Building, Synthesis) vorsehen, sodass einzelne Bestandteile unabhängig angepasst oder ausgetauscht werden können.

NFA-04 Reproduzierbarkeit

Das System muss hinreichend deterministisch sein, damit zwei aufeinanderfolgende Analyseläufe auf derselben Anwendungsversion im Wesentlichen dieselben Befunde erzeugen. Reproduzierbarkeit wird durch eine feste Temperatur-Einstellung (`temperature = 0.05`) und durch strukturierte Ausgabeformate (JSON-Schema im Prompt) sichergestellt.¹⁷¹

5.4 Lokale Inferenzinfrastruktur und Modellkandidaten

Die Wahl der Inferenzinfrastruktur ist durch NFA-01 (Lokal-First) weitgehend vorgegeben: Cloudbasierte Modell-APIs scheiden aus, da ihre Nutzung die Übermittlung potenziell schützenswerter Quellcodeausschnitte und DOM-Inhalte an externe Server erfordert.¹⁷²

5.4.1 Ollama als Laufzeitumgebung

Ollama ist ein quelloffenes Werkzeug, das die lokale Ausführung von LLMs über eine einheitliche REST-API ermöglicht.¹⁷³ Die REST-API ist kompatibel mit der OpenAI-API-Spezifikation, wodurch sich bestehende Node.js-Bibliotheken ohne größeren Anpassungsaufwand integrieren lassen. Darüber hinaus unterstützt Ollama quantisierte Modellvarianten (z. B. Q4_K_M), die eine speichereffiziente Ausführung auf Central Processing Unit (CPU)-basierten Systemen ohne dedizierte Grafikkarte ermöglichen. Dies ist insbesondere für Entwicklungsrechner der BITBW relevant, bei denen nicht von Graphics Processing Unit (GPU)-Hardware ausgegangen werden kann.

¹⁶⁹Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28 und Art. 32

¹⁷⁰Siehe Kapitel 2.4.2

¹⁷¹Vgl. OpenAI 2026, Temperature; Vgl. Anthropic 2026a, Temperature

¹⁷²Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28 und Art. 44 ff.

¹⁷³Vgl. Ollama 2026

5.4.2 Anforderungen an das Sprachmodell

Unabhängig von der konkreten Modellwahl lassen sich vier zentrale Anforderungen an das Sprachmodell ableiten. Es muss (1) **JSX und TypeScript** verarbeiten können (FA-03), (2) **strukturierte Ausgaben** in einem vorgegebenen Format erzeugen (FA-05, NFA-04), (3) innerhalb eines **Kontextfensters von mindestens 8.192 Tokens** arbeiten (FA-04) und (4) **auf CPU-basierten Systemen** mit akzeptabler Latenz betreibbar sein, was die Modellgröße auf ca. 7–32 Milliarden Parameter begrenzt (NFA-02).¹⁷⁴

Die Auswahl konkreter Modellkandidaten sowie deren vergleichende Evaluation werden in Abschnitt 6.2 bzw. Kapitel 8 behandelt.

5.4.3 Ausgewählte Modellkandidaten für die Evaluation

Für die Evaluation in Kapitel 8 werden vier lokal ausführbare Modellkandidaten betrachtet, soweit die verfügbaren Hardware-Ressourcen eine Ausführung zulassen: `qwen2.5-coder:7b`, `qwen2.5-coder:14b`¹⁷⁵, `qwen3:32b`¹⁷⁶ und `deepseek-r1:70b`¹⁷⁷. Die Auswahl folgt dem Ziel, unterschiedliche Modellgrößen systematisch miteinander zu vergleichen: 7B und 14B repräsentieren ressourcenschonende Modelle für den operativen Lokalbetrieb, 32B einen mittleren Kompromiss aus Qualität und Laufzeit, 70B ein leistungsstarkes Referenzmodell mit höherem Rechenbedarf. DeepSeek-R1:70b wurde als Kandidat betrachtet, konnte jedoch aufgrund der verfügbaren Hardware-Ressourcen und der damit verbundenen Laufzeitanforderungen nicht in die finale Benchmarkserie aufgenommen werden; die Evaluation in Kapitel 8 basiert daher auf den drei verbleibenden Modellen.

5.4.4 Zusammenfassung der Anforderungen

Tabelle 4 fasst die spezifizierten Anforderungen zusammen und ordnet ihnen jeweils Ursprung und Priorität zu.

Die Anforderungen FA-01 bis FA-05 sowie NFA-01 bis NFA-04 bilden die verbindliche Grundlage für das Systemdesign in Kapitel 6. NFA-01 (Lokal-First) hat dabei Vorrang vor allen anderen Anforderungen und schränkt den Lösungsraum strukturell ein.

¹⁷⁴Vgl. Hui et al. 2024, S. 3

¹⁷⁵Vgl. Hui et al. 2024

¹⁷⁶Vgl. Yang et al. 2025

¹⁷⁷Vgl. Guo et al. 2025

ID	Beschreibung	Ursprung	Priorität
Funktionale Anforderungen			
FA-01	Regelbasierte Accessibility-Erkennung via axe-core	Forschungsliteratur (Kap. 3)	Muss
FA-02	Dynamische Zustandserfassung via Playwright	Forschungslücke 1 (Kap. 3.5)	Muss
FA-03	Statische Quellcodeanalyse (grep-basiert)	Forschungslücke 2 (Kap. 3.5)	Muss
FA-04	LLM-gestützte Analyse und Handlungsempfehlung	Forschungsliteratur (Kap. 3)	Muss
FA-05	Strukturierter JSON-Output	Praxisanforderung BITBW	Muss
Nicht-funktionale Anforderungen			
NFA-01	Datenschutz und Lokal-First (kein externer API-Aufruf)	Institutionell (DSGVO, BITBW)	Muss
NFA-02	Laufzeit < 10 min pro Seite (ohne GPU)	Praxisanforderung BITBW	Soll
NFA-03	Wartbarkeit und Erweiterbarkeit (modulare Architektur)	Software-Engineering Best Practice	Soll
NFA-04	Reproduzierbarkeit (Temperature = 0.05, JSON-Schema)	Forschungsmethodik	Soll

Tab. 4: Anforderungsübersicht: funktionale und nicht-funktionale Anforderungen mit Ursprung und Priorität.

6 Konzeption des Assistenzsystems

Dieses Kapitel beschreibt den konzeptionellen Entwurf des Assistenzsystems *Accessibility Context Engine* (ACE). Die Entwurfsentscheidungen leiten sich aus den Anforderungen (Kapitel 5), den theoretischen Grundlagen (Kapitel 2) sowie den in Kapitel 3 identifizierten Forschungslücken ab.

6.1 Zielarchitektur und Datenpipeline

Die Systemarchitektur folgt einer sequenziellen Datenpipeline, die vier Analysequellen zusammenführt, deren Ergebnisse normalisiert und an ein lokales Sprachmodell übergibt. Eine schematische Darstellung der Verarbeitungsschritte findet sich in Anhang 5 (Abbildung 7).

Die Pipeline gliedert sich in fünf Phasen: Datenerhebung durch *axe-core*, *Playwright* und *grep*, LLM-gestützte Quellcodeanalyse, Normalisierung der Rohdaten in das Unified Finding Schema (UFS), Prompt-Konstruktion mit Token-Budget-Steuerung sowie Inferenz durch das lokale Sprachmodell mit anschließender Ausgabe.

6.1.1 Analysequellen

Die in Abschnitt 3.5 identifizierten Forschungslücken bilden den Ausgangspunkt der Konzeption und werden im Folgenden in konkrete Analysequellen und Verarbeitungsschritte überführt.

axe-core wird über einen gesteuerten Chromium-Browser ausgeführt und analysiert den vollständig gerenderten DOM der Zielanwendung auf Verstöße gegen WCAG-Erfolgskriterien. Da Rich-Client-Webanwendungen ihre Inhalte erst zur Laufzeit per JavaScript erzeugen (vgl. Abschnitt 2.3), ist die Ausführung in einem echten Browser unerlässlich.¹⁷⁸ *axe-core* deckt primär *syntaktische* Verstöße ab und ist daher als alleinige Quelle nicht ausreichend.¹⁷⁹

Playwright führt sieben generische Interaktionschecks aus, die unabhängig von der konkreten Anwendungslogik lauffähig sind. Geprüft werden unter anderem die Erreichbarkeit interaktiver Elemente per Tastatur, die Sichtbarkeit des Fokusindikators sowie das Vorhandensein eines Skip-Links. Diese Checks adressieren gezielt das in Abschnitt 2.3.2 beschriebene Problem *zustandsabhängiger Barrieren*: Ein Modalfenster ohne Fokus-Trap oder ein per Tastatur nicht navigierbares Dropdown-Menü ist erst durch tatsächliche Browser-Interaktion sichtbar.¹⁸⁰

¹⁷⁸Vgl. Fathallah, Hernández, Staab 2025

¹⁷⁹Vgl. Kodithuwakku, Wickramaarachchi 2025

¹⁸⁰Vgl. Huang, C. et al. 2024

grep übernimmt zwei komplementäre Aufgaben: Erstens reichert es bestehende axe-core- und Playwright-Findings um ihren Entstehungskontext im Quellcode an (*Kontextanreicherung*); zweitens erkennt es eigenständig bekannte Anti-Patterns im React-Quellcode anhand regulärer Ausdrücke (*Pattern-Erkennung*). Beide Aufgaben adressieren die in Kapitel 3.5 beschriebene Lücke zwischen DOM-Analyse und Quellcode, die Fathallah et al. als offenes Problem von AccessGuru benennen.¹⁸¹

Die Entscheidung für reguläre Ausdrücke anstelle eines vollständigen Abstract Syntax Tree (AST)-Parsers beruht darauf, dass die implementierten Pattern-Checks syntaktischer Natur sind und nach textuell erkennbaren Mustern im JSX-Markup suchen, nicht nach semantischen Abhängigkeiten oder Kontrollflüssen im Code. Der regex-basierte Ansatz gewährleistet zudem Sprachunabhängigkeit über `.tsx`, `.jsx`, `.ts` und `.js`-Dateien ohne sprachspezifische Parser-Abhängigkeiten.¹⁸² Die Kontextanreicherung erfolgt selektiv und durchsucht ausschließlich Dateien, auf die bestehende Findings verweisen; die Pattern-Erkennung arbeitet breiter über die relevante Codebasis (vgl. Abschnitt 7.4).

LLM-Codeanalyse ergänzt die regelbasierten Quellen um eine semantische Detektionsstufe. Das lokale Modell liest Quellcode-Chunks mit Zeilennummern und erzeugt strukturierte Findings im UFS-Schema. Diese Findings werden als zusätzliche Quelle eingebunden und anschließend mit axe-core-, Playwright- und grep-Befunden fusioniert.¹⁸³

Tabelle 5 fasst die Zuordnung der vier Analysequellen zu den adressierten Problemen und den jeweiligen Verstößtypen zusammen.

Quelle	Adressiertes Problem	Verstößtypen	Theoriebezug
axe-core	Begrenzte Regelabdeckung ¹⁸⁴	primär syntaktisch	Abschnitt 2.2.2
Playwright	Fehlende Dynamik bei zustandsabhängigen Barrieren	semantisch, layout	Abschnitt 2.3.2
grep	DOM/Quellcode-Diskrepanz ¹⁸⁵	alle (Anreicherung)	Abschnitte 2.2.2 und 3.5
LLM-Codeanalyse	Semantische Muster jenseits fester Regex-Regeln	semantisch, syntaktisch, layout	Abschnitte 2.4 und 3.5

Tab. 5: Zuordnung der Analysequellen zu adressierten Problemen, Verstößtypen und theoretischer Herleitung.

¹⁸¹Vgl. Fathallah, Hernández, Staab 2025

¹⁸²Vgl. Bassi et al. 2025, S. 12

¹⁸³Vgl. Fathallah, Hernández, Staab 2025

¹⁸⁴Vgl. Kodithuwakku, Wickramaarachchi 2025.

¹⁸⁵Vgl. Fathallah, Hernández, Staab 2025.

6.1.2 Unified Finding Schema

Die Notwendigkeit einer Normalisierungsschicht ergibt sich aus der Heterogenität der Ausgangsformate.¹⁸⁶ Konkret liefert axe-core Objekte mit **impact**-Stufen und DOM-Selektorketten, Playwright gibt boolesche **passed**-Felder zurück, grep produziert Datei-Zeilen-Tupel, und die LLM-Codeanalyse liefert JSON-Objekte mit Evidenz- und Zeilenfeldern. Eine quellenübergreifende Deduplizierung und Priorisierung wäre auf dieser Grundlage nicht möglich. Zudem enthalten rohe axe-core-Ausgaben vollständige DOM-Snapshots und ausführliche Hilfetexte, die für die LLM-Inferenz das Token-Budget unnötig belasten.

Das UFS überführt daher alle Findings in eine einheitliche TypeScript-Schnittstelle. Listing 6.1 zeigt das Interface, wie es im PoC implementiert ist.

Listing 6.1: TypeScript-Interface des Unified Finding Schema

```

1 export type FindingSource = "axe" | "playwright" | "grep" | "llm";
2 export type Severity = "critical" | "serious" | "moderate" | "minor";
3 export type WcagLevel = "A" | "AA" | "AAA";
4 export type ViolationCategory = "syntaktisch" | "semantisch" | "layout";
5
6 export interface UnifiedFinding {
7   id: string;
8   source: FindingSource;
9   ruleId: string;
10  description: string;
11  severity: Severity;
12  wcagCriteria: string[];
13  wcagLevel: WcagLevel;
14  category: ViolationCategory;
15  selector: string | null;
16  componentPath: string | null;
17  codeSnippet: string | null;
18  lineRange: [number, number] | null;
19 }

```

Die Felder lassen sich in drei Funktionsgruppen einteilen: **Identifikation und Klassifikation** (id, source, ruleId, severity, category), **WCAG-Bezug** (wcagCriteria, wcagLevel) und **Quellcode-Kontext** (selector, componentPath, codeSnippet, lineRange). Felder, die eine Quelle nicht liefern kann, bleiben null und werden gegebenenfalls durch die grep-Anreicherung oder die LLM-Codeanalyse ergänzt.

Das Feld **category** bildet die in Abschnitt 2.2.3 eingeführte Taxonomie nach Fathallah et al. direkt im Datenmodell ab.¹⁸⁷ Das Feld **componentPath** bildet das entscheidende Bindeglied zwischen DOM-Analyse und Quellcode: Erst durch die Verknüpfung eines axe-Findings mit dem konkreten Dateipfad und der Zeilennummer wird aus einer generischen Meldung („*Element hat keinen alternativen Text*“) eine entwicklernahe Handlungsanweisung („*In LoginForm.tsx, Zeile 46: Füge alt-Attribut hinzu*“).

¹⁸⁶Vgl. Pool 2023

¹⁸⁷Vgl. Fathallah, Hernández, Staab 2025

6.2 Prompt-Strategie

Dieser Abschnitt beschreibt die Interaktionsstrategie mit dem lokalen Sprachmodell. Im Fokus stehen der zweistufige Prompt-Ansatz, die eingesetzten Prompt-Engineering-Strategien sowie die Steuerung des Token-Budgets.

6.2.1 Zweistufige Prompt-Strategie

Konzeptionell wäre ein stärker iterativer Ansatz denkbar; entsprechende Vorarbeiten wie Access-Guru und ACCESS wurden in Kapitel 3 bereits eingeordnet.

Für den vorliegenden Anwendungsfall wird eine zweistufige Strategie gewählt: In Stufe 1 erzeugt das Modell als LLM-Detektor strukturierte Code-Findings aus Quellcode-Chunks; in Stufe 2 priorisiert ein kombinierter Prompt alle Quellen (axe-core, Playwright, grep, LLM). Der zentrale Grund gegen weitere iterative Schleifen liegt im verfügbaren Latenzbudget: Ein LLM-Aufruf auf einem lokalen CPU-basierten System benötigt je nach Modellgröße bereits mehrere Minuten. Zusätzliche ReAct- oder Corrective-Re-Prompting-Schleifen würden die Laufzeit weiter erhöhen und damit NFA-02 gefährden – eine Einschränkung, die bei cloudbasierten Systemen mit GPU-Beschleunigung in dieser Form nicht besteht.

Der kombinierte Priorisierungsprompt in Stufe 2 ermöglicht es dem Sprachmodell, Zusammenhänge zwischen den einzelnen Quellen unmittelbar zu erkennen: Meldet axe-core ein fehlendes `aria-label`, liefert grep den zugehörigen Quellcodeausschnitt und bestätigt die LLM-Codeanalyse dieselbe Stelle, kann das Modell diese Evidenz in einem konkreten Vorschlag zusammenführen.

6.2.2 Wahl der Prompt-Engineering-Strategien

Role-Play Prompting wird eingesetzt, um dem Modell die Rolle eines Barrierefreiheitsexperten und React-Entwicklers zuzuweisen. Njifenjou et al. und Fathallah et al. belegen, dass diese Strategie die Qualität accessibility-spezifischer Ausgaben verbessert.¹⁸⁸

Few-Shot Prompting wird ergänzend eingesetzt, um dem Modell das erwartete Ausgabeformat anhand konkreter Beispiele zu demonstrieren. Few-Shot-Beispiele verbessern nachweislich die Konsistenz und Formatkonformität der Modellausgaben¹⁸⁹ – besonders relevant bei kleineren Modellen, die ohne Beispiele häufiger vom vorgegebenen Schema abweichen.

¹⁸⁸Vgl. Njifenjou et al. 2024; Vgl. Fathallah, Hernández, Staab 2025

¹⁸⁹Vgl. Brown et al. 2020, S. 5

Chain-of-Thought wird bewusst nicht eingesetzt: CoT kann zwar Halluzinationen bei komplexen Schlussfolgerungen reduzieren¹⁹⁰, verbraucht jedoch zusätzliche Output-Tokens und verlängert die Inferenzzeit auf lokaler Hardware. Die konkrete Umsetzung des Prompts wird in Kapitel 7 beschrieben.

6.2.3 Token-Budget-Steuerung

Jedes Modell verfügt über ein festes *Kontextfenster*, das die maximale Anzahl verarbeitbarer Tokens definiert – dabei müssen sowohl Eingabe (Prompt) als auch Ausgabe (Antwort) in dieses Fenster passen. Da der System-Prompt und die Ausgabe-Reserve einen festen Anteil des Kontextfensters belegen, steht für die Findings ein variables Budget zur Verfügung (vgl. Abbildung 8 im Anhang).

Übersteigt die Gesamtmenge der serialisierten Findings dieses Budget, greift eine Priorisierungsstrategie: Alle Findings werden quellenübergreifend nach Schweregrad sortiert, wobei Einträge niedriger Priorität bei Bedarf entfernt werden. Auf diese Weise wird sichergestellt, dass gesetzlich relevante Verstöße (*critical/serious*) stets im Prompt enthalten sind.

Ergänzend wird *grep* selektiv eingesetzt: Statt die gesamte Codebasis zu durchsuchen, werden vorrangig jene Dateien analysiert, die durch bestehende Befunde als relevant erscheinen. Die LLM-Codeanalyse arbeitet mit chunkbasiertem Retrieval – Quelldateien werden mit Zeilennummern versehen und in Textsegmente fester Zeichenlänge (*Chunks*) unterteilt (vgl. Abschnitt 7.5). Konzeptionell folgt dieses selektive Retrieval dem Grundprinzip von RAG-Systemen:¹⁹¹ relevanter Kontext wird zur Laufzeit gezielt abgerufen und in den Prompt eingebettet.

6.3 Ausgabeformat: entwicklernahe To-Do-Liste

Das Ausgabeformat des Systems ist als priorisierte To-Do-Liste für Entwicklerinnen und Entwickler konzipiert. Die Wahl dieses Formats ergibt sich aus FA-05 sowie aus der Beobachtung, dass bestehende Violation Reports das Problem auf Ebene des gerenderten DOM beschreiben, ohne Bezug zur Quellcodestruktur herzustellen (vgl. Abschnitt 2.2.2).¹⁹²

Die Liste ist in zwei Prioritätsstufen gegliedert: *Must-have*-Einträge bezeichnen Verstöße gegen Erfolgskriterien der Konformitätsstufen A und AA (gesetzliche Konformitätspflicht nach BITV und BfSG); *Nice-to-have*-Einträge umfassen Empfehlungen über den gesetzlichen Mindeststandard hinaus. Jeder Eintrag enthält – sofern durch *grep*-Anreicherung oder LLM-Codeanalyse verfügbar – den Dateipfad, die Zeilennummer sowie einen konkreten Code-Fix-Vorschlag.

¹⁹⁰Vgl. Wei, Wang, X. et al. 2022, S. 2

¹⁹¹Vgl. Lewis et al. 2020, S. 1–2

¹⁹²Vgl. Fathallah, Hernández, Staab 2025

Zu berücksichtigen ist, dass das Ausgabeformat zwar durch einen System-Prompt vorgegeben wird, dessen Einhaltung jedoch nicht garantiert werden kann – insbesondere bei kleineren Modellen der 7B-Klasse.¹⁹³ Die Evaluation in Kapitel 8 untersucht, inwieweit die getesteten Modelle das vorgegebene Schema einhalten; Limitationen werden in Abschnitt 8.6 diskutiert.

Zusammenfassend ergibt sich aus den Anforderungen, den theoretischen Grundlagen und den identifizierten Forschungslücken ein System, das vier komplementäre Analysequellen über ein einheitliches Datenmodell (UFS) zusammenführt und in einer zweistufigen LLM-Strategie verarbeitet: Stufe 1 für LLM-basierte Code-Findings und Stufe 2 für die quellenübergreifende Priorisierung. Kapitel 7 beschreibt die prototypische Umsetzung dieser Konzeption.

¹⁹³Vgl. Brown et al. 2020, S. 5

7 Implementierung des Proof of Concept

Dieses Kapitel beschreibt die prototypische Umsetzung des in Kapitel 6 konzipierten Assistenzsystems. Zunächst werden der technische Rahmen und die Projektstruktur dargestellt. Anschließend wird die Implementierung der einzelnen Pipeline-Module erläutert. Abschließend wird die Umsetzung der Prompt-Strategie einschließlich des System-Prompts beschrieben.

7.1 Technischer Rahmen und Projektstruktur

Der PoC ist als Node.js-Anwendung auf Basis von TypeScript implementiert.¹⁹⁴ Die Wahl von TypeScript ist durch die technologische Nähe zum Untersuchungsgegenstand begründet: Der BWEC ist in ein TypeScript-basiertes Monorepo eingebettet, sodass auch der PoC im selben technologischen Kontext entwickelt wird. Dies erleichtert sowohl die Integration als auch eine mögliche Übertragung auf weitere Anwendungen innerhalb vergleichbarer Architekturen. Darüber hinaus ermöglicht TypeScript eine typsichere Definition des UFS (vgl. Listing 6.1). Als Laufzeitumgebung wird Node.js ab Version 24 eingesetzt; die Ausführung erfolgt über **tsx**, einen TypeScript-Runner, der Kompilierung und Ausführung in einem Schritt verbindet. Auf ein zusätzliches Framework wurde bewusst verzichtet, um den PoC minimal zu halten.

Als Kernabhängigkeiten dienen **playwright@1.50** für die Browser-Automatisierung,¹⁹⁵ **@axe-core/playwright@4.10** für die axe-core-Integration sowie **dotenv@17.3** für die Konfigurationsverwaltung und **tsx@6.5.7** als TypeScript-Runner.

Die Projektstruktur folgt einer flachen Modulorganisation. Jedes Modul in **src/modules/** übernimmt sowohl die Datenerhebung als auch die Normalisierung in das UFS:

Listing 7.1: Projektstruktur des PoC

```
1 ace/  
2   src/  
3     index.ts  
4     types.ts  
5     config.ts  
6     prompt.ts  
7     ollama.ts  
8     output.ts  
9     modules/  
10    axe.ts  
11    playwright.ts  
12    code.ts  
13    llmDetector.ts  
14  results/
```

¹⁹⁴Vgl. *JavaScript With Syntax For Types*. 2026

¹⁹⁵Vgl. Microsoft 2026

Die Pipeline wird über `src/index.ts` orchestriert und unterstützt fünf Command Line Interface (CLI)-Flags: `--url` (Ziel-URL), `--src-dir` (Quellcode-Pfad), `--skip-llm` (Prompt speichern ohne finale Priorisierung), `--axe-only` (nur axe-core ausführen) und `--llm-detect` (zusätzliche LLM-Codeanalyse aktivieren). Die Konfiguration – einschließlich Ollama-URL, Modellname und Timeout-Werte – erfolgt zentral in `config.ts` und kann über Umgebungsvariablen überschrieben werden.

7.2 axe-core-Modul

Das axe-core-Modul realisiert FA-01 und übernimmt die regelbasierte Accessibility-Erkennung. Es startet über Playwright einen headless Chromium-Browser, navigiert zur Ziel-URL und wartet auf `networkidle`, um sicherzustellen, dass die React-Anwendung vollständig gerendert ist. Anschließend wird über `AxeBuilder` eine axe-core-Analyse ausgeführt, die auf die Tags `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, `wcag22a` und `wcag22aa` beschränkt ist – entsprechend dem gesetzlichen Mindeststandard (vgl. Abschnitt 2.1).

Die Normalisierung überführt jede Violation in ein `UnifiedFinding`. Dabei erfolgen drei zentrale Transformationen:

- **Severity-Mapping:** Das `impact`-Feld von axe-core (`critical`, `serious`, `moderate`, `minor`) wird direkt auf das `severity`-Feld des UFS abgebildet.
- **WCAG-Extraktion:** Aus den axe-Tags (z. B. `wcag143`) werden die Erfolgskriterien (z. B. 1.4.3) und die Konformitätsstufe extrahiert.
- **Kategorie-Heuristik:** Jede Regel wird anhand ihres `ruleId` einer Kategorie der in Abschnitt 2.2.3 eingeführten Taxonomie zugeordnet. Farbkontrast-Regeln werden als `layout` klassifiziert, ARIA-Strukturregeln als `semantisch`, alle übrigen als `syntaktisch`.

Die Felder `componentPath`, `codeSnippet` und `lineRange` bleiben nach der axe-Normalisierung zunächst `null` und werden erst in der Code-Phase (Abschnitt 7.4) angereichert.

Zu berücksichtigen ist, dass axe-core im PoC nur einmal beim initialen Seitenaufruf ausgeführt wird und nicht nach jeder Playwright-Interaktion. Eine Analyse pro Zustandswechsel wäre technisch realisierbar, würde jedoch bei sieben Checks mit je mehreren Zustandsänderungen die Laufzeit um bis zu 168 Sekunden erhöhen und damit NFA-02 verletzen. Diese Einschränkung wird in Abschnitt 8.6 als Limitation diskutiert.

7.3 Playwright-Modul

Das Playwright-Modul implementiert FA-02 (dynamische Zustandserfassung). Es definiert sieben generische Interaktionschecks, die jeweils ein spezifisches WCAG-Erfolgskriterium adressieren (vgl. Anhang 6, Tabelle 15).

Die Checks sind bewusst anwendungsunabhängig konzipiert: Sie setzen keine Kenntnis der konkreten Anwendungslogik voraus und sind auf jeder Web-App ausführbar. Der Keyboard-Trap-Check beispielsweise simuliert 40 Tab-Schritte und prüft, ob der Fokus in den letzten zehn Schritten auf demselben Element verbleibt. Der Check `focus-visible` navigiert per Tab durch die Seite und prüft für jedes fokussierte Element, ob ein sichtbarer Fokusindikator (`outline` oder `box-shadow`) vorhanden ist – eine Prüfung, die Tafreshipour et al. als Schwachstelle bestehender Tools identifizieren.¹⁹⁶

Fehlgeschlagene Checks werden über ein statisches Mapping (`CHECK_DEFINITIONS`) in `UnifiedFinding`-Objekte überführt. Jede Check-Definition enthält die zugehörige `ruleId`, `severity`, WCAG-Kriterien und Kategorie. Checks, die erfolgreich bestanden werden, erzeugen kein Finding.

7.4 Code-Modul

Das Code-Modul übernimmt zwei Aufgaben: die Anreicherung bestehender Findings mit Quellcode-Kontext (FA-03) und die eigenständige Erkennung von Anti-Patterns im React-Quellcode.

7.4.1 Anreicherung bestehender Findings

Für jedes axe-core- oder Playwright-Finding, das einen CSS-Selektor enthält, aber noch keinen `componentPath` besitzt, wird der Quellcode der React-Anwendung durchsucht. Der Algorithmus extrahiert Suchbegriffe aus dem Selektor – IDs, CSS-Klassen, ARIA-Attribute – und sucht diese in den `.tsx/.jsx/.ts/.js`-Dateien des angegebenen Quellverzeichnis. Bei einem Treffer werden `componentPath`, `codeSnippet` (zehn Zeilen Kontext um die Fundstelle) und `lineRange` im Finding ergänzt.

Diese Anreicherung adressiert die in Abschnitt 2.3.2 beschriebene Diskrepanz zwischen gerendertem DOM und Quellcode. Durchsucht werden ausschließlich Dateien, die durch axe-core- oder Playwright-Findings referenziert werden – nicht die gesamte Codebasis. Dieses selektive Vorgehen reduziert den Token-Verbrauch im späteren Prompt (vgl. Abschnitt 6.2).

¹⁹⁶Vgl. Tafreshipour et al. 2024, S. 110

7.4.2 Pattern-Findings

Zusätzlich zur Anreicherung erkennt das Modul sechs Anti-Patterns durch reguläre Ausdrücke (vgl. Anhang 6, Tabelle 16): `div-span-onclick`, `img-missing-alt`, `label-missing-htmlfor`, `role-button-non-semantic`, `tabindex-removes-element` und `autofocus-usage`.

Treffer werden nach der Normalisierung dedupliziert und auf maximal acht Treffer pro Pattern begrenzt, um eine Überrepräsentation im Token-Budget zu vermeiden (vgl. Abschnitt 6.2).

7.5 LLM-Detektor-Modul

Das LLM-Detektor-Modul ergänzt die regelbasierten Quellen um eine modellbasierte Quellcodeanalyse. Es wird nur ausgeführt, wenn der CLI-Parameter `--llm-detect` gesetzt ist und ein Quellcodepfad via `--src-dir` vorliegt. Ziel ist nicht die Ersetzung bestehender Tools, sondern die Ergänzung um eine zusätzliche Detektionsquelle mit semantischem Fokus.

Die Verarbeitung erfolgt in fünf Schritten:

1. **Dateiselektion:** Das Modul sammelt rekursiv Dateien mit den Endungen `.tsx`, `.ts`, `.jsx`, `.js` und `.css`. Die Anzahl der analysierten Dateien ist über `LLM_DETECT_MAX_FILES` begrenzt (Default: 60; in den Benchmarkläufen auf 25 reduziert).
2. **Chunking:** Die Dateien werden mit Zeilennummern versehen und in Text-Chunks aufgeteilt. Die Chunk-Größe ist über `LLM_DETECT_CHUNK_CHARS` konfigurierbar (Benchmarkläufe: 4 000 Zeichen).
3. **Detektionsprompt:** Für jeden Chunk wird ein eigener Prompt erzeugt, der ausschließlich JSON im fest definierten Schema verlangt.¹⁹⁷
4. **Parsing und Validierung:** Antworten werden als JSON extrahiert und validiert; ein Fallback überführt Formatabweichungen in das Zielformat.
5. **Normalisierung und Deduplizierung:** Valide Einträge werden als `source="llm"` in das UFS überführt und über den Schlüssel `ruleId|filePath|startLine` dedupliziert.

Dieses Vorgehen reduziert das Risiko von Halluzinationen, weil Findings ohne belastbare Evidenz oder ohne verwertbaren Datei-/Zeilenbezug verworfen werden. Gleichzeitig bleibt die Nachvollziehbarkeit erhalten, da jedes LLM-Finding einem konkreten Quellcodeausschnitt zugeordnet ist.

¹⁹⁷Der vollständige System- und User-Prompt der Detektor-Phase ist in Anhang 8 dokumentiert.

7.5.1 Abgrenzung zu grep und Nutzen der Kombination

Die beiden Detektionsansätze unterscheiden sich grundlegend in ihrer Arbeitsweise:

- **grep** arbeitet regelbasiert und deterministisch. Es erkennt bekannte Muster zuverlässig und reproduzierbar, kann aber nur das finden, was vorher als Regel modelliert wird.
- **LLM-Detektor** arbeitet kontextsensitiv. Er kann auch semantisch ähnliche Problemstellen erkennen, die nicht exakt einem festen Regex-Muster entsprechen, ist jedoch anfälliger für Formatabweichungen und Halluzinationen.

Der PoC kombiniert daher beide Ansätze in komplementärer Weise: grep als stabile Baseline und den LLM-Detektor als zusätzliche Quelle mit potenziellem Recall-Gewinn.

7.5.2 Beispielhafter Ablauf eines LLM-Findings

Ein typischer Detektionsfall läuft wie folgt ab: In einer Komponente enthält ein `<div>` einen `onClick`-Handler ohne tastaturäquivalentes Ereignis. Das Modell meldet dazu ein Finding mit Datei- und Zeilenbezug sowie Evidenztext, das normalisiert, dedupliziert und als `source="llm"` in das UFS überführt wird, bevor es in der Priorisierungsphase verarbeitet wird.

Listing 7.2: Beispiel eines validierten LLM-Detektor-Findings (vereinfacht)

```
1 {  
2   "title": "Klickbare div-Komponente ohne Tastaturbedienung",  
3   "ruleId": "keyboard-click-only",  
4   "wcagCriteria": ["2.1.1"],  
5   "wcagLevel": "A",  
6   "severity": "serious",  
7   "category": "semantisch",  
8   "filePath": "components/ContactForm.tsx",  
9   "startLine": 45,  
10  "endLine": 49,  
11  "evidence": "<div ... onClick={() => setTopic(t)}>",  
12  "fixSuggestion": "button verwenden oder onKeyDown fuer Enter/Space ergaenzen"  
13 }
```

7.6 Prompt-Builder und System-Prompt

Der Prompt-Builder setzt die in Abschnitt 6.2 beschriebene Prompt-Strategie technisch um. Er besteht aus drei Komponenten: dem System-Prompt, der Token-Budget-Steuerung und der Serialisierung der Findings aus vier Quellen (axe-core, Playwright, grep, LLM-Codeanalyse).

7.6.1 System-Prompt

Der System-Prompt implementiert die in Kapitel 6 gewählten Prompt-Engineering-Strategien. Das **Role-Play Prompting** erfolgt durch die Anweisung „*Du bist ein erfahrener Barrierefreiheitsexperte für React/TypeScript-Webanwendungen*“, die das Modell in eine domänenspezifische Expertenrolle versetzt (vgl. Abschnitt 2.4.2). Das **Few-Shot Prompting** wird durch zwei konkrete Beispiele realisiert: ein **critical**-Finding (Farbkontrast) mit vollständigem Code-Fix als Must-have-Eintrag und ein **moderate**-Finding (**tabIndex**) als Nice-to-have-Eintrag. Beide Beispiele verdeutlichen dem Modell das erwartete Ausgabeformat einschließlich der WCAG-Angabe, des Dateipfads und eines konkreten Code-Fix-Vorschlags.

Ergänzend enthält der System-Prompt sechs Priorisierungsregeln, mit denen das Must-have/Nice-to-have-Schema operationalisiert wird:

1. Must-have: Severity **critical**/**serious** und WCAG Level A/AA
2. Nice-to-have: Severity **moderate**/**minor** oder Level AAA
3. Findings zum selben Element zusammenfassen
4. Codekontext für konkrete React/TypeScript-Fixes nutzen
5. Ausschließlich auf Deutsch antworten
6. Keine Findings erfinden, die nicht in den Daten vorhanden sind

Regel 6 adressiert gezielt das in Abschnitt 2.4 beschriebene Halluzinationsproblem von LLMs. Der vollständige System-Prompt einschließlich der Few-Shot-Beispiele ist in Anhang 7 dokumentiert.

7.6.2 Token-Budget-Steuerung

Wie in Abschnitt 6.2 beschrieben, ist das Kontextfenster lokaler Modelle eine begrenzte Ressource. Der Prompt-Builder reserviert feste Anteile für den System-Prompt und die Modellausgabe; der verbleibende Platz steht für die serialisierten Findings zur Verfügung.¹⁹⁸

Übersteigt die Gesamtmenge der Findings dieses Budget, werden alle Findings quellenübergreifend nach Schweregrad sortiert; Einträge niedriger Priorität werden bei Bedarf entfernt. Findings mit Severity **critical** und **serious** haben dabei Vorrang. Der User-Prompt enthält einen expliziten Hinweis, wenn Findings aus Budgetgründen weggelassen werden.

¹⁹⁸Beim eingesetzten Qwen2.5-Coder beträgt das Kontextfenster 32.768 Tokens. Nach Abzug von System-Prompt und Output-Reserve verbleiben ca. 24.000 Tokens für die Findings. Vgl. Hui et al. 2024 sowie Anhang 9

7.6.3 Serialisierung

Jedes Finding wird in ein kompaktes Textformat serialisiert, das die wesentlichen Informationen auf wenige Zeilen verdichtet. Beschreibungen werden auf 300 Zeichen und Code-Snippets auf 20 Zeilen begrenzt, um den Token-Verbrauch je Finding vorhersagbar zu halten. Das vollständige Serialisierungsformat ist im System-Prompt-Beispiel in Anhang 7 illustriert.

7.7 Ollama-Client und Output-Formatter

Der Ollama-Client kapselt den HTTP-Aufruf an die lokale Ollama-Instanz (`localhost:11434`). Die Anfrage nutzt den `/api/generate`-Endpunkt mit `temperature: 0.05` (NFA-04).¹⁹⁹ Die niedrige `temperature`-Einstellung reduziert den Nicht-Determinismus der Ausgaben (vgl. Abschnitt 2.4), ohne ihn vollständig zu eliminieren. Der Client verwendet ein konfigurierbares Timeout (Default: drei Stunden), da die Inferenz auf CPU-basierten Systemen je nach Modellgröße mehrere Minuten in Anspruch nehmen kann. Die Antwort von Ollama enthält neben dem generierten Text auch die tatsächliche Anzahl der Prompt- und Output-Tokens (`prompt_eval_count`, `eval_count`), die für die Evaluation in Kapitel 8 erfasst werden.

Der Output-Formatter erzeugt pro Analysedurchlauf zwei Ausgabedateien: einen Markdown-Report für die menschliche Lesbarkeit sowie einen JSON-Report für die maschinelle Weiterverarbeitung (FA-05). Der Markdown-Report enthält einen Metadaten-Header (URL, Modell, Zeitstempel, Token-Statistiken), die Roh-Ausgabe des Sprachmodells sowie eine Metriken-Tabelle mit den Laufzeiten aller Pipeline-Phasen. Der JSON-Report speichert zusätzlich das Parsing-Ergebnis der LLM-Antwort: Der Formatter versucht, die Markdown-Ausgabe in Must-have- und Nice-to-have-Einträge zu zerlegen. Schlägt dieses Parsing fehl, etwa weil das Modell das vorgegebene Format nicht einhält, wird dies im Report dokumentiert (`parsed.success: false`).

7.8 Pipeline-Orchestrierung

Die Pipeline-Orchestrierung in `index.ts` verkettet alle Module in der im Anhang dargestellten Reihenfolge (vgl. Anhang 5, Abbildung 7). Jede Phase wird einzeln zeitgemessen, um die in NFA-02 geforderte Laufzeitanalyse zu ermöglichen (für die Laufzeitanalyse siehe Kapitel 8). Die Metriken werden in einem `PipelineMetrics`-Objekt gesammelt, das neben den Phasenzeiten auch Token-Schätzungen, Anreicherungsquoten und Deduplizierungsstatistiken enthält.

Ein vollständiger Durchlauf umfasst folgende Schritte:

1. `axe-core`: Browser starten, DOM analysieren, Findings normalisieren

¹⁹⁹Der Standardwert `num_predict = 6144` gilt für den operativen Betrieb. In den Benchmarkläufen wurden suiteabhängige Werte eingesetzt: 6144 (test-12), 8192 (test-50) und 12288 (test-100); vgl. Tabelle 6 und Abschnitt 8.6.2.

2. Playwright: sieben Interaktionschecks ausführen, fehlgeschlagene normalisieren
3. Code-Anreicherung: axe-/Playwright-Findings mit Quellcode-Kontext ergänzen
4. Code-Patterns: sechs Anti-Patterns im Quellcode suchen, deduplizieren
5. LLM-Codeanalyse (`--llm-detect`): semantische Quellcodeanalyse ausführen und zusätzliche Findings normalisieren
6. Prompt-Builder: Findings serialisieren, Token-Budget anwenden
7. Ollama: kombinierten Priorisierungsprompt senden, Antwort empfangen
8. Output: Markdown- und JSON-Report generieren und speichern

Der `--skip-llm`-Modus speichert den fertigen Prompt als Textdatei, ohne Ollama aufzurufen. Dieser Modus wird während der Entwicklung genutzt, um den Prompt unabhängig von der Modellverfügbarkeit zu iterieren und für die Evaluation zu dokumentieren.

7.9 Beispielabläufe am Untersuchungsobjekt

Ein vollständiger Durchlauf auf der test-12-Suite erzeugt konsistent Findings aus allen vier Quellen: axe-core meldet vier Violations (u. a. fehlende Alternativtexte und Farbkontrastverstöße), zwei Playwright-Checks schlagen fehl (Fokusindikator-Verlust und fehlende Dialog-Fokusführung) und grep erkennt sechs Anti-Pattern-Treffer im Quellcode (u. a. `<div>`-Elemente mit `onClick`-Handler ohne Tastaturäquivalent). Der LLM-Detektor ergänzt je nach Modellgröße durchschnittlich 11 bis 23 weitere semantische Findings. Eine vollständige Aufstellung der erkannten Violations findet sich in Anhang 17, Screenshots der Suite in Anhang 29.

Ein konkretes Beispiel für den Datenpfad ist der folgende Fall: Der Playwright-Check `focus-after-dialog` schlägt fehl, weil ein modaler Dialog den Tastaturfokus beim Öffnen nicht übernimmt. Der grep-Anreicherungsschritt lokalisiert die zugehörige Komponente im Quellcode und ergänzt den relevanten Code-Ausschnitt als Kontext. Der Prompt-Builder serialisiert dieses angereicherte Finding gemeinsam mit den übrigen Quellen und übergibt es an das lokale Modell. `qwen3:32b` klassifiziert den Befund als Must-have und generiert eine Empfehlung, die die betroffene Datei namentlich referenziert und einen React-konformen `useEffect`-Hook für die Fokussteuerung vorschlägt. Die Rohdaten und Benchmark-Ergebnisse aller 30 Runs sind in Anhang 20 und Anhang 21 dokumentiert.

Auf der test-50-Suite (50 Violations) identifiziert axe-core 14 Violations, zwei Playwright-Checks schlagen fehl und grep liefert 21 Anti-Pattern-Treffer. Die größere Komponentenanzahl erhöht die Token-Last für den Detektor-Prompt spürbar, weshalb das Kontextfenster des 7b-Modells gelegentlich durch Truncation begrenzt wird. Screenshots der Testanwendung, Rohdaten und Modellvergleich finden sich in Anhang 43, Anhang 34 und Anhang 35.

Die test-100-Suite skaliert auf 100 eingebettete Violations über 11 Komponenten. Der erhöhte Umfang zwingt das 7b-Modell regelmäßig zu Truncation und senkt damit den effektiven Recall merklich, während `qwen3:32b` eine stabile Erkennungsleistung beibehält. Details zur Recall-Degradation, Screenshots und Rohdaten finden sich in Anhang 61, Anhang 48 sowie Anhang 49.

Für den BWEK, eine produktiv eingesetzte React-Anwendung des Landes Baden-Württemberg, erzeugt die Pipeline zehn axe-core-Violations, zwei Playwright-Checks und acht grep-Treffer. Da die Codebasis weit größer als die Codebasis der synthetischen Suites ist, wird sie in Chunks aufgeteilt; bei `maxfiles = 100` (66 Chunks) erzeugt der LLM-Detektor im Mittel 15 bis 32 Findings pro Durchlauf. Bei `maxfiles = 500` (330 Chunks) steigt der Gesamtumfang entsprechend. Rohdaten und Modellübersichten beider Konfigurationen befinden sich in Anhang 73 und Anhang 74 (`maxfiles = 100`) sowie in Anhang 75 und Anhang 76 (`maxfiles = 500`).

Der erzeugte Markdown-Report enthält die priorisierten Handlungsempfehlungen strukturiert als Must-have- und Nice-to-have-Listen mit Datei- und WCAG-Bezug; der maschinenlesbare JSON-Report speichert zusätzlich Token-Statistiken, Phasenlaufzeiten und das Parsing-Ergebnis. Die vollständige quantitative Auswertung mit Recall-Matrix, Rohdaten und Modellvergleich ist in Kapitel 8 dokumentiert.

8 Evaluation und Diskussion

Dieses Kapitel evaluiert das ACE-System entlang der in Abschnitt 1.3 formulierten Forschungsfragen. Zunächst werden Versuchsdesign und Metriken beschrieben. Anschließend werden die Ergebnisse zu Detektionsqualität, Priorisierung, Fix-Qualität und Effizienz dargestellt und diskutiert. Abschließend folgen Limitationen, die Beantwortung der Forschungsfragen sowie daraus abgeleitete Einsatzempfehlungen.

8.1 Versuchsdesign

Die Evaluation folgt einer mehrstufigen Versuchslogik: Die *test-12-Suite* dient als methodische **Kalibrierungsphase** – bei vollständig bekannter Ground Truth lässt sich prüfen, ob die Pipeline prinzipiell korrekt funktioniert und alle Verstoßkategorien grundsätzlich erkennt. Die *test-100-Suite* bildet die zentrale **Belastungsprüfung**: Erst hier gerät das Token-Budget systematisch unter Druck, treten Skalierungseffekte auf und werden Modellunterschiede unter realistischerer Last sichtbar. Die *test-50-Suite* fungiert als Zwischenstufe. Ergänzend wird das reale Untersuchungsobjekt BWEC qualitativ evaluiert; mangels Ground Truth erfolgt dort jedoch keine quantitative Recall-Messung. Die folgenden Unterabschnitte beschreiben die Untersuchungsobjekte und die Versuchslogik im Überblick.

8.1.1 Testobjekte und Ground Truth

Die *test-12-Suite* ist eine eigens entwickelte React-SPA mit zwölf gezielt eingebetteten WCAG-Verstößen (Tabelle 23), die alle drei Kategorien der in Kapitel 2.2.3 eingeführten Taxonomie abdecken: vier syntaktische (V1, V2, V3, V10), drei layout-bezogene (V4, V5, V12) und fünf semantische Verstöße (V6, V7, V8, V9, V11). Da die Ground Truth vollständig bekannt und kontrollierbar ist, eignet sich diese Suite als **Kalibrierungsphase**: Es lässt sich exakt prüfen, ob alle Detektionsquellen korrekt angebunden sind, die Normalisierung funktioniert und das LLM die Pipeline-Ausgaben grundsätzlich verarbeiten kann. Aufgrund der geringen Komplexität der Suite sind die erzielten Recall-Werte jedoch nicht als repräsentativ für reale Anwendungen zu interpretieren. Die visuelle Gestalt der Testanwendung ist in Abbildung 9 und 10 dokumentiert.

Die *test-50-Suite* (50 Violations) dient als Zwischenstufe zwischen Kalibrierung und Belastungsprüfung. Die *test-100-Suite* (100 Violations, 11 Komponenten, vgl. Anhang 45) bildet die eigentliche **Belastungsprüfung**: Erst bei 100 Violations gerät das Token-Budget systematisch unter Druck, treten Trunkierungseffekte auf und werden Modellunterschiede unter annähernd realistischer Last sichtbar. Als reales Untersuchungsobjekt dient ergänzend der BWEC der BITBW – eine produktiv betriebene React-SPA mit unbekannter Ground Truth (vgl. Abschnitt 8.7.3).

8.1.2 Modelle, Konfiguration und Runs

Es werden drei lokal ausgeführte Sprachmodelle verglichen: `qwen2.5-coder:7b`, `qwen2.5-coder:14b`²⁰⁰ und `qwen3:32b`²⁰¹. Das in Kapitel 5 ebenfalls genannte `deepseek-r1:70b`²⁰² kann im Rahmen dieser Arbeit nicht evaluiert werden, da das Modell mit 70 Milliarden Parametern die verfügbare Random-Access Memory (RAM)-Kapazität der Testumgebung überschreitet; es bleibt Gegenstand künftiger Evaluation auf leistungsstärkerer Hardware (vgl. Abschnitt 8.7).

Auf jeder der drei synthetischen Suites absolvierte jedes Modell zehn unabhängige Pipeline-Durchläufe, was insgesamt 90 Benchmarkläufe auf synthetischen Testsuiten ergibt. Ergänzend wurden 18 Läufe auf dem realen BWEC durchgeführt (zwei Konfigurationen mit je 9 Runs). Die Temperatur wurde auf 0.05 gesetzt, um den Nicht-Determinismus zu minimieren. Der LLM-Detektor operiert mit separaten Parametern: 2048 Tokens für die Code-Analyse, 4000 Zeichen pro Chunk und maximal 25 Dateien pro Durchlauf.

Modell	Typ	test-12	test-50	test-100
<code>qwen2.5-coder:7b</code>	Code	10 Runs	10 Runs	10 Runs
<code>qwen2.5-coder:14b</code>	Code	10 Runs	10 Runs	10 Runs
<code>qwen3:32b</code>	Generalist ²⁰³	10 Runs	10 Runs	10 Runs
<code>deepseek-r1:70b</code>	Reasoning ²⁰⁴	–	–	–
Gesamt (durchgeführt)		30	30	30

Tab. 6: Verteilung der Benchmark-Runs auf Modelle und Testsuiten (je 10 Runs pro Modell und Suite). `temperature` = 0.05; `num_predict`: test-12 = 6144; test-50 = 8192; test-100 = 12288. `deepseek-r1:70b` konnte aufgrund fehlender RAM-Kapazität sowie prohibitiv hoher CPU- und GPU-Last und resultierender Laufzeit nicht evaluiert werden.

8.1.3 Metriken und Messverfahren

Ausgewertet werden vier Metriken: (1) **Recall** (erkannte Violations relativ zur Ground Truth), (2) **Priorisierungsqualität** (Korrektheit der Must-have/Nice-to-have-Klassifikation), (3) **Fix-Qualität** nach der dreistufigen Skala aus Tabelle 18 sowie (4) **Effizienz** (Laufzeit und Konsistenz über mehrere Runs). Eine Violation gilt als erkannt, wenn sie in der Mehrzahl der zehn Runs (> 5) im finalen LLM-Report erscheint. Diese Schwelle bietet Robustheit gegenüber einzelnen LLM-Ausreißern, ohne bei probabilistischen Modellen zu restriktiv zu sein. Die vollständigen Rohdaten aller Benchmarkläufe finden sich in den Anhangstabellen der jeweiligen Suites.

²⁰⁰Vgl. Hui et al. 2024

²⁰¹Vgl. Yang et al. 2025

²⁰²Vgl. Guo et al. 2025

²⁰⁴Vgl. Yang et al. 2025

²⁰⁴Vgl. Guo et al. 2025

8.2 Ergebnisse: Detektionsqualität (Recall)

Dieser Abschnitt bewertet, welche der 12 Ground-Truth-Violations die Pipeline erkennt – zunächst ohne LLM-Detektor (Baseline), dann mit LLM-Detektor je Modell und abschließend differenziert nach Verstoßkategorie. Die UI-Screenshots der test-12-Suite finden sich in Anhang 9 und 10.

8.2.1 Baseline: Tool-Pipeline ohne LLM-Detektor

Ohne LLM-Detektor erkennt die Tool-Pipeline (axe-core + Playwright + grep) in jedem Durchlauf stabil **8 von 12 Violations** (Recall: 66,7%). Nicht erkannt werden V2 (fehlendes aria-label), V8 (modaler Fokus-Trap), V11 (DOM-Reihenfolge) und V12 (Mindestgröße) – Fälle, die überwiegend semantische oder kontextabhängige Interpretation erfordern. Ein generischer Fokus-Trap-Test für V8 ließ sich ohne anwendungsspezifische Selektoren nicht robust implementieren (vgl. Abschnitt 9.3).

8.2.2 Recall nach LLM-Detektor-Augmentierung

Tabelle 7 zeigt den Recall nach Hinzunahme des LLM-Detektors. Mit `qwen2.5-coder:14b` und `qwen3:32b` werden alle vier zuvor fehlenden Violations konsistent erkannt. `qwen2.5-coder:7b` gewinnt V8 und V12 hinzu, verfehlt jedoch weiterhin V2 und V11; zusätzlich wird V3 im finalen Report nur instabil ausgegeben und unterschreitet dadurch die Konsistenzschwelle.

Bedingung	Erkannte V.	Recall	Neu ggü. Baseline	Nicht erkannt
Baseline (Tools only)	8 / 12	66,7 %	–	V2, V8, V11, V12
+ <code>qwen2.5-coder:7b</code>	9 / 12	75,0 %	+V8, +V12; V3*	V2, V3*, V11
+ <code>qwen2.5-coder:14b</code>	12 / 12	100 %	+V2, +V8, +V11, +V12	–
+ <code>qwen3:32b</code>	12 / 12	100 %	+V2, +V8, +V11, +V12	–

Tab. 7: Recall der ACE-Pipeline auf der test-12-Suite (Kalibrierungsphase; 12 kontrollierte Violations mit bekannter Ground Truth). *V3 wird von `qwen2.5-coder:7b` nur instabil im finalen Report übernommen und verfehlt dadurch die Konsistenzschwelle.

Die detaillierte Aufschlüsselung für alle 12 Violations mit der jeweiligen Erkennungsquelle findet sich in Tabelle 24, die vollständige Recall-Matrix in Tabelle 25.

8.2.3 Erkennungsleistung nach Verstoßkategorie

Für TF1 ist nicht nur der Gesamt-Recall relevant, sondern die Frage, welche *Typen* von Violations erkannt werden. Tabelle 8 schlüsselt den Recall nach den drei in Kapitel 2.2.3 eingeführten Kategorien auf.

Kategorie	Violations	Baseline	+ 7b	+ 14b	+ 32b
Syntaktisch	V1, V2, V3, V10	3 / 4 (75 %)	2 / 4 (50 %)	4 / 4 (100 %)	4 / 4 (100 %)
Layout	V4, V5, V12	2 / 3 (67 %)	3 / 3 (100 %)	3 / 3 (100 %)	3 / 3 (100 %)
Semantisch	V6, V7, V8, V9, V11	3 / 5 (60 %)	4 / 5 (80 %)	5 / 5 (100 %)	5 / 5 (100 %)
Gesamt	12	8 / 12 (67 %)	9 / 12 (75 %)	12 / 12 (100 %)	12 / 12 (100 %)

Tab. 8: Erkennungsleistung differenziert nach Verstoßkategorie (test-12-Suite). Werte basieren auf der Mehrzahl der zehn Runs je Modell.

Aus dieser Aufschlüsselung ergeben sich drei zentrale Beobachtungen. Erstens zeigt die Baseline die **größte Lücke bei semantischen Violations** (60 %): Diese Kategorie umfasst konzeptionell die anspruchsvollsten Prüffälle – Zustandsabhängigkeit, fehlende Keyboard-Interaktion und DOM-Konsistenz – wird von regelbasierten Tools am unzuverlässigsten abgedeckt. Zweitens trägt der LLM-Detektor seinen **größten Recall-Gewinn im semantischen Bereich** bei: Von 60 % auf 100 % mit 14b und 32b, also eine Steigerung um 40 Prozentpunkte. Drittens zeigt 7b, dass eine zusätzliche LLM-Stufe nicht automatisch jede Kategorie stabilisiert: Obwohl V8 und V12 hinzugewonnen werden, wird V3 im finalen Report nur instabil übernommen, sodass der syntaktische Recall vorübergehend auf 50 % sinkt. Erst ab 14b sind alle drei Kategorien vollständig abgedeckt.

8.2.4 Precision und F1-Score

Neben dem Recall ist Precision – der Anteil der Report-Items, der tatsächlichen WCAG-Violations entspricht – entscheidend für den praktischen Nutzen: Hohe Precision bedeutet, dass Entwicklerinnen und Entwickler den Findings vertrauen können, ohne jeden Eintrag manuell verifizieren zu müssen. Gemessen wird auf Item-Ebene: Ein Report-Item gilt als True Positive, wenn es mindestens einem der bekannten Ground-Truth-Verstöße zuzuordnen ist; andernfalls als False Positive (Halluzination oder Fehlinterpretation). Die Bewertung erfolgt über alle zehn Runs je Modell und Suite.²⁰⁵

Die Precision-Werte liegen über alle Modelle und Suites zwischen 96,8 % und 99,1 % und sind damit konsistent hoch – im Einklang mit der Stichproben-Plausibilisierung in Abschnitt 8.6.3, die auf test-100 nur 1 von 176 geprüften Einträgen als Halluzination einstufte. Der PoC erzeugt damit kaum False-Positive-Einträge, unabhängig von Modellgröße oder Suitekomplexität. Die wesentlichen Unterschiede zwischen den Modellen zeigen sich im Recall und folglich auch im F1-Score: **qwen3:32b** erreicht auf test-50 einen F1 von 99,1 %, während **qwen2.5-coder:7b** auf test-100 mit einem F1 von 57,0 % deutlich zurückfällt. Dieser Unterschied ist ausschließlich durch die Recall-Differenz bedingt, nicht durch mangelnde Genauigkeit.

²⁰⁵test-12 wird in der Precision-Analyse nicht gesondert ausgewiesen: Mit nur 12 Ground-Truth-Violations ist die Stichprobe zu klein, um stabile Precision-Schätzungen zu liefern – ein einzelnes FP-Item würde bereits eine Abweichung von über 8 Prozentpunkten erzeugen.

Suite	Modell	Precision (%)	Recall (%)	F1 (%)
test-50	qwen3:32b	98,2	100,0	99,1
	qwen2.5-coder:14b	98,4	96,0	97,2
	qwen2.5-coder:7b	97,5	72,0	82,8
test-100	qwen3:32b	96,8	63,0	76,3
	qwen2.5-coder:14b	98,9	53,0	69,0
	qwen2.5-coder:7b	99,1	40,0	57,0

Tab. 9: Precision, Recall und F1 auf test-50 und test-100 (Mittelwerte über 10 Runs je Modell). Precision gemessen auf Item-Ebene: Anteil der Report-Einträge mit nachweisbarem Bezug zu einer Ground-Truth-Violation.

Ein direkter Benchmark gegen **GenA11y**²⁰⁶ (Precision 94,5 %, Recall 87,6 %, F1 91,0 %) ist aufgrund abweichender Datensätze, Metriken und Evaluationsumgebungen nur eingeschränkt vergleichbar und daher als orientierend zu verstehen: ACE übertrifft GenA11y bei der Precision, liegt aber bei dem Recall – insbesondere auf test-100 – darunter. Vollständige Per-Run-Auflistungen der Precision je Modell und Run befinden sich in Anhang 39 (test-50) sowie Anhang 54 (test-100).

8.3 Ergebnisse: Priorisierungsqualität

Neben dem Recall ist die Korrektheit der Must-have/Nice-to-have-Klassifikation für den praktischen Nutzen des Systems entscheidend: Eine falsch priorisierte Empfehlung kann dazu führen, dass gesetzlich verpflichtende Anforderungen als optional behandelt werden. Dieser Abschnitt vergleicht das Priorisierungsverhalten der drei Modelle.

8.3.1 Modellvergleich Must-have und Nice-to-have

Das Prompt-Design schreibt dem Modell vor, Findings nach Must-have und Nice-to-have zu kategorisieren. Da die Priorisierungsphase sämtliche Quellen (axe, Playwright, grep und LLM-Detektor) zusammenführt, ist die Gesamtzahl der Report-Items größer als die Zahl der 12 Ground-Truth-Violations. Tabelle 10 zeigt die mittleren Zählungen über 10 Runs.

Die Spalte *Bewertung* fasst das Verhalten qualitativ zusammen. Zu beachten ist, dass Findings am Listenende – typischerweise Nice-to-have – bei 7b und 14b durch das Output-Token-Limit systematisch häufiger abgeschnitten werden; die Nice-to-have-Werte sind daher als untere Schranke zu interpretieren.

qwen2.5-coder:7b zeigt ein **fehlerhaftes Priorisierungsverhalten**: Pflichtanforderungen wie Fokusindikator oder Skip-Link werden nicht stabil im Must-have-Bereich gehalten. Für einen BITV-konformen Einsatz ist dieses Modell daher *nicht geeignet*.

²⁰⁶Vgl. He, Huq, Malek 2025, FSE101:20

Modell	Must-have Ø	Nice-to-have Ø	Gesamt-Items Ø	Bewertung
qwen2.5-coder:7b	9,1	23,1	32,2	Fehlerhaft
qwen2.5-coder:14b	27,0	8,9	35,9	Undifferenziert
qwen3:32b	14,6	8,5	23,1	Korrekt

Tab. 10: Priorisierungsverhalten der drei Modelle (Mittelwerte über 10 Runs, test-12-Suite). Nice-to-have-Werte bei 7b und 14b sind durch Token-Limit-Effekte als untere Schranke zu interpretieren.

qwen2.5-coder:14b trennt die beiden Kategorien zwar formal, **überbetont aber den Must-have-Bereich** deutlich. Das Modell ist damit *undifferenziert*, aber nicht so fehlerhaft wie 7b.

qwen3:32b zeigt mit durchschnittlich 14,6 Must-have- und 8,5 Nice-to-have-Einstufungen insgesamt die **plausibelste Trennung** und stellt damit im Kalibrierungsfall das priorisierungsseitig stärkste Modell dar.

8.4 Ergebnisse: Fix-Qualität (qualitativ)

Ein hoher Recall allein ist für den Entwicklungsalltag nicht ausreichend: Entscheidend ist, ob die generierten Handlungsempfehlungen konkret und direkt umsetzbar sind. Dieser Abschnitt bewertet die Fix-Qualität anhand der dreistufigen Skala (Stufe 0–2) und illustriert die Unterschiede zwischen den Modellen an konkreten Fallbeispielen.

8.4.1 Bewertungsmaßstab

Die Bewertungsskala (Tabelle 18) unterscheidet drei Stufen: Stufe 2 (konkret: Dateiname, Zeilennummer, umsetzbarer Code-Vorschlag), Stufe 1 (teilweise: richtiger Fix-Typ ohne konkreten Kontext) und Stufe 0 (generisch: nur Problembeschreibung). Die vollständige Bewertung aller zwölf Violations findet sich in Tabelle 34. Im Ergebnis erreicht **qwen3:32b** für 11 von 12 Violations Stufe 2; **qwen2.5-coder:14b** erreicht überwiegend Stufe 1; **qwen2.5-coder:7b** schwankt zwischen Stufe 0 und 1.

8.4.2 Fallbeispiele nach Modell

Fallbeispiel 1 – Konkrete Empfehlung (Stufe 2, qwen3:32b). Für V8 (modaler Dialog ohne Fokus-Trap) generiert **qwen3:32b** eine Empfehlung, die **ContactForm.tsx** namentlich referenziert, einen React-konformen **useEffect**-Hook für **modalOpen** vorschlägt und die Fokusführung inklusive Tab- und Shift+Tab-Logik mit sauberem Event-Listener-Cleanup beschreibt. Die Empfehlung ist damit direkt in den Quellcode übertragbar und erreicht Stufe 2.

Fallbeispiel 2 – Teilweise Empfehlung (Stufe 1, qwen2.5-coder:14b). `qwen2.5-coder:14b` empfiehlt für V4 (Kontrastverhältnis) sinngemäß, Hintergrund- oder Textfarbe anzupassen, um den Kontrast zu verbessern, liefert jedoch keinen ausreichend konkreten Bezug zum tatsächlichen Farbpaar oder die betroffene Regel im Stylesheet. Der Fix-Typ stimmt, der konkrete Umsetzungskontext bleibt jedoch zu vage – Stufe 1.

Fallbeispiel 3 – Fehlklassifikation (Stufe 0, qwen2.5-coder:7b). `qwen2.5-coder:7b` behandelt den Fokusindikator-Verlust (V5, WCAG 2.4.7 AA) nicht stabil als Pflichtanforderung und formuliert den Fix nur generisch. Damit fehlt sowohl das korrekte Priorisierungssignal als auch die unmittelbare Umsetzbarkeit im Quellcode – Stufe 0.

8.5 Ergebnisse: Effizienz und Robustheit

Neben der inhaltlichen Qualität ist für den produktiven Einsatz entscheidend, wie schnell und konsistent die Pipeline arbeitet. Dieser Abschnitt prüft die Konformität mit Anforderung NFA-02 (Laufzeit), analysiert die Varianz der Ergebnisse über mehrere Runs und bewertet die Token-Nutzung als Indikator für systematische Antwortabbrüche.

8.5.1 Laufzeit und NFA-02-Konformität

Anforderung NFA-02 fordert eine Gesamtlaufzeit von unter zehn Minuten:

- `qwen2.5-coder:7b`: durchschnittlich 231s; erfüllt Anforderung NFA-02 deutlich; Durchsatz: 0,048 Findings/s
- `qwen2.5-coder:14b`: durchschnittlich 420s; erfüllt Anforderung NFA-02 deutlich; Durchsatz: 0,054 Findings/s (trotz höherer Laufzeit leicht höher als bei 7b)
- `qwen3:32b`: durchschnittlich 701s (Ausreißer bis 937s); überschreitet die Zehn-Minuten-Grenze und ist daher für zeitkritische Entwicklungszyklen nicht als Standardmodell geeignet, stellt jedoch weiterhin das qualitativ stärkste Referenzmodell dar; Durchsatz: 0,028 Findings/s

Die vollständigen Laufzeit- und Effizienztabellen finden sich in Anhang 23.

8.5.2 Konsistenz und Parsing-Stabilität

Die Standardabweichung der LLM-Detektor-Findings variiert deutlich zwischen den Modellen. Auf test-12 erreicht `qwen2.5-coder:14b` mit $\sigma = 0,42$ die höchste Konsistenz; `qwen3:32b` liegt mit $\sigma = 0,53$ nur geringfügig darüber, während `qwen2.5-coder:7b` mit $\sigma = 3,28$ eine deutlich höhere Varianz zeigt.

Die festen Detektoren (axe, Playwright, grep) bleiben über alle Benchmarkläufe vollständig deterministisch, sodass die Varianz ausschließlich aus der LLM-Stufe stammt. In allen 30 Runs der test-12-Suite wird der LLM-Output erfolgreich geparkt (`parsed.success: true`, 100 %). Das Prompt-Design mit Role-Play- und Few-Shot-Prompting zeigt damit eine hohe Formatstabilität über alle drei Modelle hinweg und unterstützt Anforderung NFA-04.

8.5.3 Token-Nutzung

Die Output-Token-Nutzung weist ein deutliches Trunkierungsprofil auf: `qwen2.5-coder:7b` erreicht in 5 von 10 Runs das Output-Limit von 6 144 Tokens, `qwen2.5-coder:14b` in 4 von 10 Runs; `qwen3:32b` liegt im Mittel bei 3 491 Output-Tokens und wird in lediglich 1 von 10 Runs abgeschnitten. Das Token-Budget ist damit für 7b und 14b messbar unter Druck, während 32b noch ausreichend Reserve besitzt.

Tabelle 28 schlüsselt Input- und Output-Tokens je Modell auf. Die größeren Modelle konsumieren tendenziell weniger Output-Tokens bei höherem oder vergleichbarem Recall, was auf eine höhere Fokussierung der Ausgabe hindeutet.

8.6 Limitationen und Validität

Die vorliegenden Ergebnisse sind vor dem Hintergrund der methodischen Rahmenbedingungen zu interpretieren. Dieser Abschnitt benennt die wesentlichen Einschränkungen hinsichtlich externer Validität, interner Validität sowie weiterer Einzelaspekte des Versuchsaufbaus.

8.6.1 Externe Validität

Die Evaluation stützt sich primär auf drei synthetisch konstruierte Testsuiten (test-12, test-50, test-100). Obwohl alle drei Verstößkategorien vertreten und die Ground Truths vollständig bekannt sind, lässt sich die Übertragbarkeit auf produktive Anwendungen mit unkontrollierter Ground Truth nicht direkt ableiten. Die ergänzende Evaluation am realen BWEC reduziert diese Lücke, ersetzt jedoch keine formale Ground-Truth-basierte Recall-Messung.

8.6.2 Interne Validität und Token-Limit-Effekt

Die Priorisierungszählungen (Must-have/Nice-to-have) sind durch das Token-Limit beeinflusst: Bei 7b endeten 5 von 10 Runs auf test-12 am Output-Limit, bei 14b 4 von 10 und bei 32b nur 1 von 10. Auf größeren Suites verstärkt sich dieser Effekt deutlich. Dadurch werden systematische Abschneidungs-Bias begünstigt, insbesondere zulasten später Listenanteile (typischerweise Nice-to-have). Die Recall-Messung bleibt davon weniger stark betroffen, da zentrale Violations tendenziell früher im Output erscheinen.

Zusätzlich birgt die Operationalisierung „Violation erkannt“ ein Konstruktvaliditätsrisiko. Die Erkennung wird über Report-Präsenz (Mehrheit der Runs) gemessen, nicht über eine formale semantische Äquivalenzklasse. Paraphrasierte Findings können dadurch konservativ oder liberal zugeordnet werden.

8.6.3 Weitere Einschränkungen

Grep-Limitierungen. Die regex-basierten grep-Patterns erkennen bekannte Anti-Patterns zuverlässig auf Textebene, sind aber weder erschöpfend noch struktursensitiv. Insbesondere Click-Patterns ohne Tastaturäquivalent können in Einzelfällen legitime Implementierungen zu aggressiv markieren.

Ursachen der Über-Detektion. Eine Stichproben-Plausibilisierung an zwei Läufen von `qwen3:32b` auf `test-100` zeigt, dass bestätigte Halluzinationen die Ausnahme sind (1 von 176 geprüften Einträgen). Die dominanten Überdetektions-Effekte sind Quell-Duplikate, Mehrfachinstanzen und Grenzfälle der Zuordnung (vgl. Anhang 53). **Über-Detektion ist damit überwiegend ein strukturelles und nicht primär ein inhaltliches Qualitätsproblem.**

Hardware-Abhängigkeit. Alle Laufzeitmessungen wurden auf einem einzelnen CPU-basierten System durchgeführt. Auf GPU-beschleunigter Hardware oder mit weiter optimierten Quantisierungen wären deutlich kürzere Laufzeiten zu erwarten.

8.7 Belastungsprüfung: test-50 und test-100

Die in den vorherigen Abschnitten präsentierten Kalibrierungsergebnisse (`test-12`) zeigen, dass die Pipeline korrekt funktioniert und alle Verstoßkategorien grundsätzlich erkennbar sind. Dieser Abschnitt dokumentiert die eigentliche Belastungsprüfung: die `test-50`-Suite (50 Violations) als Zwischenstufe sowie die `test-100`-Suite (100 Violations) als primäres Skalierungsobjekt, an dem Tokenbudget-Effekte, Modellunterschiede und Erkennungsgrenzen unter realistischerer Last sichtbar werden.

8.7.1 Evaluation an test-50

Die `test-50`-Messreihe wurde mit identischer Grundkonfiguration wie `test-12` durchgeführt: drei Modelle, je zehn Runs, `temperature` = 0,05, CPU-basierte lokale Ollama-Instanz. Das Output-Token-Limit wurde auf `num_predict` = 8192 angehoben. Die vollständigen Rohdaten aller 30 Runs finden sich in Tabelle 41.

Token-Budget unter Druck. Die test-50-Messreihe zeigt als wichtigste Beobachtung: das Token-Budget wird nun systematisch erschöpft. Die mittleren Input-Tokens steigen auf ca. 13 200–14 200. Trotz des angehobenen Output-Limits schöpfen 60–90 % aller Runs das Budget vollständig aus: `qwen2.5-coder:7b` in 6 / 10 Runs, `qwen2.5-coder:14b` in 7 / 10 Runs und `qwen3:32b` in 9 / 10 Runs.

Detektionsleistung und Recall. Die Baseline-Tools liefern auf test-50 stabil 14 axe-core-, 2 Playwright- und 21 grep-Findings. Der LLM-Detektor erzielt durchschnittlich:

- `qwen2.5-coder:7b`: 46,6 Findings ($\sigma \approx 3,20$); Recall: 36 / 50
- `qwen2.5-coder:14b`: 51,3 Findings ($\sigma \approx 2,21$); Recall: 48 / 50
- `qwen3:32b`: 60,7 Findings ($\sigma \approx 1,25$); Recall: 50 / 50

Ein strukturell bedeutsamer Befund ist in der Recall-Matrix (Tabelle 40) direkt ablesbar: Die Baseline (Tool-only) erzielt 43 / 50 Violations, während `qwen2.5-coder:7b` mit der vollständigen Pipeline lediglich 36 / 50 erreicht. Bei 50 Violations wird die LLM-Zusammenfassungsphase für 7b somit zum **Engpass statt zum Mehrwert**, da vorhandene Tool-Findings unter Informationsverlust komprimiert werden, anstatt sie zuverlässig zu ergänzen. Erst `qwen2.5-coder:14b` (48 / 50) und `qwen3:32b` (50 / 50) übertreffen das Baseline-Niveau.

Priorisierungsqualität. `qwen3:32b` erzielt mit im Mittel 47,6 Must-have die plausibelste Klassifikation relativ zur Ground Truth von 50 Violations. `qwen2.5-coder:14b` zeigt ein bimodales Muster mit einzelnen Runs, die fast ausschließlich Must-have ausgeben. `qwen2.5-coder:7b` bleibt instabil. Die vollständigen Priorisierungszählungen finden sich in Tabelle 42 im Anhang.

Effizienz und NFA-02. Die Gesamtlaufzeit skaliert auf test-50 substanziell:

- `qwen2.5-coder:7b`: durchschnittlich 413 s; erfüllt Anforderung NFA-02
- `qwen2.5-coder:14b`: durchschnittlich 826 s; überschreitet die Zehn-Minuten-Grenze
- `qwen3:32b`: durchschnittlich 1 897 s; überschreitet die Zehn-Minuten-Grenze deutlich und ist damit für zeitkritische Szenarien nicht geeignet.

8.7.2 Evaluation an test-100

Die test-100-Suite (100 eingebettete Violations, 11 Komponenten, vgl. Anhang 45) wurde mit allen drei Modellen evaluiert. Pro Modell wurden zehn Runs mit `num_predict = 12 288` durchgeführt, insgesamt somit 30 Runs.

Token-Budget und Truncation. Das Output-Limit von 12 288 Tokens bewährt sich modellabhängig sehr unterschiedlich. `qwen2.5-coder:7b` schöpft das Budget in 80 % der Runs aus (8 / 10). `qwen2.5-coder:14b` erreicht in allen 10 Runs das Token-Limit (100 % Truncation) – kein Run liefert einen vollständigen Bericht. `qwen3:32b` verhält sich gegensätzlich: Nur 1 / 10 Runs ist trunziert; der mittlere Output liegt bei 7 360 Tokens.

Detektionsleistung und Recall. Die Baseline-Tools erkennen stabil 34 / 100 Violations. Der LLM-Detektor erzielt durchschnittlich:

- `qwen2.5-coder:7b`: 104,0 Findings ($\sigma = 5,75$); Recall: 40 / 100
- `qwen2.5-coder:14b`: 94,3 Findings ($\sigma = 3,23$); Recall: 53 / 100
- `qwen3:32b`: 99,1 Findings ($\sigma = 5,63$); Recall: 63 / 100

Der deutliche Recall-Zuwachs von `qwen3:32b` gegenüber der Baseline (+29 Violations) spricht auch bei 100 Violations für einen substanziellen Mehrwert der LLM-Erweiterung. Die hohe Truncation-Rate bei 7b und 14b begrenzt deren Erkennungsleistung hingegen systematisch.

Die 37 nicht erkannten Violations bei `qwen3:32b` verteilen sich auf 16 syntaktische, 13 semantische und 8 layout-bezogene Fälle (vgl. Abbildung 15 im Anhang). Strukturell dominieren drei Ursachenklassen: Interaktions- oder Zustandsfälle ohne robuste Playwright-Instrumentierung (z. B. V78), rein manuell prüfbare semantische Fälle (z. B. Status nur durch Farbe) sowie Befunde, die die > 5/10-Konsistenzschwelle verfehlen.

Einordnung der Pareto-Perspektive. Für die Ursachenanalyse wird ein Pareto-Diagramm²⁰⁷ gezielt auf **test-100 mit `qwen3:32b`** angewendet, da dieses Modell den höchsten Recall bei zugleich nur 1 / 10 trunzierten Runs erreicht. Die verbleibenden Lücken lassen sich damit weniger auf offensichtliche Modellschwächen als vielmehr auf strukturell schwierige Violations zurückführen.

Laufzeit und Einsatzkontext. Die Gesamtlaufzeit steigt auf test-100 deutlich an; alle drei Modelle überschreiten Anforderung NFA-02:

- `qwen2.5-coder:7b`: durchschnittlich 765 s
- `qwen2.5-coder:14b`: durchschnittlich 1 550 s
- `qwen3:32b`: durchschnittlich 2 616 s

²⁰⁷ Ein Pareto-Diagramm kombiniert ein nach Häufigkeit absteigend sortiertes Balkendiagramm mit einer kumulativen Linie und macht so sichtbar, welche Ursachenklassen den größten Anteil am Gesamtproblem ausmachen (Pareto-Prinzip: ca. 80 % der Effekte entstehen durch 20 % der Ursachen).

Dies ist als **Erkenntnis über den geeigneten Einsatzkontext** zu interpretieren: ACE eignet sich bei komplexen Anwendungen strukturell nicht für per-Commit-CI/CD-Läufe, wohl aber für Release-Gates oder Nightly-Workflows.

Erkennungsrate, Empfehlungsqualität und CSV-Auszug. Die vollständige Violation-für-Violation-Auswertung findet sich in den Anhang-Tabellen 52, 53 und 62. In Run 01 erhalten alle 73 erkannten Violations mit `qwen3:32b` die Bewertung Stufe 2. Der konsistente Recall über alle 10 Runs beträgt 63 / 100.

Gesamteinordnung test-100. Für die test-100-Suite ergibt sich ein klarer Trade-off: `qwen3:32b` liefert den höchsten Recall (63 %) und die stärkste Empfehlungsqualität, ist aber mit deutlichem Laufzeitnachteil verbunden; `qwen2.5-coder:14b` liegt bei der Erkennungsleistung darunter (53 %), bleibt jedoch als pragmatischer Kompromiss aus Qualität und Aufwand relevant.

8.7.3 Evaluation am BWEC-Untersuchungsobjekt

Ergänzend wurde das reale Untersuchungsobjekt BWEC der BITBW evaluiert. Da keine bekannte Ground Truth vorliegt, beschränkt sich die Evaluation auf Systemstabilität, Skalierungseffekte und qualitative Plausibilität der generierten Befunde.

Konfiguration und Tool-Baseline. Es wurden zwei Konfigurationen mit je 9 Runs durchgeführt: `bwec` mit `maxfiles=100` (66 Chunks) und `bwec-500` mit `maxfiles=500` (330 Chunks) als Skalierungsexperiment. Die deterministischen Quellen liefern über alle 18 Runs hinweg konstant `axe-core` 10, `Playwright` 2 und `grep` 8 Findings (Summe: 20), was die deterministische Eigenschaft dieser Quellen auch am realen Untersuchungsobjekt bestätigt (vgl. Anhang 72).

Ergebnisse bei `maxfiles = 100 (66 Chunks)`. Mit der für den produktiven Einsatz vorgesehenen Konfiguration verhält sich das System stabil; alle Reports sind parsebar. Der LLM-Detektor liefert:

- `qwen2.5-coder:7b`: durchschnittlich 14,5 Findings, Laufzeit durchschnittlich 328 s; erfüllt Anforderung NFA-02
- `qwen2.5-coder:14b`: durchschnittlich 22,0 Findings, Laufzeit durchschnittlich 824 s; überschreitet Anforderung NFA-02
- `qwen3:32b`: durchschnittlich 32,0 Findings, Laufzeit durchschnittlich 2 128 s; überschreitet Anforderung NFA-02

Skalierungsexperiment: maxfiles = 500 (330 Chunks). Das Experiment adressiert bewusst einen Grenzfall außerhalb des vorgesehenen Einsatzkontexts. Der LLM-Detektor erzeugt deutlich mehr Findings:

- `qwen2.5-coder:7b`: durchschnittlich 257 Findings
- `qwen2.5-coder:14b`: durchschnittlich 271 Findings
- `qwen3:32b`: durchschnittlich 415 Findings

Kein Run von `qwen3:32b` liefert einen parsebaren Report; `qwen2.5-coder:14b` erschöpft in zwei von drei Runs das Output-Budget. Dieses Ergebnis ist daher weniger als Systemfehler denn als **Use-Case-Mismatch** zu interpretieren: ACE ist nicht darauf ausgelegt, eine gewachsene Produktivapplikation vollständig nachzuauditieren, sondern den Entwicklungsprozess bei einer begrenzten Anzahl geänderter Komponenten je Iteration zu begleiten.

Qualitative Plausibilitätsprüfung. Die im BWEC-Lauf (`maxfiles = 100`) generierten Findings mit `qwen3:32b` referenzieren ausschließlich real im Quellcode vorhandene Dateien und Komponenten. Die benannten Violations entsprechen bekannten React-Accessibility-Anti-Patterns und sind inhaltlich plausibel. Eine formale Expertenvalidierung steht aus; die Dateibezüge und Fix-Vorschläge legen jedoch eine hohe praktische Verwertbarkeit nahe.

8.8 Beantwortung der Forschungsfragen

Auf Basis der in den Abschnitten 8.2–8.6 dokumentierten Ergebnisse können die in Abschnitt 1.3 formulierten Forschungsfragen nun beantwortet werden.

8.8.1 Übergeordnete Forschungsfrage

Die übergeordnete Forschungsfrage kann positiv beantwortet werden, mit modell- und lastabhängigen Einschränkungen: Das kontextsensitive Assistenzsystem erkennt einen substanziellen Teil relevanter Barrierefreiheitsverstöße automatisiert und erzeugt entwicklernahe Empfehlungen mit konkreten Datei-/Zeilenbezügen. Insbesondere Token-Limits und Laufzeitrestriktionen begrenzen die Ergebnisse jedoch auf größeren Suites.

8.8.2 TF1: Erkennungsleistung nach Verstoßtyp

Ja – mit modellabhängigen Einschränkungen. Mit `qwen2.5-coder:14b` und `qwen3:32b` erkennt die vollständige Pipeline auf der test-12-Suite alle drei Verstoßkategorien vollständig: syntaktische, layout-bezogene und semantische Violations jeweils zu 100 % (vgl. Tabelle 8). Die größte methodische Herausforderung liegt im semantischen Bereich: Hier ist der Baseline-Recall

mit 60 % am niedrigsten, und der LLM-Detektor leistet den größten Einzelbeitrag (+40 Prozentpunkte). Bei `qwen2.5-coder:7b` bestehen hingegen modellspezifische Grenzen.

Über die drei kontrollierten Testsuiten zeigt sich konsistent ein Mehrwert durch die LLM-Erweiterung: auf test-12 steigt der Recall von 66,7 % auf bis zu 100 %, auf test-50 von 43/50 auf 48/50 bzw. 50/50 und auf test-100 von 34/100 auf 53/100 bzw. 63/100. Ein direkter Benchmark gegen Literaturwerte bleibt methodisch eingeschränkt, da Datensätze, Ground-Truth-Konstruktion, Metriken und Laufzeitumgebungen nicht identisch sind.

8.8.3 TF2: Mehrwert der Kontextkombination und Ablationsanalyse

Ja, der Mehrwert ist messbar und zeigt sich sowohl quantitativ als auch qualitativ. Die Tool-Pipeline (axe + Playwright + grep) erreicht auf test-12 einen Recall von 66,7 % (8/12). Durch den LLM-Detektor steigt der Recall auf bis zu 100 %. Dabei betrifft der Zugewinn genau jene Violations, die durch die regelbasierten Quellen nicht zuverlässig erreichbar sind: V8 (modaler Fokus-Trap), V11 (DOM-/visuelle Reihenfolge) und V12 (Mindestgröße interaktiver Elemente) werden erst durch die LLM-gestützte Analyse konsistent erfasst; V2 (fehlendes `aria-label` auf Icon-Button) ebenfalls ab 14b.

Ablationsanalyse: Beitrag des LLM-Detektors. Um den Recall-Beitrag der einzelnen Pipeline-Stufen zu isolieren, werden zwei empirisch gemessene Bedingungen um eine architekturell begründete Bedingung B ergänzt:

Bed.	Konfiguration	Recall test-12	Quelle
A	Tool-Baseline (axe + PW + grep, kein LLM)	8 / 12 (66,7 %)	gemessen
B	Stufe-2-Priorisierer ohne LLM-Detektor (Stufe 1)	$\leq 8 / 12$	architekturell
C	Vollständige Pipeline (Stufe 1 + 2, <code>qwen3:32b</code>)	12 / 12 (100 %)	gemessen

Tab. 11: Ablationsanalyse auf test-12. Bedingung B ist architekturell begründet: Der Priorisierer (Stufe 2) verarbeitet ausschließlich die ihm übergebenen Findings und kann daher per Konstruktion keine neuen Violations erkennen.

Bedingung B ist konstruktiv ableitbar: Der LLM-Priorisierer in Stufe 2 erhält als Eingabe ausschließlich die von den Tool-Quellen gemeldeten Findings – er kann keine Violations erkennen, die nicht bereits in diesem Input enthalten sind. Die empirischen Daten stützen diese architekturelle Ableitung: In keinem der 30 Runs auf test-12 tritt eine Violation erstmals im Stufe-2-Output auf, die nicht bereits durch die Quellen in Stufe 1 gemeldet wurde. Folglich gilt $\text{Recall}(B) \leq \text{Recall}(A)$. Der empirisch messbare Recall-Zuwachs von 8/12 auf 12/12 ist damit kausal dem LLM-Detektor (Stufe 1) zuzuschreiben, der den React-Quellcode chunkweise semantisch analysiert und dabei V2, V8, V11 und V12 aufdeckt. Diese Violations entziehen sich der regelbasierten Prüfung, weil

sie entweder semantisches Kontextwissen (V2: inhaltlich unzureichendes `aria-label`), Zustand-sabhängigkeit (V8: modale Fokusführung) oder Layout-Semantik (V11: DOM-/visuelle Reihen-folgedivergenz) erfordern. Die Aussage bleibt bewusst vorsichtig: Da keine vollständige Ablati-onsstudie mit isolierten Bedingungen auf allen drei Suites durchgeführt wurde, handelt es sich um eine auf test-12 beschränkte Partialablation.

Zusätzlich verbessert die Kontextkombination die Umsetzbarkeit der Empfehlungen: Tool-Befunde werden mit Quellcodekontext verbunden, wodurch konkrete, entwicklungsnahe Fix-Vorschläge entstehen (Stufe 2 für 11 von 12 Violations mit 32b auf test-12). Gegenüber einer reinen Tool-/DOM-Sicht steigt damit nicht nur die Erkennungsbreite, sondern auch die praktische Anschlussfähigkeit.

8.9 Modellwahl und Einsatzempfehlungen

Aus den vorstehenden Ergebnissen zu Recall, Priorisierungsqualität, Fix-Qualität und Laufzeit lassen sich folgende Einsatzempfehlungen ableiten. Eine suiteübergreifende Einordnung findet sich ergänzend in den Anhang-Tabellen 17 und 64.

Szenario	Empfehlung
Regulärer Entwicklungszyklus / Compliance-Gate	<code>qwen2.5-coder:14b</code> als Standardmodell für kleine bis mittlere Befundmengen; guter Trade-off aus Qualität und Laufzeit
CI/CD-Rauchtest (nicht compliance-relevant)	<code>qwen2.5-coder:7b</code> ; schnellstes Feedback, aber nicht für verbindliche Bewertungen geeignet
Endaudit / Referenzläufe	<code>qwen3:32b</code> ; höchste inhaltliche Abdeckung, aber deutlich höhere Laufzeit

Tab. 12: Modellwahl nach Anwendungsszenario.

`qwen2.5-coder:14b` stellt unter Kalibrierungs- und mittleren Lastbedingungen den besten Trade-off dar. Das Modell erreicht 100 % Recall auf test-12, 96 % auf test-50 und bleibt dort noch in einem vertretbaren Aufwand-Korridor. Auf test-100 wird jedoch deutlich, dass 14b bei großen Befundmengen systematisch an das Output-Limit stößt: 100 % Truncation und 53 % Recall zeigen, dass das Modell für sehr große Reports strukturell überfordert ist.

`qwen3:32b` stellt weiterhin das methodisch stärkste Referenzmodell dar, wenn maximale Stabilität im finalen Bericht und möglichst hohe semantische Abdeckung im Vordergrund stehen. Der Zugewinn gegenüber 14b fällt bei kleinen Suites jedoch geringer aus als der zusätzliche Zeitaufwand. Für regelmäßige Entwicklungszyklen ist 32b daher eher als Referenz- oder Endaudit-Modell zu verstehen.

`qwen2.5-coder:7b` eignet sich vor allem für schnelle Vorprüfungen in CI/CD-nahen Szenarien. Die Laufzeit ist am niedrigsten, zugleich bleibt die Detektionsleistung deutlich hinter 14b und

32b zurück, und die Priorisierung ist für compliance-relevante Einsätze nicht hinreichend verlässlich. Für rechtlich oder fachlich verbindliche Accessibility-Bewertungen sollte 7b daher nicht als Primärmodell eingesetzt werden.

9 Fazit

Dieses abschließende Kapitel fasst die Ergebnisse der Arbeit zusammen (Abschnitt 9.1), reflektiert deren Aussagekraft kritisch (Abschnitt 9.2) und skizziert Ansatzpunkte für künftige Weiterentwicklungen (Abschnitt 9.3).

9.1 Zusammenfassung der Ergebnisse

9.1.1 Beitrag zur übergeordneten Forschungsfrage

Mit ACE wurde ein DSR-Artefakt entwickelt, das regelbasierte Prüfung, Interaktionsanalyse, Quellcodekontext und LLM-gestützte Auswertung in einer Pipeline zusammenführt. Die Hauptevaluation umfasste 90 Benchmarkläufe (3 Modelle $\times$ 10 Runs je Testsuite) über die drei synthetischen Testsuiten test-12, test-50 und test-100. Die Beantwortung beider TFs fällt dabei je nach Evaluationssuite unterschiedlich aus und wird im Folgenden differenziert dargestellt.

Im Rahmen der synthetischen Evaluation kann **TF1** grundsätzlich bestätigt werden. Die kombinierte Pipeline erzielt auf der test-100-Suite mit **qwen3:32b** einen konsistenten Recall von 63 / 100 Violations und liegt damit deutlich über der Tool-only-Baseline von 34 / 100. Die Arbeit zeigt damit keinen vollautomatischen Ersatz manueller Accessibility-Prüfung, sondern vielmehr eine substanzielle Erweiterung bestehender Toolketten.

TF2 wird auf der Kalibrierungssuite vollständig bestätigt. Auf den Belastungssuiten wird der Effekt jedoch durch Token-Budget-Restriktionen begrenzt, sowohl hinsichtlich der Erkennungsleistung als auch der Empfehlungsqualität. Die Ablationsanalyse in Tabelle 11 zeigt, dass der Recall-Zuwachs von der Tool-Baseline (8 / 12; 66,7%) auf die vollständige Pipeline (12 / 12; 100%) kausal auf den LLM-Detektor (Stufe 1) zurückzuführen ist. Der Priorisierer in Stufe 2 kann konstruktionsbedingt keine Violations erkennen, die nicht bereits in seinem Input enthalten sind, und beeinflusst somit ausschließlich die Priorisierungsqualität, nicht jedoch den Recall. Hinsichtlich der Empfehlungsqualität erreicht **qwen3:32b** auf test-12 für 11 / 12 Violations die Stufe 2. Dies entspricht konkreten, entwicklungsnahen Verbesserungsvorschlägen mit Dateibezug und unmittelbarem Umsetzungsbezug im Code. Diese Aussage ist jedoch bewusst vorsichtig zu interpretieren, da sich die Ablationsanalyse auf test-12 beschränkt. Eine Generalisierung auf größere Suites müsste durch weitere Experimente abgesichert werden. Detaillierte Werte finden sich in den Tabellen 11, 25, 27, 53 und 34.

9.1.2 Erfüllung der funktionalen und nicht-funktionalen Anforderungen

Die fünf funktionalen Anforderungen FA-01 bis FA-05 wurden vollständig umgesetzt. Axe-core übernimmt die regelbasierte Basiserkennung (FA-01), Playwright erfasst zustandsabhängige Bar-

rieren (FA-02), die grep-gestützte Quellcodeanalyse erweitert die Sicht auf statische Muster (FA-03), die zweistufige LLM-Verarbeitung erzeugt priorisierte Handlungsempfehlungen (FA-04) und der strukturierte JSON-Report ermöglicht maschinelle Weiterverarbeitung (FA-05).

NFA-01 (Lokal-First) wird vollständig erfüllt. NFA-02 (Laufzeit) wird nur teilweise erreicht: Auf test-12 erfüllen 7b und 14b die Grenze, 32b nur eingeschränkt; auf test-100 überschreiten alle drei Modelle sie deutlich – ACE eignet sich dort eher für release-nahe oder nächtliche Analysezyklen als für per-Commit-Läufe. NFA-03 (Wartbarkeit) wird durch die modulare Architektur unterstützt. NFA-04 (Reproduzierbarkeit) wird teilweise erreicht: Die Varianz steigt mit der Suite-Größe, ist aber für einen experimentellen PoC ausreichend. Eine tabellarische Gesamtübersicht findet sich in Tabelle 19 im Anhang.

9.2 Kritische Reflexion

9.2.1 Grenzen der externen Validität

Die belastbarste Evidenz dieser Arbeit stammt aus den drei synthetischen Testsuiten. Synthetisch eingebrachte Violations sind in ihrer Struktur kontrollierter und klarer isolierbar als Barrieren in realen Anwendungssystemen, in denen technische Altlasten, Projektkonventionen und historisch gewachsene UI-Muster zusammenwirken. Der auf test-100 mit **qwen3:32b** erreichte Recall von 63 % ist daher als belastbarer Laborwert innerhalb der gewählten Evaluationslogik zu verstehen, nicht jedoch als direkt übertragbarer Erwartungswert für produktive Systeme.

Die ergänzende Erprobung am BVEC zeigt, dass das System grundsätzlich auf ein reales Untersuchungsobjekt übertragen werden kann und dort plausible, datei- und zeilennahe Befunde erzeugt. Zugleich treten die Skalierungsprobleme der Priorisierungsphase in realen Codebasen deutlicher hervor als in den synthetischen Suiten. Bei **maxfiles** = 500 zeigen sich Truncation, kollabierte Outputs und instabile Priorisierung deutlich. Der BVEC ist daher als Feldtest der praktischen Einsetzbarkeit und Systemgrenzen zu verstehen, nicht als quantitative Recall-Validierung.

9.2.2 Methodische Einschränkungen

Erstens beeinflusst das verfügbare Token-Budget die Priorisierungsqualität systematisch: Auf test-100 schöpft 14b in allen 10 Runs das Output-Limit aus, 7b in 80 % der Runs. **qwen3:32b** liefert die methodisch robusteste Basis (nur 1 / 10 trunkierte Runs), löst das Skalierungsproblem jedoch nicht vollständig.

Zweitens beruht die Erkennungsbewertung auf einer Operationalisierung über die Report-Präsenz (Mehrheit der Runs), nicht auf einer formalen semantischen Äquivalenzklasse. Semantisch äquivalente, anders formulierte Findings können dadurch zu Unterschätzungen führen. Die Ergebnisse sind deskriptiv belastbar, aber nicht inferenzstatistisch verallgemeinerbar.

Drittens wurde keine vollständige Precision-Evaluation über alle Runs der großen Suiten durchgeführt. Die ergänzende Stichprobenprüfung für `qwen3:32b` auf test-100 zeigt, dass echte Halluzinationen selten auftreten (1 von 176 geprüften Einträgen), der größere Teil der Überdetektion also auf Quell-Duplikate und Paraphrasen zurückgeht. Für den produktiven Einsatz bleibt dennoch eine nachgelagerte Plausibilisierung erforderlich.

9.3 Ausblick

9.3.1 Kurz- und mittelfristige Weiterentwicklungen

Kurzfristig bieten sich vier Weiterentwicklungen an:

1. **Manuelle Plausibilisierung des BWEC-Blocks:** Expertenabgleich der Pipeline-Ergebnisse mit einer referenziellen Baseline.
2. **Gezielte Playwright-Interaktionsprüfungen:** Erweiterung der Checks um Fokus-Traps und Dialog-Navigation.
3. **Deduplikations- und Konsolidierungslogik:** Nachgelagerte Zusammenführung überlappender Findings aus den vier Detektionsquellen.
4. **Formalisierung der Stichprobenprüfung:** Integration als fester Workflow-Bestandteil zur systematischen Quantifizierung von Halluzinationsraten und Deduplikationsbedarf.

Mittelfristig stehen Verbesserungen der Skalierbarkeit im Vordergrund: Die Priorisierungsphase könnte hierarchisch umgebaut werden – statt eines einzigen großen Prompts wären mehrstufige Cluster- oder Zusammenfassungsverfahren denkbar, da die BWEC-Messungen den zentralen technischen Engpass in diesem Schritt erkennen lassen. Ergänzend könnte `axe-core` auf mehreren systematisch durchlaufenen DOM-Zuständen ausgeführt werden, und der strukturierte JSON-Report könnte als Grundlage für nachgelagerte Automatisierung dienen – etwa die direkte Überführung priorisierter Findings in Issue-Tracker oder Code-Assistenzsysteme.

9.3.2 Langfristige Forschungsperspektiven

Langfristig erscheint insbesondere eine stärkere Spezialisierung des verwendeten Modells vielversprechend. Ein domänenspezifisch feinabgestimmtes Modell auf Accessibility-relevanten React-/TypeScript-Beispielen und typischen BITV-Kontexten könnte systematische Fehlklassifikationen reduzieren und die Priorisierungslogik stabilisieren – eine Verschiebung von der reinen Modellgrößenstrategie hin zu einer wissens- und domänenspezifischen Optimierung.

Realistischer als eine vollständige Analyse großer Codebasen in jeder Pipeline-Ausführung erscheint ein gestuftes Betriebsmodell: leichte Prüfungen auf Komponenten- oder Änderungsbasis während der Entwicklung, tiefere Analysen in qualitätssichernden Release- oder Nightly-Kontexten. In dieser differenzierten Form – als *Continuous-Accessibility-Loop* aus automatischer Erkennung, konsolidierter Priorisierung, unterstützter Behebung und erneuter Prüfung – läge der Beitrag von ACE in der nachhaltigen Reduktion manuellen Prüfaufwands sowie in der frühzeitigen Sichtbarmachung relevanter Barrieren im Entwicklungsprozess.

Anhang

Anhangverzeichnis

Anhang 1	Übersicht der Anhangsstruktur	70
Anhang 2	Komplexität von Home Pages	71
Anhang 3	Framework-Popularität (Frontend)	71
Anhang 4	Erweiterte Konzeptmatrix	72
Anhang 5	Schematische Übersicht der ACE-Pipeline	74
Anhang 6	Playwright-Checks und grep-Patterns	75
Anhang 7	System-Prompt des ACE-Assistenzsystems	76
Anhang 8	Prompts der LLM-Detektor-Phase	78
Anhang 9	Aufteilung des Token-Budgets	80
Anhang 10	Empfehlung nach Anwendungsszenario (konsolidiert)	81
Anhang 11	Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben	82
Anhang 12	Erfüllungsgrad der funktionalen und nicht-funktionalen Anforderungen	83
Anhang 13	Reproduzierbarkeitsblatt	85
Anhang 14	Console-Output-Auszüge eines Benchmark-Runs	87
Anhang 15	Threats to Validity	88
Anhang 16	test-12-Suite (kompletter Block)	90
Anhang 17	Accessibility-Verstöße der Testanwendung und zugehörige Prüfquellen (test-12-Suite)	91
Anhang 18	Erkennungsrate der Violations im Proof of Concept (test-12-Suite)	92
Anhang 19	Recall-Matrix: Violation × Bedingung (test-12-Suite)	93
Anhang 20	Rohdaten: alle 30 Benchmark-Runs (test-12-Suite)	94
Anhang 21	Modell-Leistungsvergleich: test-12-Suite Zusammenfassung	95
Anhang 22	Token-Nutzung pro Modell: Input/Output (test-12-Suite)	96
Anhang 23	Laufzeit und Effizienz der Pipeline-Modelle (test-12-Suite)	97
Anhang 24	Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-12-Suite)	97
Anhang 25	Über-Detektion relativ zur Ground Truth (test-12-Suite)	98
Anhang 26	Detektor-Konsistenz über alle 30 Runs (test-12-Suite)	98
Anhang 27	Bewertung der Empfehlungsqualität pro Violation (test-12-Suite)	99
Anhang 28	CSV-Auszug: Benchmark-Rohdaten (test-12-Suite)	100
Anhang 29	Ansichten der Testanwendung (test-12-Suite)	101
Anhang 30	test-50-Suite (kompletter Block)	103
Anhang 31	Accessibility-Verstöße der Testanwendung (test-50-Suite)	104
Anhang 32	Erkennungsrate der Violations im Proof of Concept (test-50-Suite)	107
Anhang 33	Recall-Matrix: Violation × Bedingung (test-50-Suite)	110
Anhang 34	Rohdaten: alle 30 Benchmark-Runs (test-50-Suite)	113
Anhang 35	Modell-Leistungsvergleich: test-50-Suite Zusammenfassung	114

Anhang 36	Token-Nutzung pro Modell: Input/Output (test-50-Suite)	115
Anhang 37	Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-50-Suite)	115
Anhang 38	Über-Detektion relativ zur Ground Truth (test-50-Suite)	116
Anhang 39	Precision und F1-Score je Run (test-50-Suite)	117
Anhang 40	Detektor-Konsistenz über alle 30 Runs (test-50-Suite)	117
Anhang 41	Bewertung der Empfehlungsqualität pro Violation (test-50-Suite)	118
Anhang 42	CSV-Auszug: Benchmark-Rohdaten (test-50-Suite)	121
Anhang 43	Ansichten der Testanwendung (test-50-Suite)	122
Anhang 44	test-100-Suite (kompletter Block)	125
Anhang 45	Accessibility-Verstöße der Testanwendung (test-100-Suite)	126
Anhang 46	Erkennungsrate der Violations im Proof of Concept (test-100-Suite)	130
Anhang 47	Recall-Matrix: Violation $\times$ Bedingung (test-100-Suite)	135
Anhang 48	Rohdaten: alle 30 Benchmark-Runs (test-100-Suite)	140
Anhang 49	Modell-Leistungsvergleich: test-100-Suite Zusammenfassung	141
Anhang 50	Token-Nutzung pro Modell: Input/Output (test-100-Suite)	142
Anhang 51	Effizienzmetriken (test-100-Suite)	142
Anhang 52	Über-Detektion relativ zur Ground Truth (test-100-Suite)	143
Anhang 53	Stichproben-Plausibilisierung des finalen Reports (qwen3:32b , Run 01 & Run 03, test-100)	144
Anhang 54	Precision und F1-Score je Run (test-100-Suite)	145
Anhang 55	Detektor-Konsistenz über alle 30 Runs (test-100-Suite)	146
Anhang 56	Bewertung der Empfehlungsqualität pro Violation (test-100-Suite)	147
Anhang 57	CSV-Auszug: Benchmark-Rohdaten (test-100-Suite)	153
Anhang 58	Pareto-Diagramm: Nicht erkannte Violations (test-100, qwen3:32b)	154
Anhang 59	Boxplot der Laufzeiten (test-100, 10 Runs je Modell)	155
Anhang 60	Pipeline-Diagramm der Findings-Verarbeitung	156
Anhang 61	Ansichten der Testanwendung (test-100-Suite)	157
Anhang 62	Lesehinweis zu den Vergleichsgrafiken	160
Anhang 63	Vergleich über alle Suites	161
Anhang 64	Recall-Degradation über alle Testsuites	162
Anhang 65	Gesamtlaufzeit-Skalierung über alle Testsuites	163
Anhang 66	LLM-Konsistenz (σ) im Suite-Vergleich	164
Anhang 67	Recall und Überschuss - nach Suite gruppiert	165
Anhang 68	Truncation-Rate im Suite-Vergleich	166
Anhang 69	Heatmap der Recall-Werte	167
Anhang 70	Gestapeltes Balkendiagramm: Must-have vs. Nice-to-have	168
Anhang 71	BWEC-Block (kompletter Block)	169
Anhang 72	Detektionsquellen-Übersicht: BWEC	170
Anhang 73	Rohdaten: BWEC-Messreihe (maxfiles = 100, 66 Chunks)	171
Anhang 74	Modell-Leistungsvergleich: BWEC (maxfiles = 100)	172
Anhang 75	Rohdaten: BWEC-Messreihe (maxfiles = 500, 330 Chunks)	173

Anhang 76	Modell-Leistungsvergleich: BWEC (<code>maxfiles</code> = 500)	174
-----------	--	-----

Anhang 1: Übersicht der Anhangsstruktur

Tab. 13: Übersicht der Anhangsstruktur mit Trennung zwischen generellen und suite-spezifischen Inhalten.

Bereich	Inhalte
Generell	Komplexität von Home Pages, Framework-Popularität, Konzeptmatrix, Pipeline-Schema, Playwright-Checks & grep-Patterns, System-Prompt, LLM-Detektor-Prompts, Token-Budget, konsolidierte Empfehlung, Bewertungsskala, FA/NFA-Erfüllungsgrad, Reproduzierbarkeitsblatt, Console-Output-Auszüge, Threats to Validity
Vergleichsgrafiken	Suite-Vergleich (Tabelle), Recall-Degradation, Laufzeit-Skalierung, Konsistenz (σ), Recall/Überschuss (Suite-Gruppen), Truncation-Rate, Heatmap Recall, Must-/Nice-to-have-Verteilung
test-12-Block	Violations-Katalog, Erkennungsrate, Recall-Matrix, Rohdaten, Modellvergleich, Token-Nutzung, Laufzeit, Effizienz, Über-Detektion, Konsistenz, Empfehlungsqualität, CSV-Auszug, Ansichten der Testsuite
test-50-Block	Violations-Katalog, Erkennungsrate, Recall-Matrix, Rohdaten, Modellvergleich, Token-Nutzung, Effizienz, Über-Detektion, Precision/F1 je Run, Konsistenz, Empfehlungsqualität, CSV-Auszug, Ansichten der Testsuite
test-100-Block	Violations-Katalog, Erkennungsrate, Recall-Matrix, Rohdaten, Modellvergleich, Token-Nutzung, Effizienz, Über-Detektion, Stichproben-Plausibilisierung, Precision/F1 je Run, Konsistenz, Empfehlungsqualität, CSV-Auszug, Pareto-Diagramm, Boxplot Laufzeiten, Pipeline-Diagramm, Ansichten der Testsuite
BWEC-Block	Detektionsquellen-Übersicht, Rohdaten (maxfiles=100), Modellvergleich (maxfiles=100), Rohdaten (maxfiles=500), Modellvergleich (maxfiles=500)

Anhang 2: Komplexität von Home Pages

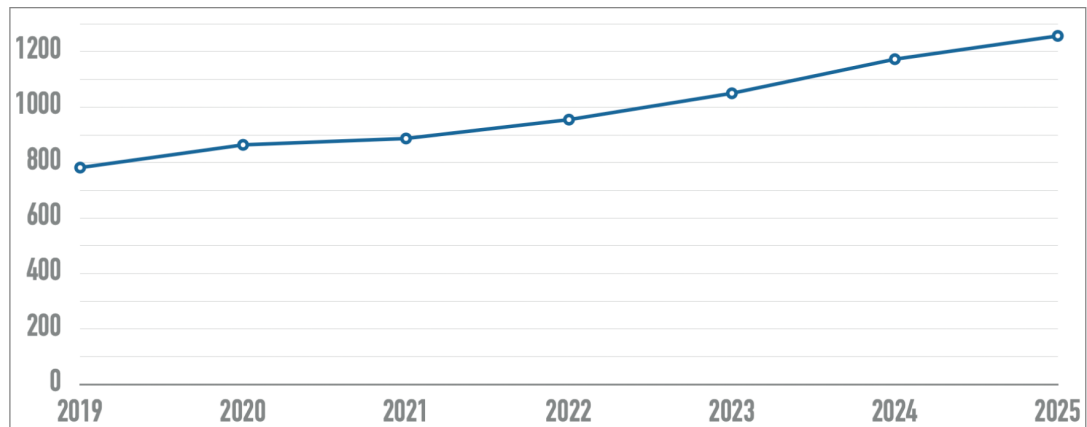


Abb. 5: Entwicklung der Komplexität von Home Pages in den letzten sechs Jahren. Die X-Achse zeigt die Jahre, die Y-Achse gibt die Anzahl der Elemente an.²⁰⁸

Anhang 3: Framework-Popularität (Frontend)

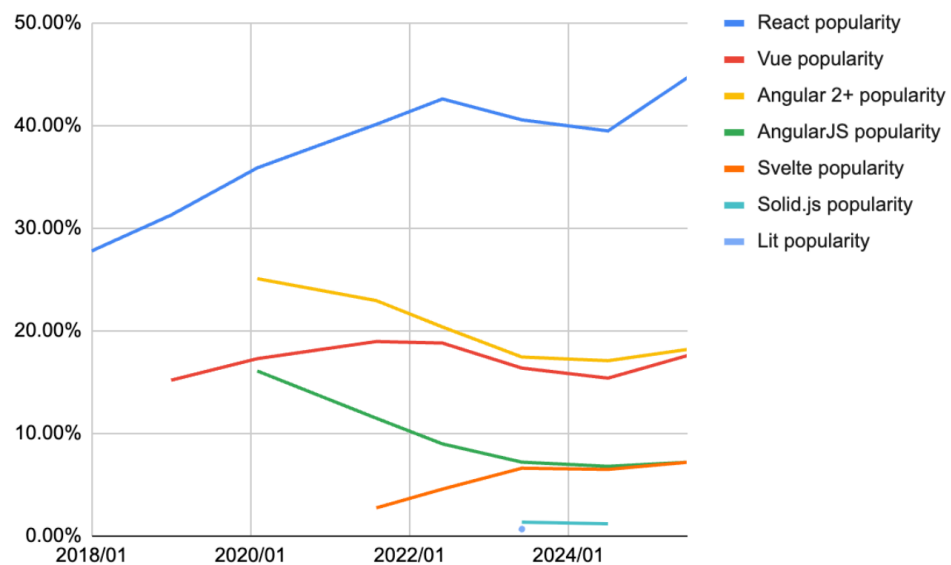


Abb. 6: Framework Popularity über die letzten Jahre²⁰⁹

²⁰⁸Entnommen aus WebAIM 2025

²⁰⁹Entnommen aus Krotoff 2026

Anhang 4: Erweiterte Konzeptmatrix

²¹⁰Vgl. Webster, Watson 2002.

Studie		Konzepte													
Autor(en) & Jahr	System	Ziel			Datenquellen			KI- / LLM-Ansatz			Evaluation			Kontext	
		WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA	Rechtl. Rahmen (BFSG etc.)
Hoch priorisierte Studien															
He et al. (2025)	GenA11y	x			x			x			x				
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x				
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x		x	
Paternò et al. (2025)	TeVal	x		x	x			x	x			x			
Tafreshipour et al. (2024)	Ma11y	x			x	x					x				
Fernández-Navarro & Chicano (2026)	LLM Remediation	x	x	x	x		x	x			x		x		
Mittel priorisierte Studien															
Yu et al. (2025)	GenAI Screen Reader	x	x		x		x	x	x		x	x	x		
Huang et al. (2024)	ACCESS	x	x	x	x	x		x			x				
Pool (2023)	Testaro	x			x	x					x				
Mowar et al. (2024)	Copilot Study	x		x				x					x		
Abu Doush & Kassem (2024)	AI Code Study	x	x	x				x			x				
Guriță & Vatavu (2025)	AI UI Study	x						x			x				
Kodithuwakku & Wick. (2025)	Tool Eval.	x			x						x				
Manca et al. (2023)	Transparency	x		x	x							x	x		
Jiang & Nam (2025)	Cursor Rules			x			x	x				x			
Gubbi Mohanbabu & Pavel (2024)	Context-Aware Img.	x	x					x	x		x	x	x		
Campoverde-Molina & Luján-Mora (2025)	AI A11y SMS	x	x	x	x			x				x			
Leedy et al. (2025)	GenAI Builder Eval.	x			x			x			x	x			
Patel & Melcer (2025)	AIDA Study	x		x				x				x	x		
Aljedaani et al. (2024)	ChatGPT A11y Code	x	x	x			x	x			x				
Niedrig priorisierte Studien															
Abou-Zahra et al. (2018)	AI for A11y	x						x				x		x	
Delnevo et al. (2024)	LLM Interaction	x	x					x				x		x	
Draffan et al. (2020)	Web2Access+AI	x			x			x				x			
Eigener Proof of Concept															
Martínez Campo (2026)	ACE (PoC)	x	x	x	x	x	x	x	x	x	x	x		x	

Tab. 14: Konzeptmatrix nach Webster Watson²¹⁰: vollständige Übersicht aller analysierten Studien im Vergleich zum eigenen Proof of Concept (PoC). **x** = Konzept vorhanden/adressiert; leer = nicht adressiert. „Barrieren korrigieren“ schließt die Generierung konkreter Code-Fix-Vorschläge ein, nicht notwendigerweise automatisches Patchen.

Anhang 5: Schematische Übersicht der ACE-Pipeline

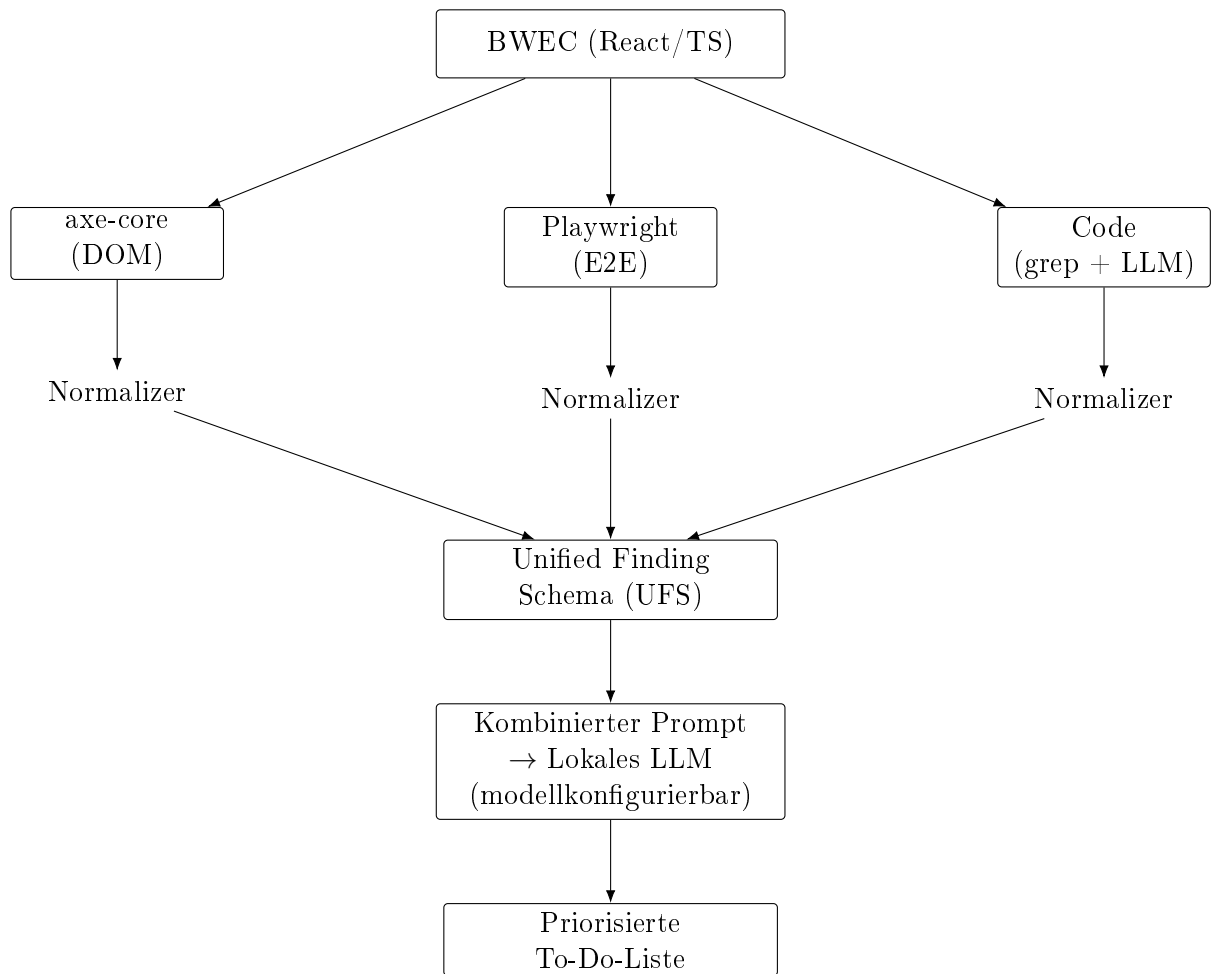


Abb. 7: Schematische Übersicht der im PoC implementierten ACE-Pipeline von der Datenerhebung bis zur priorisierten Ausgabe. Die Code-Schiene umfasst regex-basierte Pattern (grep) und die LLM-basierte Codeanalyse. Das verwendete LLM ist in der Pipeline bewusst abstrahiert; die konkrete Modellwahl erfolgt je Einsatzszenario (vgl. Tabelle 17).

Anhang 6: Playwright-Checks und grep-Patterns

Check-ID	Prüfgegenstand	WCAG	Kategorie
keyboard-tab-reachable	Interaktive Elemente per Tab erreichbar	2.1.1 (A)	semantisch
keyboard-trap-free	Kein Keyboard-Trap nach 40 Tab-Presses	2.1.2 (A)	semantisch
focus-visible	Sichtbarer Fokusindikator vorhanden	2.4.7 (AA)	layout
skip-link-present	Erstes Tab-Ziel ist ein Skip-Link	2.4.1 (A)	semantisch
page-title-present	Seite hat einen nicht-leeren <title>	2.4.2 (A)	syntaktisch
html-lang-present	<html> hat ein lang-Attribut	3.1.1 (A)	syntaktisch
focus-after-dialog	Fokus wandert beim Dialogöffnen in den Dialog	2.4.3 (A)	semantisch

Tab. 15: Implementierte Playwright-Checks mit WCAG-Zuordnung und Kategorie

Pattern-ID	Beschreibung	WCAG	Kategorie
div-span-onclick	onClick auf <div>/ ohne Keyboard-Handler	2.1.1 (A)	semantisch
img-missing-alt	 ohne alt="..."	1.1.1 (A)	syntaktisch
label-missing-htmlfor	<label> ohne htmlFor	1.3.1 (A)	syntaktisch
role-button-non-semantic	role="button" auf Nicht-Button	4.1.2 (A)	semantisch
tabindex-removes-element	tabIndex={-1} entfernt Element aus Tab-Reihenfolge	2.1.1 (A)	semantisch
autofocus-usage	autoFocus kann den Fokusfluss stören	2.4.3 (A)	semantisch

Tab. 16: Implementierte grep-Patterns mit WCAG-Zuordnung

Anhang 7: System-Prompt des ACE-Assistenzsystems

Der folgende System-Prompt wird dem Sprachmodell bei jedem Pipeline-Durchlauf übergeben. Er implementiert die in Abschnitt 6.2 beschriebenen Prompt-Engineering-Strategien Role-Play Prompting und Few-Shot Prompting (vgl. Abschnitt 7.6).

Listing 9.1: Vollständiger System-Prompt des ACE-Assistenzsystems (aus `src/prompt.ts`)

```

1 Du bist ein erfahrener Barrierefreiheitsexperte für React/TypeScript-Webanwendungen.
2 Deine Aufgabe ist es, Findings aus vier Analysequellen (axe-core, Playwright, grep
3 und LLM-Codeanalyse) zu einer priorisierten, entwicklernahen To-Do-Liste zusammenzufassen
4
5 AUSGABEFORMAT (strikt einhalten):
6
7 ## Must-have
8 *WCAG 2.x Level A und AA, Severity "critical" oder "serious" -- gesetzlich relevant*
9
10 ### [ID] Kurztitel des Problems
11 - **WCAG:** X.X.X (Level X)
12 - **Schweregrad:** critical | serious
13 - **Element/Datei:** CSS-Selektor oder Dateipfad:Zeile
14 - **Problem:** Ein Satz -- was ist konkret falsch?
15 - **Fix:** Konkreter Code-Vorschlag in JSX/TypeScript
16
17 ## Nice-to-have
18 *Severity "moderate" oder "minor", Level AAA -- empfohlen aber nicht gesetzlich verpflichtig*
19
20 ### [ID] Kurztitel des Problems
21 [gleiche Struktur]
22
23 PRIORISIERUNGSREGELN:
24 1. Must-have: Severity "critical" oder "serious" UND WCAG Level A oder AA
25 2. Nice-to-have: Severity "moderate" oder "minor" ODER Level AAA
26 3. Wenn mehrere Findings dasselbe Element betreffen, fasse sie zusammen
27 4. Nutze den Codekontext für konkrete React/TypeScript-Fixes
28 5. Antworte ausschließlich auf Deutsch
29 6. Erfinde keine Findings, die nicht in den Daten vorhanden sind
30
31 BEISPIEL (Few-Shot):
32
33 Eingabe-Finding:
34 [axe-007] CRITICAL | color-contrast | WCAG 1.4.3 (AA)
35 Beschreibung: Element hat unzureichenden Farbkontrast (2.8:1 statt 4.5:1)
36 Element: .sidebar > .nav-link
37 Datei: components/Sidebar.tsx:42-52
38 Code:
39 42: <a className="nav-link" href="/dashboard">
40 43:   Dashboard
41 44: </a>
42
43 Erwartete Ausgabe:
44
45 ## Must-have
46
47 ### [axe-007] Farbkontrast der Sidebar-Navigation unzureichend

```

```
48 - **WCAG:** 1.4.3 (Level AA)
49 - **Schweregrad:** critical
50 - **Element/Datei:** components/Sidebar.tsx:42
51 - **Problem:** Der Link ".nav-link" hat ein Kontrastverhältnis von 2.8:1,
52   gefordert sind mindestens 4.5:1 (WCAG 1.4.3 AA).
53 - **Fix:**
54   ``tsx
55   // Sidebar.tsx -- Farbe des Links anpassen
56   <a className="nav-link" style={{ color: "#1a1a2e" }} href="/dashboard">
57     Dashboard
58   </a>
59   // Alternativ: CSS-Klasse mit ausreichendem Kontrast definieren
60   ``
61
62   ---
63
64   Eingabe-Finding:
65   [grep-002] MODERATE | tabindex-removes-element | WCAG 2.1.1 (A)
66   Beschreibung: tabIndex=-1 entfernt Elemente aus der Tab-Reihenfolge.
67   Datei: components/Modal.tsx:18-28
68   Code:
69   18: <div className="modal-overlay" tabIndex={-1}>
70   19:   <div className="modal-content" role="dialog">
71   20:     <button className="close-btn">Schließen</button>
72
73   Erwartete Ausgabe:
74
75   ## Nice-to-have
76
77   ### [grep-002] tabIndex=-1 auf Modal-Overlay
78   - **WCAG:** 2.1.1 (Level A)
79   - **Schweregrad:** moderate
80   - **Element/Datei:** components/Modal.tsx:18
81   - **Problem:** Das Modal-Overlay hat tabIndex={-1}. Wenn das Overlay selbst
82     fokussierbar sein soll (z.B. für Escape-Handling), ist das korrekt.
83     Andernfalls entfernt es das Element unnötig aus der Tab-Reihenfolge.
84   - **Fix:**
85     ``tsx
86     // Falls Overlay nicht fokussierbar sein muss -- tabIndex entfernen:
87     <div className="modal-overlay">
88       <div className="modal-content" role="dialog" aria-modal="true">
89         <button className="close-btn" autoFocus>Schließen</button>
90     ``
```

Anhang 8: Prompts der LLM-Detektor-Phase

Die LLM-Detektor-Phase (Stufe 1 der zweistufigen LLM-Strategie, vgl. Abschnitt 7.5) verwendet einen eigenen, vom Priorisierungs-Prompt unabhängigen Prompt-Satz. Er wird für jeden Codechunk separat aufgerufen. In der Implementierung gilt `num_predict=640` als *Default* (vgl. `src/modules/llmDetector.ts`); im Benchmark wird dieser Wert explizit per Umgebungsvariable auf `LLM_DETECT_NUM_PREDICT=2048` gesetzt (vgl. `src/benchmark.ts`), um den Recall in größeren Suites zu stabilisieren.

Damit ergibt sich für Stufe 1 kein globales Einzelbudget, sondern ein **chunkweises Budget**: $\text{Budget}_{\text{Detektor}} = \text{Anzahl Chunks} \times \text{num_predict}_{\text{Detektor}}$. Für die im Benchmark beobachteten mittleren Chunk-Zahlen (test-12: ≈ 4 , test-50: ≈ 8 , test-100: ≈ 12) entspricht dies bei `LLM_DETECT_NUM_PREDICT=2048` einem theoretischen Output-Rahmen von ca. 8,192 / 16,384 / 24,576 Tokens in Stufe 1.

System-Prompt (LLM-Detektor):

Listing 9.2: System-Prompt der LLM-Detektor-Phase (aus `src/modules/llmDetector.ts`)

```

1 Du bist ein präziser Accessibility-Reviewer für React/TypeScript.
2 Analysiere NUR den gegebenen Codechunk.
3
4 Ausgabeformat: Nur JSON (kein Markdown), ein Objekt mit Feld "findings".
5 Schema je Finding:
6 {
7   "title": string,
8   "ruleId": string,
9   "wcagCriteria": string[],
10  "wcagLevel": "A" | "AA" | "AAA",
11  "severity": "critical" | "serious" | "moderate" | "minor",
12  "category": "syntaktisch" | "semantisch" | "layout",
13  "filePath": string,
14  "startLine": number,
15  "endLine": number,
16  "evidence": string,
17  "fixSuggestion": string
18 }
19
20 Regeln:
21 1. Melde nur Findings mit klarer Evidenz im Codechunk.
22 2. Erfinde keine Dateien oder Zeilen.
23 3. Wenn unsicher: nicht melden.
24 4. Maximal 12 Findings pro Chunk.
25 5. Nur JSON ausgeben.
```

User-Prompt-Template (LLM-Detektor, vereinfacht):

Listing 9.3: User-Prompt-Template der LLM-Detektor-Phase; `<Chunk-Nr>`, `<Dateiliste>` und `<Code>` werden zur Laufzeit eingesetzt (aus `src/modules/llmDetector.ts`)

```

1 Aufgabe: Finde Barrierefreiheitsprobleme im folgenden Codechunk.
2 Berücksichtige WCAG-relevante Probleme bei Semantik, Keyboard, Fokus,
```

```
3 Labels, ARIA und visuellem Kontrast nur wenn im Code direkt ableitbar.  
4  
5 Chunk: <Chunk-Nr>  
6 Dateien im Chunk:  
7 - <Datei-1>  
8 - <Datei-2>  
9 ...  
10  
11 Code:  
12 <Inhalt des Chunks (Quelldateien mit Zeilennummern, max. 4000 Zeichen)>  
13  
14 Gib nur JSON zurück: {"findings": [...]}
```

Anhang 9: Aufteilung des Token-Budgets



Abb. 8: Aufteilung des Kontextfensters gemäß Implementierung in `src/prompt.ts`: feste Reserven für System (400 Tokens) und Output, variabler Bereich für serialisierte Findings mit Schweregrad-basierter Kürzung bei Budgetüberschreitung. Die Abbildung zeigt ausschließlich Stufe 2 (Priorisierung) mit den Code-Standardkonstanten (`TOTAL_CONTEXT_WINDOW = 32 768`, `RESERVED_FOR_OUTPUT = 8 192`). In den Benchmarks wurde `num_predict` suite-spezifisch überschrieben: 6 144 (test-12), 8 192 (test-50), 12 288 (test-100); für BWEC galt `OLLAMA_NUM_CTX = 65 536` mit `num_predict = 24 576`. Stufe 1 (LLM-Detektor) besitzt ein separates, chunkweises Budget: Default 640, Benchmarks test-12–100: 2 048, BWEC: 4 096 Tokens. Vgl. Abschnitt 6.2.

Anhang 10: Empfehlung nach Anwendungsszenario (konsolidiert)

Leseanleitung: Die nachfolgende Tabelle konsolidiert die Evaluationsergebnisse aus test-12, test-50 und test-100 (Kapitel 8). `qwen2.5-coder:7b` erscheint in der Spalte *Primär* ausschließlich in Szenarien, in denen **kein** BITV-/WCAG-konformes Ergebnis erforderlich ist (CI/CD-Rauchtest, Prototyp). Für jede rechtlich relevante Barrierefreiheitsprüfung gilt unverändert: `qwen2.5-coder:7b` ist aufgrund der nachgewiesenen inversen bzw. instabilen Priorisierung (vgl. Abschnitt 8.3.1) *nicht geeignet*. Für Enterprise-Kontexte wird ein zweistufiges Betriebsmodell angenommen: schneller, nicht verbindlicher CI-Check und ein verbindlicher Compliance-Gate mit geeignetem Modell. Die Spalte *Eignung BITV-Einsatz* in Tabelle 27, Tabelle 42 und Tabelle 55 ist das übergeordnete Kriterium.

Tab. 17: Konsolidierte Modellwahl in Abhängigkeit vom Einsatzkontext auf Basis von test-12, test-50 und test-100. Primär: bevorzugtes Modell; Sekundär: Alternative. Die Empfehlungen für 7b gelten ausschließlich für nicht-compliance-relevante Szenarien.

Szenario	Primär	Sekundär	Begründung
CI/CD-Rauchtest (nicht compliance-relevant)	7b	14b	Schnellstes Feedback; inverse Priorisierung hier tolerierbar
Regulärer Entwicklungszyklus	14b	32b	Bester Kompromiss aus Recall, Priorisierung und Laufzeit
BITV-Compliance-Audit	14b	32b	14b liefert robustes Qualitäts-/Laufzeitprofil über die Suites; 32b als Zweitmeinung
Abnahmeprüfung / Endaudit	32b	14b	Höchste inhaltliche Abdeckung; 14b als zeitlich stabilere Gegenprüfung
Prototyp / MVP	7b	–	Schnelle Iteration; für finale Abnahme durch 14b/32b ersetzen
Enterprise-Deployment	14b	32b	Produktionstauglich im zweistufigen Betrieb: CI-Schnellcheck + verbindlicher Compliance-Gate

Anhang 11: Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben

Tab. 18: Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben über alle drei Test-suites (test-12, test-50 und test-100).

Stufe	Bezeichnung	Kriterium
2	Konkret	Enthält Dateiname, Zeilennummer und einen konkreten Fix-Vorschlag (z. B. <code>aria-label="..."</code>).
1	Teilweise	Enthält den richtigen Fix-Typ, jedoch ohne konkreten Codekontext (z. B. „Füge ein <code>aria-label</code> hinzu“).
0	Generisch	Beschreibt lediglich das Problem, ohne einen umsetzbaren Lösungsvorschlag.

Anhang 12: Erfüllungsgrad der funktionalen und nicht-funktionalen Anforderungen

Tab. 19: Dargestellt ist der Erfüllungsgrad der funktionalen (FA) und nicht-funktionalen (NFA) Anforderungen im realisierten PoC. Die Bewertung basiert auf der konsolidierten Evaluation über test-12, test-50 und test-100. ✓ = vollständig erfüllt; (∼) = bedingt erfüllt; × = nicht erfüllt.

Anf.	Beschreibung	Status	Bemerkung
FA-01	Regelbasierte Accessibility-Erkennung via axe-core	✓	axe-core ist vollständig integriert und liefert auf allen Suites stabile, deterministische Baseline-Findings; die Abdeckung bleibt jedoch auf automatisierbare Regelverstöße begrenzt.
FA-02	Dynamische Zustandserfassung via Playwright	✓	Generische Interaktionschecks (z. B. Skip-Link, Fokusindikator) sind umgesetzt und liefern stabile Findings; modalspezifische Fokus-Traps werden jedoch nicht in allen Suites zuverlässig erfasst.
FA-03	Statische Quellcodeanalyse via grep	✓	Pattern-Matching und Kontextanreicherung liefern datei- und zeilenspezifischen Kontext für grep-basierte bzw. angereicherte Befunde; eine vollständige Quellcodeabdeckung aller Violations ist damit nicht verbunden.
FA-04	LLM-gestützte Analyse und Handlungsempfehlung	✓	Zweistufige Strategie (Detektor + Priorisierung) umgesetzt; Fix-Qualität mit <code>qwen3:32b</code> auf test-12 überwiegend Stufe 2.
FA-05	Strukturierter JSON-Report	✓	Maschinenlesbarer JSON-Output und separate Report-Artefakte sind implementiert; Parsing-Erfolg in den Benchmarkläufen 100 %.
NFA-01	Lokal-First / Datenschutz (kein Cloud-Transfer)	✓	Ollama-Infrastruktur; kein Quellcode verlässt das lokale System.
NFA-02	Laufzeit < 10 Min.	(∼)	In test-12 erfüllen 7b und 14b die Anforderung, in test-50 nur 7b; in test-100 überschreiten alle drei Modelle die Zehn-Minuten-Grenze.
NFA-03	Wartbarkeit (modulare Architektur)	✓	Trennung in Detection, Context-Building und Synthesis; unabhängig erweiterbar.
NFA-04	Reproduzierbarkeit (stabile Ausgaben)	(∼)	Temperature 0,05 und JSON-Schema-Prompting sichern Grundstabilität; auf test-12 zeigt 14b mit $\sigma = 0,42$ die höchste Konsistenz, auf größeren Suites steigt die Varianz jedoch an.

Anhang 13: Reproduzierbarkeitsblatt

Tab. 20: Umgebungs- und Konfigurationsparameter zur Reproduktion der Benchmark-Runs über alle drei Testsuites.

Parameter	Wert
Hardware	
CPU	Apple M4 Max
RAM	128 GB
GPU	integrierte GPU nicht genutzt; CPU-basierte Ausführung
Speicher	4TB
Betriebssystem	
OS	macOS 26.4 (25E246)
Modelle und Infrastruktur	
Ollama-Version	0.19.0
Modellinstanz 7b	qwen2.5-coder:7b, Quantisierung
Modellinstanz 14b	qwen2.5-coder:14b, Quantisierung
Modellinstanz 32b	qwen3:32b, Quantisierung
zentrale Parameter	
temperature	0,05 für alle Runs
num_predict (test-12)	6 144 Tokens
num_predict (test-50)	8 192 Tokens
num_predict (test-100)	12 288 Tokens
Chunk-Größe (Codeanalyse)	4000 Zeichen max.
Max. Dateien (Codeanalyse)	25
Runs pro Modell	10 Runs je Testsuite
Toolchain	
Node.js-Version	24.12.0
TypeScript-Version	5.7.3
axe-core-Version	4.10.3
Playwright-Version	1.50.1

Anhang 14: Console-Output-Auszüge eines Benchmark-Runs

Hinweis: Im Folgenden wird ein repräsentativer Console-Output eines erfolgreichen Runs dokumentiert. Sie zeigt die typische Pipeline-Ausführung: Initialisierung, Tool-Ausführungen, LLM-Detektor-Phase, Priorisierung und finaler Report-Export.

Listing 9.4: Console-Output eines erfolgreichen Benchmark-Runs

```
1 [INFO] Pipeline initialized
2 Muss ich noch füllen, falsche Version genommen!!!!
```

Anhang 15: Threats to Validity

Tab. 21: Systematische Analyse der internen, externen und Konstruktvalidität des PoC.

Validitätsebene	Threat	Mitigationsmaßnahmen / Aussagekraft
Interne Validität: Korrektheit der Ergebnisse		
Modell-abhängigkeit	Die drei Modellgrößen liefern teils deutlich unterschiedliche Ergebnisse; auf test-100 reicht der Recall von 40–63 %	Der Mehrmodellvergleich wird offengelegt. Für produktive Einsätze sind größere Modelle besser geeignet als 7b.
Prompt-Abhängigkeit	System-Prompt und Few-Shot-Beispiele wurden fest vorgegeben; alternative Prompt-Varianten wurden nicht untersucht	Der Prompt wird als fester Bestandteil des PoC behandelt. Eine Sensitivitätsanalyse war out of Scope der Arbeit.
Konfigurationsparameter	Einstellungen wie Temperatur (0,05), num_predict und Chunk-Größe (4000 Zeichen) beeinflussen das Antwortverhalten der Modelle	Alle Parameter sind dokumentiert und reproduzierbar. Eine nachträgliche Optimierung zugunsten einzelner Ergebnisse fand nicht statt.
Ground-Truth-Genauigkeit	Die 100 Violations wurden manuell in die Testanwendung eingebracht und nicht durch externe Accessibility-Audits validiert	Innerhalb der Testumgebung ist die Ground Truth konsistent. Die externe Absicherung bleibt jedoch begrenzt.
Externe Validität: Generalisierbarkeit		
Testanwendungs-Spezifik	Die Hauptevaluation basiert auf drei synthetischen React-Testsuites mit bekannter Ground Truth; der reale BWEC wurde ergänzend ohne Ground Truth qualitativ betrachtet	Die Ergebnisse sind primär auf vergleichbare React-/SPA-Umgebungen übertragbar. Eine breitere Generalisierung erfordert zusätzliche Ground-Truth-basierte Tests an realen Anwendungssystemen.
WCAG-Coverage	Abgedeckt werden zentrale WCAG-Kriterien aus den Bereichen 1.x bis 4.1.x, jedoch kein vollständiger Kriterienkatalog	Für eine an WCAG AA orientierte Bewertung ist die Abdeckung ausreichend. Aussagen zu AAA bleiben nur eingeschränkt möglich.
<i>Fortsetzung auf der nächsten Seite</i>		

Tab. 21: Threats to Validity (Fortsetzung)

Validitätsebene	Threat	Mitigationsmaßnahmen / Aussagekraft
Modell-verfügbarkeit	Evaluiert wurden ausschließlich lokal ausgeführte Modelle (<code>qwen2.5-coder</code> , <code>qwen3</code>)	Der Fokus liegt bewusst auf einem datenschutzfreundlichen Local-First-Ansatz. Die Ergebnisse lassen sich daher nicht unmittelbar auf Cloud-Modelle übertragen.
Skalierbarkeit	Ground-Truth-basiert wurden die Testsuites <code>test-50</code> und <code>test-100</code> untersucht; ergänzend wurde mit <code>BWEC-500</code> auch eine große reale Codebasis qualitativ erprobt, jedoch ohne Recall-Metrik	Das Verfahren ist durch Chunking grundsätzlich skalierbar. Praktische Grenzen bei Laufzeit, Kontextfenster und Priorisierungsstabilität wurden im <code>BWEC-500</code> -Experiment sichtbar und sollten in künftigen Studien weiter untersucht werden.
Konstruktvalidität: Repräsentation der Konstrukte		
Recall vs. Precision	Der Schwerpunkt der Evaluation liegt auf dem Recall; Precision wurde für <code>test-50</code> und <code>test-100</code> per Run berechnet, jedoch ohne vollständige manuelle Plausibilisierung aller Einträge	Per-Run-Precision-Werte für alle drei Modelle finden sich in Anhang 39 (<code>test-50</code>) und Anhang 54 (<code>test-100</code>). Für <code>qwen3:32b</code> auf <code>test-100</code> wurde ergänzend eine Stichprobenprüfung durchgeführt.
Metrik-Einschränkungen	Das gesetzte Zeitlimit von 10 Minuten wird in <code>test-100</code> nicht eingehalten	Die Bewertung erfolgt deshalb sowohl aus strenger als auch aus pragmatischer Perspektive. Ein Einsatz ist eher im Release-Kontext denkbar als in einer kurzen CI/CD-Pipeline.
Priorisierungsqualität	Die Fix-Konkretheit wurde über eine Skala von 0 bis 2 bewertet; eine automatisierte Validierung erfolgte nicht	Es wurden einzelne Checks durchgeführt. Ob sich diese Einschätzung auf alle Runs übertragen lässt, bleibt offen. ²¹¹

²¹¹Vgl. Anhang 27, Anhang 41 und Anhang 56.

Anhang 16: test-12-Suite (kompletter Block)

Tab. 22: Navigationsübersicht in der standardisierten Reihenfolge für die test-12-Suite.

Nr.	Baustein
1	Violations-Katalog
2	Erkennungsrate
3	Recall-Matrix
4	Rohdaten
5	Modellvergleich
6	Token-Nutzung
7	Laufzeit & Effizienz
8	Effizienz (Kosten-Nutzen)
9	Über-Detektion
10	Konsistenz
11	Empfehlungsqualität
12	CSV-Auszug
13	Ansichten der Testsuite

Anhang 17: Accessibility-Verstöße der Testanwendung und zugehörige Prüfquellen (test-12-Suite)

Tab. 23: Alle 12 eingebetteten Accessibility-Verstöße mit Kategorie, erwarteter Erkennungsquelle und zugehörigem WCAG-Kriterium.

#	Violation	Kategorie	Erwartete Quelle	WCAG
1	 ohne alt="..."	syntaktisch	axe-core	1.1.1
2	<button> ohne sichtbaren Text und ohne aria-label	syntaktisch	axe-core	4.1.2
3	Formularfeld ohne <label>	syntaktisch	axe-core	1.3.1
4	Kontrastverhältnis < 4.5 : 1 (Text auf Hintergrund)	layout	axe-core	1.4.3
5	Fokus-Outline via CSS entfernt (outline: none)	layout	axe-core + Playwright	2.4.7
6	onClick-Handler ohne onKeyDown-Pendant	semantisch	grep + LLM-Codeanalyse	2.1.1
7	role="button" auf <div> ohne tabIndex	semantisch	axe-core + grep + LLM-Codeanalyse	4.1.2
8	Modaler Dialog ohne Fokus-Trap	semantisch	Playwright	2.1.2
9	Skip-Link fehlt	semantisch	Playwright	2.4.1
10	aria-hidden="true" auf fokussierbarem Element	syntaktisch	axe-core	4.1.2
11	Reihenfolge im DOM weicht von visueller Reihenfolge ab	semantisch	axe-core	1.3.2
12	Interaktives Element zu klein (< 24 × 24px)	layout	grep + LLM-Codeanalyse	2.5.8

Anhang 18: Erkennungsrate der Violations im Proof of Concept (test-12-Suite)

Tab. 24: Erkennungsrate aller 12 Violations über alle Modelle (Konsistenzschwelle: $> 5/10$ Runs). *Erkannt* = konsistent über alle Modelle hinweg; modellabhängige Abweichungen sind in *Anmerkung* dokumentiert.²¹²

#	Violation	Erkannt (✓/×)	Tatsächliche Quelle	Anmerkung
1	<code></code> ohne <code>alt="..."</code>	✓	axe-core	Alle Modelle
2	<code><button></code> ohne <code>aria-label</code>	✓	LLM-Detektor	Nur 14b/32b; 7b miss
3	Formularfeld ohne <code><label></code>	✓	axe-core + grep	7b instabil; 14b/32b konsistent
4	Kontrastverhältnis $< 4.5 : 1$	✓	axe-core	2 axe-Findings (nav + submit)
5	Fokus-Outline entfernt (<code>outline: none</code>)	✓	axe-core + Playwright	Alle Modelle
6	onClick ohne onKeyDown	✓	grep	Alle Modelle
7	<code>role="button"</code> auf <code><div></code> ohne <code>tabIndex</code>	✓	axe-core + grep	Alle Modelle
8	Modal ohne Fokus-Trap	✓	LLM-Detektor	Playwright-Check greift nicht; nur mit LLM
9	Skip-Link fehlt	✓	Playwright	Alle Modelle
10	<code>aria-hidden="true"</code> auf fokussierbarem Element	✓	axe-core	Alle Modelle
11	DOM-Reihenfolge vs. visuelle Reihenfolge	✓	LLM-Detektor	14b/32b konsistent; 7b miss
12	Interaktives Element zu klein ($< 24 \times 24$ px)	✓	LLM-Detektor	Alle Modelle

Anhang 19: Recall-Matrix: Violation × Bedingung (test-12-Suite)

Tab. 25: Erkennungsstatus je Violation. ✓ = konsistent erkannt (> 5/10 Runs); ~ = instabil (3–5/10 Runs); × = nicht erkannt (0–2/10 Runs).

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exklusiv
1	 ohne alt="..."	✓	✓	✓	✓	
2	<button> ohne aria-label (Icon-Button)	×	×	✓	✓	✓
3	Formularfeld ohne <label>	✓	~	✓	✓	
4	Kontrastverhältnis < 4.5 : 1	✓	✓	✓	✓	
5	Fokus-Outline entfernt (outline: none)	✓	✓	✓	✓	
6	onClick ohne onKeyDown	✓	✓	✓	✓	
7	role="button" auf <div> ohne tabIndex	✓	✓	✓	✓	
8	Modaler Dialog ohne Fokus-Trap	×	✓	✓	✓	✓
9	Skip-Link fehlt	✓	✓	✓	✓	
10	aria-hidden="true" auf fokussierbarem Element	✓	✓	✓	✓	
11	DOM-Reihenfolge weicht von visueller Reihenfolge ab	×	×	✓	✓	✓
12	Interaktives Element zu klein (< 24 × 24 px)	×	✓	✓	✓	✓
Recall		8 / 12	9 / 12	12 / 12	12 / 12	4 Violations

Anhang 20: Rohdaten: alle 30 Benchmark-Runs (test-12-Suite)

Tab. 26: Vollständige Messdaten der 30 Pipeline-Durchläufe. LLM-Det. = Findings des LLM-Detektors; Axe/PW/Grep = Tool-Findings (konstant); Tokens = Output-Tokens; Dauer = Priorisierungslaufzeit in Sekunden.

Run	Modell	LLM-Det.	Axe	PW	Grep	Gesamt	Tokens	Dauer (s)
01	qwen2.5-coder:7b	11	4	2	6	23	6144	112
02	qwen2.5-coder:7b	11	4	2	6	23	6144	134
03	qwen2.5-coder:7b	11	4	2	6	23	6144	139
04	qwen2.5-coder:7b	4	4	2	6	16	2448	53
05	qwen2.5-coder:7b	10	4	2	6	22	6144	133
06	qwen2.5-coder:7b	16	4	2	6	28	3132	76
07	qwen2.5-coder:7b	14	4	2	6	26	2744	65
08	qwen2.5-coder:7b	9	4	2	6	21	6144	137
09	qwen2.5-coder:7b	14	4	2	6	26	3108	71
10	qwen2.5-coder:7b	11	4	2	6	23	2179	51
11	qwen2.5-coder:14b	23	4	2	6	35	3090	149
12	qwen2.5-coder:14b	23	4	2	6	35	6144	285
13	qwen2.5-coder:14b	23	4	2	6	35	5395	258
14	qwen2.5-coder:14b	23	4	2	6	35	6142	291
15	qwen2.5-coder:14b	23	4	2	6	35	5254	253
16	qwen2.5-coder:14b	22	4	2	6	34	6144	290
17	qwen2.5-coder:14b	23	4	2	6	35	5403	241
18	qwen2.5-coder:14b	23	4	2	6	35	2679	127
19	qwen2.5-coder:14b	22	4	2	6	34	6144	266
20	qwen2.5-coder:14b	23	4	2	6	35	6144	267
21	qwen3:32b	19	4	2	6	31	2846	290
22	qwen3:32b	20	4	2	6	32	3153	324
23	qwen3:32b	19	4	2	6	31	4233	440
24	qwen3:32b	19	4	2	6	31	2794	300
25	qwen3:32b	20	4	2	6	32	2908	312
26	qwen3:32b	19	4	2	6	31	2649	301
27	qwen3:32b	20	4	2	6	32	2963	311
28	qwen3:32b	20	4	2	6	32	4033	398
29	qwen3:32b	20	4	2	6	32	3191	321
30	qwen3:32b	19	4	2	6	31	6144	609

Anhang 21: Modell-Leistungsvergleich: test-12-Suite

Zusammenfassung

Tab. 27: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle (je $n = 10$ Runs, `num_predict = 6144`). σ = Stichproben-Standardabweichung. Der Recall basiert auf Konsistenzschwelle $> 5/10$ Runs je Modell.

Metrik	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Recall (konsistent)	9 / 12 (75 %)	12 / 12 (100 %)	12 / 12 (100 %)
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	11,1 $\pm$ 3,28	22,8 $\pm$ 0,42	19,5 $\pm$ 0,53
LLM-Detektor-Findings (Min–Max)	4–16	22–23	19–20
Konsistenz σ	3,28	0,42	0,53
Priorisierungsqualität	Fehlerhaft	Undifferenziert	Korrekt
Output-Tokens ($\bar{O}$)	4,433	5,254	3,491
Output-Limit ausgeschöpft	5 / 10 (50 %)	4 / 10 (40 %)	1 / 10 (10 %)
Parsing-Erfolg	100 %	100 %	100 %
NFA-02-Konformität	Erfüllt	Erfüllt	Bedingt
Eignung BITV-Einsatz	Nicht geeignet	Empfohlen	Eingeschränkt
<i>Suite-spezifische Zusatzmetriken</i>			
Fix-Qualität (Stufe)	0–1	0–2	1–2
Tool-Findings (axe/PW/grep)	4 / 2 / 6	4 / 2 / 6	4 / 2 / 6
Must-have im Bericht ($\bar{O}$)	9,1	27,0	14,6
Nice-to-have ($\bar{O}$) Bericht	23,1	8,9	8,5
Gesamt-Report-Items ($\bar{O}$)	32,2	35,9	23,1
LLM-Detektor-Laufzeit ($\bar{O}$)	139 s	231 s	374 s
Priorisierungslaufzeit ($\bar{O}$)	92 s	189 s	327 s
Priorisierungslaufzeit (Min–Max)	51–139 s	127–291 s	290–609 s
Gesamtlaufzeit ($\bar{O}$)	231 s	420 s	701 s
Gesamtlaufzeit (Min–Max)	191–284 s	300–480 s	617–937 s

Anhang 22: Token-Nutzung pro Modell: Input/Output (test-12-Suite)

Tab. 28: Durchschnittliche Token-Nutzung je Run und Modell auf Basis der test-12-Suite (`num_predict = 6144`). Input-Tokens umfassen System-Prompt, serialisierte Findings und Konfigurationsparameter. 5 / 10 Runs (7b) und 4 / 10 Runs (14b) erreichten das Output-Limit; 32b-Runs blieben mit Ø 3491 deutlich darunter.

Modell	Input-Tokens (Ø)	Output-Tokens (Ø)	Truncation
qwen2.5-coder:7b	4 451	4 433	5/10 Runs (50 %)
qwen2.5-coder:14b	5 547	5 254	4/10 Runs (40 %)
qwen3:32b	5 214	3 491	1/10 Runs (10 %)
<i>Gesamt (30 Runs): Input $\approx 152\,100$ / Output 131 784 Tokens</i>			

Anhang 23: Laufzeit und Effizienz der Pipeline-Modelle (test-12-Suite)

Modell	LLM-Detektor Ø	LLM-Priorisierung Ø	Gesamt Ø	Min	Max
qwen2.5-coder:7b	139 s	92 s	231 s	191 s	284 s
qwen2.5-coder:14b	231 s	189 s	420 s	300 s	480 s
qwen3:32b	374 s	327 s	701 s	617 s	937 s

Tab. 29: Mittlere Laufzeiten je Pipeline-Phase ($n = 10$ je Modell, test-12-Suite).

Modell	LLM-Findings Ø	Dauer Ø	Findings/s	Mehrgewinn ggü. 7b
qwen2.5-coder:7b	11,1	231 s	0,048	–
qwen2.5-coder:14b	22,8	420 s	0,054	+105 % bei +82 % Zeitaufwand
qwen3:32b	19,5	701 s	0,028	+76 % bei +203 % Zeitaufwand

Tab. 30: Effizienzvergleich: LLM-Findings je Sekunde Gesamtlaufzeit (test-12-Suite).

Anhang 24: Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-12-Suite)

Tab. 31: Effizienzvergleich auf Basis der 30 Benchmark-Runs der test-12-Suite. *LLM-Findings/s* bezieht sich auf LLM-Detektor-Findings pro Sekunde Gesamtlaufzeit.

Modell	LLM-Findings Ø	Gesamtlaufzeit Ø (s)	LLM-Findings/s	Zusatznutzen ggü. 7b
qwen2.5-coder:7b	11,1	231	0,048	Referenz
qwen2.5-coder:14b	22,8	420	0,054	+105 % bei +82 % Zeitaufwand
qwen3:32b	19,5	701	0,028	+76 % bei +203 % Zeitaufwand

Anhang 25: Über-Detektion relativ zur Ground Truth (test-12-Suite)

Tab. 32: Vergleich der durchschnittlichen LLM-Detektor-Findings pro Run mit der Ground Truth (12 Violations). Die LLM-only-Betrachtung vermeidet Doppelzählungen zwischen überlappenden Detektoren.

Modell	LLM-Findings Ø	Ground Truth	Quote	Interpretation
qwen2.5-coder:7b	11,1	12	92,5 %	nahe an der Ground Truth; niedrigste LLM-Überdetektion, aber instabile Priorisierung
qwen2.5-coder:14b	22,8	12	190 %	deutlicher Überschuss; hoher Recall bei erhöhter Redundanz
qwen3:32b	19,5	12	162,5 %	moderater Überschuss; konsistenter als 7b und mit besserer Priorisierung

Anhang 26: Detektor-Konsistenz über alle 30 Runs (test-12-Suite)

Tab. 33: Konsistenzanalyse der Detektoren über alle 30 Benchmark-Runs. Da axe-core, Playwright und grep vollständig deterministisch bleiben, wird die relevante Varianz auf Modellebene innerhalb des LLM-Detektors ausgewiesen.

Detektor / Modell	Runs	Gesamt-Findings	Ø pro Run	Min-Max	σ^2 / σ	Interpretation
axe-core	30	120	4,0	4–4	0,00 / 0,00	vollständig deterministisch
Playwright	30	60	2,0	2–2	0,00 / 0,00	vollständig deterministisch
grep	30	180	6,0	6–6	0,00 / 0,00	vollständig deterministisch
qwen2.5-coder:7b	10	111	11,1	4–16	10,76 / 3,28	höchste Streuung
qwen2.5-coder:14b	10	228	22,8	22–23	0,18 / 0,42	hohe Konsistenz bei höchster Trefferzahl
qwen3:32b	10	195	19,5	19–20	0,28 / 0,53	stabil, aber mit höherer Laufzeit
LLM-Detektor gesamt	30	534	17,8	4–23	modell-abhängig	–

Die Tabelle zeigt, dass die Unterschiede zwischen den Detektoren selbst methodisch kaum relevant sind: axe-core, Playwright und grep liefern über alle 30 Runs exakt konstante Ergebnisse. Die tatsächlich relevante Varianz entsteht ausschließlich innerhalb des LLM-Detektors und dort vor allem durch die Modellwahl. **qwen2.5-coder:14b** weist mit $\sigma = 0,42$ die höchste Konsistenz bei zugleich höchster durchschnittlicher Fundzahl auf, während **qwen2.5-coder:7b** mit $\sigma = 3,28$ deutlich volatiler reagiert. **qwen3:32b** ist ebenfalls sehr stabil ($\sigma = 0,53$), erkaufte diese Stabilität jedoch mit einer wesentlich höheren Laufzeit. Für die Interpretation der Gesamtpipeline ist daher nicht die Frage entscheidend, welcher Detektor variiert, sondern welches Modell im LLM-Detektor eingesetzt wird.

Anhang 27: Bewertung der Empfehlungsqualität pro Violation (test-12-Suite)

Kürzel: DT = DataTable, SR = SearchResults, FU = FileUpload, N.= Notifications, Cal.= Calendar, Dash.= Dashboard, User = UserProfile.

Die Bewertung basiert auf Run 01 von **qwen3:32b** (Tabelle 26) und verwendet dieselbe dreistufige Skala wie Tabelle 18. Stufe 2 liegt vor, wenn Dateiname, Zeilennummer und ein konkreter Code-Vorschlag vorhanden sind; Stufe 1, wenn der richtige Fix-Typ erkannt, aber kein konkreter Codekontext geliefert wird.

Tab. 34: Bewertung der Fix-Qualität für alle 12 Violations (**qwen3:32b**, Run 01).

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
1	<code></code> ohne <code>alt="..."</code>	2	Datei App.tsx:37, konkreter <code>alt</code> -Text angegeben
2	<code><button></code> ohne <code>aria-label</code>	2	<code>aria-label</code> -Wert mit entsprechendem Text vorgeschlagen
3	Formularfeld ohne <code><label></code>	2	<code>label</code> + <code>htmlFor</code> + <code>id</code> vollständig ausgearbeitet
4	Kontrastverhältnis $< 4.5 : 1$	2	CSS-Regel mit konkreten Hex-Farben je betroffener Klasse
5	Fokus-Outline entfernt (<code>outline: none</code>)	2	CSS-Klasse mit <code>outline: 2px solid</code> als Ersatz
6	<code>onClick</code> ohne <code>onKeyDown</code>	1	<code>onKeyDown</code> -Muster korrekt, aber generisch; kein Dateikontext
7	<code>role="button"</code> auf <code><div></code> ohne <code>tabIndex</code>	2	<code>tabIndex={0}</code> + <code>onKeyDown</code> JSX-Snippet mit Kontext
8	Modal ohne Fokus-Trap	2	Vollständiger <code>useEffect</code> -Fokus-Trap mit <code>Shift+Tab</code> -Logik
9	Skip-Link fehlt	2	<code></code> -Snippet mit CSS-Klasse
10	<code>aria-hidden="true"</code> auf fokussierbarem Element	2	Korrektur (<code>aria-hidden</code> entfernen oder <code>tabIndex=-1</code>) mit Kontext
11	DOM-Reihenfolge vs. visuelle Reihenfolge	2	CSS <code>order</code> -Property als Lösung; DOM-Reihenfolge erhalten
12	Interaktives Element zu klein ($< 24 \times 24$ px)	2	CSS <code>width/height</code> auf 24px mit Klasse <code>.btn-tiny</code>

Anhang 28: CSV-Auszug: Benchmark-Rohdaten (test-12-Suite)

Die Benchmark-Rohdaten wurden zusätzlich als CSV-Datei exportiert (**results/benchmark/summary.csv**). Tabelle 35 dokumentiert das reale Spaltenschema, Tabelle 36 zeigt exemplarische Zeilen (je ein Run pro Modell).

Tab. 35: Spaltenschema des CSV-Exports **results/benchmark/summary.csv**.

Spalte	Bedeutung
model	Verwendetes Modell (z. B. qwen2.5-coder:14b)
suite	Testsuite (hier: test-12)
run	Laufnummer innerhalb einer Modellserie
success / parseSuccess	Pipeline-Erfolg und Parsing-Erfolg der Ausgabe
mustHaveCount / niceToHaveCount	Anzahl Must-have- bzw. Nice-to-have-Findings im finalen Report
durationMs	Gesamtlaufzeit der Pipeline in Millisekunden
promptTokens / outputTokens	Tatsächlich verwendete Prompt- und Output-Tokens
numPredict	Verwendetes Output-Token-Limit der Priorisierungsphase
Axe / Playwright / Grep / LLM	Anzahl erkannter Findings je Detektor
error	Fehlermeldung (leer bei erfolgreichem Run)

Tab. 36: Repräsentative Beispielzeilen aus **results/benchmark/summary.csv** (test-12-Suite; jeweils Run 01 pro Modell).

model	run	must	nice	LLM	Axe	PW	Grep	durationMs	prompt	output	parse
qwen2.5-coder:7b	1	14	33	11	4	2	6	111817	4405	6144	true
qwen2.5-coder:14b	1	6	16	23	4	2	6	148920	5635	3090	true
qwen3:32b	1	12	8	19	4	2	6	289630	5200	2846	true

Anhang 29: Ansichten der Testanwendung (test-12-Suite)

The screenshot shows a web application interface for a login window. At the top, there is a blue header bar with the text "120 x 40" on the left and two buttons, "Kontakt" and "Login", on the right. The main content area is light gray. In the center, there is a white rectangular box titled "Anmelden". Inside this box, there are two input fields: "E-Mail-Adresse" with the placeholder text "name@beispiel.de" and "Passwort eingeben". Both fields have a small red and green icon to their right. Below the password field is a button labeled "Anmelden" and a circular button with a question mark. To the right of the "Anmelden" button is a link that says "Passwort vergessen?". At the bottom of the page, there is a small gray footer bar with the text "ACE Synthetische Testkomponente – Nur für Evaluationszwecke".

Abb. 9: Anmeldefenster der test-12-Suite

Kontaktformular

Name

Ihr Name

Betreff auswählen

Allgemein Technisch

Abrechnung Sonstiges

Nachricht

Ihre Nachricht...

 Zurücksetzen

Abb. 10: Kontaktformular der test-12-Suite

Anhang 30: test-50-Suite (kompletter Block)

Tab. 37: Navigationsübersicht in der standardisierten Reihenfolge für die test-50-Suite.

Nr.	Baustein
1	Violations-Katalog
2	Erkennungsrate
3	Recall-Matrix
4	Rohdaten
5	Modellvergleich
6	Token-Nutzung
7	Effizienz
8	Über-Detektion
9	Konsistenz
10	Empfehlungsqualität
11	CSV-Auszug
12	Ansichten der Testsuite

Anhang 31: Accessibility-Verstöße der Testanwendung (test-50-Suite)

Tab. 38: Alle 50 eingebetteten Accessibility-Verstöße mit Kategorie, erwarteter Erkennungsquelle und zugehörigem WCAG-Kriterium. *LLM* = ausschließlich LLM-Codeanalyse; *manuell* = keine automatische Erkennungsquelle erwartet.

#	Violation	Kat.	Erwartete Quelle	WCAG
V1	Skip-Link fehlt	sem.	Playwright	2.4.1
V2	<html> ohne lang-Attribut	syn.	axe-core	3.1.1
V3	Logo ohne alt="..."	syn.	axe-core + grep	1.1.1
V4	Deko-Bild ohne alt= / role="presentation"	syn.	axe-core + grep	1.1.1
V5	Hamburger-Button ohne zugänglichen Namen	syn.	axe-core	4.1.2
V6	<a> ohne href – nicht fokussierbar	sem.	axe-core	2.1.1
V7	Badge-Kontrast zu niedrig (#aac auf #0057b8)	lay.	axe-core	1.4.3
V8	CSS order weicht von DOM-Reihenfolge ab	sem.	LLM-Codeanalyse	1.3.2
V9	<nav> ohne aria-label	syn.	axe-core	1.3.1
V10	onClick auf ohne onKeyDown (1)	sem.	grep	2.1.1
V11	onClick auf ohne onKeyDown (2)	sem.	grep	2.1.1
V12	role="button" ohne tabIndex (Sidebar)	sem.	axe-core + grep	4.1.2
V13	onClick auf ohne onKeyDown (3)	sem.	grep	2.1.1
V14	Button 16×16 px < 24×24 px (Sidebar)	lay.	grep (CSS)	2.5.8
V15	aria-hidden="true" auf fokussierbarem Element	syn.	axe-core	4.1.2
V16	Überschriften-Hierarchie übersprungen (h3 statt h1)	sem.	axe-core	1.3.1
V17	KPI-Icon ohne alt="..." (1)	syn.	axe-core + grep	1.1.1
V18	KPI-Wert Kontrast zu niedrig (#bbb auf #fff)	lay.	axe-core	1.4.3
V19	KPI-Icon ohne alt="..." (2)	syn.	axe-core + grep	1.1.1
V20	onClick auf <div> ohne onKeyDown	sem.	grep	2.1.1
Fortsetzung auf der nächsten Seite				

Tab. 38: Accessibility-Verstöße der test-50-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V21	<code>role="button"</code> auf <code><div></code> ohne <code>tabIndex</code>	sem.	axe-core + grep	4.1.2
V22	Chart-Bild ohne <code>alt="..."</code>	syn.	axe-core + grep	1.1.1
V23	Tabelle ohne <code><caption></code> oder <code>aria-label</code>	sem.	axe-core	1.3.1
V24	<code><th></code> ohne <code>scope</code> -Attribut	syn.	axe-core	1.3.1
V25	Status nur durch Farbe kodiert (Dashboard)	sem.	LLM-Codeanalyse (manuell)	1.4.1
V26	Avatar <code></code> ohne <code>alt="..."</code>	syn.	axe-core + grep	1.1.1
V27	Input ohne <code><label></code> (Anzeigename)	syn.	axe-core	1.3.1
V28	Input ohne <code><label></code> (E-Mail)	syn.	axe-core	1.3.1
V29	Textarea ohne <code><label></code> (Bio)	syn.	axe-core	1.3.1
V30	Pflichtfeld ohne <code>required/aria-required</code>	syn.	LLM-Codeanalyse (manuell)	3.3.2
V31	Fehlermeldung nicht programmatisch verknüpft	sem.	LLM-Codeanalyse (manuell)	3.3.1
V32	Button-Kontrast zu niedrig (#999 auf #fff)	lay.	axe-core	1.4.3
V33	<code>outline: none</code> auf Button (Fokus entfernt)	lay.	Playwright	2.4.7
V34	Statusmeldung ohne <code>aria-live</code>	sem.	LLM-Codeanalyse (manuell)	4.1.3
V35	Duplikat-ID <code>help-text</code>	syn.	axe-core	4.1.1
V36	<code>onClick</code> auf <code></code> ohne <code>onKeyDown</code>	sem.	grep	2.1.1
V37	Suchfeld ohne sichtbares Label	syn.	axe-core	1.3.1
V38	Tabelle ohne <code><caption></code> (DataTable)	sem.	axe-core	1.3.1
V39	Checkbox ohne Label (Header)	syn.	axe-core	1.3.1
V40	Checkbox ohne Label (Zeilen)	syn.	axe-core	1.3.1
V41	Status nur durch Farbe kodiert (DataTable)	sem.	LLM-Codeanalyse (manuell)	1.4.1
V42	Icon-Button ohne zugänglichen Namen	syn.	axe-core	4.1.2
V43	Paginierung: <code>onClick</code> auf <code><div></code> ohne Keyboard	sem.	grep	2.1.1
V44	<code></code> mit <code>onClick</code> ohne <code>onKeyDown</code>	sem.	grep	2.1.1
Fortsetzung auf der nächsten Seite				

Tab. 38: Accessibility-Verstöße der test-50-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V45	Ungelesen nur durch Farbe kodiert	sem.	LLM-Codeanalyse (manuell)	1.4.1
V46	Zeitangabe Kontrast zu niedrig (#bbb auf #fff)	lay.	axe-core	1.4.3
V47	Button $16 \times 16 \text{ px} < 24 \times 24 \text{ px}$ (Notif.)	lay.	grep (CSS)	2.5.8
V48	Modaler Dialog ohne Fokus-Trap	sem.	Playwright	2.1.2
V49	Dialog ohne aria-labelledby	syn.	axe-core	4.1.2
V50	outline: none im Modal-Button (Fokus entfernt)	lay.	Playwright	2.4.7

Kat.: syn. = syntaktisch, sem. = semantisch, lay. = layout.

Anhang 32: Erkennungsrate der Violations im Proof of Concept (test-50-Suite)

Tab. 39: Erkennungsrate aller 50 Violations (Konsistenzschwelle: > 5/10 Runs je Modell). *Erkannt* = konsistent über alle Modelle hinweg; modellabhängige Abweichungen sind in *Anmerkung* dokumentiert.²¹³

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V1	Skip-Link fehlt	✓	Playwright	14b/32b kons.; 7b inst. (4/10)
V2	<html> ohne lang	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V3	Logo ohne alt="..."	✓	axe-core + grep	Alle Modelle
V4	Deko-Bild ohne alt=	✓	axe-core + grep	Alle Modelle
V5	Hamburger-Button ohne Namen	✓	axe-core	Alle Modelle
V6	<a> ohne href	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V7	Badge-Kontrast zu niedrig	✓	axe-core	Alle Modelle
V8	CSS order ≠ DOM-Reihenfolge	✓	LLM-Detektor	nur 32b kons.; 14b nie erkannt; 7b inst. (3/10)
V9	<nav> ohne aria-label	✓	axe-core	Alle Modelle
V10	onClick auf o. onKeyDown (1)	✓	grep	Alle Modelle
V11	onClick auf o. onKeyDown (2)	✓	grep	Alle Modelle
V12	role="button" o. tabIndex (Sidebar)	✓	axe-core + grep	Alle Modelle
V13	onClick auf o. onKeyDown (3)	✓	grep	Alle Modelle
V14	Button < 24 px (Sidebar)	✓	grep (CSS)	Alle Modelle
V15	aria-hidden auf fokussierbarem Element	✓	axe-core	Alle Modelle
V16	Überschriften-Hierarchie übersprungen	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V17	KPI-Icon ohne alt="..." (1)	✓	axe-core + grep	14b/32b kons.; 7b inst. (5/10)
V18	KPI-Wert Kontrast zu niedrig	✓	axe-core	Alle Modelle
V19	KPI-Icon ohne alt="..." (2)	✓	axe-core + grep	14b/32b kons.; 7b inst. (5/10)
V20	onClick auf <div> o. onKeyDown	✓	grep	7b/14b kons.; 32b inst. (5/10)
Fortsetzung auf der nächsten Seite				

Tab. 39: Erkennungsrate der Violations (test-50-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V21	role="button" auf <div> o. tabIndex	✓	axe-core + grep	7b/14b kons.; 32b inst. (5/10)
V22	Chart-Bild ohne alt="..."	✓	axe-core + grep	Alle Modelle
V23	Tabelle o. <caption> (Dashboard)	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V24	<th> ohne scope	✓	axe-core	14b/32b kons.; 7b inst. (3/10)
V25	Status nur durch Farbe (Dashboard)	✓	LLM-Detektor	14b/32b kons.; 7b inst. (4/10)
V26	Avatar ohne alt="..."	✓	axe-core + grep	Alle Modelle
V27	Input ohne <label> (Anzeigename)	✓	axe-core	Alle Modelle
V28	Input ohne <label> (E-Mail)	✓	axe-core	Alle Modelle
V29	Textarea ohne <label> (Bio)	✓	axe-core	Alle Modelle
V30	Pflichtfeld ohne required/aria-req.	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V31	Fehlermeldung nicht progr. verknüpft	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V32	Button-Kontrast zu niedrig	✓	axe-core	Alle Modelle
V33	outline: none auf Button	✓	Playwright	Alle Modelle
V34	Statusmeldung ohne aria-live	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V35	Duplikat-ID help-text	✓	axe-core	7b/14b kons.; 32b inst. (5/10)
V36	onClick auf o. onKeyDown	✓	grep	32b kons.; 7b/14b inst. (4-5/10)
V37	Suchfeld ohne sichtbares Label	✓	axe-core	32b kons.; 14b inst.; 7b selten (2/10)
V38	Tabelle o. <caption> (DataTable)	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V39	Checkbox ohne Label (Header)	✓	axe-core	14b/32b kons.; 7b inst. (3/10)
V40	Checkbox ohne Label (Zeilen)	✓	axe-core	14b/32b kons.; 7b inst. (3/10)
V41	Status nur durch Farbe (DataTable)	✓	LLM-Detektor	14b/32b kons.; 7b inst. (3/10)
V42	Icon-Button ohne zugängl. Namen	✓	axe-core	14b kons.; 7b/32b inst. (3/10)
Fortsetzung auf der nächsten Seite				

Tab. 39: Erkennungsrate der Violations (test-50-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V43	Paginierung <code><div></code> ohne Keyboard	✓	grep	Alle Modelle
V44	<code></code> mit <code>onClick</code> ohne <code>onKeyDown</code>	✓	grep	Alle Modelle
V45	Ungelesen nur durch Farbe	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V46	Zeitangabe Kontrast zu niedrig	✓	axe-core	32b kons.; 14b inst.; 7b selten (1/10)
V47	Button <code>< 24 px</code> (Notif.)	✓	grep (CSS)	7b/32b kons.; 14b inst. (5/10)
V48	Modaler Dialog ohne Fokus-Trap	✓	Playwright	14b kons.; 32b inst. (5/10); 7b selten
V49	Dialog ohne <code>aria-labelledby</code>	✓	axe-core	7b kons. (8/10); 14b/32b inst. (5/10)
V50	<code>outline: none</code> im Modal-Button	✓	Playwright	32b kons.; 14b inst. (4/10); 7b selten

Anhang 33: Recall-Matrix: Violation × Bedingung (test-50-Suite)

Tab. 40: Erkennungsstatus je Violation. ✓ = konsistent erkannt (> 5/10 Runs); ~ = instabil (3–5/10 Runs); × = nicht erkannt (0–2/10 Runs).

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V1	Skip-Link fehlt	✓	~	✓	✓	
V2	<html> ohne lang	✓	~	✓	✓	
V3	Logo ohne alt="..."	✓	✓	✓	✓	
V4	Deko-Bild ohne alt=	✓	✓	✓	✓	
V5	Hamburger-Button ohne Namen	✓	✓	✓	✓	
V6	<a> ohne href	✓	~	×	✓	
V7	Badge-Kontrast zu niedrig	✓	✓	✓	✓	
V8	CSS order ≠ DOM-Reihenfolge	×	~	×	✓	✓
V9	<nav> ohne aria-label	✓	✓	✓	✓	
V10	onClick auf o. onKeyDown (1)	✓	✓	✓	✓	
V11	onClick auf o. onKeyDown (2)	✓	✓	✓	✓	
V12	role="button" o. tabIndex (Sb.)	✓	✓	✓	✓	
V13	onClick auf o. onKeyDown (3)	✓	✓	✓	✓	
V14	Button <24px (Sidebar)	✓	✓	✓	✓	
V15	aria-hidden auf fokussierbarem Element	✓	✓	✓	✓	
V16	Überschriften-Hierarchie übersprungen	✓	~	✓	✓	
V17	KPI-Icon ohne alt="..." (1)	✓	✓	✓	✓	
V18	KPI-Wert Kontrast zu niedrig	✓	✓	✓	✓	
V19	KPI-Icon ohne alt="..." (2)	✓	✓	✓	✓	
V20	onClick auf <div> o. onKeyDown	✓	✓	✓	✓	
Fortsetzung auf der nächsten Seite						

Tab. 40: Recall-Matrix test-50-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V21	role="button" auf <div> o. tabIndex	✓	✓	✓	✓	
V22	Chart-Bild ohne alt="..."	✓	✓	✓	✓	
V23	Tabelle o. <caption> (Dash- board)	✓	~	✓	✓	
V24	<th> ohne scope	✓	~	✓	✓	
V25	Status nur durch Farbe (Das- hboard)	×	✓	✓	✓	✓
V26	Avatar ohne alt="..."	✓	✓	✓	✓	
V27	Input ohne <label> (Anzei- gename)	✓	✓	✓	✓	
V28	Input ohne <label> (E-Mail)	✓	✓	✓	✓	
V29	Textarea ohne <label> (Bio)	✓	✓	✓	✓	
V30	Pflichtfeld ohne required/aria-req.	×	✓	✓	✓	✓
V31	Fehlermeldung nicht pro- gr. verknüpft	×	✓	✓	✓	✓
V32	Button-Kontrast zu niedrig	✓	✓	✓	✓	
V33	outline: none auf Button	✓	✓	✓	✓	
V34	Statusmeldung ohne aria-live	×	✓	✓	✓	✓
V35	Duplikat-ID help-text	✓	✓	✓	✓	
V36	onClick auf o. onKeyDown	✓	~	✓	✓	
V37	Suchfeld ohne sichtbares La- bel	✓	~	✓	✓	
V38	Tabelle o. <caption> (Data- Table)	✓	~	✓	✓	
V39	Checkbox ohne Label (Hea- der)	✓	~	✓	✓	
V40	Checkbox ohne Label (Zeilen)	✓	~	✓	✓	
V41	Status nur durch Farbe (Data- Table)	×	~	✓	✓	✓
Fortsetzung auf der nächsten Seite						

Tab. 40: Recall-Matrix test-50-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V42	Icon-Button ohne zugängl. Namen	✓	~	✓	✓	
V43	Paginierung <div> ohne Keyboard	✓	✓	✓	✓	
V44	mit onClick o. onKeyDown	✓	✓	✓	✓	
V45	Ungelesen nur durch Farbe	×	✓	✓	✓	✓
V46	Zeitangabe Kontrast zu niedrig	✓	~	✓	✓	
V47	Button < 24 px (Notif.)	✓	✓	✓	✓	
V48	Modaler Dialog ohne Fokus-Trap	✓	✓	✓	✓	
V49	Dialog ohne aria-labelledby	✓	✓	✓	✓	
V50	outline: none im Modal-Button	✓	✓	✓	✓	
Recall		43 / 50	36 / 50	48 / 50	50 / 50	7 Violations

Hinweis: Die Werte 14/2/21 sind Detektor-*Finding*-Counts (axe/Playwright/grep), während 43/50 die Abdeckung der 50 definierten Ground-Truth-*Violations* angibt; beide Kennzahlen sind daher nicht direkt gegeneinander austauschbar.

Anhang 34: Rohdaten: alle 30 Benchmark-Runs (test-50-Suite)

Tab. 41: Vollständige Messdaten der 30 Pipeline-Durchläufe (`num_predict` = 8192). LLM-Det. = Findings des LLM-Detektors; Axe/PW/Grep = Tool-Findings (konstant); Tokens = Output-Tokens; Dauer = Priorisierungslaufzeit in Sekunden.

Run	Modell	LLM-Det.	Axe	PW	Grep	Must	Nice	Tokens	Dauer (s)
01	qwen2.5-coder:7b	47	14	2	21	10	17	3409	123
02	qwen2.5-coder:7b	47	14	2	21	14	51	8192	228
03	qwen2.5-coder:7b	47	14	2	21	27	41	8192	232
04	qwen2.5-coder:7b	47	14	2	21	26	34	8192	234
05	qwen2.5-coder:7b	40	14	2	21	10	4	1996	75
06	qwen2.5-coder:7b	44	14	2	21	13	59	8192	236
07	qwen2.5-coder:7b	47	14	2	21	24	49	8192	241
08	qwen2.5-coder:7b	47	14	2	21	24	23	5683	180
09	qwen2.5-coder:7b	53	14	2	21	54	10	7291	205
10	qwen2.5-coder:7b	47	14	2	21	13	68	8192	224
11	qwen2.5-coder:14b	45	14	2	21	50	8	8192	457
12	qwen2.5-coder:14b	52	14	2	21	14	48	8192	462
13	qwen2.5-coder:14b	52	14	2	21	14	45	8192	458
14	qwen2.5-coder:14b	52	14	2	21	14	48	8192	480
15	qwen2.5-coder:14b	52	14	2	21	14	45	8192	473
16	qwen2.5-coder:14b	52	14	2	21	59	0	8192	465
17	qwen2.5-coder:14b	52	14	2	21	14	33	6014	352
18	qwen2.5-coder:14b	52	14	2	21	14	34	6239	361
19	qwen2.5-coder:14b	52	14	2	21	14	47	8192	459
20	qwen2.5-coder:14b	52	14	2	21	14	49	8046	470
21	qwen3:32b	60	14	2	21	36	38	8192	1084
22	qwen3:32b	60	14	2	21	37	46	8192	1030
23	qwen3:32b	62	14	2	21	66	0	8192	1014
24	qwen3:32b	60	14	2	21	66	12	8192	1004
25	qwen3:32b	63	14	2	21	73	7	8192	1008
26	qwen3:32b	60	14	2	21	49	22	8192	996
27	qwen3:32b	61	14	2	21	32	46	8192	998
28	qwen3:32b	60	14	2	21	30	36	7179	885
29	qwen3:32b	62	14	2	21	49	30	8192	995
30	qwen3:32b	59	14	2	21	38	36	8192	987

Anhang 35: Modell-Leistungsvergleich: test-50-Suite

Zusammenfassung

Tab. 42: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle (je $n = 10$ Runs, `num_predict = 8192`). σ = Stichproben-Standardabweichung. Der Recall basiert auf Konsistenzschwelle $> 5/10$ Runs je Modell.

Metrik	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Recall (konsistent)	36 / 50 (72 %)	48 / 50 (96 %)	50 / 50 (100 %)
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	46,6 $\pm$ 3,20	51,3 $\pm$ 2,21	60,7 $\pm$ 1,25
LLM-Detektor-Findings (Min–Max)	40–53	45–52	59–63
Konsistenz σ	3,20	2,21	1,25
Priorisierungsqualität	Instabil	Bimodal	Plausibel
Output-Tokens ($\bar{O}$)	6 753	7 764	8 091
Output-Limit ausgeschöpft	6 / 10 (60 %)	7 / 10 (70 %)	9 / 10 (90 %)
Parsing-Erfolg	100 %	100 %	100 %
NFA-02-Konformität	Erfüllt	Erfüllt	Nicht erfüllt
Eignung BITV-Einsatz	Nicht geeignet	Empfohlen	Eingeschränkt
<i>Suite-spezifische Zusatzmetriken</i>			
Fix-Qualität (Stufe)	0–1	0–2	1–2
Tool-Findings (axe/PW/grep)	14 / 2 / 21	14 / 2 / 21	14 / 2 / 21
Must-have im Bericht ($\bar{O}$)	21,5	22,1	47,6
Nice-to-have ($\bar{O}$) Bericht	35,6	35,7	27,3
Gesamt-Report-Items ($\bar{O}$)	57,1	57,8	74,9
LLM-Detektor-Laufzeit ($\bar{O}$)	215 s	382 s	897 s
Priorisierungslaufzeit ($\bar{O}$)	198 s	444 s	1 000 s
Priorisierungslaufzeit (Min–Max)	75–241 s	352–480 s	885–1 084 s
Gesamtlaufzeit ($\bar{O}$)	413 s	826 s	1 897 s
Gesamtlaufzeit (Min–Max)	291–465 s	733–871 s	1 752–2 011 s

Anhang 36: Token-Nutzung pro Modell: Input/Output (test-50-Suite)

Tab. 43: Durchschnittliche Token-Nutzung je Run und Modell auf Basis der test-50-Suite (`num_predict` = 8 192). Input-Tokens umfassen System-Prompt, serialisierte Findings und Konfigurationsparameter. Der höhere Input gegenüber test-12 reflektiert die größere Anzahl serialisierter Violations.

Modell	Input-Tokens (Ø)	Output-Tokens (Ø)	Truncation
qwen2.5-coder:7b	13 196	6 753	6 / 10 Runs (60 %)
qwen2.5-coder:14b	13 556	7 764	7 / 10 Runs (70 %)
qwen3:32b	14 231	8 091	9 / 10 Runs (90 %)
<i>Gesamt (30 Runs): Input $\approx$ 409 500 / Output 226 740 Tokens</i>			

Anhang 37: Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-50-Suite)

Tab. 44: Effizienzvergleich auf Basis der 30 Benchmark-Runs der test-50-Suite. *LLM-Findings/s* bezieht sich auf LLM-Detektor-Findings pro Sekunde Gesamtlaufzeit.

Modell	LLM-Findings Ø	Gesamtlaufzeit Ø (s)	LLM-Findings/s	Zusatznutzen ggü. 7b
qwen2.5-coder:7b	46,6	413	0,113	Referenz
qwen2.5-coder:14b	51,3	826	0,062	+10,1 % bei +99 % Zeitaufwand
qwen3:32b	60,7	1897	0,032	+30,3 % bei +359 % Zeitaufwand

Anhang 38: Über-Detektion relativ zur Ground Truth (test-50-Suite)

Tab. 45: Vergleich der durchschnittlichen LLM-Detektor-Findings pro Run mit der Ground Truth (50 Violations). Die LLM-only-Betrachtung vermeidet Doppelzählungen zwischen überlappenden Detektoren.

Modell	LLM-Findings $\bar{O}$	Ground Truth	Quote	Interpretation
qwen2.5-coder:7b	46,6	50	93,2 %	nahe an der Ground Truth; LLM liefert teils Underdetektion bei einzelnen Runs
qwen2.5-coder:14b	51,3	50	102,6 %	sehr nahe an der Ground Truth; geringste Abweichung im Mittel
qwen3:32b	60,7	50	121,4 %	erhöhter Überschuss; zugleich konsistenteste LLM-Detektion ($\sigma \approx 1,25$)

Anhang 39: Precision und F1-Score je Run (test-50-Suite)

Per-Run-Auflistung der Precision für alle 30 Benchmark-Runs auf der test-50-Suite. Precision gemessen auf Item-Ebene: Ein Report-Item gilt als True Positive (TP), wenn es per Modell-ID-Präfix einem Tool-Fund (axe, Playwright, grep) zugeordnet werden kann oder per Inhalt auf ein WCAG-Kriterium der Ground Truth verweist; andernfalls als False Positive (FP). Konsistenz-Recall und F1 beziehen sich auf die modellübergreifenden Werte aus Tabelle 9.

Tab. 46: Precision je Run auf test-50 für alle drei evaluierten Modelle ($n = 10$ Runs je Modell). TP = True-Positive-Items, FP = False-Positive-Items (keine nachweisbare Entsprechung in der Ground Truth). Modell-Gesamtwerte in der letzten Zeile.

Run	qwen2.5-coder:7b			qwen2.5-coder:14b			qwen3:32b		
	Items	FP	Prec. %	Items	FP	Prec. %	Items	FP	Prec. %
01	27	0	100,0	58	0	100,0	74	0	100,0
02	65	3	95,4	62	0	100,0	83	0	100,0
03	68	0	100,0	59	2	96,6	66	0	100,0
04	60	1	98,3	62	0	100,0	78	0	100,0
05	14	0	100,0	59	0	100,0	80	0	100,0
06	72	0	100,0	59	0	100,0	71	0	100,0
07	73	0	100,0	47	3	93,6	78	1	98,7
08	47	0	100,0	48	3	93,8	66	0	100,0
09	64	0	100,0	61	0	100,0	79	1	98,7
10	81	3	96,3	63	2	96,8	74	1	98,6
Ø	571	7	99,0	578	10	98,1	749	3	99,6
Kons.-Recall: 72 % F1: 83,4 % Kons.-Recall: 96 % F1: 97,0 % Kons.-Recall: 100 % F1: 99,8 %									

Anhang 40: Detektor-Konsistenz über alle 30 Runs (test-50-Suite)

Tab. 47: Konsistenzanalyse der Detektoren über alle 30 Benchmark-Runs. Die festen Detektoren bleiben vollständig deterministisch; die LLM-Varianz ist modellabhängig.

Detektor / Modell	Runs	Gesamt-Findings	Ø pro Run	Min-Max	σ^2 / σ	Interpretation
axe-core	30	420	14,0	14–14	0,00 / 0,00	vollständig deterministisch
Playwright	30	60	2,0	2–2	0,00 / 0,00	vollständig deterministisch
grep	30	630	21,0	21–21	0,00 / 0,00	vollständig deterministisch
qwen2.5-coder:7b	10	466	46,6	40–53	10,27 / 3,20	höchste Streuung
qwen2.5-coder:14b	10	513	51,3	45–52	4,90 / 2,21	gute Konsistenz (1 Ausreißer)
qwen3:32b	10	607	60,7	59–63	1,57 / 1,25	höchste Konsistenz aller Modelle
LLM-Detektor gesamt	30	1586	52,9	40–63	modellabhängig	–

Anhang 41: Bewertung der Empfehlungsqualität pro Violation (test-50-Suite)

Kürzel: DT = DataTable, SR = SearchResults, FU = FileUpload, N.= Notifications, Cal.= Calendar, Dash.= Dashboard, User = UserProfile.

Die Bewertung basiert auf Run 01 von **qwen3:32b** (Tabelle 41) und verwendet dieselbe dreistufige Skala wie Tabelle 18. Stufe 2 liegt vor, wenn Dateiname, Zeilennummer und ein konkreter Code-Vorschlag vorhanden sind; Stufe 1, wenn der richtige Fix-Typ erkannt, aber kein konkreter Codekontext geliefert wird.

Tab. 48: Bewertung der Fix-Qualität für alle 50 Violations (**qwen3:32b**, Run 01).

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V1	Skip-Link fehlt	2	App.tsx, <code></code> -Snippet mit CSS-Klasse
V2	<code><html></code> ohne <code>lang</code>	2	index.html, <code><html lang="de"></code> direkt angegeben
V3	Logo <code></code> ohne <code>alt="..."</code>	2	App.tsx:29, beschreibender <code>alt</code> -Text vorgeschlagen
V4	Deko-Bild ohne <code>alt=/role</code>	2	App.tsx:32, <code>alt=role="presentation"</code> korrekt
V5	Hamburger-Button ohne Namen	2	App.tsx:36, <code>aria-label="Menü öffnen"</code> mit vollst. JSX
V6	<code><a></code> ohne <code>href</code>	2	App.tsx:41, <code>href="/help"</code> mit konkretem Pfad
V7	Badge-Kontrast	2	App.tsx:44, CSS <code>.header-badge</code> mit Hex-Farben
V8	CSS <code>order</code> vs. DOM-Reihenfolge	1	App.tsx:48; nur allgemeiner Hinweis („Vermeide CSS order“), kein konkreter Code
V9	<code><nav></code> ohne <code>aria-label</code>	2	Sidebar.tsx:11, <code>aria-label="Hauptnavigation"</code>
V10	<code>onClick</code> li ohne <code>onKeyDown</code> (1)	2	Sidebar.tsx:14, vollst. <code>onKeyDown</code> -Handler mit Enter-Logik
V11	<code>onClick</code> li ohne <code>onKeyDown</code> (2)	2	Sidebar.tsx:21, Querverweis auf V10-Fix
Fortsetzung auf der nächsten Seite			

Tab. 48: Bewertung der Empfehlungsqualität pro Violation (test-50) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V12	<code>role="button"</code> ohne <code>tabIndex</code>	2	Sidebar.tsx:30, <code>tabIndex={0}</code> + vollst. JSX-Snippet
V13	<code>onClick</code> li ohne <code>onKeyDown</code> (3)	2	Sidebar.tsx:38, Querverweis auf V10-Fix
V14	Button 16×16 px Sidebar	2	styles.css, <code>.btn-tiny-sidebar</code> auf 32×32 px mit CSS
V15	<code>aria-hidden</code> auf fokus. Element	2	Sidebar.tsx:55, <code>aria-hidden</code> entfernt; vollst. JSX
V16	Überschriften-Hierarchie	2	Dashboard.tsx:7, <code><h1></code> -Ersatz mit Dateikontext
V17	KPI-Icon ohne <code>alt="..."</code> (1)	2	Dashboard.tsx:12, beschreibender <code>alt</code> -Text
V18	KPI-Wert Kontrast	2	Dashboard.tsx:16, CSS-Fix mit Hex-Farben
V19	KPI-Icon ohne <code>alt="..."</code> (2)	2	Dashboard.tsx:22, Querverweis auf V17-Fix
V20	<code>onClick</code> div ohne <code>onKeyDown</code>	2	Dashboard.tsx:30, <code>onKeyDown</code> + <code>tabIndex</code> vollst.
V21	<code>role="button"</code> div ohne <code>tabIndex</code>	2	Dashboard.tsx:38, <code>tabIndex={0}</code> mit JSX-Snippet
V22	Chart-Bild ohne <code>alt="..."</code>	2	Dashboard.tsx:49, inhaltsbeschreibender <code>alt</code> -Text
V23	Tabelle ohne <code>caption/aria-label</code>	2	Dashboard.tsx:53, <code>aria-label="Dashboard-Daten"</code>
V24	<code><th></code> ohne <code>scope</code>	2	Dashboard.tsx:57, <code>scope="col"</code> ergänzt
V25	Status nur Farbe (Dashboard) (<i>manuell</i>)	2	styles.css:247, CSS <code>::after</code> mit Textalternative
V26	Avatar ohne <code>alt="..."</code>	2	UserProfile.tsx:17, <code>alt="Profilbild des Nutzers"</code>
V27	Input ohne <code><label></code> (Name)	2	UserProfile.tsx:20, <code>htmlFor="name"</code> + id vollst.
V28	Input ohne <code><label></code> (E-Mail)	2	UserProfile.tsx:32, Querverweis auf V27-Fix
Fortsetzung auf der nächsten Seite			

Tab. 48: Bewertung der Empfehlungsqualität pro Violation (test-50) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V29	Textarea ohne <code><label></code>	2	UserProfile.tsx:44, Querverweis auf V27-Fix
V30	Pflichtfeld ohne <code>required</code> (<i>manuell</i>)	2	UserProfile.tsx:56, <code>required aria-required="true"</code>
V31	Fehlermeldung nicht verknüpft (<i>manuell</i>)	2	UserProfile.tsx:64, <code>aria-describedby-Snippet</code>
V32	Button-Kontrast	2	UserProfile.tsx:73, CSS-Fix mit Hex-Farben
V33	<code>outline: none</code> Button	2	UserProfile.tsx:78, <code>outline: "2px solid #000"</code>
V34	Statusmeldung ohne <code>aria-live</code> (<i>manuell</i>)	2	UserProfile.tsx:83, <code>aria-live="polite"</code>
V35	Duplikat-ID <code>help-text</code>	2	UserProfile.tsx:86, <code>id="help-text-1"/-2</code>
V36	<code>onClick</code> span ohne <code>onKeyDown</code>	2	DataTable.tsx:24, Umwandlung in <code><button></code> -Element
V37	Suchfeld ohne Label	2	DataTable.tsx:29, <code><label htmlFor> + id</code>
V38	Tabelle ohne <code><caption></code>	2	DataTable.tsx:33, <code><caption></code> -Element
V39	Checkbox ohne Label (Header)	2	DataTable.tsx:37, <code>htmlFor + id</code> vollst.
V40	Checkbox ohne Label (Zeilen)	2	DataTable.tsx:50, Querverweis auf V39-Fix
V41	Status nur Farbe (DataTable) (<i>manuell</i>)	2	DataTable.tsx:63 + styles.css:405, CSS <code>::after</code>
V42	Icon-Button ohne Namen	2	DataTable.tsx:71, <code>aria-label="Bearbeiten"</code>
V43	Paginierung <code><div></code> ohne Keyboard	2	DataTable.tsx:80, Umwandlung in <code><button></code>
V44	<code></code> <code>onClick</code> ohne <code>onKeyDown</code>	2	Notifications.tsx:21, <code>onKeyDown-Enter-Snippet</code>
V45	Ungelesen nur Farbe (<i>manuell</i>)	2	styles.css:485, CSS <code>::after</code> mit Textalternative
Fortsetzung auf der nächsten Seite			

Tab. 48: Bewertung der Empfehlungsqualität pro Violation (test-50) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V46	Zeitangabe Kontrast	2	Notifications.tsx:34, CSS-Fix mit Dateikontext
V47	Button 16×16 px Notifications	2	styles.css + Notifications.tsx:41, CSS 32×32 px
V48	Modaler Dialog ohne Fokus-Trap	2	Notifications.tsx:46, useEffect-Fokus-Snippet
V49	Dialog ohne aria-labelledby	2	Notifications.tsx:49, aria-labelledby="modal-title"
V50	outline: none Modal-Button	2	Notifications.tsx:54, autoFocus + korrekter Button
Gesamt Stufe 2		49 / 50	98 % konkrete Fixes mit Dateikon-text

Anhang 42: CSV-Auszug: Benchmark-Rohdaten (test-50-Suite)

Tab. 49: Repräsentative Beispielzeilen aus `results-50/benchmark/summary.csv` (test-50-Suite; jeweils Run 01 pro Modell). Spaltenschema analog zu Tabelle 35.

model	run	must	nice	LLM	Axe	PW	Grep	durationMs	prompt	output	parse
qwen2.5-coder:7b	1	10	17	47	14	2	21	123084	13109	3409	true
qwen2.5-coder:14b	1	50	8	45	14	2	21	456526	13053	8192	true
qwen3:32b	1	36	38	60	14	2	21	1083991	14171	8192	true

Anhang 43: Ansichten der Testanwendung (test-50-Suite)

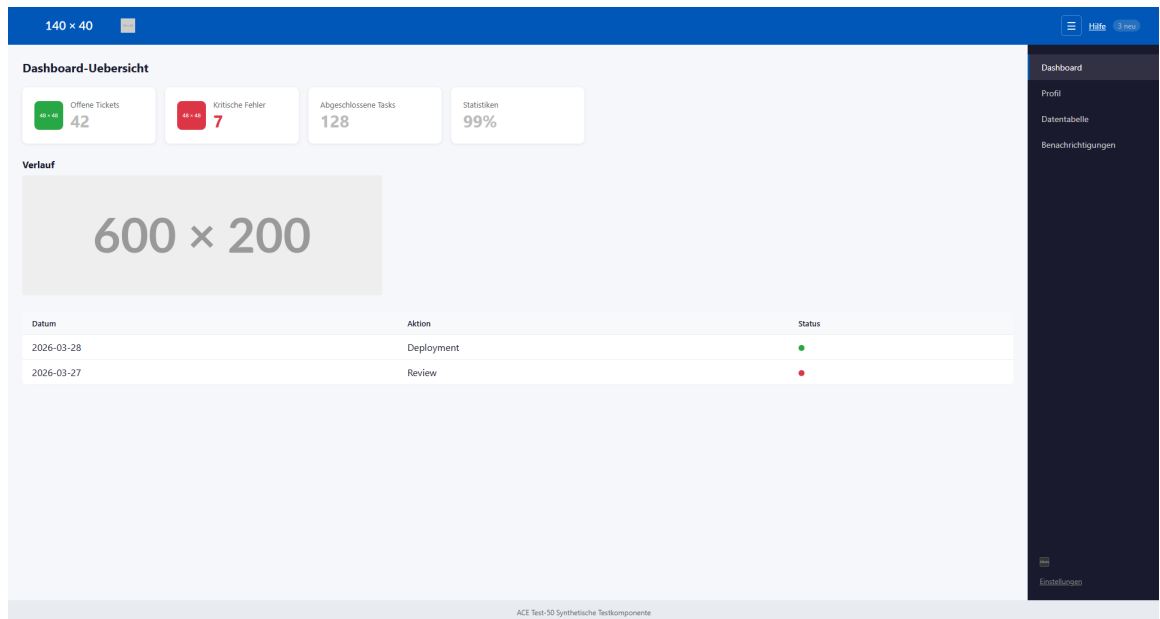


Abb. 11: Dashboard-Übersicht der test-50-Suite

140 x 40

Hilfe

Profil bearbeiten

80 x 80

Anzeigename
Max Mustermann

E-Mail
max@bitbw.de

Bio
Über mich...

Abteilung *
Pflichtfeld

Telefon
Bitte gültige Nummer eingeben

Speichern

Abbrechen

Dashboard

Profil

Datentabelle

Benachrichtigungen

Einstellungen

ACE Test-50 Synthetische Testkomponente

Abb. 12: Profil-Tab (Profil bearbeiten) der test-50-Suite

140 x 40

Serverstatus

Sortierung: A-Z Suchen ...

☐	Name	Status	Last (%)	Aktionen
☐	Server Alpha	●	42	
☐	Server Beta	●	0	
☐	Server Delta	●	15	
☐	Server Gamma	●	87	

◀ Zurück 1 2 Weiter ▶

ACE Test-50 Synthetische Testkomponente

Dashboard
Profil
Datentabelle
Benachrichtigungen

Benachrichtigungen

Abb. 13: Serverstatus der test-50-Suite

140 x 40

Benachrichtigungen

- Deployment abgeschlossen vor 3 Std.
- Neuer Kommentar vor 1 Std.
- Sicherheitswarnung vor 3 Std.

ACE Test-50 Synthetische Testkomponente

Dashboard
Profil
Datentabelle
Benachrichtigungen

Abb. 14: Benachrichtigungen der test-50-Suite

Anhang 44: test-100-Suite (kompletter Block)

Tab. 50: Navigationsübersicht in der standardisierten Reihenfolge für die test-100-Suite.

Nr.	Baustein
1	Violations-Katalog
2	Erkennungsrate
3	Recall-Matrix
4	Rohdaten
5	Modellvergleich
6	Token-Nutzung
7	Effizienz
8	Über-Detektion
9	Konsistenz
10	Empfehlungsqualität
11	CSV-Auszug
12	Pareto-Diagramm
13	Boxplot Laufzeiten
14	Pipeline-Diagramm
15	Ansichten der Testsuite

Anhang 45: Accessibility-Verstöße der Testanwendung (test-100-Suite)

Tab. 51: Alle 100 eingebetteten Accessibility-Verstöße mit Kategorie, erwarteter Erkennungsquelle und zugehörigem WCAG-Kriterium. *LLM* = ausschließlich LLM-Codeanalyse; *manuell* = keine automatische Erkennungsquelle erwartet.

#	Violation	Kat.	Erwartete Quelle	WCAG
V1	Skip-Link fehlt	sem.	Playwright	2.4.1
V2	<html> ohne lang	syn.	axe-core	3.1.1
V3	Logo ohne alt="..."	syn.	axe-core + grep	1.1.1
V4	Deko-Bild ohne alt=	syn.	axe-core + grep	1.1.1
V5	Button ohne zugänglichen Namen	syn.	axe-core	4.1.2
V6	Badge Kontrast zu niedrig	lay.	axe-core	1.4.3
V7	<a> ohne href	sem.	axe-core	2.1.1
V8	CSS order weicht von DOM ab	sem.	axe-core	1.3.2
V9	<nav> ohne aria-label	syn.	axe-core	1.3.1
V10	onClick auf o. onKeyDown	sem.	grep	2.1.1
V11	Headings übersprungen (h3)	sem.	axe-core	1.3.1
V12	KPI-Icon ohne alt="..."	syn.	axe-core + grep	1.1.1
V13	KPI-Wert Kontrast niedrig	lay.	axe-core	1.4.3
V14	KPI-Icon ohne alt="..."	syn.	axe-core + grep	1.1.1
V15	onClick auf <div> o. onKeyDown	sem.	grep	2.1.1
V16	role="button" ohne tabIndex	sem.	axe-core + grep	4.1.2
V17	Chart ohne alt="..."	syn.	axe-core + grep	1.1.1
V18	Tabelle ohne <caption>	sem.	axe-core	1.3.1
V19	<th> ohne scope	syn.	axe-core	1.3.1
V20	Status nur durch Farbe	sem.	manuell	1.4.1
V21	Avatar ohne alt="..."	syn.	axe-core + grep	1.1.1
V22	Input ohne <label>	syn.	axe-core	1.3.1
V23	Input ohne <label>	syn.	axe-core	1.3.1
V24	Textarea ohne <label>	syn.	axe-core	1.3.1
V25	Pflichtfeld ohne required	syn.	manuell	3.3.2
V26	Fehlermeldung nicht verknüpft	sem.	manuell	3.3.1
V27	Button Kontrast niedrig	lay.	axe-core	1.4.3
V28	outline: none auf Button	lay.	Playwright	2.4.7

Fortsetzung auf der nächsten Seite

Tab. 51: Accessibility-Verstöße der test-100-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V29	Statusmeldung ohne <code>aria-live</code>	sem.	manuell	4.1.3
V30	Duplikat-ID	syn.	axe-core	4.1.1
V31	<code>onClick</code> auf <code></code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V32	Suchfeld ohne Label	syn.	axe-core	1.3.1
V33	Tabelle ohne <code><caption></code>	sem.	axe-core	1.3.1
V34	Checkbox ohne Label	syn.	axe-core	1.3.1
V35	Checkbox ohne Label (Zeilen)	syn.	axe-core	1.3.1
V36	Status nur durch Farbe	sem.	manuell	1.4.1
V37	Icon-Button ohne Namen	syn.	axe-core	4.1.2
V38	Paginierung <code>onClick</code> o. Keyboard	sem.	grep	2.1.1
V39	Paginierung <code>onClick</code> o. Keyboard	sem.	grep	2.1.1
V40	Paginierung <code>onClick</code> o. Keyboard	sem.	grep	2.1.1
V41	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V42	Ungelesen nur durch Farbe	sem.	manuell	1.4.1
V43	Kontrast Zeitangabe niedrig	lay.	axe-core	1.4.3
V44	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V45	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V46	Button 16×16 px < 24×24 px	lay.	grep (CSS)	2.5.8
V47	Modal ohne Fokus-Trap	sem.	Playwright	2.1.2
V48	Dialog ohne <code>aria-labelledby</code>	syn.	axe-core	4.1.2
V49	<code>outline: none</code> auf Button	lay.	Playwright	2.4.7
V50	<code>aria-hidden</code> auf fokussierbarem	syn.	axe-core	4.1.2
V51	Tab <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V52	Tab <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V53	Tab <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V54	Toggle ohne zugänglichen Namen	syn.	axe-core	4.1.2
V55	Select ohne Label	syn.	axe-core	1.3.1
V56	Input ohne Label	syn.	axe-core	1.3.1
V57	Passwortfeld ohne Label	syn.	axe-core	1.3.1
V58	Passwortfeld ohne Label	syn.	axe-core	1.3.1
V59	Button Kontrast niedrig	lay.	axe-core	1.4.3
V60	Checkbox ohne Label	syn.	axe-core	1.3.1
V61	Suchfeld ohne Label	syn.	axe-core	1.3.1
Fortsetzung auf der nächsten Seite				

Tab. 51: Accessibility-Verstöße der test-100-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V62	Button ohne Namen	syn.	axe-core	4.1.2
V63	Filter-Chips <code>onClick</code> o. <code>Keyboard</code>	sem.	grep	2.1.1
V64	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V65	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V66	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V67	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V68	Typ-Badge nur durch Farbe	sem.	manuell	1.4.1
V69	Kontrast Datum niedrig	lay.	axe-core	1.4.3
V70	Paginierung <code>role="button"</code> o. <code>tabIndex</code>	sem.	axe-core + grep	4.1.2
V71	Dropzone <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V72	Upload-Icon ohne <code>alt="..."</code>	syn.	axe-core + grep	1.1.1
V73	File-Input ohne Label	syn.	axe-core	1.3.1
V74	Löschen-Button ohne Namen	syn.	axe-core	4.1.2
V75	Progressbar ohne <code>aria-valuenow</code>	syn.	axe-core	4.1.2
V76	Kontrast Hilfetext niedrig	lay.	axe-core	1.4.3
V77	Button $16 \times 16 \text{ px} < 24 \times 24 \text{ px}$	lay.	grep (CSS)	2.5.8
V78	Modal ohne Fokus-Trap	sem.	Playwright	2.1.2
V79	Dialog ohne <code>aria-labelledby</code>	syn.	axe-core	4.1.2
V80	<code>outline: none</code> auf Button	lay.	Playwright	2.4.7
V81	Tabelle ohne <code><caption></code>	sem.	axe-core	1.3.1
V82	<code><th></code> ohne <code>scope</code>	syn.	axe-core	1.3.1
V83	<code><td> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V84	Event nur durch Farbe	sem.	manuell	1.4.1
V85	<code>role="button"</code> ohne <code>tabIndex</code>	sem.	axe-core + grep	4.1.2
V86	Detail ohne <code>aria-live</code>	sem.	manuell	4.1.3
V87	Banner-Bild ohne <code>alt="..."</code>	syn.	axe-core + grep	1.1.1
V88	Chat ohne <code>aria-live</code>	sem.	manuell	4.1.3
V89	Kontrast Zeitstempel niedrig	lay.	axe-core	1.4.3
V90	Chat-Input ohne Label	syn.	axe-core	1.3.1
V91	Senden-Button ohne Namen	syn.	axe-core	4.1.2
V92	Emoji <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V93	Online-Status nur durch Farbe	sem.	manuell	1.4.1
Fortsetzung auf der nächsten Seite				

Tab. 51: Accessibility-Verstöße der test-100-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V94	Kontrast Hilfetext niedrig	lay.	axe-core	1.4.3
V95	Suchfeld ohne Label	syn.	axe-core	1.3.1
V96	Accordion <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V97	Accordion ohne <code>aria-expanded</code>	sem.	manuell	4.1.2
V98	Kontaktbild ohne <code>alt="..."</code>	syn.	axe-core + grep	1.1.1
V99	Kontrast Kontaktinfo niedrig	lay.	axe-core	1.4.3
V100	<code>aria-hidden</code> auf fokussierbarem	syn.	axe-core	4.1.2

Kat.: syn. = syntaktisch, sem. = semantisch, lay. = layout.

Anhang 46: Erkennungsrate der Violations im Proof of Concept (test-100-Suite)

Tab. 52: Erkennungsrate aller 100 Violations (Gesamterkennungsquote durch Vollpipeline). *Erkannt* = konsistent über alle Modelle hinweg; modellabhängige Abweichungen sind in *Anmerkung* dokumentiert.²¹⁴

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V1	Skip-Link fehlt	✓	Playwright	Alle Modelle
V2	<html> ohne lang	✓	axe-core	Alle Modelle
V3	Logo ohne alt="..."	✓	axe-core	Alle Modelle
V4	Deko-Bild ohne alt=	✓	axe-core	Alle Modelle
V5	Button ohne zugängl. Namen	✓	axe-core	Alle Modelle
V6	Badge Kontrast zu niedrig	✓	axe-core	Alle Modelle
V7	<a> ohne href	✓	LLM-Detektor	nur 32b kons.; 7b/14b inst.
V8	CSS order weicht von DOM ab	×	–	nicht erkannt
V9	<nav> ohne aria-label	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V10	onClick o. onKeyDown	✓	grep	Alle Modelle
V11	Headings übersprungen (h3)	×	–	nicht erkannt
V12	KPI-Icon ohne alt="..." (1)	✓	axe-core	Alle Modelle
V13	KPI-Wert Kontrast niedrig	✓	axe-core	Alle Modelle
V14	KPI-Icon ohne alt="..." (2)	✓	axe-core	Alle Modelle
V15	onClick <div> o. onKeyDown	✓	grep	Alle Modelle
V16	role="button" o. tabIndex (D.)	✓	grep	Alle Modelle
V17	Chart ohne alt="..."	✓	axe-core	Alle Modelle
V18	Tabelle ohne <caption> (Dash.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V19	<th> ohne scope (Dash.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V20	Status nur durch Farbe (Dash.)	✓	LLM-Detektor	nur 32b kons.; manuell
V21	Avatar ohne alt="..."	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V22	Input ohne <label> (Anzeige)	×	–	nicht erkannt
Fortsetzung auf der nächsten Seite				

Tab. 52: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V23	Input ohne <label> (E-Mail)	×	–	nicht erkannt
V24	Textarea ohne <label>	×	–	nicht erkannt
V25	Pflichtfeld ohne required	×	–	nicht erkannt (manuell)
V26	Fehlermeldung nicht verknüpft	×	–	nicht erkannt (manuell)
V27	Button Kontrast niedrig (User)	×	–	nicht erkannt
V28	outline: none auf Button (User)	×	–	nicht erkannt
V29	Statusmeldung ohne aria-live	✓	LLM-Detektor	nur 32b kons.; manuell
V30	Duplikat-ID	✓	LLM-Detektor	Alle Modelle (LLM)
V31	onClick o. onKeyDown	✓	grep	Alle Modelle
V32	Suchfeld ohne Label (DT)	×	–	nicht erkannt
V33	Tabelle ohne <caption> (DT)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V34	Checkbox ohne Label (Header)	×	–	nicht erkannt
V35	Checkbox ohne Label (Zeilen)	×	–	nicht erkannt
V36	Status nur durch Farbe (DT)	✓	LLM-Detektor	nur 32b kons.; manuell
V37	Icon-Button ohne Namen (DT)	✓	LLM-Detektor	Alle Modelle (LLM)
V38	Paginierung onClick o. Keyboard (1)	✓	grep	Alle Modelle
V39	Paginierung onClick o. Keyboard (2)	✓	grep	Alle Modelle
V40	Paginierung onClick o. Keyboard (3)	✓	grep	Alle Modelle
V41	onClick o. onKeyDown (N.)	✓	grep	Alle Modelle
V42	Ungelesen nur durch Farbe	×	–	nicht erkannt (manuell)
V43	Kontrast Zeitangabe niedrig	✓	LLM-Detektor	nur 32b kons.
Fortsetzung auf der nächsten Seite				

Tab. 52: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V44	onClick o. onKeyDown (2)	✓	grep	Alle Modelle
V45	onClick o. onKeyDown (3)	✓	grep	Alle Modelle
V46	Button 16×16 px (Notif.)	✓	grep	Alle Modelle
V47	Modal ohne Fokus-Trap (Notif.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V48	Dialog ohne aria-labelledby (N.)	✓	LLM-Detektor	Alle Modelle (LLM)
V49	outline: none (Notif.)	×	–	nicht erkannt
V50	aria-hidden auf fokuss. El. (N.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V51	Tab onClick o. onKeyDown (1)	✓	grep	Alle Modelle
V52	Tab onClick o. onKeyDown (2)	✓	grep	Alle Modelle
V53	Tab onClick o. onKeyDown (3)	✓	grep	Alle Modelle
V54	Toggle ohne zugängl. Namen	✓	LLM-Detektor	Alle Modelle (LLM)
V55	Select ohne Label	✓	LLM-Detektor	Alle Modelle (LLM)
V56	Input ohne Label (Settings)	✓	LLM-Detektor	Alle Modelle (LLM)
V57	Passwortfeld ohne Label (1)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V58	Passwortfeld ohne Label (2)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V59	Button Kontrast niedrig (Settings)	×	–	nicht erkannt
V60	Checkbox ohne Label (Settings)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V61	Suchfeld ohne Label (Search)	✓	LLM-Detektor	Alle Modelle (LLM)
V62	Button ohne Namen (Search)	✓	LLM-Detektor	Alle Modelle (LLM)
V63	Filter-Chips onClick o. Keyboard	✓	grep	Alle Modelle
Fortsetzung auf der nächsten Seite				

Tab. 52: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V64	onClick o. onKeyDown (4)	✓	grep	Alle Modelle
V65	onClick o. onKeyDown (5)	✓	grep	Alle Modelle
V66	onClick o. onKeyDown (6)	✓	grep	Alle Modelle
V67	onClick o. onKeyDown (7)	✓	grep	Alle Modelle
V68	Typ-Badge nur durch Farbe	×	–	nicht erkannt (manuell)
V69	Kontrast Datum niedrig	✓	LLM-Detektor	nur 32b kons.
V70	Paginierung ohne tabIndex (SR)	✓	LLM-Detektor	nur 32b kons.
V71	Dropzone onClick o. onKeyDown	✓	grep	Alle Modelle
V72	Upload-Icon ohne alt="..."	✓	LLM-Detektor	nur 32b kons.
V73	File-Input ohne Label	×	–	nicht erkannt
V74	Löschen-Button ohne Namen	✓	LLM-Detektor	nur 32b kons.
V75	Progressbar ohne aria-valuenow	✓	LLM-Detektor	32b inst.; 7b/14b selten
V76	Kontrast Hilfetext niedrig (FU)	✓	LLM-Detektor	nur 32b kons.
V77	Button 16×16 px (FileU- pload)	✓	grep	Alle Modelle
V78	Modal ohne Fokus-Trap (FU)	×	–	nicht erkannt
V79	Dialog ohne aria-labelledby (FU)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V80	outline: none (FU)	✓	LLM-Detektor	32b inst.; 7b/14b selten
V81	Tabelle ohne <caption> (Cal.)	×	–	nicht erkannt
V82	<th> ohne scope (Cal.)	✓	LLM-Detektor	32b inst.; 7b/14b selten
V83	<td> onClick o. onKeyDown	✓	grep	Alle Modelle
V84	Event nur durch Farbe (Cal.)	×	–	nicht erkannt (manuell)
Fortsetzung auf der nächsten Seite				

Tab. 52: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V85	role="button" o. tabIndex (Cal.)	×	–	nicht erkannt
V86	Detail ohne aria-live (Cal.)	✓	LLM-Detektor	32b inst.; manuell
V87	Banner-Bild ohne alt="..." (Cal.)	✓	LLM-Detektor	32b inst.; 7b/14b selten
V88	Chat ohne aria-live	×	–	nicht erkannt (manuell)
V89	Kontrast Zeitstempel niedrig	×	–	nicht erkannt
V90	Chat-Input ohne Label	✓	LLM-Detektor	32b inst.; 7b/14b selten
V91	Senden-Button ohne Namen	✓	LLM-Detektor	32b inst.; 7b/14b selten
V92	Emoji onClick o. onKeyDown	✓	grep	Alle Modelle
V93	Online-Status nur durch Farbe	×	–	nicht erkannt (manuell)
V94	Kontrast Hilfetext niedrig (HC)	×	–	nicht erkannt
V95	Suchfeld ohne Label (HC)	×	–	nicht erkannt
V96	Accordion onClick o. onKeyDown	✓	grep	Alle Modelle
V97	Accordion ohne aria-expanded	×	–	nicht erkannt (manuell)
V98	Kontaktbild ohne alt="..."	✓	LLM-Detektor	32b inst.; 7b/14b selten
V99	Kontrast Kontaktinfo niedrig	✓	LLM-Detektor	32b inst.; 7b/14b selten
V100	aria-hidden auf fokuss. El. (HC)	✓	LLM-Detektor	32b inst.; 7b/14b selten
Erkannt gesamt			73 / 100 (Baseline: 34 / 100; LLM-exklusiv: 39 / 100)	

Anhang 47: Recall-Matrix: Violation × Bedingung (test-100-Suite)

Hinweis: 63 / 100 bezeichnet den **konsistenten** Recall über 10 Runs (Schwelle > 5/10), 99,1 den mittleren **Roh-Finding-Output pro Run**, und 73 / 100 einen **Einzelrun** (Run 01).

Tab. 53: Erkennungsstatus je Violation. ✓ = konsistent erkannt (> 5/10 Runs); ~ = instabil (3–5/10 Runs); × = nicht erkannt (0–2/10 Runs).

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V1	Skip-Link fehlt	✓	✓	✓	✓	
V2	<html> ohne lang	✓	✓	✓	✓	
V3	Logo ohne alt="..."	✓	✓	✓	✓	
V4	Deko-Bild ohne alt=	✓	✓	✓	✓	
V5	Button ohne zugängl. Namen	✓	✓	✓	✓	
V6	Badge Kontrast zu niedrig	✓	✓	✓	✓	
V7	<a> ohne href	×	×	×	✓	✓
V8	CSS order weicht von DOM ab	×	×	×	×	
V9	<nav> ohne aria-label	×	×	✓	✓	✓
V10	onClick o. onKeyDown	✓	✓	✓	✓	
V11	Headings übersprungen (h3)	×	×	×	×	
V12	KPI-Icon ohne alt="..." (1)	✓	✓	✓	✓	
V13	KPI-Wert Kontrast niedrig	✓	✓	✓	✓	
V14	KPI-Icon ohne alt="..." (2)	✓	✓	✓	✓	
V15	onClick <div> o. onKeyDown	✓	✓	✓	✓	
V16	role="button" o. tabIndex (D.)	✓	✓	✓	✓	
V17	Chart ohne alt="..."	✓	✓	✓	✓	
V18	Tabelle ohne <caption> (Dash.)	×	×	✓	✓	✓
V19	<th> ohne scope (Dash.)	×	×	✓	✓	✓
V20	Status nur durch Farbe (Dash.)	×	×	×	✓	✓
V21	Avatar ohne alt="..."	×	×	✓	✓	✓
V22	Input ohne <label> (Anzeige)	×	×	×	×	
V23	Input ohne <label> (E-Mail)	×	×	×	×	

Fortsetzung auf der nächsten Seite

Tab. 53: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V24	Textarea ohne <label>	×	×	×	×	
V25	Pflichtfeld ohne required	×	×	×	×	
V26	Fehlermeldung nicht verknüpft	×	×	×	×	
V27	Button Kontrast niedrig (User)	×	×	×	×	
V28	outline: none auf Button (User)	×	×	×	×	
V29	Statusmeldung ohne aria-live	×	×	×	✓	✓
V30	Duplikat-ID	×	✓	✓	✓	✓
V31	onClick o. onKeyDown	✓	✓	✓	✓	
V32	Suchfeld ohne Label (DT)	×	×	×	×	
V33	Tabelle ohne <caption> (DT)	×	×	✓	✓	✓
V34	Checkbox ohne Label (Header)	×	×	×	×	
V35	Checkbox ohne Label (Zeilen)	×	×	×	×	
V36	Status nur durch Farbe (DT)	×	×	×	✓	✓
V37	Icon-Button ohne Namen (DT)	×	✓	✓	✓	✓
V38	Paginierung onClick o. Keyboard (1)	✓	✓	✓	✓	
V39	Paginierung onClick o. Keyboard (2)	✓	✓	✓	✓	
V40	Paginierung onClick o. Keyboard (3)	✓	✓	✓	✓	
V41	onClick o. onKeyDown (N.)	✓	✓	✓	✓	
V42	Ungelesen nur durch Farbe	×	×	×	×	
V43	Kontrast Zeitangabe niedrig	×	×	×	✓	✓
V44	onClick o. onKeyDown (2)	✓	✓	✓	✓	
V45	onClick o. onKeyDown (3)	✓	✓	✓	✓	
V46	Button 16×16 px (Notif.)	✓	✓	✓	✓	
Fortsetzung auf der nächsten Seite						

Tab. 53: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V47	Modal ohne Fokus-Trap (Notif.)	×	×	✓	✓	✓
V48	Dialog ohne aria-labelledby (N.)	×	✓	✓	✓	✓
V49	outline: none (Notif.)	×	×	×	×	
V50	aria-hidden auf fokuss. El. (N.)	×	×	✓	✓	✓
V51	Tab onClick o. onKeyDown (1)	✓	✓	✓	✓	
V52	Tab onClick o. onKeyDown (2)	✓	✓	✓	✓	
V53	Tab onClick o. onKeyDown (3)	✓	✓	✓	✓	
V54	Toggle ohne zugängl. Namen	×	✓	✓	✓	✓
V55	Select ohne Label	×	✓	✓	✓	✓
V56	Input ohne Label (Settings)	×	✓	✓	✓	✓
V57	Passwortfeld ohne Label (1)	×	×	✓	✓	✓
V58	Passwortfeld ohne Label (2)	×	×	✓	✓	✓
V59	Button Kontrast niedrig (Settings)	×	×	×	×	
V60	Checkbox ohne Label (Settings)	×	×	✓	✓	✓
V61	Suchfeld ohne Label (Search)	×	✓	✓	✓	✓
V62	Button ohne Namen (Search)	×	✓	✓	✓	✓
V63	Filter-Chips onClick o. Keyboard	✓	✓	✓	✓	
V64	onClick o. onKeyDown (4)	✓	✓	✓	✓	
V65	onClick o. onKeyDown (5)	✓	✓	✓	✓	
V66	onClick o. onKeyDown (6)	✓	✓	✓	✓	
V67	onClick o. onKeyDown (7)	✓	✓	✓	✓	
V68	Typ-Badge nur durch Farbe	×	×	×	×	
V69	Kontrast Datum niedrig	×	×	×	✓	✓
Fortsetzung auf der nächsten Seite						

Tab. 53: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V70	Paginierung ohne <code>tabIndex</code> (SR)	×	×	×	✓	✓
V71	Dropzone <code>onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
V72	Upload-Icon ohne <code>alt="..."</code>	×	×	×	✓	✓
V73	File-Input ohne Label	×	×	×	×	
V74	Löschen-Button ohne Namen	×	×	×	✓	✓
V75	Progressbar ohne <code>aria-valuenow</code>	×	×	×	~	✓
V76	Kontrast Hilfetext niedrig (FU)	×	×	×	✓	✓
V77	Button 16×16 px (FileUpload)	✓	✓	✓	✓	
V78	Modal ohne Fokus-Trap (FU)	×	×	×	×	
V79	Dialog ohne <code>aria-labelledby</code> (FU)	×	×	✓	✓	✓
V80	<code>outline: none</code> (FU)	×	×	×	~	✓
V81	Tabelle ohne <code><caption></code> (Cal.)	×	×	×	×	
V82	<code><th></code> ohne <code>scope</code> (Cal.)	×	×	×	~	✓
V83	<code><td> onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
V84	Event nur durch Farbe (Cal.)	×	×	×	×	
V85	<code>role="button"</code> o. <code>tabIndex</code> (Cal.)	×	×	×	×	
V86	Detail ohne <code>aria-live</code> (Cal.)	×	×	×	~	✓
V87	Banner-Bild ohne <code>alt="..."</code> (Cal.)	×	×	×	~	✓
V88	Chat ohne <code>aria-live</code>	×	×	×	×	
V89	Kontrast Zeitstempel niedrig	×	×	×	×	
V90	Chat-Input ohne Label	×	×	×	~	✓
V91	Senden-Button ohne Namen	×	×	×	~	✓
V92	Emoji <code>onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
V93	Online-Status nur durch Farbe	×	×	×	×	
Fortsetzung auf der nächsten Seite						

Tab. 53: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V94	Kontrast Hilfetext niedrig (HC)	×	×	×	×	
V95	Suchfeld ohne Label (HC)	×	×	×	×	
V96	Accordion onClick o. onKeyDown	✓	✓	✓	✓	
V97	Accordion ohne aria-expanded	×	×	×	×	
V98	Kontaktbild ohne alt="..."	×	×	×	~	✓
V99	Kontrast Kontaktinfo niedrig	×	×	×	~	✓
V100	aria-hidden auf fokuss. El. (HC)	×	×	×	~	✓
Recall (konsistent)		34 / 100	40 / 100	53 / 100	63 / 100	39 LLM-exkl.

Anhang 48: Rohdaten: alle 30 Benchmark-Runs (test-100-Suite)

Tab. 54: Vollständige Messdaten der 30 Pipeline-Durchläufe (`num_predict` = 12288). LLM-Det. = Findings des LLM-Detektors; Axe/PW/Grep = Tool-Findings (konstant je Run); Tokens = Output-Tokens; Dauer = Priorisierungslaufzeit in Sekunden.

Run	Modell	LLM-Det.	Axe	PW	Grep	Gesamt-Det.	Tokens	Dauer (s)
01	qwen2.5-coder:7b	107	11	2	24	144	12288	424
02	qwen2.5-coder:7b	99	11	2	24	136	12288	395
03	qwen2.5-coder:7b	98	11	2	24	135	12288	401
04	qwen2.5-coder:7b	97	11	2	24	134	7902	261
05	qwen2.5-coder:7b	110	11	2	24	147	12288	408
06	qwen2.5-coder:7b	109	11	2	24	146	12288	388
07	qwen2.5-coder:7b	109	11	2	24	146	12288	372
08	qwen2.5-coder:7b	96	11	2	24	133	8007	258
09	qwen2.5-coder:7b	106	11	2	24	143	12288	371
10	qwen2.5-coder:7b	109	11	2	24	146	12288	369
11	qwen2.5-coder:14b	93	11	2	24	130	12288	759
12	qwen2.5-coder:14b	91	11	2	24	128	12288	790
13	qwen2.5-coder:14b	93	11	2	24	130	12288	865
14	qwen2.5-coder:14b	91	11	2	24	128	12288	867
15	qwen2.5-coder:14b	100	11	2	24	137	12288	864
16	qwen2.5-coder:14b	93	11	2	24	130	12288	818
17	qwen2.5-coder:14b	94	11	2	24	131	12288	821
18	qwen2.5-coder:14b	95	11	2	24	132	12288	822
19	qwen2.5-coder:14b	100	11	2	24	137	12288	844
20	qwen2.5-coder:14b	93	11	2	24	130	12288	759
21	qwen3:32b	94	11	2	24	131	8936	1220
22	qwen3:32b	104	11	2	24	141	6698	960
23	qwen3:32b	104	11	2	24	141	12288	1708
24	qwen3:32b	106	11	2	24	143	6367	971
25	qwen3:32b	94	11	2	24	131	7355	1000
26	qwen3:32b	104	11	2	24	141	6890	976
27	qwen3:32b	94	11	2	24	131	5476	781
28	qwen3:32b	93	11	2	24	130	4206	634
29	qwen3:32b	94	11	2	24	131	8571	1126
30	qwen3:32b	104	11	2	24	141	6816	954

Anhang 49: Modell-Leistungsvergleich: test-100-Suite

Zusammenfassung

Tab. 55: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle (je $n = 10$ Runs, `num_predict = 12 288`). σ = Stichproben-Standardabweichung. Der Recall basiert auf Konsistenzschwelle $> 5/10$ Runs je Modell.

Metrik	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Recall (konsistent)	40 / 100 (40 %)	53 / 100 (53 %)	63 / 100 (63 %)
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	104,0 $\pm$ 5,75	94,3 $\pm$ 3,23	99,1 $\pm$ 5,63
LLM-Detektor-Findings (Min–Max)	96–110	91–100	93–106
Konsistenz σ	5,75	3,23	5,63
Priorisierungsqualität	Fehlerhaft	Bimodal / instabil	Bimodal / instabil
Output-Tokens ($\bar{O}$)	11 421	12 288	7 360
Output-Limit ausgeschöpft	8 / 10 (80 %)	10 / 10 (100 %)	1 / 10 (10 %)
Parsing-Erfolg	100 %	100 %	100 %
NFA-02-Konformität	Nicht erfüllt	Nicht erfüllt	Nicht erfüllt
Eignung BITV-Einsatz	Nicht geeignet	Empfohlen	Eingeschränkt
<i>Suite-spezifische Zusatzmetriken</i>			
Fix-Qualität (Stufe)	0–1	0–2	0–2 (73 % Stufe 2)
Tool-Findings (axe/PW/grep)	11 / 2 / 24	11 / 2 / 24	11 / 2 / 24
Must-have im Bericht ($\bar{O}$)	46,1	58,9	39,2
Nice-to-have ($\bar{O}$) Bericht	45,4	27,9	18,9
Gesamt-Report-Items ($\bar{O}$)	91,5	86,8	58,1
LLM-Detektor-Laufzeit ($\bar{O}$)	394 s	723 s	1 576 s
Priorisierungslaufzeit ($\bar{O}$)	365 s	821 s	1 033 s
Priorisierungslaufzeit (Min–Max)	258–424 s	759–867 s	634–1 708 s
Gesamtlaufzeit ($\bar{O}$)	765 s	1 550 s	2 616 s
Gesamtlaufzeit (Min–Max)	648–839 s	1 418–1 688 s	2 172–3 314 s

Hinweis: Die Bewertung in der Spalte *Eignung BITV-Einsatz* priorisiert die Balance zwischen Erkennungsqualität und Zeitaufwand und kann daher auch bei Verletzung von NFA-02 positiv ausfallen. Bei strikter Auslegung von NFA-02 (Laufzeit < 10 Min.) wäre auf test-100 hingegen kein Modell uneingeschränkt geeignet.

Anhang 50: Token-Nutzung pro Modell: Input/Output (test-100-Suite)

Tab. 56: Durchschnittliche Token-Nutzung je Run und Modell auf Basis der test-100-Suite (`num_predict` = 12 288). Input-Tokens umfassen System-Prompt, serialisierte Findings und Konfigurationsparameter. 14b schöpfte in allen 10 Runs das Output-Limit aus; 32b blieb mit Ø 7 360 deutlich darunter.

Modell	Input-Tokens (Ø)	Output-Tokens (Ø)	Truncation
qwen2.5-coder:7b	17 503	11 421	8 / 10 Runs (80 %)
qwen2.5-coder:14b	17 043	12 288	10 / 10 Runs (100 %)
qwen3:32b	17 197	7 360	1 / 10 Runs (10 %)
<i>Gesamt (30 Runs): Input $\approx$ 517 400 / Output 310 700 Tokens</i>			

Anhang 51: Effizienzmetriken (test-100-Suite)

Tab. 57: Effizienzvergleich auf Basis der 30 Benchmark-Runs der test-100-Suite. *LLM-Findings/s* bezieht sich auf LLM-Detektor-Findings pro Sekunde Gesamtlaufzeit.

Modell	LLM-Findings Ø	Gesamtlaufzeit Ø (s)	LLM-Findings/s	Zusatznutzen ggü. 7b
qwen2.5-coder:7b	104,0	765	0,136	Referenz
qwen2.5-coder:14b	94,3	1 550	0,061	−9,3 % bei +103 % Zeitaufwand
qwen3:32b	99,1	2 616	0,038	−4,7 % bei +242 % Zeitaufwand

Anhang 52: Über-Detektion relativ zur Ground Truth (test-100-Suite)

Tab. 58: Vergleich der durchschnittlichen LLM-Detektor-Findings pro Run mit der Ground Truth (100 Violations). Die LLM-only-Betrachtung vermeidet Doppelzählungen mit Tool-Detektoren.

Modell	LLM-Findings Ø	Ground Truth	Quote	Interpretation
qwen2.5-coder:7b	104,0	100	104 %	Leichte Überdetektion; hohe LLM-Finding-Zahl trotz inverser Priorisierung und 80 % Truncation
qwen2.5-coder:14b	94,3	100	94,3 %	Leichte Unterdetektion; alle 10 Runs bei 12 288 Output-Tokens; Vollständigkeit durch Truncation begrenzt
qwen3:32b	99,1	100	99,1 %	Nahe an Ground Truth; 1 / 10 Runs trunziert; höchste absolute Nähe zur Ziellanzahl

Anhang 53: Stichproben-Plausibilisierung des finalen Reports (qwen3:32b, Run 01 & Run 03, test-100)

Da die Überdetections-Quote ausschließlich eine Mengenrelation zwischen LLM-Detektor-Rohoutput und Ground Truth misst, wurde eine manuelle Stichprobenprüfung an zwei Runs durchgeführt, um die inhaltliche Qualität des finalen Reports einzuschätzen. Grundlage ist `qwen3:32b` auf `test-100`: Run 01 ist vollständig (keine Truncation, 72 Report-Einträge), Run 03 ist trunziert (12 288 Output-Tokens ausgeschöpft, 104 Einträge). Jeder Eintrag wurde dem Quellcode unter `test/test-100/src/` sowie dem Ground-Truth-Katalog (Tabelle 51) zugeordnet.

Tab. 59: Kategorisierung aller Report-Einträge aus Run 01 (vollständig) und Run 03 (trunziert) von `qwen3:32b` auf `test-100`. *Quell-Duplikat* = axe-core und LLM-Detektor finden dieselbe Violation unabhängig voneinander; *Instanz-Mehrfach* = zweite DOM-Instanz einer Violation, die im Ground Truth als ein Eintrag zählt; *Gebündelt* = ein Finding umfasst mehrere GT-Violations.

Kategorie	Beschreibung	Run-01	Run-03
True Positive	Unique Finding, eindeutig einer GT-Violation zugeordnet	51	82
Quell-Duplikat	Dieselbe Violation durch axe-core <i>und</i> LLM-Detektor unabhängig gemeldet	10	2
Instanz-Mehrfach	Zweite DOM-Instanz; GT zählt Violationsklasse einmalig	2	–
Gebündelt	Ein Finding deckt mehrere GT-Violations ab (z.B. 11m-076: V51+V52+V53)	1	2
Halluzination	Kein GT-Match, kein Basis im Quellcode	0	1
Nicht eindeutig	Mögliche Paraphrase oder Violation außerhalb der 100 GT-Einträge	8	17
Gesamt		72	104

Fazit: Von 176 untersuchten Einträgen ($72 + 104$) ist 1 als Halluzination bestätigt ([11m-061] in Run 03 meldet eine Tabelle ohne `caption` in `Notifications.tsx`, obwohl dort keine Tabelle existiert). Die Halluzinationsrate liegt damit unter 1 %. Die dominanten “Über-Detektion”-Effekte sind Quell-Duplikate und nicht eindeutig zugeordnete Findings – keine inhaltlichen Fehler, sondern strukturelle Artefakte der zweistufigen Pipeline.

Anhang 54: Precision und F1-Score je Run (test-100-Suite)

Per-Run-Auflistung der Precision für alle 30 Benchmark-Runs auf der test-100-Suite. Methodik identisch zu Anhang 39: TP = Item referenziert ein in der Ground Truth vorhandenes WCAG-Kriterium oder stammt von einem Tool-Detektor (axe/Playwright/grep); FP = kein nachweisbarer Ground-Truth-Bezug. Die Stichproben-Plausibilisierung (Tabelle 59) bestätigt eine Halluzinationsrate von $< 1\%$ für **qwen3:32b**. Konsistenz-Recall und F1 gemäß Tabelle 9.

Tab. 60: Precision je Run auf test-100 für alle drei evaluierten Modelle ($n = 10$ Runs je Modell). TP = True-Positive-Items, FP = False-Positive-Items. Modell-Gesamtwerte in der letzten Zeile.

Run	qwen2.5-coder:7b			qwen2.5-coder:14b			qwen3:32b		
	Items	FP	Prec. %	Items	FP	Prec. %	Items	FP	Prec. %
01	94	4	95,7	86	0	100,0	72	0	100,0
02	110	9	91,8	69	1	98,6	49	0	100,0
03	94	4	95,7	91	0	100,0	98	0	100,0
04	62	1	98,4	84	0	100,0	52	0	100,0
05	109	7	93,6	90	0	100,0	68	0	100,0
06	93	2	97,8	89	0	100,0	52	0	100,0
07	104	0	100,0	91	0	100,0	41	0	100,0
08	64	2	96,9	87	0	100,0	29	0	100,0
09	93	2	97,8	89	0	100,0	68	0	100,0
10	92	1	98,9	91	0	100,0	52	1	98,1
Ø	915	32	96,7	867	1	99,9	581	1	99,8
Kons.-Recall: 40 % F1: 57,4 % Kons.-Recall: 53 % F1: 69,2 % Kons.-Recall: 63 % F1: 76,9 %									

Anhang 55: Detektor-Konsistenz über alle 30 Runs (test-100-Suite)

Tab. 61: Konsistenzanalyse der Detektoren über alle 30 Benchmark-Runs. Axe-core, Playwright und grep bleiben vollständig deterministisch; die Varianz entsteht ausschließlich im LLM-Detektor.

Detektor / Modell	Runs	Gesamt-Findings	Ø/Run	Min–Max	σ^2 / σ	Interpretation
axe-core	30	330	11,0	11–11	0,00 / 0,00	vollständig deterministisch
Playwright	30	60	2,0	2–2	0,00 / 0,00	vollständig deterministisch
grep	30	720	24,0	24–24	0,00 / 0,00	vollständig deterministisch
qwen2.5-coder:7b	10	1 040	104,0	96–110	33,11 / 5,75	höchste Streuung; 80 % Truncation
qwen2.5-coder:14b	10	943	94,3	91–100	10,46 / 3,23	moderate Konsistenz; 100 % Truncation
qwen3:32b	10	991	99,1	93–106	31,66 / 5,63	moderate Streuung; deutlich höhere Laufzeit
LLM-Detektor gesamt	30	2 974	99,1	91–110	modell-abhängig	–

Anhang 56: Bewertung der Empfehlungsqualität pro Violation (test-100-Suite)

Kürzel: DT = DataTable, SR = SearchResults, FU = FileUpload, N.= Notifications, Cal.= Calendar, Dash.= Dashboard, User = UserProfile.

Die Bewertung basiert auf Run 01 von **qwen3:32b** und verwendet dieselbe dreistufige Skala wie Tabelle 18. Stufe 2 liegt vor, wenn Dateiname, Zeilennummer und ein konkreter Code-Vorschlag vorhanden sind. Violations, die in Run 01 nicht erkannt wurden (27 von 100), werden mit Stufe 0 ausgewiesen. Violations mit (*manuell*) waren als rein manuelle Prüffälle klassifiziert – die LLM-Erkennung stellt einen Mehrwert über die Erwartung hinaus dar.

Tab. 62: Bewertung der Fix-Qualität für alle 100 Violations (**qwen3:32b**, Run 01). Stufe 0 = nicht erkannt; Stufe 2 = konkreter Dateiname, Zeile und Code-Snippet.

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V1	Skip-Link fehlt	2	App.tsx, <code></code> -Snippet mit CSS-Positionierung
V2	<code><html></code> ohne <code>lang</code>	2	index.html, <code><html lang="de"></code> direkt angegeben
V3	Logo ohne <code>alt="..."</code>	2	App.tsx, beschreibender <code>alt="Company Logo"</code> vorge-schlagen
V4	Deko-Bild ohne <code>alt=</code>	2	App.tsx, <code>alt=</code> <code>role="presentation"</code> kor- rekt ergänzt
V5	Button ohne zugängl. Namen	2	App.tsx, <code>aria-label="Menü öffnen"</code> mit JSX-Snippet
V6	Badge Kontrast zu niedrig	2	App.tsx, CSS <code>.badge</code> mit kor- rektem Hex-Farbpaar
V7	<code><a></code> ohne <code>href</code>	2	App.tsx, <code>href="/help"</code> mit <code>role="link"</code> ergänzt
V8	CSS <code>order</code> weicht von DOM ab	0	nicht erkannt
V9	<code><nav></code> ohne <code>aria-label</code>	2	App.tsx, <code>aria-label="Hauptnavigation"</code>
V10	<code>onClick </code> o. <code>onKeyDown</code>	2	App.tsx, vollst. <code>onKeyDown</code> - Handler mit Enter-Logik
V11	Headings übersprungen (<code>h3</code>)	0	nicht erkannt
Fortsetzung auf der nächsten Seite			

Tab. 62: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V12	KPI-Icon ohne <code>alt="..."</code> (1)	2	Dashboard.tsx, beschreibender <code>alt</code> -Text je KPI-Typ
V13	KPI-Wert Kontrast niedrig	2	Dashboard.tsx, CSS-Fix mit WCAG-konformem Hex-Farbpaar
V14	KPI-Icon ohne <code>alt="..."</code> (2)	2	Dashboard.tsx, Querverweis auf V12-Fix
V15	<code>onClick <div> o. onKeyDown</code>	2	Dashboard.tsx, <code>onKeyDown + tabIndex={0}</code> vollst.
V16	<code>role="button" o. tabIndex</code> (D.)	2	Dashboard.tsx, <code>tabIndex={0}</code> + JSX-Snippet
V17	Chart ohne <code>alt="..."</code>	2	Dashboard.tsx, inhaltsbeschreibender <code>alt</code> -Text
V18	Tabelle ohne <code><caption></code> (Dash.)	2	Dashboard.tsx, <code><caption>Dashboard-Übersicht</caption></code>
V19	<code><th></code> ohne <code>scope</code> (Dash.)	2	Dashboard.tsx, <code>scope="col"</code> für alle <code><th></code> ergänzt
V20	Status nur durch Farbe (Dash.)	2	Dashboard.tsx, Icon + <code>aria-label</code> zusätzlich zu Farbe (manuell)
V21	Avatar ohne <code>alt="..."</code>	2	UserProfile.tsx, <code>alt="Profilbild von {name}"</code>
V22	Input ohne <code><label></code> (Anzeige)	0	nicht erkannt
V23	Input ohne <code><label></code> (E-Mail)	0	nicht erkannt
V24	Textarea ohne <code><label></code>	0	nicht erkannt
V25	Pflichtfeld ohne <code>required</code>	0	nicht erkannt (manuell)
V26	Fehlermeldung nicht verknüpft	0	nicht erkannt (manuell)
V27	Button Kontrast niedrig (User)	0	nicht erkannt
V28	<code>outline: none</code> auf Button (User)	0	nicht erkannt
V29	Statusmeldung ohne <code>aria-live</code>	2	UserProfile.tsx, <code>aria-live="polite"</code> auf Status-Container (manuell)
Fortsetzung auf der nächsten Seite			

Tab. 62: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V30	Duplikat-ID	2	UserProfile.tsx, eindeutige id-Werte per Index generiert
V31	onClick o. onKeyDown	2	DataTable.tsx, vollst. Handler mit role="button"
V32	Suchfeld ohne Label (DT)	0	nicht erkannt
V33	Tabelle ohne <caption> (DT)	2	DataTable.tsx, <caption>Datentabelle</caption> ergänzt
V34	Checkbox ohne Label (Header)	0	nicht erkannt
V35	Checkbox ohne Label (Zeilen)	0	nicht erkannt
V36	Status nur durch Farbe (DT)	2	DataTable.tsx, Icon-Symbol + aria-label zusätzl. (manuell)
V37	Icon-Button ohne Namen (DT)	2	DataTable.tsx, aria-label für Aktions-Buttons
V38	Paginierung onClick o. Keyboard (1)	2	DataTable.tsx, onKeyDown + tabIndex vollst.
V39	Paginierung onClick o. Keyboard (2)	2	DataTable.tsx, Querverweis auf V38-Fix
V40	Paginierung onClick o. Keyboard (3)	2	DataTable.tsx, Querverweis auf V38-Fix
V41	onClick o. onKeyDown (N.)	2	Notifications.tsx, vollst. Handler-Snippet
V42	Ungelesen nur durch Farbe	0	nicht erkannt (manuell)
V43	Kontrast Zeitangabe niedrig	2	Notifications.tsx, CSS-Fix mit Hex-Farbpaar
V44	onClick o. onKeyDown (2)	2	Notifications.tsx, Querverweis auf V41-Fix
V45	onClick o. onKeyDown (3)	2	Notifications.tsx, Querverweis auf V41-Fix
V46	Button 16×16 px (Notif.)	2	styles.css, .btn-notif auf mind. 24×24 px
V47	Modal ohne Fokus-Trap (Notif.)	2	Notifications.tsx, Fokus-Trap-Implementierung mit useRef
Fortsetzung auf der nächsten Seite			

Tab. 62: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V48	Dialog ohne aria-labelledby (N.)	2	Notifications.tsx, aria-labelledby auf Dialog- Titel
V49	outline: none (Notif.)	0	nicht erkannt
V50	aria-hidden auf fokuss. El. (N.)	2	Notifications.tsx, aria-hidden entfernt; vollst. JSX
V51	Tab onClick o. onKeyDown (1)	2	Settings.tsx, vollst. onKeyDown- Handler
V52	Tab onClick o. onKeyDown (2)	2	Settings.tsx, Querverweis auf V51-Fix
V53	Tab onClick o. onKeyDown (3)	2	Settings.tsx, Querverweis auf V51-Fix
V54	Toggle ohne zugängl. Namen	2	Settings.tsx, aria-label="Be- nachrichtigungen aktivieren"
V55	Select ohne Label	2	Settings.tsx, <label for="lang-select» mit JSX
V56	Input ohne Label (Settings)	2	Settings.tsx, <label> + htmlFor korrekt verknüpft
V57	Passwortfeld ohne Label (1)	2	Settings.tsx, <label for="pw1»Neues Passwort</label>
V58	Passwortfeld ohne Label (2)	2	Settings.tsx, <label for="pw2»Passwort bestätigen</label>
V59	Button Kontrast niedrig (Set- tings)	0	nicht erkannt
V60	Checkbox ohne Label (Set- tings)	2	Settings.tsx, <label> mit htmlFor ergänzt
V61	Suchfeld ohne Label (Search)	2	SearchResults.tsx, <label for=search-input» + Snippet
V62	Button ohne Namen (Search)	2	SearchResults.tsx, aria-label=SSuche starten"
V63	Filter-Chips onClick o. Key- board	2	SearchResults.tsx, onKeyDown + role="button"
Fortsetzung auf der nächsten Seite			

Tab. 62: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V64	onClick o. onKeyDown (4)	2	SearchResults.tsx, vollst. Handler
V65	onClick o. onKeyDown (5)	2	SearchResults.tsx, Querverweis auf V64-Fix
V66	onClick o. onKeyDown (6)	2	SearchResults.tsx, Querverweis auf V64-Fix
V67	onClick o. onKeyDown (7)	2	SearchResults.tsx, Querverweis auf V64-Fix
V68	Typ-Badge nur durch Farbe	0	nicht erkannt (manuell)
V69	Kontrast Datum niedrig	2	SearchResults.tsx, CSS-Fix mit WCAG-konformem Grouton
V70	Paginierung ohne tabIndex (SR)	2	SearchResults.tsx, tabIndex={0} + role="button"
V71	Dropzone onClick o. onKeyDown	2	FileUpload.tsx, vollst. onKeyDown-Handler
V72	Upload-Icon ohne alt="..."	2	FileUpload.tsx, alt="Datei hochladen"
V73	File-Input ohne Label	0	nicht erkannt
V74	Löschen-Button ohne Namen	2	FileUpload.tsx, aria-label="Datei {name} entfernen"
V75	Progressbar ohne aria-valuenow	2	FileUpload.tsx, aria-valuenow={progress} + aria-valuemin/max
V76	Kontrast Hilfetext niedrig (FU)	2	FileUpload.tsx, CSS-Fix für Hilfetext-Farbe
V77	Button 16×16 px (FileU- pload)	2	styles.css, .btn-upload auf mind. 24×24 px
V78	Modal ohne Fokus-Trap (FU)	0	nicht erkannt
V79	Dialog ohne aria-labelledby (FU)	2	FileUpload.tsx, aria-labelledby auf Vorschau- Dialog
V80	outline: none (FU)	2	FileUpload.tsx, outline: none entfernt; Fokus-Stil ergänzt
Fortsetzung auf der nächsten Seite			

Tab. 62: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V81	Tabelle ohne <code><caption></code> (Cal.)	0	nicht erkannt
V82	<code><th></code> ohne <code>scope</code> (Cal.)	2	Calendar.tsx, <code>scope="col"</code> für Wochentage ergänzt
V83	<code><td></code> <code>onClick</code> o. <code>onKeyDown</code>	2	Calendar.tsx, vollst. <code>onKeyDown</code> + <code>tabIndex</code>
V84	Event nur durch Farbe (Cal.)	0	nicht erkannt (manuell)
V85	<code>role="button"</code> o. <code>tabIndex</code> (Cal.)	0	nicht erkannt
V86	Detail ohne <code>aria-live</code> (Cal.)	2	Calendar.tsx, <code>aria-live="polite"</code> auf Event-Detail (manuell)
V87	Banner-Bild ohne <code>alt="..."</code> (Cal.)	2	Calendar.tsx, <code>alt="Kalender-Banner"</code>
V88	Chat ohne <code>aria-live</code>	0	nicht erkannt (manuell)
V89	Kontrast Zeitstempel niedrig	0	nicht erkannt
V90	Chat-Input ohne Label	2	Chat.tsx, <code><label for="chat-input">Nachricht</label></code>
V91	Senden-Button ohne Namen	2	Chat.tsx, <code>aria-label="Nachricht senden"</code>
V92	Emoji <code>onClick</code> o. <code>onKeyDown</code>	2	Chat.tsx, vollst. <code>onKeyDown</code> -Handler
V93	Online-Status nur durch Farbe	0	nicht erkannt (manuell)
V94	Kontrast Hilfetext niedrig (HC)	0	nicht erkannt
V95	Suchfeld ohne Label (HC)	0	nicht erkannt
V96	Accordion <code>onClick</code> o. <code>onKeyDown</code>	2	HelpCenter.tsx, vollst. <code>onKeyDown</code> -Handler
V97	Accordion ohne <code>aria-expanded</code>	0	nicht erkannt (manuell)
V98	Kontaktbild ohne <code>alt="..."</code>	2	HelpCenter.tsx, <code>alt="Kontaktperson {name}"</code>
Fortsetzung auf der nächsten Seite			

Tab. 62: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V99	Kontrast Kontaktinfo niedrig	2	HelpCenter.tsx, CSS-Fix mit konformem Grauton
V100	aria-hidden auf fokuss. El. (HC)	2	HelpCenter.tsx, aria-hidden entfernt; vollst. JSX
Stufe-2-Bewertungen		73 Violations (73 %); 27 nicht erkannt (Stufe 0)	

Anhang 57: CSV-Auszug: Benchmark-Rohdaten (test-100-Suite)

Die Benchmark-Rohdaten wurden als CSV-Datei exportiert. Tabelle 63 zeigt repräsentative Zeilen für alle drei Modelle (jeweils Run 01).

Tab. 63: Repräsentative Beispielzeilen aus `results/benchmark/summary.csv` (test-100-Suite; jeweils Run 01 pro Modell). `num_predict` = 12 288; Dauer in Millisekunden.

model	run	must	nice	LLM	Axe	PW	Grep	durationMs	prompt	output	parse
qwen2.5-coder:7b	1	94	0	107	11	2	24	423 845	17 763	12 288	true
qwen2.5-coder:14b	1	86	0	93	11	2	24	759 432	17 025	12 288	true
qwen3:32b	1	59	13	94	11	2	24	1 219 721	16 672	8 936	true

Anhang 58: Pareto-Diagramm: Nicht erkannte Violations (test-100, qwen3:32b)

Das Pareto-Diagramm zeigt die 37 in test-100 nicht konsistent erkannten Violations für qwen3:32b (aus der Recall-Matrix). Verwendete Kategorien: syntaktisch, semantisch, layout. Ergebnis: Der größte Anteil entfällt auf syntaktische und semantische Fälle.

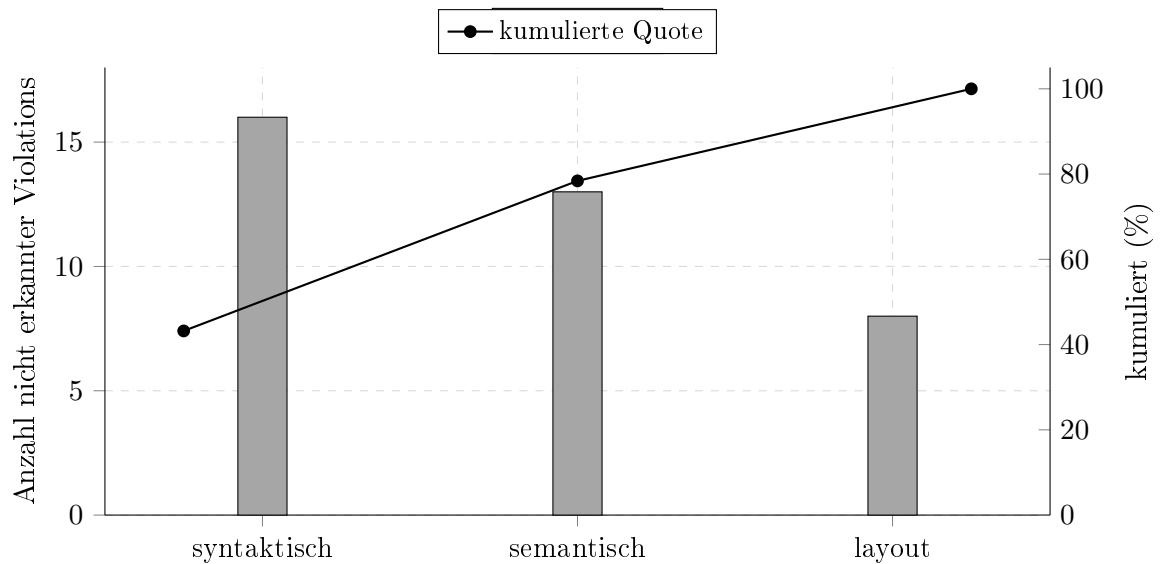


Abb. 15: Pareto-Auswertung der nicht konsistent erkannten Violations in test-100 (qwen3:32b). 78,4% der Ausfälle liegen bereits in den ersten zwei Kategorien (syntaktisch + semantisch).

Anhang 59: Boxplot der Laufzeiten (test-100, 10 Runs je Modell)

Die Boxplots zeigen die Laufzeitverteilung je Modell auf test-100 (Rohdaten aus Tabelle 54). Dargestellt sind Min, Q1, Median, Q3 und Max. Die dicke Linie innerhalb der Boxen ist die Median-Linie (50%-Quantil).

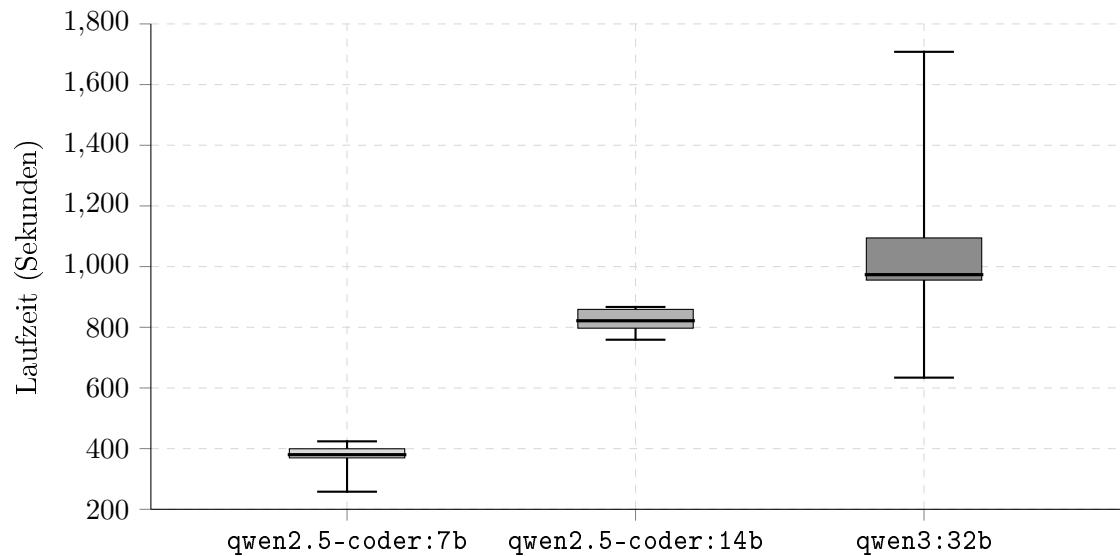


Abb. 16: Boxplot der Laufzeitverteilung je Modell (test-100). **qwen3:32b** zeigt den größten oberen Whisker bis 1 708 s, während **qwen2.5-coder:14b** die engste IQR-Box besitzt.

Anhang 60: Pipeline-Diagramm der Findings-Verarbeitung

Das folgende Diagramm zeigt den strukturierten Verarbeitungsfluss der Findings für test-100 mit `qwen3:32b` (Run 01). Es visualisiert die Zusammenführung der Tool- und LLM-basierten Findings, deren Konsolidierung sowie die anschließende Priorisierung in Must- und Nice-to-have-Kategorien.

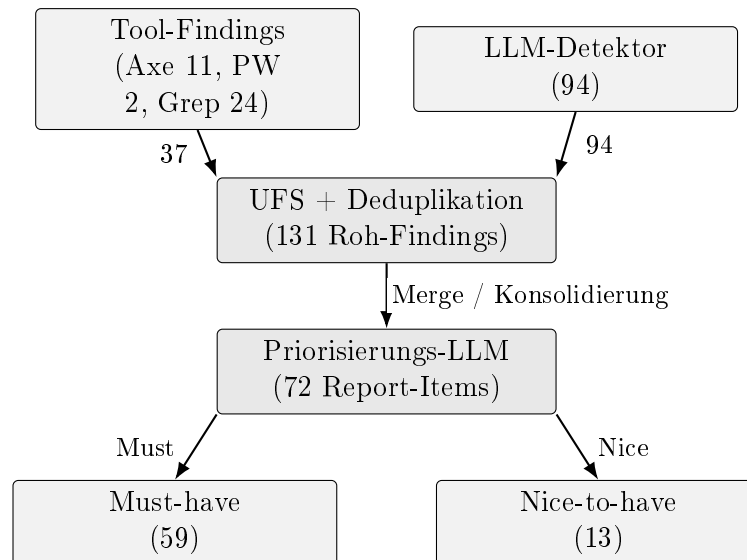


Abb. 17: Pipeline der Findings-Verarbeitung für test-100 (`qwen3:32b`, Run 01). Die Pfeilbeschriftungen geben die jeweiligen Mengen bzw. Kategorien zwischen den Verarbeitungsschritten an.

Anhang 61: Ansichten der Testanwendung (test-100-Suite)

Anmerkung: Teile der test-100-Suite verwenden dieselben bzw. strukturell sehr ähnliche Dashboard-Muster wie test-50. Zur Vollständigkeit werden hier die entsprechenden test-100-Ansichten dokumentiert, ergänzt um suitespezifische Screens, ohne daraus eine inhaltliche Gleichsetzung beider Evaluationen abzuleiten.

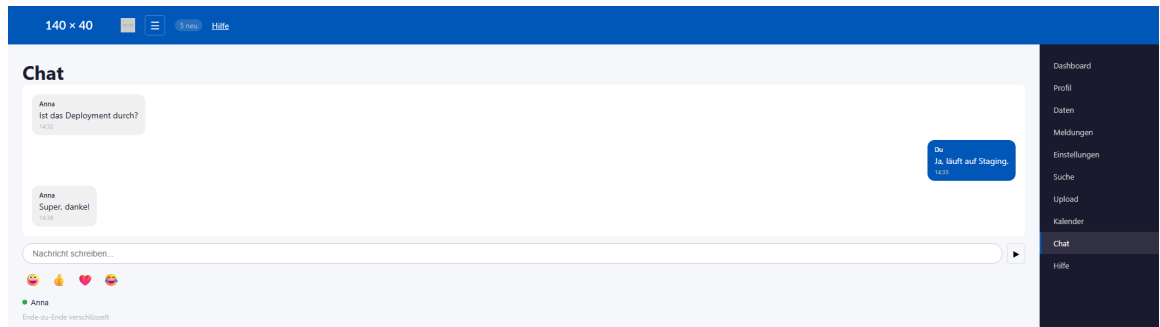


Abb. 18: Chat-Modul der test-100-Suite

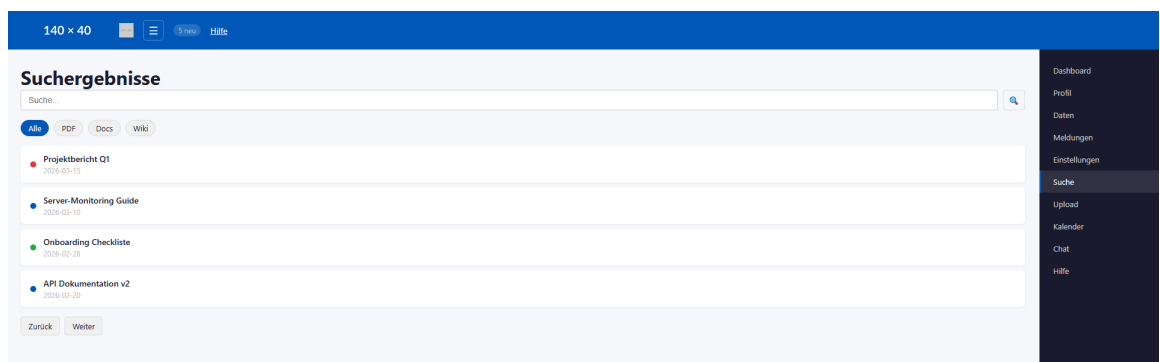


Abb. 19: Suche-Modul der test-100-Suite

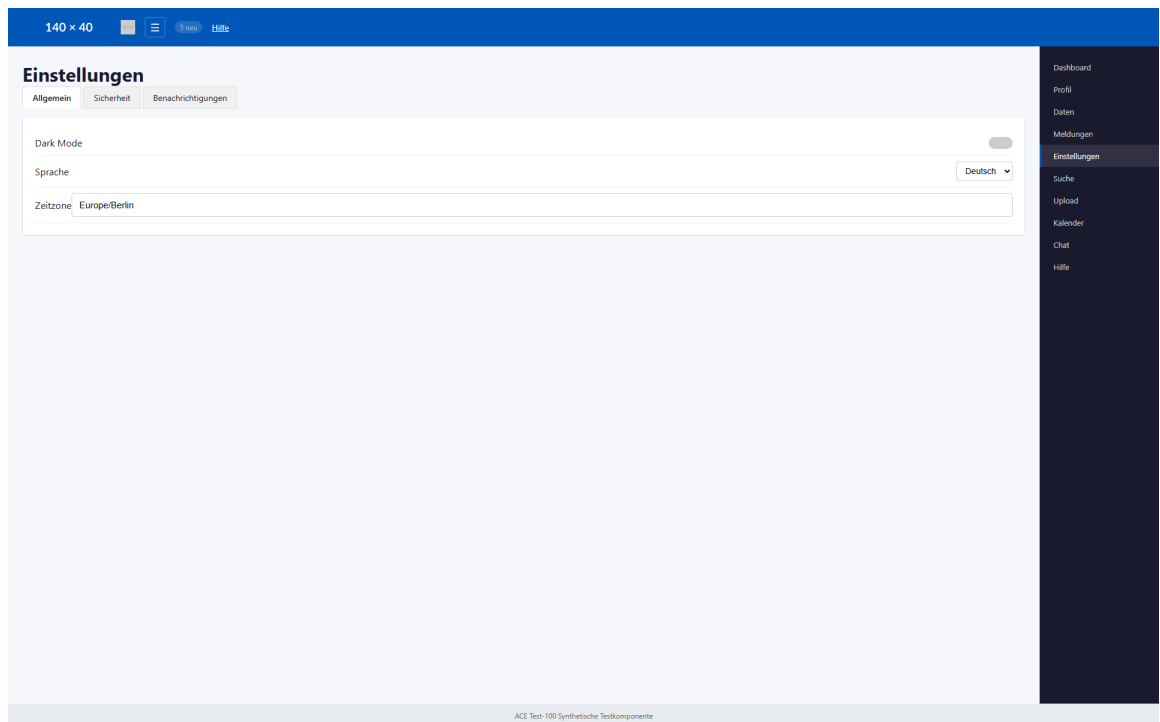


Abb. 20: Einstellungen-Modul der test-100-Suite

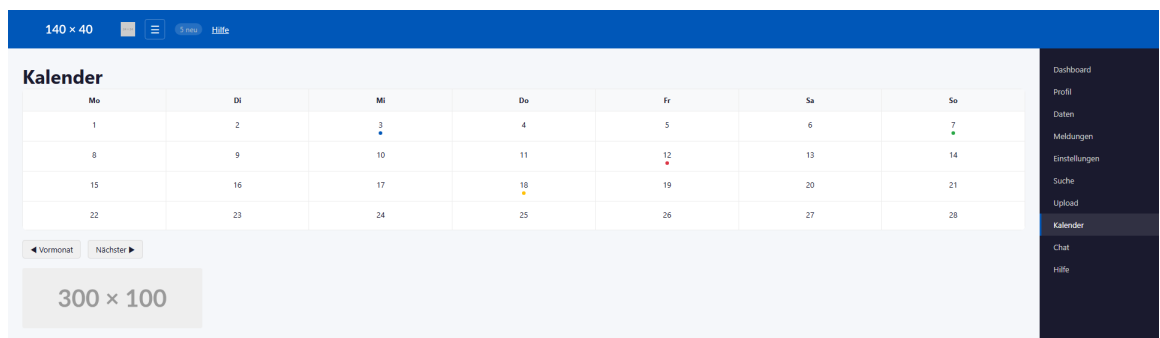


Abb. 21: Kalender-Modul der test-100-Suite

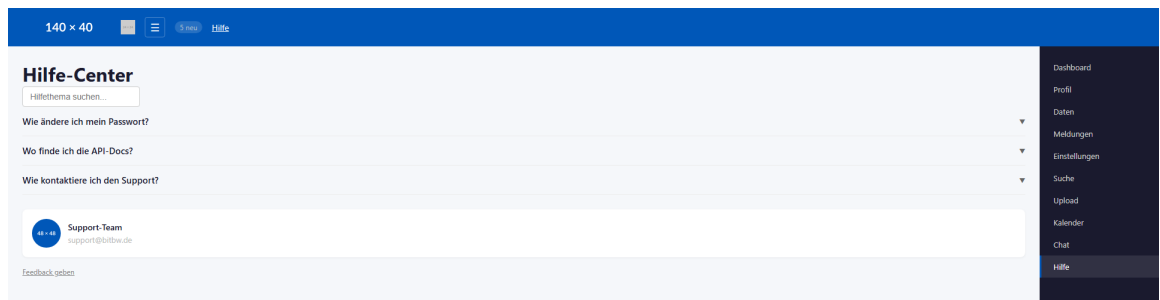


Abb. 22: Hilfe-Center-Modul der test-100-Suite

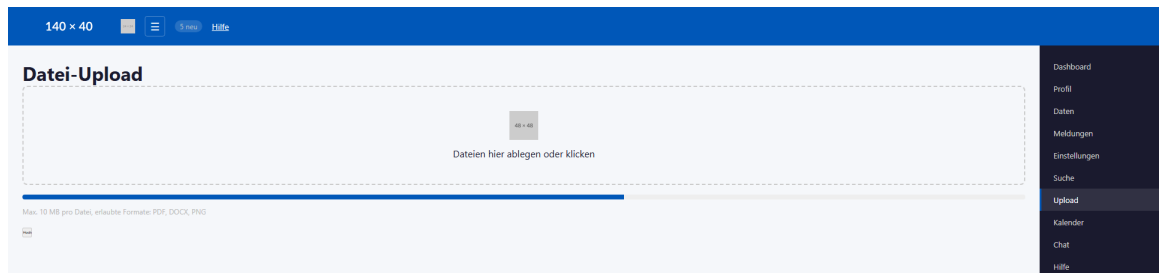


Abb. 23: Datei-Upload-Modul der test-100-Suite

Anhang 62: Lesehinweis zu den Vergleichsgrafiken

Die folgenden Grafiken fassen die Ergebnisse *suite-übergreifend* (test-12, test-50, test-100) zusammen und sind damit nicht einem einzelnen Suite-Block zugeordnet. Nach den Ansichtsabschnitten dienen sie als komprimierte Vergleichsebene für Recall, Laufzeit, Konsistenz, Über-/Unterdetektion und Truncation.

Anhang 63: Vergleich über alle Suites

Tab. 64: Kompakter Orientierungsvergleich der zentralen Kennzahlen über test-12, test-50 und test-100.

Metrik	test-12	test-50	test-100
Ground-Truth-Violations	12	50	100
Runs je Modell	10 je Modell und Suite		
Output-Limit num_predict	6 144	8 192	12 288
Beste LLM-Konsistenz σ	0,42 (14b)	1,25 (32b)	3,23 (14b)
NFA-02-Konformität (strenge Sicht)	7b, 14b: ✓	7b: ✓	keines: ×
Pragmatische BITV-Eignung	14b: empfohlen, 32b: eingeschränkt (<i>alle Suites</i>)		

Hinweis: Im Suite-Vergleich werden zwei Perspektiven getrennt ausgewiesen: *strenge Sicht* (NFA-02 als Ausschlusskriterium) und *pragmatische Sicht* (Balance aus Erkennungsqualität und Zeitaufwand).

Anhang 64: Recall-Degradation über alle Testsuites

Alle drei Modelle erzielen in test-12 hohe Recall-Werte ($\geq 75\%$); der markante Einbruch findet vor allem zwischen test-50 und test-100 statt. **qwen3:32b** hält mit 63 % den höchsten Recall in test-100. Datenbasis: Tabellen 27, 42, 55.

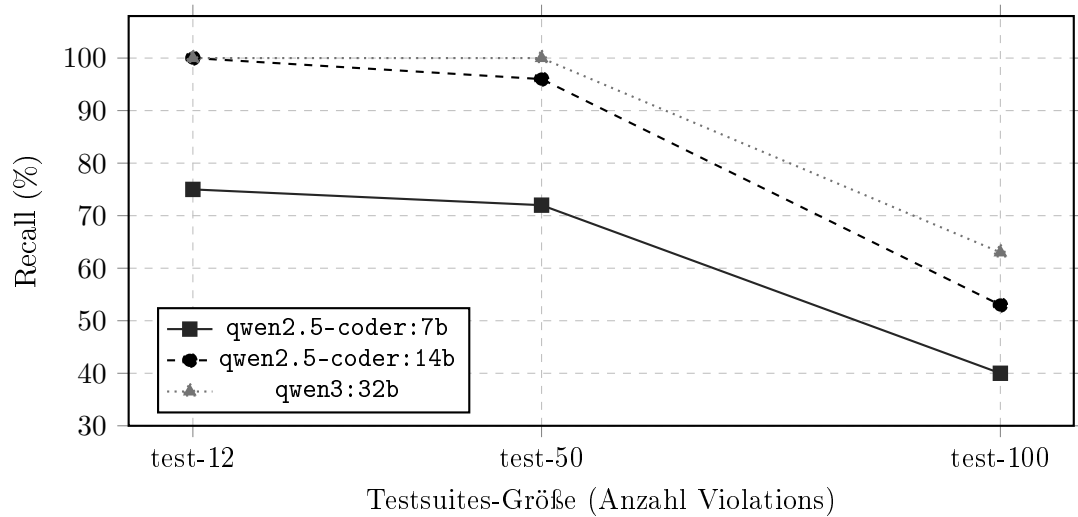


Abb. 24: Recall in Prozent je Modell über alle drei Testsuites (Konsistenzschwelle $> 5/10$ Runs). Der Knick zwischen test-50 und test-100 ist bei allen Modellen sichtbar. Datenbasis: Tabellen 27, 42, 55.

Anhang 65: Gesamtlaufzeit-Skalierung über alle Testsuites

Dargestellt ist die mittlere Gesamtlaufzeit je Modell über die drei Testsuites. Die gestrichelte Referenzlinie markiert den NFA-02-Grenzwert (600 s = 10 min). In test-12 unterschreiten alle Modelle diese Grenze; in test-100 liegt kein Modell mehr darunter. Datenbasis: Tabellen 27, 42, 55.

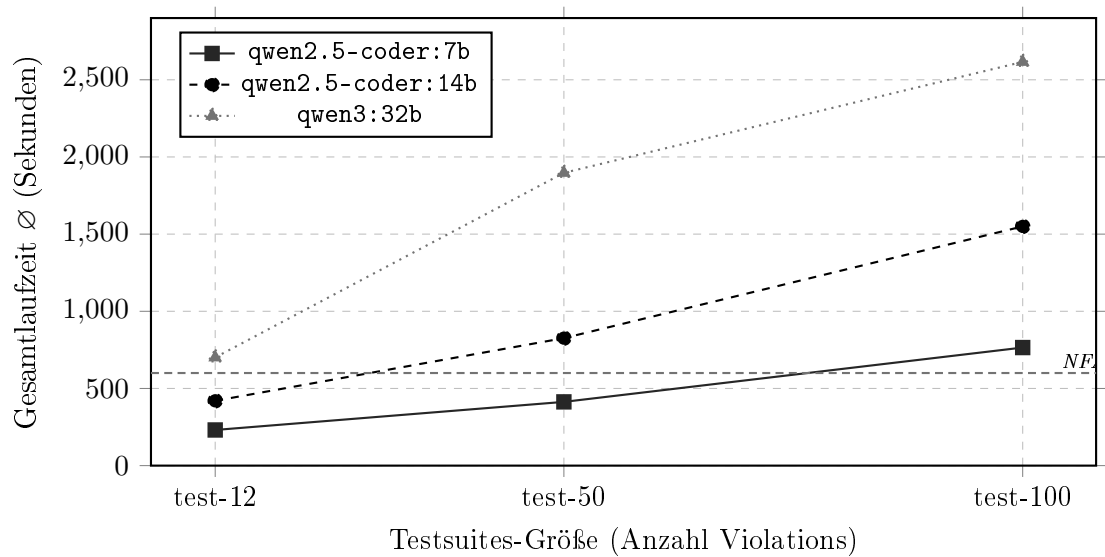


Abb. 25: Mittlere Gesamtlaufzeit in Sekunden je Modell und Testsuite. Die NFA-02-Grenzlinie markiert 600 s (10 Minuten). Datenbasis: Tabellen 27, 42, 55.

Anhang 66: LLM-Konsistenz (σ) im Suite-Vergleich

In test-12 liefert `qwen2.5-coder:14b` mit $\sigma = 0,42$ die stabilste Ausgabe. In test-100 steigt die Streuung bei 7b ($\sigma = 5,75$) und 32b ($\sigma = 5,63$) stark an, während 14b mit $\sigma = 3,23$ am stabilsten bleibt. Datenbasis: Tabellen 33, 47, 61.

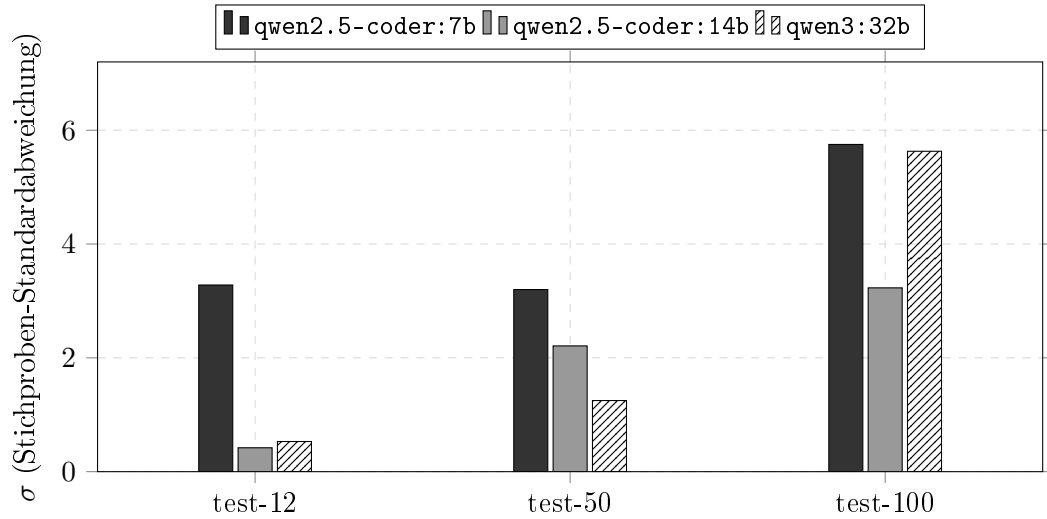


Abb. 26: Stichproben-Standardabweichung σ der LLM-Detektor-Findings je Modell und Testsuite ($n = 10$ Runs). Niedrigere Werte bedeuten höhere Run-zu-Run-Konsistenz. Datenbasis: Tabellen 33, 47, 61.

Anhang 67: Recall und Überschuss - nach Suite gruppiert

Das Diagramm zeigt für jedes Modell, welcher Anteil der LLM-Detektor-Rohfindings dem tatsächlichen Recall entspricht (dunkler Balkenanteil) und welcher Anteil darüber hinausgeht (heller Anteil: Quell-Duplikate, Mehrfachinstanzen, nicht eindeutig zugeordnete Findings). In test-12 liegt der Überschuss bei 14b und 32b besonders hoch, weil axe-core und LLM-Detektor viele der 12 Violations unabhängig voneinander finden (Quell-Duplikate). In test-100 bleibt der Recall trotz annähernd gleichem Gesamtvolumen auf 40–63 %, da der Überschuss einen großen Teil der Rohfindings ausmacht. Die Linie bei 100 % markiert die Ground-Truth-Menge. Datenbasis: Recall aus Tabellen 27, 42, 55; Überschuss = LLM-Detektor-Rohfindings minus Recall (vgl. Tabellen 32, 45, 58).

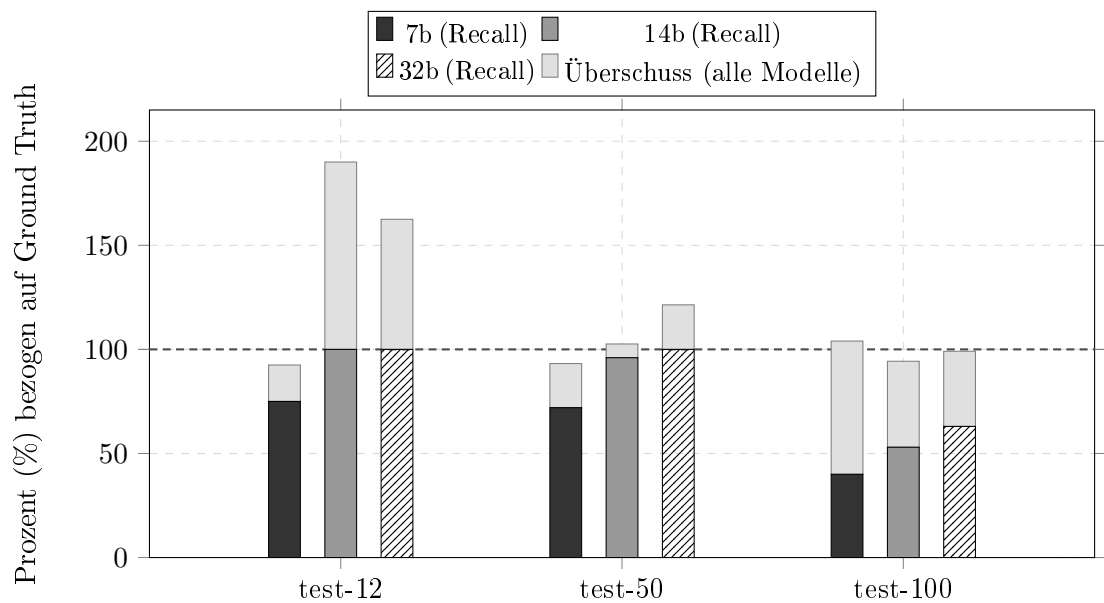


Abb. 27: Gestapeltes Balkendiagramm: **dunkler Anteil** = Recall (konsistent erkannte Violations, Schwelle > 5/10 Runs); **heller Anteil** = Überschuss des LLM-Detektor-Rohoutputs über den Recall hinaus (Quell-Duplikate, Mehrfachinstanzen, nicht zugeordnete Findings). Die Linie bei 100 % markiert die Ground-Truth-Menge. Innerhalb jeder Gruppe: links = 7b, Mitte = 14b, rechts = 32b.

Anhang 68: Truncation-Rate im Suite-Vergleich

Das Diagramm zeigt, in wie vielen der jeweils 10 Runs das Output-Token-Limit ausgeschöpft wurde. Dass **qwen3:32b** in test-50 bei 90 %, in test-100 jedoch bei nur 10 % liegt, ist kein Widerspruch: Das **num_predict**-Budget wurde proportional zur Suite-Größe erhöht (6 144 / 8 192 / 12 288), und 32b produziert kompaktere JSON-Ausgaben als 14b. Datenbasis: Tabellen 27, 42, 55.

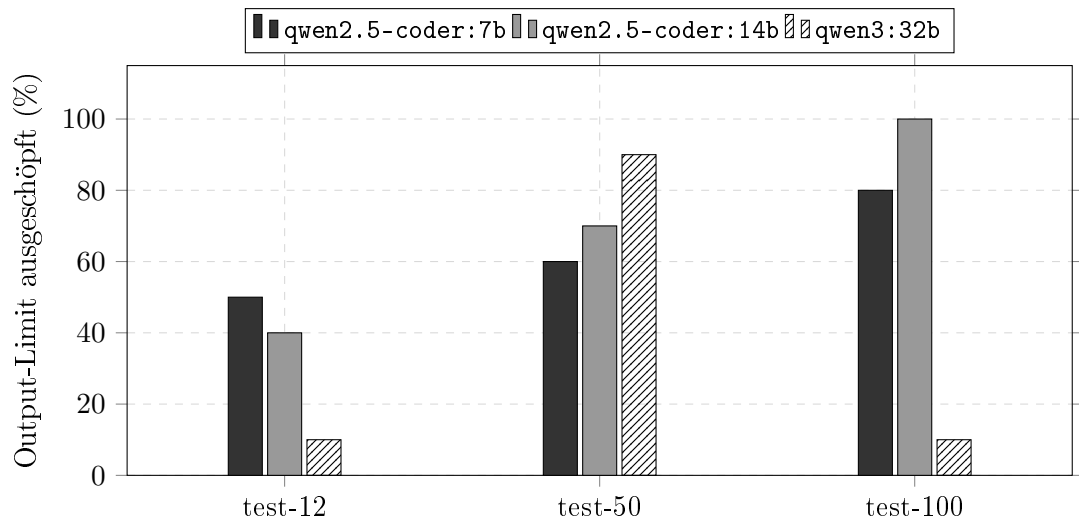


Abb. 28: Anteil der Runs mit ausgeschöpftem Output-Token-Limit je Modell ($n = 10$). Das **num_predict**-Budget wurde skaliert: test-12: 6 144, test-50: 8 192, test-100: 12 288 Tokens. Der Rückgang bei **qwen3:32b** von 90 % (test-50) auf 10 % (test-100) erklärt sich durch dessen kompaktere JSON-Ausgabestruktur. Datenbasis: Tabellen 27, 42, 55.

Anhang 69: Heatmap der Recall-Werte

Die folgende Heatmap zeigt die Recall-Werte (in %) je Modell und Testsuite in kompakter Form. Sie eignet sich als schnelle Übersicht, bevor die detaillierten Recall-Matrizen gelesen werden. Datenbasis: Tabellen 27, 42, 55.

Modell	test-12	test-50	test-100
qwen2.5-coder:7b	75 %	72 %	40 %
qwen2.5-coder:14b	100 %	96 %	53 %
qwen3:32b	100 %	100 %	63 %

 niedrig  mittel  hoch

Abb. 29: Heatmap der Recall-Werte über alle drei Suites. Sie macht die Degradation auf test-100 visuell unmittelbar sichtbar.

Anhang 70: Gestapeltes Balkendiagramm: Must-have vs. Nice-to-have

Das Diagramm zeigt die mittleren Must-have- und Nice-to-have-Anteile pro Modell und Suite. Datenbasis: Tabellen 27, 42, 55.

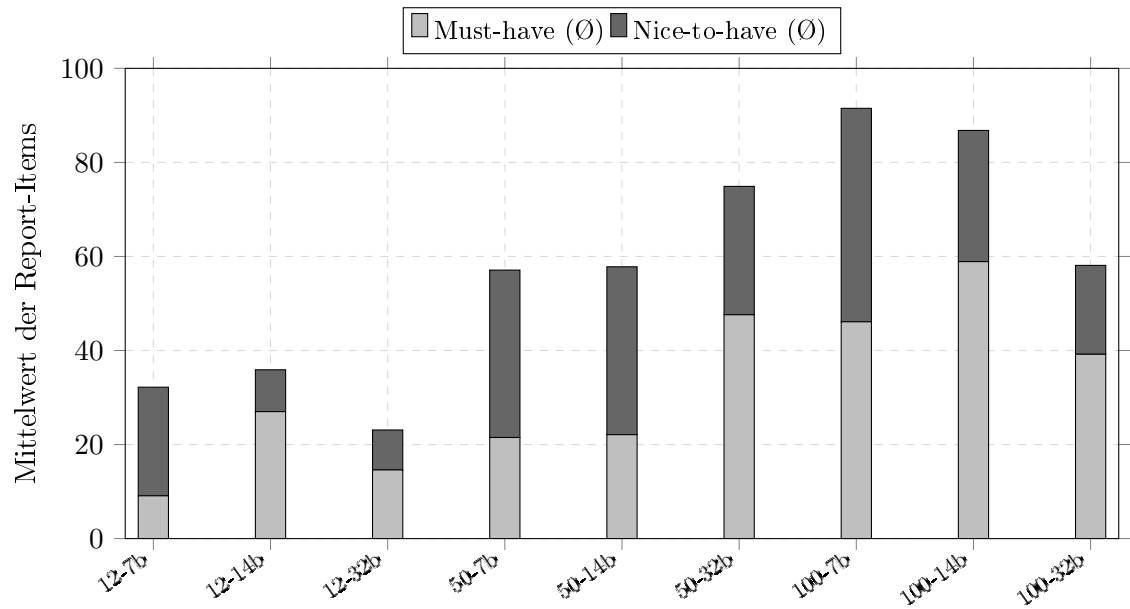


Abb. 30: Gestapelter Vergleich der mittleren Must-have- und Nice-to-have-Items. Die Verlagerung Richtung Must-have bei größeren Suites wird besonders bei `qwen2.5-coder:14b` sichtbar.

Anhang 71: BWEC-Block (kompletter Block)

Hinweis: Der BWEC-Block dokumentiert die explorative Erprobung des PoC am realen Untersuchungsobjekt ohne Ground Truth. Im Unterschied zu test-12, test-50 und test-100 stehen hier daher vor allem Laufzeit-, Skalierungs- und Robustheitsbeobachtungen im Vordergrund.

Tab. 65: Navigationsübersicht in der standardisierten Reihenfolge für den BWEC-Block.

Nr.	Baustein
1	Detektionsquellen-Übersicht
2	Rohdaten (maxfiles = 100)
3	Modell-Leistungsvergleich (maxfiles = 100)
4	Rohdaten (maxfiles = 500)
5	Modell-Leistungsvergleich (maxfiles = 500)

Anhang 72: Detektionsquellen-Übersicht: BWEC

Die Tool-basierten Findings (axe-core, Playwright, grep) sind über alle Runs und Modelle konstant. Der LLM-Detektor liefert variable Ergebnisse je nach Modell und Run. Mangels Ground Truth wird kein Recall ausgewiesen; die Tabellen dokumentieren stattdessen die Zusammensetzung und Kategorisierung der Detektionsergebnisse.

Tab. 66: Überblick über die Findings je Detektionsquelle für den BWEC. Tool-basierte Findings sind laufzeit- und modellunabhängig konstant; die Findings des LLM-Detektors variieren je nach Modell.

Quelle	Findings	WCAG-Kriterien	Typische Befunde	Detektionsart
Tool-basierte Detektion				
axe-core	10	1.1.1 A (2×), 1.4.3 AA (8×)	Farbkontrast, fehlendes alt	statisch
Playwright	2	2.4.1 A (1×), 2.4.7 AA (1×)	Skip-Link, Fokusindikator	dynamisch
grep	8	1.1.1 A (3×), 1.3.1 A (1×), 2.1.1 A (4×)	fehlendes alt, onClick ohne Keyboard, <label> ohne htmlFor	statisch
Gesamt (Tool)	20	Konstante, modellunabhängige Basis-Findings		
LLM-gestützte Detektion				
LLM-Detektor	14–35	verschiedene	kontextuelle und strukturelle Muster	modellbasiert

Tab. 67: Vollständiger Katalog der 20 tool-basierten Findings im BWEC. Konstant über alle Runs und Modelle; LLM-Detektor-Findings nicht aufgeführt (modellabhängig variabel).

ID	Befund	Quelle	WCAG	Schweregrad
axe-001–008	Unzureichender Farbkontrast (8 Elemente)	axe-core	1.4.3 AA	serious
axe-009–010	Fehlendes alt-Attribut bei / .bitbwLogoSvg	axe-core	1.1.1 A	critical
pw-001	Fehlender sichtbarer Fokusindikator (input#password)	Playwright	2.4.7 AA	serious
pw-002	Fehlender Skip-Link	Playwright	2.4.1 A	serious
grep-001–004	<div> mit onClick ohne onKeyDown/role (4 Komponenten)	grep	2.1.1 A	serious
grep-005–007	Fehlendes alt-Attribut bei (3 Dateien)	grep	1.1.1 A	critical
grep-008	<label> ohne htmlFor	grep	1.3.1 / 4.1.2 A	serious

Anhang 73: Rohdaten: BWEC-Messreihe (`maxfiles = 100`, 66 Chunks)

Konfiguration: `OLLAMA_NUM_CTX = 65 536`; `OLLAMA_NUM_PREDICT = 24 576`; `LLM_DETECT_NUM_PREDICT = 4`
 Chunk-Größe = 4 000 Zeichen. Tool-Findings über alle Runs konstant: `axe-core = 10`, `Playwright = 2`, `grep = 8`. Run 03 von 7b wird als Ausreißer markiert, da die Priorisierungsphase 205 Nice-to-have-Einträge erzeugte und das Output-Budget vollständig ausschöpfte.

Tab. 68: Rohdaten der BWEC-Messreihe bei `maxfiles = 100`. Det. = LLM-Detektor-Findings; OutTok. = Output-Tokens; Trunc. = Truncation der Priorisierungsphase.

Run	Modell	Det.	OutTok.	Trunc.	Must	Nice	Dauer (s)
01	7b	14	3 369	nein	5	22	319
02	7b	15	3 400	nein	5	23	337
03 [†]	7b	16	24 576	nein*	4	205	907
04	14b	22	6 557	nein	42	0	833
05	14b	22	6 485	nein	37	5	828
06	14b	22	6 230	nein	42	0	811
07	32b	30	7 302	nein	36	14	2 104
08	32b	31	7 152	nein	37	14	2 088
09	32b	35	7 616	nein	39	16	2 194

Modellkürzel: 7b = `qwen2.5-coder:7b`, 14b = `qwen2.5-coder:14b`, 32b = `qwen3:32b`.

[†] Ausreißer; aus Mittelwertberechnungen ausgeschlossen: Priorisierungsphase erzeugte 205 Nice-to-have-Einträge und schöpfte das Output-Budget (24 576 Tokens) vollständig aus.

* Das `tokensBudgetTruncated`-Flag zeigte dennoch `false` (Tracking-Lücke im Code).

Anhang 74: Modell-Leistungsvergleich: BWEC (`maxfiles` = 100)

Tab. 69: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle auf dem realen Untersuchungsobjekt BWEC (`maxfiles` = 100). Aufgrund fehlender Ground Truth werden keine Recall-Metriken ausgewiesen.

Metrik	7b	14b	32b
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	14,5 $\pm$ 0,7	22,0 $\pm$ 0,0	32,0 $\pm$ 2,6
LLM-Detektor-Findings (Min–Max)	14–15	22–22	30–35
Output-Tokens ($\bar{O}$)	3 385	6 424	7 357
Must-have im Bericht ($\bar{O}$)	5,0	40,3	37,3
Nice-to-have im Bericht ($\bar{O}$)	22,5	1,7	14,7
Gesamt-Report-Items ($\bar{O}$)	27,5	42,0	52,0
Gesamtlaufzeit ($\bar{O}$)	328 s	824 s	2 129 s
Gesamtlaufzeit (Min–Max)	319–337 s	811–833 s	2 088–2 194 s
NFA-02-Konformität	Erfüllt	Nicht erfüllt	Nicht erfüllt
Priorisierungsstatus	2/3 stabil, 1 Ausreißer	stabil	stabil

Modellkürzel: 7b = `qwen2.5-coder:7b`, 14b = `qwen2.5-coder:14b`, 32b = `qwen3:32b`.

Für 7b wurden die Mittelwerte ohne Run 03 berechnet, da dieser Run als dokumentierter Ausreißer klassifiziert wurde.

Gesamt-Report-Items = Must-have + Nice-to-have. σ = Stichproben-Standardabweichung der LLM-Detektor-Findings.

Anhang 75: Rohdaten: BWEC-Messreihe (`maxfiles = 500`, 330 Chunks)

Konfiguration: Identisch zu Anhang 73; lediglich `maxfiles = 500` (330 LLM-Detektor-Chunks statt 66). Tool-Findings konstant: `axe-core = 10`, `Playwright = 2`, `grep = 8`. † markiert partielle Priorisierung (nur Must-have), ‡ vollständige Truncation der Priorisierungsphase, § einen fehlgeschlagenen bzw. kollabierten Priorisierungsooutput.

Tab. 70: Rohdaten der BWEC-Messreihe bei `maxfiles = 500`. PromptTok. = Prompt-Tokens; OutTok. = Output-Tokens.

Run	Modell	Det.	PromptTok.	OutTok.	Must	Nice	Dauer (s)
01	7b	258	—	6 499	5	38	2 219
02	7b	253	—	8 014	7	47	2 226
03 [†]	7b	261	—	2 022	10	0	2 169
04 [‡]	14b	268	32 080	24 576	192	0	5 524
05 [‡]	14b	271	32 252	24 576	176	0	5 525
06 [§]	14b	274	32 768	2 213	0	0	4 242
07 [§]	32b	418	40 960	11 450	0	0	13 653
08 [§]	32b	426	40 960	3 442	0	0	13 481
09 [§]	32b	400	40 960	24 576	0	0	17 517

Modellkürzel: 7b = `qwen2.5-coder:7b`, 14b = `qwen2.5-coder:14b`, 32b = `qwen3:32b`.

[†] Priorisierungsphase partiell: nur Must-have ausgegeben, Nice-to-have abgeschnitten.

[‡] Output-Budget (24 576 Tokens) vollständig ausgeschöpft; Report unvollständig.

[§] Priorisierungsphase fehlgeschlagen bzw. kollabiert (Must = 0, Nice = 0, kein parsebarer Report). Bei 32b gilt PromptTok. = 40 960 = NUM_CTX – NUM_PREDICT; der Eingabekontext war damit vollständig gesättigt.

Anhang 76: Modell-Leistungsvergleich: BWEC (maxfiles = 500)

Die Konfiguration war identisch zu `maxfiles = 100`; lediglich der Dateiraum wurde auf 500 Quelldateien (330 Chunks) ausgedehnt. Die Faktoren beziehen sich auf die jeweiligen Basiswerte aus Tabelle 69.

Tab. 71: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle auf dem realen Untersuchungsobjekt BWEC bei `maxfiles = 500`.

Metrik	7b	14b	32b
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	257,3 $\pm$ 4,0	271,0 $\pm$ 3,0	414,7 $\pm$ 13,3
LLM-Detektor-Findings (Min–Max)	253–261	268–274	400–426
Prompt-Tokens ($\bar{O}$)	–	32 367	40 960
Output-Tokens ($\bar{O}$)	5 512	17 122	13 156
Gesamtlaufzeit ($\bar{O}$)	2 205 s	5 097 s	14 884 s
Gesamtlaufzeit (Min–Max)	2 169–2 226 s	4 242–5 525 s	13 481–17 517 s
Findings-Faktor ggü. <code>maxfiles=100</code>	17,7 $\times$	12,3 $\times$	13,0 $\times$
Zeit-Faktor ggü. <code>maxfiles=100</code>	6,7 $\times$	6,2 $\times$	7,0 $\times$
NFA-02-Konformität	Nicht erfüllt	Nicht erfüllt	Nicht erfüllt
Priorisierungsstatus	partiell (1/3)	Truncation (2/3), Kollaps (1/3)	fehlgeschlagen (3/3)

Modellkürzel: 7b = `qwen2.5-coder:7b`, 14b = `qwen2.5-coder:14b`, 32b = `qwen3:32b`.

Findings-Faktor = $\bar{O}$ LLM-Detektor-Findings bei `maxfiles = 500` relativ zu `maxfiles = 100`.

Zeit-Faktor = $\bar{O}$ Gesamtlaufzeit bei `maxfiles = 500` relativ zu `maxfiles = 100`.

Die Ergebnisse zeigen eine deutlich nicht-lineare Skalierung der Priorisierungsphase: Während die Laufzeit etwa 6–7-fach steigt, wächst die Zahl der LLM-Detektor-Findings je nach Modell um den Faktor 12–18.

Literaturverzeichnis

Wissenschaftliche Artikel und Konferenzbeiträge

- Abou-Zahra, Shadi; Brewer, Judy; Cooper, Michael (2018):** Artificial Intelligence (AI) for Web Accessibility: Is Conformance Evaluation a Way Forward? In: *Proceedings of the 15th International Web for All Conference*. W4A '18. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 978-1-4503-5651-0. DOI: 10.1145/3192714.3192834. URL: <https://dl.acm.org/doi/10.1145/3192714.3192834> (Abruf: 24.02.2026).
- Abuaddous, Hayfa Y.; Jali, Mohd Zalisham; Basir, Nurlida (2016):** Web Accessibility Challenges. In: *International Journal of Advanced Computer Science and Applications (ijacsa)* 7.10. ISSN: 2156-5570. DOI: 10.14569/IJACSA.2016.071023. URL: <https://thesai.org/Publications/ViewPaper?Volume=7&Issue=10&Code=ijacsa&SerialNo=23> (Abruf: 04.03.2026).
- Bai, Aiden (2022):** A Fast Compiler-Augmented Virtual DOM for the Web. In: *arXiv*. URL: <https://arxiv.org/abs/2202.08409> (Abruf: 05.03.2026).
- Baskerville, Richard; Myers, Michael D. (2004):** Special Issue on Action Research in Information Systems: Making IS Research Relevant to Practice Foreword. In: *MIS Quarterly* 28.3, S. 329–335.
- Baskerville, Richard L. (1999):** Investigating Information Systems with Action Research. In: *Communications of the Association for Information Systems* 2.19, S. 1–32.
- Bassi, Barry; Delnevo, Giovanni; Franco, Mirko; Gaggi, Ombretta; Gatto, Salvatore; Mirri, Silvia; Olaiya, Kelvin (2025):** An Assessment of LLM-Based Auditing and Validation for Web Accessibility. In: *Proceedings of the 2025 International Conference on Information Technology for Social Good*. GoodIT '25. New York, NY, USA: Association for Computing Machinery, S. 297–305. ISBN: 979-8-4007-2089-5. DOI: 10.1145/3748699.3749805. URL: <https://dl.acm.org/doi/10.1145/3748699.3749805> (Abruf: 24.02.2026).
- Bender, Emily M.; Gebru, Timnit; McMillan-Major, Angelina; Shmitchell, Shmargaret (2021):** On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. FAccT '21. New York, NY, USA: Association for Computing Machinery, S. 610–623. ISBN: 978-1-4503-8309-7. DOI: 10.1145/3442188.3445922. URL: <https://dl.acm.org/doi/10.1145/3442188.3445922> (Abruf: 05.03.2026).
- Brown, Tom; Mann, Benjamin; Ryder, Nick; Subbiah, Melanie; Kaplan, Jared D; Dhariwal, Prafulla; Neelakantan, Arvind; Shyam, Pranav; Sastry, Girish; Askell, Amanda; Agarwal, Sandhini; Herbert-Voss, Ariel; Krueger, Gretchen; Henighan, Tom; Child, Rewon; Ramesh, Aditya; Ziegler, Daniel; Wu, Jeffrey; Winter, Clemens; Hesse, Chris; Chen, Mark; Sigler, Eric; Litwin, Mateusz; Gray, Scott; Chess, Benjamin; Clark, Jack; Berner, Christopher; McCandlish, Sam; Radford, Alec; Sutskever, Ilya; Amodei, Dario (2020):** Language Models Are Few-Shot Learners. In: *Advances in Neural Information Processing Systems*. Bd. 33. Curran Associates,

- Inc., S. 1877–1901. URL: https://papers.nips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html (Abruf: 06.04.2026).
- Brynjolfsson, Erik; Li, Danielle; Raymond, Lindsey (2025):** Generative AI at Work*. In: *The Quarterly Journal of Economics* 140.2, S. 889–942. ISSN: 0033-5533. DOI: 10.1093/qje/qjae044. URL: <https://doi.org/10.1093/qje/qjae044> (Abruf: 06.04.2026).
- Chiou, Paul T.; Alotaibi, Ali S.; Halfond, William G. J. (2021):** Detecting and localizing keyboard accessibility failures in web applications. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, S. 855–867. ISBN: 978-1-4503-8562-6. DOI: 10.1145/3468264.3468581. URL: <https://dl.acm.org/doi/10.1145/3468264.3468581> (Abruf: 17.03.2026).
- Delnevo, Giovanni; Andruccioli, Manuel; Mirri, Silvia (2024):** On the Interaction with Large Language Models for Web Accessibility: Implications and Challenges. In: *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*. 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC), S. 1–6. DOI: 10.1109/CCNC51664.2024.10454680. URL: <https://ieeexplore.ieee.org/document/10454680> (Abruf: 24.02.2026).
- Doush, Iyad Abu; Kassem, Reem (2025):** Evaluating AI-Generated Web Code for Accessibility Compliance: A Metric-Driven Approach. In: *Proceedings of the 11th International Conference on Software Development and Technologies for Enhancing Accessibility and Fighting Info-exclusion*. DSAI '24. New York, NY, USA: Association for Computing Machinery, S. 338–344. ISBN: 979-8-4007-0729-2. DOI: 10.1145/3696593.3696614. URL: <https://dl.acm.org/doi/10.1145/3696593.3696614> (Abruf: 24.02.2026).
- Fathallah, Nadeen; Hernández, Daniel; Staab, Steffen (2025):** AccessGuru: Leveraging LLMs to Detect and Correct Web Accessibility Violations in HTML Code. In: *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '25. New York, NY, USA: Association for Computing Machinery, S. 1–22. ISBN: 979-8-4007-0676-9. DOI: 10.1145/3663547.3746360. URL: <https://dl.acm.org/doi/10.1145/3663547.3746360> (Abruf: 24.02.2026).
- Fernandes, Nádia; Costa, Daniel; Duarte, Carlos; Carriço, Luís (2012):** Evaluating the Accessibility of Web Applications. In: *Procedia Computer Science*. Proceedings of the 4th International Conference on Software Development for Enhancing Accessibility and Fighting Info-exclusion (DSAI 2012) 14, S. 28–35. ISSN: 1877-0509. DOI: 10.1016/j.procs.2012.10.004. URL: <https://www.sciencedirect.com/science/article/pii/S1877050912007661> (Abruf: 05.03.2026).
- Fernández-Navarro, Carla; Chicano, Francisco (2026):** Automated LLM-Based Accessibility Remediation: From Conventional Websites to Angular Single-Page Applications. In: arXiv: 2602.17887 [cs.SE]. URL: <https://arxiv.org/abs/2602.17887>.
- Gubbi Mohanbabu, Ananya; Pavel, Amy (2024):** Context-Aware Image Descriptions for Web Accessibility. In: *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '24. New York, NY, USA: Association for Computing

- Machinery, S. 1–17. ISBN: 979-8-4007-0677-6. DOI: 10.1145/3663548.3675658. URL: <https://dl.acm.org/doi/10.1145/3663548.3675658> (Abruf: 24.02.2026).
- Guo, Daya et al. (2025):** DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. In: *Nature* 645.8081, S. 633–638. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09422-z. URL: <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- Guriță, Alexandra-Elena; Vatavu, Radu-Daniel (2025):** Insights and Implications of Evaluating Accessibility Compliance in AI-Generated Web Interfaces. In: *Companion Proceedings of the ACM on Web Conference 2025*. WWW '25. New York, NY, USA: Association for Computing Machinery, S. 996–1000. ISBN: 979-8-4007-1331-6. DOI: 10.1145/3701716.3715552. URL: <https://dl.acm.org/doi/10.1145/3701716.3715552> (Abruf: 24.02.2026).
- He, Ziyao; Huq, Syed Fatiul; Malek, Sam (2025):** Enhancing Web Accessibility: Automated Detection of Issues with Generative AI. In: *Proc. ACM Softw. Eng.* 2 (FSE), FSE101:2264–FSE101:2287. DOI: 10.1145/3729371. URL: <https://dl.acm.org/doi/10.1145/3729371> (Abruf: 24.02.2026).
- Hevner, Alan R.; March, Salvatore T.; Park, Jinsoo; Ram, Sudha (2004):** Design Science in Information Systems Research. In: *MIS Quarterly* 28.1, S. 75–105.
- Huang, Calista; Ma, Alyssa; Vyasamudri, Suchir; Puype, Eugenie; Kamal, Sayem; Garcia, Juan Belza; Cheema, Salar; Lutz, Michael (2024):** ACCESS: Prompt Engineering for Automated Web Accessibility Violation Corrections. In: arXiv:2401.16450. arXiv. DOI: 10.48550/arXiv.2401.16450. arXiv: 2401.16450[cs]. URL: <http://arxiv.org/abs/2401.16450> (Abruf: 25.02.2026).
- Huang, Lei; Yu, Weijiang; Ma, Weitao; Zhong, Weihong; Feng, Zhangyin; Wang, Haotian; Chen, Qianglong; Peng, Weihua; Feng, Xiaocheng; Qin, Bing; Liu, Ting (2025):** A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. In: *ACM Transactions on Information Systems* 43.2, S. 1–55. ISSN: 1046-8188, 1558-2868. DOI: 10.1145/3703155. arXiv: 2311.05232[cs]. URL: <http://arxiv.org/abs/2311.05232> (Abruf: 05.03.2026).
- Hui, Binyuan; Yang, Jian; Cui, Zeyu; Yang, Jiaxi; Liu, Dayiheng; Zhang, Lei; Liu, Tianyu; Zhang, Jiajun; Yu, Bowen; Lu, Keming; Dang, Kai; Fan, Yang; Zhang, Yichang; Yang, An; Men, Rui; Huang, Fei; Zheng, Bo; Miao, Yibo; Quan, Shanghaoran; Feng, Yunlong; Ren, Xingzhang; Ren, Xuancheng; Zhou, Jingren; Lin, Junyang (2024):** Qwen2.5-Coder Technical Report. In: arXiv:2409.12186. arXiv. DOI: 10.48550/arXiv.2409.12186. arXiv: 2409.12186[cs]. URL: <http://arxiv.org/abs/2409.12186> (Abruf: 15.03.2026).
- Kodithuwakku, Tashmi; Wickramaarachchi, Dilani (2025):** Assessing the Limits of Automated Accessibility Testing: Insights for QA Engineers and Tool Developers. In: *2025 5th International Conference on Advanced Research in Computing (ICARC)*. 2025 5th International Conference on Advanced Research in Computing (ICARC), S. 1–6. DOI: 10.1109/ICARC64760.2025.10963173. URL: <https://ieeexplore.ieee.org/document/10963173> (Abruf: 24.02.2026).

- Krok, Ewa (2024)**: Digital Accessibility as an Essential Element of an Inclusive Organization. In: *European Research Studies XXVII* (Special A), S. 857–876. ISSN: 1108-2976. URL: <https://ersj.eu/journal/3752> (Abruf: 17.03.2026).
- Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra; Petroni, Fabio; Karpukhin, Vladimir; Goyal, Naman; Küttler, Heinrich; Lewis, Mike; Yih, Wen-tau; Rocktäschel, Tim; Riedel, Sebastian; Kiela, Douwe (2020)**: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*. Hrsg. von H. Larochelle; M. Ranzato; R. Hadsell; M.F. Balcan; H. Lin. Bd. 33. Curran Associates, Inc., S. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
- Manca, Marco; Palumbo, Vanessa; Paternò, Fabio; Santoro, Carmen (2023)**: The Transparency of Automatic Web Accessibility Evaluation Tools: Design Criteria, State of the Art, and User Perception. In: *ACM Trans. Access. Comput.* 16.1, 3:1–3:36. ISSN: 1936-7228. DOI: 10.1145/3556979. URL: <https://dl.acm.org/doi/10.1145/3556979> (Abruf: 24.02.2026).
- Morris, Meredith Ringel; Zolyomi, Annuska; Yao, Catherine; Bahram, Sina; Bigham, Jeffrey P.; Kane, Shaun K. (2016)**: "With most of it being pictures now, I rarely use it: Understanding Twitter's Evolving Accessibility to Blind Users. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. CHI '16. New York, NY, USA: Association for Computing Machinery, S. 5506–5516. ISBN: 978-1-4503-3362-7. DOI: 10.1145/2858036.2858116. URL: <https://dl.acm.org/doi/10.1145/2858036.2858116> (Abruf: 04.03.2026).
- Morrison, Cecily; Cutrell, Edward; Dhareshwar, Anupama; Doherty, Kevin; Thieme, Anja; Taylor, Alex (2017)**: Imagining Artificial Intelligence Applications with People with Visual Disabilities using Tactile Ideation. In: *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '17. New York, NY, USA: Association for Computing Machinery, S. 81–90. ISBN: 978-1-4503-4926-0. DOI: 10.1145/3132525.3132530. URL: <https://dl.acm.org/doi/10.1145/3132525.3132530> (Abruf: 04.03.2026).
- Mowar, Peya (2024)**: Accessibility in AI-Assisted Web Development. In: *Proceedings of the 21st International Web for All Conference*. W4A '24. New York, NY, USA: Association for Computing Machinery, S. 123–125. ISBN: 979-8-4007-1030-8. DOI: 10.1145/3677846.3679054. URL: <https://dl.acm.org/doi/10.1145/3677846.3679054> (Abruf: 24.02.2026).
- Njifenjou, Ahmed; Sucal, Virgile; Jabaian, Bassam; Lefèvre, Fabrice (2024)**: Role-Play Zero-Shot Prompting with Large Language Models for Open-Domain Human-Machine Conversation. In: arXiv:2406.18460. arXiv. DOI: 10.48550/arXiv.2406.18460. arXiv: 2406.18460[cs]. URL: <http://arxiv.org/abs/2406.18460> (Abruf: 28.02.2026).
- OpenAI (2023)**: GPT-4 Technical Report. In: arXiv: 2303.08774 [cs.CL]. URL: <https://arxiv.org/abs/2303.08774> (Abruf: 01.03.2026).
- Paternò, Fabio; Vinci, Manuela; Manca, Marco; Iannuzzi, Nicola (2025)**: How an LLM Can Improve Automatic Web Accessibility Validation ? In: *Proceedings of the 16th Biannual*

- Conference of the Italian SIGCHI Chapter*. CHIItaly '25. New York, NY, USA: Association for Computing Machinery, S. 1–8. ISBN: 979-8-4007-2102-1. DOI: 10.1145/3750069.3750310. URL: <https://dl.acm.org/doi/10.1145/3750069.3750310> (Abruf: 24.02.2026).
- Peffer, Ken; Tuunanen, Tuure; Rothenberger, Marcus A.; Chatterjee, Samir (2007):** A Design Science Research Methodology for Information Systems Research. In: *Journal of Management Information Systems* 24.3, S. 45–77. DOI: 10.2753/MIS0742-1222240302.
- Pérez, J. Eduardo; Arrue, Myriam; Valencia, Xabier; Abascal, Julio (2020):** Longitudinal Study of Two Virtual Cursors for People With Motor Impairments: A Performance and Satisfaction Analysis on Web Navigation. In: *IEEE Access*. DOI: 10.1109/ACCESS.2020.3001766.
- Pool, Jonathan Robert (2023):** Testaro: Efficient Ensemble Testing for Web Accessibility. In: *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '23. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 979-8-4007-0220-4. DOI: 10.1145/3597638.3614505. URL: <https://dl.acm.org/doi/10.1145/3597638.3614505> (Abruf: 24.02.2026).
- Purwandari, Nuraini; Dewi, Ratna Kusuma; Rinaldo; Sucipto, Purwo Agus (2025):** Policy in Practice: A Systematic Review of WCAG 2.2 and ADA 2024 Effects on Web and Mobile Accessibility. In: *Digitus*. DOI: 10.61978/digitus.v3i2.957. URL: https://www.researchgate.net/publication/396366120_Policy_in_Practice_A_Systematic_Review_of_WCAG_22_and_ADA_2024_Effects_on_Web_and_Mobile_Accessibility (Abruf: 04.03.2026).
- Schulhoff, Sander; Ilie, Michael; Balepur, Nishant; Kahadze, Konstantine; Liu, Amanda; Si, Chenglei; Li, Yinheng; Gupta, Aayush; Han, HyoJung; Schulhoff, Sevien; Dulepet, Pranav Sandeep; Vidyadhara, Saurav; Ki, Dayeon; Agrawal, Sweta; Pham, Chau; Kroiz, Gerson; Li, Feileen; Tao, Hudson; Srivastava, Ashay; Da Costa, Hevander; Gupta, Saloni; Rogers, Megan L.; Goncarenco, Inna; Sarli, Giuseppe; Galynker, Igor; Peskoff, Denis; Carpuat, Marine; White, Jules; Anadkat, Shyamal; Hoyle, Alexander; Resnik, Philip (2024):** The Prompt Report: A Systematic Survey of Prompting Techniques. In: arXiv: 2406.06608 [cs.CL]. URL: <https://arxiv.org/abs/2406.06608> (Abruf: 09.03.2026).
- Silvestre, Pedro F.; Pietzuch, Peter (2025):** Systems Opportunities for LLM Fine-Tuning using Reinforcement Learning. In: *Proceedings of the 5th Workshop on Machine Learning and Systems*. EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, S. 90–99. ISBN: 979-8-4007-1538-9. DOI: 10.1145/3721146.3721944. URL: <https://dl.acm.org/doi/10.1145/3721146.3721944> (Abruf: 09.03.2026).
- Souza, Aline; de Souza Filho, José Cezar; Bezerra, Carla; Alves, Victor Anthony; Lima, Lara; Marques, Anna Beatriz; Teixeira Monteiro, Ingrid (2024):** Accessibility Evaluation of Web Systems for People with Visual Impairments: Findings from a Literature Survey. In: *Proceedings of the XXIII Brazilian Symposium on Human Factors in Computing Systems*. IHC '24. New York, NY, USA: Association for Computing Machinery, S. 1–13. ISBN: 979-8-4007-1224-1. DOI: 10.1145/3702038.3702090. URL: <https://dl.acm.org/doi/10.1145/3702038.3702090> (Abruf: 04.03.2026).

- Tafreshipour, Mahan; Deshpande, Anmol; Mehralian, Forough; Ahmed, Iftekhhar; Malek, Sam (2024):** Mally: A Mutation Framework for Web Accessibility Testing. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, S. 100–111. ISBN: 979-8-4007-0612-7. DOI: 10.1145/3650212.3652113. URL: <https://dl.acm.org/doi/10.1145/3650212.3652113> (Abruf: 24.02.2026).
- Touvron, Hugo; Martin, Louis; Stone, Kevin; Albert, Peter; Almahairi, Amjad; Babaei, Yasmine; Bashlykov, Nikolay; Batra, Soumya; Bhargava, Prajjwal; Bhosale, Shruti; Bikel, Dan; Blecher, Lukas; Ferrer, Cristian Canton; Chen, Moya; Cucurull, Guillem; Esiobu, David; Fernandes, Jude; Fu, Jeremy; Fu, Wenying; Fuller, Brian; Gao, Cynthia; Goswami, Vedanuj; Goyal, Naman; Hartshorn, Anthony; Hosseini, Saghar; Hou, Rui; Inan, Hakan; Kardas, Marcin; Kerkez, Viktor; Khabsa, Madian; Kloumann, Isabel; Korenev, Artem; Koura, Punit Singh; Lachaux, Marie-Anne; Lavril, Thibaut; Lee, Jenya; Liskovich, Diana; Lu, Yinghai; Mao, Yuning; Martinet, Xavier; Mihaylov, Todor; Mishra, Pushkar; Molybog, Igor; Nie, Yixin; Poulton, Andrew; Reizenstein, Jeremy; Rungta, Rashi; Saladi, Kalyan; Schelten, Alan; Silva, Ruan; Smith, Eric Michael; Subramanian, Ranjan; Tan, Xiaoqing Ellen; Tang, Binh; Taylor, Ross; Williams, Adina; Kuan, Jian Xiang; Xu, Puxin; Yan, Zheng; Zarov, Iliyan; Zhang, Yuchen; Fan, Angela; Kambadur, Melanie; Narang, Sharan; Rodriguez, Aurelien; Stojnic, Robert; Edunov, Sergey; Scialom, Thomas (2023):** Llama 2: Open Foundation and Fine-Tuned Chat Models. In: arXiv: 2307.09288 [cs.CL]. URL: <https://arxiv.org/abs/2307.09288> (Abruf: 01.03.2026).
- Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Lukasz; Polosukhin, Illia (2023):** Attention Is All You Need. In: arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- Wang, Yuqing; Zhao, Yun (2024):** Metacognitive Prompting Improves Understanding in Large Language Models. In: arXiv:2308.05342. arXiv. DOI: 10.48550/arXiv.2308.05342. arXiv: 2308.05342[cs]. URL: <http://arxiv.org/abs/2308.05342> (Abruf: 09.03.2026).
- Webster, Jane; Watson, Richard T. (2002):** Analyzing the Past to Prepare for the Future: Writing a Literature Review. In: *MIS Quarterly* 26.2, S. 2–6.
- Wei, Jason; Tay, Yi; Bommasani, Rishi; Raffel, Colin; Zoph, Barret; Borgeaud, Sebastian; Yogatama, Dani; Bosma, Maarten; Zhou, Denny; Metzler, Donald; Chi, Ed H.; Hashimoto, Tatsunori; Vinyals, Oriol; Liang, Percy; Dean, Jeffrey; Fedus, William (2022):** Emergent Abilities of Large Language Models. In: *Transactions on Machine Learning Research*, S. 1–35. URL: <https://arxiv.org/abs/2206.07682> (Abruf: 28.02.2026).
- Wei, Jason; Wang, Xuezhi; Schuurmans, Dale; Bosma, Maarten; Ichter, brian; Xia, Fei; Chi, Ed; Le, Quoc V; Zhou, Denny (2022):** Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems*. Hrsg. von S. Koyejo; S. Mohamed; A. Agarwal; D. Belgrave; K. Cho; A. Oh. Bd. 35. Curran Associates, Inc., S. 24824–24837. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.

- Yang, An; Li, Anfeng; Yang, Baosong; Zhang, Beichen; Hui, Binyuan; Zheng, Bo; Yu, Bowen; Gao, Chang; Huang, Chengen; Lv, Chenxu; Zheng, Chujie; Liu, Dayiheng; Zhou, Fan; Huang, Fei; Hu, Feng; Ge, Hao; Wei, Haoran; Lin, Huan; Tang, Jialong; Yang, Jian; Tu, Jianhong; Zhang, Jianwei; Yang, Jianxin; Yang, Jiayi; Zhou, Jing; Zhou, Jingren; Lin, Junyang; Dang, Kai; Bao, Keqin; Yang, Kexin; Yu, Le; Deng, Lianghao; Li, Mei; Xue, Mingfeng; Li, Mingze; Zhang, Pei; Wang, Peng; Zhu, Qin; Men, Rui; Gao, Ruize; Liu, Shixuan; Luo, Shuang; Li, Tianhao; Tang, Tianyi; Yin, Wenbiao; Ren, Xingzhang; Wang, Xinyu; Zhang, Xinyu; Ren, Xuancheng; Fan, Yang; Su, Yang; Zhang, Yichang; Zhang, Yinger; Wan, Yu; Liu, Yuqiong; Wang, Zekun; Cui, Zeyu; Zhang, Zhenru; Zhou, Zhipeng; Qiu, Zihan (2025): Qwen3 Technical Report. In: arXiv: 2505.09388 [cs.CL]. URL: <https://arxiv.org/abs/2505.09388>.**
- Yao, Shunyu; Zhao, Jeffrey; Yu, Dian; Du, Nan; Shafran, Izhak; Narasimhan, Karthik R.; Cao, Yuan (2022):** ReAct: Synergizing Reasoning and Acting in Language Models. In: The Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=WE_vluYUL-X (Abruf: 06.04.2026).

Bücher und Sammelwerke

- Flanagan, David (2020):** JavaScript: The Definitive Guide. 7. Aufl. Sebastopol, CA: O'Reilly Media. ISBN: 978-1491952023.
- Heinemann, Daniela (2024):** „Barrierefreiheit“. In: *Digitalisierte Verwaltung - Vernetztes E-Government*. Hrsg. von Margrit Seckelmann. Berlin: Erich Schmidt Verlag GmbH & Co. KG, S. 469–488. ISBN: 978-3-503-23763-0. DOI: 10.37307/b.978-3-503-23763-0.15. URL: <https://doi.org/10.37307/b.978-3-503-23763-0.15> (Abruf: 24.02.2026).
- Hevner, Alan; Chatterjee, Samir (2010):** Design Research in Information Systems: Theory and Practice. Bd. 22. Integrated Series in Information Systems. New York, NY: Springer. DOI: 10.1007/978-1-4419-5653-8.
- Jurafsky, Daniel; Martin, James H. (2026):** Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. 3rd. Online manuscript released January 6, 2026. URL: <https://web.stanford.edu/~jurafsky/slp3/> (Abruf: 12.04.2026).
- Kerkmann, Friederike; Lewandowski, Dirk (2015):** Barrierefreie Informationssysteme: Zugänglichkeit für Menschen mit Behinderung in Theorie und Praxis. Walter de Gruyter GmbH & Co KG. 292 S. ISBN: 978-3-11-033729-7.
- Kouroupetroglou, Georgios (2013):** Assistive Technologies and Computer Access for Motor Disabilities. IGI Global Scientific Publishing. ISBN: 978-1-4666-4438-0.
- Lazar, Jonathan; Feng, Jinjuan Heidi; Hochheiser, Harry (2017):** Research Methods in Human-Computer Interaction. Elsevier. ISBN: 978-0-12-805390-4.
- Russell, Stuart; Norvig, Peter (2021):** Artificial Intelligence: A Modern Approach. Hoboken: Pearson. 1136 S. ISBN: 978-0-13-461099-3.

- Sommerville, Ian (2016)**: Software Engineering. 10. Aufl. Pearson. ISBN: 978-0133943030.
- Yin, Robert K. (2018)**: Case Study Research and Applications: Design and Methods. Los Angeles London New Delhi Singapore Washington DC Melbourne: SAGE Publications, Inc. 352 S. ISBN: 978-1-5063-3616-9.

Internetquellen und Medien

- BITBW (2021)**: 6 Jahre BITBW. URL: <https://bitbw.de/neuigkeiten/artikel/6-jahre-bitbw-1> (Abruf: 28.02.2026).
- Krotoff, Tanguy (2026)**: Front-end frameworks popularity (React, Vue, Angular and Svelte). Gist. URL: <https://gist.github.com/tkrotoff/b1caa4c3a185629299ec234d2314e190> (Abruf: 05.03.2026).
- Land Baden-Württemberg (2025)**: Neuer BW-Empfangsclient treibt Digitalisierung der Verwaltung voran. URL: <https://www.baden-wuerttemberg.de/de/service/presse/pressemitteilung/pid/neuer-bw-empfangsclient-treibt-digitalisierung-der-verwaltung-voran-1> (Abruf: 24.02.2026).
- Stack Overflow (2025)**: Survey index. URL: <https://survey.stackoverflow.co/2025/> (Abruf: 05.03.2026).
- WebAIM (2019)**: The WebAIM Million - 2019 report on the accessibility of the top 1,000,000 home pages. URL: <https://webaim.org/projects/million/2019> (Abruf: 24.02.2026).
- WebAIM (2025)**: The WebAIM Million - 2025 report on the accessibility of the top 1,000,000 home pages. URL: <https://webaim.org/projects/million/> (Abruf: 24.02.2026).
- WebAIM (2026b)**: WebAIM: Keyboard Accessibility. URL: <https://webaim.org/techniques/keyboard/> (Abruf: 04.03.2026).
- World Health Organization (2026)**: Disability. URL: https://www.who.int/health-topics/disability#tab=tab_1 (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2026a)**: Evaluation Tool List. URL: <https://www.w3.org/WAI/test-evaluate/tools/list/> (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2026b)**: Web Accessibility Initiative. URL: <https://www.w3.org/WAI/> (Abruf: 28.02.2026).

Technische Dokumentation und Software

- Anthropic (2026a)**: *Claude Model Overview*. URL: <https://www.anthropic.com/claude> (Abruf: 01.03.2026).
- Anthropic (2026b)**: *System Card: Claude Sonnet 4.6*. URL: <https://www-cdn.anthropic.com/78073f739564e986ff3e28522761a7a0b4484f84.pdf> (Abruf: 02.03.2026).
- Deque Systems (2026)**: *axe-core: Accessibility engine for automated Web UI testing*. Version 4.10.3. URL: <https://github.com/dequelabs/axe-core> (Abruf: 24.02.2026).

- Google (2025):** *Lighthouse: Automated auditing, performance, and accessibility tool for web pages*. URL: <https://developer.chrome.com/docs/lighthouse/overview> (Abruf: 28.02.2026).
- JavaScript With Syntax For Types*. (2026). URL: <https://www.typescriptlang.org/> (Abruf: 14.03.2026).
- Meta Open Source (2026a):** *React Documentation: Render and Commit*. URL: <https://react.dev/learn/render-and-commit> (Abruf: 05.03.2026).
- Meta Open Source (2026b):** *React: A JavaScript library for building user interfaces*. URL: <https://react.dev/> (Abruf: 05.03.2026).
- Microsoft (2026):** *Playwright: Fast and reliable end-to-end testing for modern web apps*. URL: <https://playwright.dev/docs/intro> (Abruf: 24.02.2026).
- Ollama (2026):** *Ollama Documentation: Quickstart Guide*. URL: <https://docs.ollama.com/quickstart> (Abruf: 04.03.2026).
- OpenAI (2026):** *Create completion*. URL: <https://developers.openai.com/api/reference/resources/completions/methods/create/> (Abruf: 01.03.2026).
- Robie, Jonathan (2026):** *What is the Document Object Model?* URL: <https://www.w3.org/TR/WD-DOM/introduction.html> (Abruf: 04.03.2026).
- WebAIM (2026a):** *Wave Web Accessibility Evaluation Tools*. URL: <https://wave.webaim.org/> (Abruf: 28.02.2026).

Rechtliche Grundlagen und Standards

- Bundesministerium für Digitales und Staatsmodernisierung (2025):** *Barrierefreie Informationstechnik-Verordnung (BITV 2.0)*. URL: <https://www.barrierefreiheit-dienstekonsolidierung.bund.de/Webs/PB/DE/gesetze-und-richtlinien/bitv2-0/bitv2-0-node.html> (Abruf: 24.02.2026).
- Deutscher Bundestag (2026):** *BFSG (Barrierefreiheitsstärkungsgesetz)*. URL: <https://bfsg-gesetz.de/> (Abruf: 24.02.2026).
- Europäisches Parlament und Rat der Europäischen Union (2016):** *Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates vom 27. April 2016 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten, zum freien Datenverkehr und zur Aufhebung der Richtlinie 95/46/EG (Datenschutz-Grundverordnung)*. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Abruf: 10.03.2026).
- European Commission (2026):** *European accessibility act (EAA)*. URL: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/european-accessibility-act-eaa_en (Abruf: 24.02.2026).
- Landesrecht BW (2015):** *Gesetz zur Errichtung der Landesoberbehörde BITBW*. URL: <https://www.landesrecht-bw.de/bsbw/document/jlr-ITErGBWV3P2> (Abruf: 28.02.2026).
- United Nations (2006):** *Convention on the Rights of Persons with Disabilities*. URL: <https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities> (Abruf: 28.02.2026).

World Wide Web Consortium (W3C) (2018): *Web Content Accessibility Guidelines (WCAG) 2.1*. URL: <https://www.w3.org/TR/WCAG21/> (Abruf: 24.02.2026).

World Wide Web Consortium (W3C) (2023): *Web Content Accessibility Guidelines (WCAG) 2.2*. URL: <https://www.w3.org/TR/WCAG22/> (Abruf: 24.02.2026).

Erklärung zur Verwendung generativer KI-Systeme

Bei der Erstellung der eingereichten Arbeit habe ich auf künstlicher Intelligenz (KI) basierte Systeme benutzt:

☒ ja

☐ nein²¹⁵

Falls ja: Die nachfolgend aufgeführten auf künstlicher Intelligenz (KI) basierten Systeme habe ich bei der Erstellung der eingereichten Arbeit benutzt:

1. Claude (Anthropic)
2. ChatGPT (OpenAI)
3. Microsoft Copilot

Ich erkläre, dass ich

- mich aktiv über die Leistungsfähigkeit und Beschränkungen der oben genannten KI-Systeme informiert habe,²¹⁶
- die aus den oben angegebenen KI-Systemen direkt oder sinngemäß übernommenen Passagen gekennzeichnet habe,
- überprüft habe, dass die mithilfe der oben genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- mir bewusst bin, dass ich als Autorin bzw. Autor dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Die oben genannten KI-Systeme habe ich wie im Folgenden dargestellt eingesetzt:

²¹⁵Die Erklärung ist in jedem Fall zu unterzeichnen, auch wenn Sie keine KI-Systeme genutzt haben und Ihr Kreuz bei „nein“ gesetzt haben.

²¹⁶U.a. gilt es hierbei zu beachten, dass an KI weitergegebene Inhalte ggf. als Trainingsdaten genutzt und wiederverwendet werden. Dies ist insb. für betriebliche Aspekte als kritisch einzustufen.

Arbeitsschritt	KI-System(e)	Verwendungsweise
Ideenfindung und Strukturierung	Claude (Anthropic)	Unterstützung bei der Strukturierung der Arbeit, Formulierung von Forschungsfragen sowie Gliederung einzelner Kapitel. Inhalte wurden kritisch geprüft und eigenständig überarbeitet.
Sprachliche Überarbeitung und Glättung	ChatGPT (OpenAI)	Unterstützung bei der sprachlichen Verbesserung einzelner Textpassagen (Grammatik, Stil). Inhaltliche Aussagen wurden nicht automatisiert übernommen.
Code-Unterstützung	GitHub Copilot (Microsoft)	Unterstützung bei der Erklärung und Optimierung von Codefragmenten im Rahmen der Implementierung. Der finale Code wurde eigenständig entwickelt und validiert.

(Ort, Datum)

(Unterschrift)

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: *Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

(Ort, Datum)

(Unterschrift)